DEVOPS ENGINEERING MASTERY

KUBERNETES & CONTAINER ORCHESTRATION

Kubernetes Architecture, Cluster Management, Workload Orchestration ☸ Production Operations

VOLUME 07 • FIRST EDITION

```
nahid@technova:~$ kubectl get pods -n production
✓ technova-api-7d9f4c8b6f-x2k9p   1/1   Running
nahid@technova:~$ kubectl rollout status deployment/technova-api
✓ deployment "technova-api" successfully rolled out
nahid@technova:~$ helm upgrade technova ./charts/technova
```

Kubernetes • kind • Helm • Kustomize • MERN

First Edition • Published 9 November 2026 • Dhaka, Bangladesh

24 Chapters • 24 Guided Labs • 3 Major Projects • 120 MCQs

Kubernetes & Container Orchestration

DEVOPS ENGINEERING MASTERY — Volume 07

Kubernetes Architecture, Cluster Management, Workload Orchestration ও Production Operations

First Edition — Published 9 November 2026.

Hands-on environment: kind, Minikube and k3d local clusters, kubectl, Helm and Kustomize, with Node.js 20, Docker, and a MERN (MongoDB, Express, React, Node.js) example application called TechNova. All domains, IP addresses and credentials in examples are fictional (technova.example, 203.0.113.x, CHANGE-ME-...). Sample terminal outputs are illustrative. Managed Kubernetes comparisons (AWS EKS, Microsoft Azure AKS, Google GKE, IBM Cloud Kubernetes Service) are high-level and operational; provider details vary by region and plan, and official provider documentation is the authority — no pricing or service limits are stated in this book. Kubernetes does not build container images, and Docker Engine is not a mandatory node runtime — this book uses containerd and CRI-O as the runtimes. Never expose a Kubernetes API server publicly and never practise on a production cluster.

How to use this book. Read the parts in order — each part builds on the previous one. Keep a Docker-ready machine with kind or Minikube beside you and do every hands-on exercise and every lab in the companion Lab Workbook before moving on. Do not copy YAML blindly; read the explanation under each configuration field first. Destructive operations are always flagged with a WARNING box and a safe alternative. Never practise on a production cluster: use a local cluster and a disposable environment, and never paste a real secret into an example — use the fictional placeholders shown.

A note on honesty. Reading a book alone does not make anyone an expert. This book teaches the foundations and the practical judgment to progress toward professional engineering — but the judgment itself comes from doing the labs, breaking a test cluster, fixing it, and repeating. There is no shortcut for that practice.

Conventions. Bangla is the explanatory language; technical terms, command names, APIs and configuration keys stay in English. `monospace` marks commands, file paths and code. **CONCEPT** explains an idea, **REAL WORLD** shows an industry scenario, **WARNING** marks risk, **PRO TIP** gives professional advice, and **LAB** marks hands-on work.

© 2026 DevOps Engineering Mastery series. All rights reserved. This book is for personal learning. Cluster changes should first be practised on a local cluster, never directly on a production system. The author and publisher accept no liability for data loss or service disruption caused by running commands without a backup or an approval gate.

Who this book is for

A learner who knows JavaScript, React, Next.js, Node.js, Express and MongoDB, understands basic Git and Linux concepts, has studied Linux server administration, CI/CD and Docker, and wants to design, deploy, secure, operate and troubleshoot real Kubernetes systems — from a first local cluster to a production platform with monitoring, backup and GitOps.

Learning outcomes

By the end of this book you will be able to: explain why container orchestration exists and how Kubernetes differs from Docker Compose; describe the control plane and worker node internals, the API Server request lifecycle, etcd, scheduling and reconciliation; write correct YAML manifests for Pods, Deployments, StatefulSets, Jobs, Services, Ingress, ConfigMaps, Secrets, PVs and RBAC; design Kubernetes networking, DNS, storage and traffic management; secure clusters with RBAC, Pod Security Standards and NetworkPolicies; troubleshoot CrashLoopBackOff, ImagePullBackOff, Pending Pods and cluster failures; monitor clusters with Metrics Server, logs and metrics; package applications with Helm and Kustomize; plan upgrades, etcd backup and disaster recovery; compare managed Kubernetes offerings; and operate a production Kubernetes platform with GitOps and runbooks.

সূচিপত্র — Contents

PART I — Kubernetes Foundations & Architecture

Chapter 01 — Why Container Orchestration — Docker Compose vs Kubernetes 7

Chapter 02 — Kubernetes Architecture — Control Plane and Worker Nodes 15

Chapter 03 — API Server, etcd, Scheduler, Controller Manager, kubelet, kube-proxy, and Container Runtime

Chapter 04 — Kubernetes Objects, Declarative Model, YAML Manifests, and API Resources 31 22

Chapter 05 — Local and Managed Cluster Setup — kind, Minikube, k3d, and Managed Kubernetes 39

PART II — Workloads & Application Management

Chapter 06 — Pods, Lifecycle, Probes, Init Containers, and Sidecars 48

Chapter 07 — ReplicaSets and Deployments 58

Chapter 08 — StatefulSets, DaemonSets, Jobs, and CronJobs 65

Chapter 09 — Namespaces, Labels, Selectors, and Annotations 73

Chapter 10 — ConfigMaps, Secrets, and Application Configuration 80

Chapter 11 — Resource Requests, Limits, QoS Classes, and Horizontal Pod Autoscaler 88

PART III — Networking, Storage & Traffic Management

Chapter 12 — Kubernetes Networking Model and CNI 97

Chapter 13 — Services — ClusterIP, NodePort, LoadBalancer, and ExternalName 106

Chapter 14 — Kubernetes DNS and Service Discovery 115

Chapter 15 — Ingress, Ingress Controllers, TLS, and Gateway API 123

Chapter 16 — PersistentVolumes, PersistentVolumeClaims, StorageClasses, and CSI 133

PART IV — Security & Operations

Chapter 17 — RBAC, ServiceAccounts, Authentication, and Authorization 143

Chapter 18 — Pod Security Standards, SecurityContext, and NetworkPolicies 152

Chapter 19 — Cluster Troubleshooting and Common Kubernetes Failures 161

Chapter 20 — Monitoring, Logging, Metrics Server, and Resource Observability 170

PART V — Production Kubernetes Engineering

Chapter 21 — Helm — Kubernetes Package Management 180

Chapter 22 — Kustomize and Environment Overlays 189

Chapter 23 — Cluster Upgrades, etcd Backup, Disaster Recovery, and Managed Kubernetes 199

Chapter 24 — Production Architecture, Reliability, GitOps Introduction, and Operations Runbooks 208

Companion Volumes

Lab Workbook — 24 Guided Labs • 15 Assignments • 3 Major Projects • Capstone

Assessment and Answer Guide — 120 MCQs • Short Answers • Scenarios • Case Studies • Final Exam

Quick Reference Handbook — Glossary (241 Terms) • YAML References • Checklists • Decision Trees

Guided labs (24), major projects (3) and the capstone are in the separate *Kubernetes Lab Workbook*. MCQs, assignments, examinations and the answer key are in the separate *Assessment and Answer Guide*. Glossary, YAML references, checklists and decision trees are in the separate *Quick Reference Handbook*.

PART I • DEVOPS ENGINEERING MASTERY

Kubernetes Foundations & Architecture

কন্টেইনার অর্কেস্ট্রেশনের প্রয়োজনীয়তা, Kubernetes আর্কিটেকচার, Control Plane ও Worker Node, API Server, etcd, Scheduler, YAML ম্যানিফেস্ট ও লোকাল/ম্যানেজড ক্লাস্টার সেটআপ।

অধ্যায় ১ • PART I — Kubernetes Foundations & Architecture

Chapter 1 — Why Container Orchestration — Docker Compose vs Kubernetes

১.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি ব্যাখ্যা করতে পারবেন কেন একটি কন্টেইনার চালানো আর শত শত কন্টেইনার পরিচালনা করা একই সমস্যা নয়। আপনি **Container Orchestration** (কন্টেইনার অর্কেস্ট্রেশন — অনেকগুলো কন্টেইনারকে সমন্বিতভাবে চালানো, সাজানো ও পরিচালনার ব্যবস্থা) বলতে কী বোঝায় তা বুঝবেন, Docker Compose-এর সীমা চিনতে পারবেন, এবং Kubernetes কখন প্রয়োজন আর কখন অতিরিক্ত তা যুক্তি দিয়ে সিদ্ধান্ত নিতে পারবেন।

আরও নির্দিষ্টভাবে আপনি শিখবেন: একটিমাত্র হোস্টে (host — যে কম্পিউটার বা সার্ভারে কন্টেইনার চলে) কন্টেইনার চালানোর সীমাবদ্ধতা কী; মাল্টি-হোস্ট **Scheduling** (শিডিউলিং — কোন কন্টেইনার কোন মেশিনে চলবে তা ঠিক করা), **Self-Healing** (সেলফ-হিলিং — ব্যর্থ কন্টেইনার নিজে থেকে আবার চালু হওয়া), **Scaling** (স্কেলিং — চাহিদা অনুযায়ী কন্টেইনারের সংখ্যা বাড়ানো-কমানো) এবং **Service Discovery** (সার্ভিস ডিসকভারি — একটি সার্ভিস আরেকটি সার্ভিসকে খুঁজে পাওয়ার ব্যবস্থা) কেন অপরিহার্য; এবং Compose বনাম Kubernetes-এর একটি বাস্তবসম্মত তুলনা।

১.২ কেন এই বিষয়টি গুরুত্বপূর্ণ

Docker শিখে প্রথম কন্টেইনার চালানো রোমাঞ্চকর: একটি কমান্ড, আর আপনার অ্যাপ চলছে। কিন্তু প্রোডাকশনে (production — বাস্তব ব্যবহারকারীদের জন্য চালু থাকা পরিবেশ) গল্পটা বদলে যায়। TechNova-র মতো একটি কোম্পানির কথা ভাবুন — তাদের ওয়েব অ্যাপ, API, ডেটাবেস, ক্যাশ, ব্যাকগ্রাউন্ড জব — সব মিলিয়ে হয়তো বিশটি সার্ভিস। একটি সার্ভারে সব চালালে সেই সার্ভারটি নষ্ট হলে সব থেমে যাবে। ট্রাফিক বাড়লে হাতে করে কন্টেইনার বাড়াতে হবে। একটি কন্টেইনার ক্র্যাশ করলে কেউ না দেখলে তা মৃত অবস্থায় পড়ে থাকবে।

এই সমস্যাগুলো একটিমাত্র টুল দিয়ে সমাধান হয় না — প্রয়োজন একটি সিস্টেমের, যা সার্বক্ষণিক নজর রাখে, সিদ্ধান্ত নেয় এবং নিজে থেকে সংশোধন করে। সেই সিস্টেমই অর্কেস্ট্রেটর। এই বইয়ের বাকি সব অধ্যায় Kubernetes-এর ভেতরের কাজ বোঝাবে; এই অধ্যায় সেই প্রয়োজনের ভিত্তি গড়ে দেয়। ভিত্তি না বুঝলে Kubernetes শুধু জটিল YAML-এর স্তূপ মনে হবে।

> **CONCEPT:** অর্কেস্ট্রেশন মানে কন্টেইনার চালানো নয় — অর্কেস্ট্রেশন মানে কাঙ্ক্ষিত অবস্থা (desired state) ঘোষণা করা এবং সিস্টেমকে সেই অবস্থা বজায় রাখতে দেওয়া। আপনি বলেন "তিনটি কপি চাই", সিস্টেম সার্বক্ষণিক তা নিশ্চিত করে।

১.৩ পূর্বশর্ত জ্ঞান

এই অধ্যায়ের জন্য আপনার Docker-এর মৌলিক ধারণা থাকা দরকার: ইমেজ (image — অ্যাপ ও তার নির্ভরতাসহ প্যাকেজ), কন্টেইনার (container — ইমেজ থেকে চলমান উদাহরণ), `docker run`, `docker ps`, পোর্ট ম্যাপিং এবং ভলিউম। Docker Compose-এর `docker-compose.yml` ফাইল দেখে থাকলে ভালো, না দেখলেও চলবে — এই অধ্যায়েই তা ব্যাখ্যা করা হয়েছে। Linux কমান্ড লাইন ও নেটওয়ার্কের মৌলিক ধারণা (IP, পোর্ট, DNS) সহায়ক হবে। Kubernetes-এর পূর্ব অভিজ্ঞতা প্রয়োজন নেই।

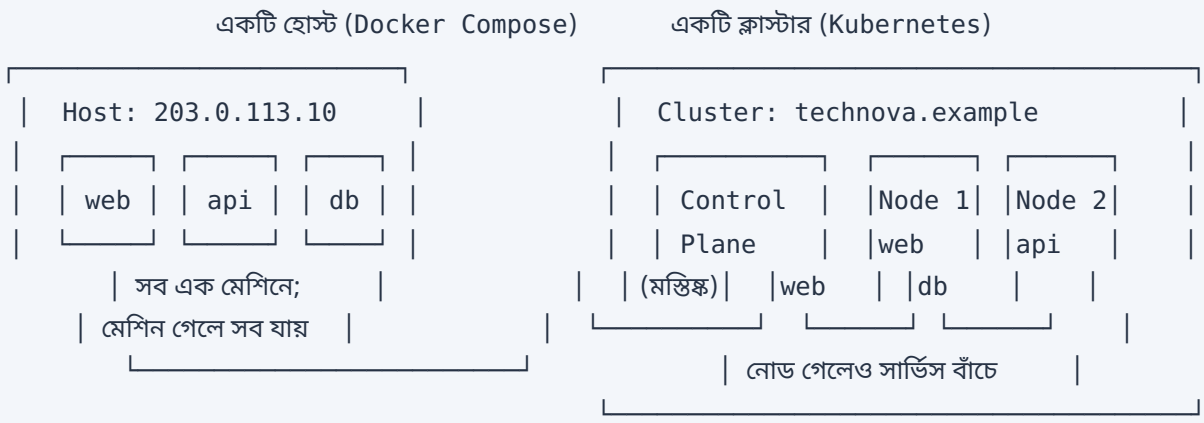
১.৪ প্রথম নীতি থেকে ব্যাখ্যা

প্রথম নীতি থেকে ভাবুন: একটি অ্যাপ চালাতে কী কী লাগে? CPU, মেমোরি, ডিস্ক, নেটওয়ার্ক — অর্থাৎ সম্পদ (resources)। একটি কম্পিউটারে সম্পদ সীমিত। একটি অ্যাপ বড় হলে তাকে একাধিক কম্পিউটারে ভাগ করতে হয়। ভাগ করলেই নতুন প্রশ্ন আসে: কোন অংশ কোন কম্পিউটারে যাবে? কোনো কম্পিউটার নষ্ট হলে তার অংশ কোথায় যাবে? অংশগুলো একে অপরকে কীভাবে খুঁজবে? লোড বাড়লে অংশের সংখ্যা কে বাড়াবে?

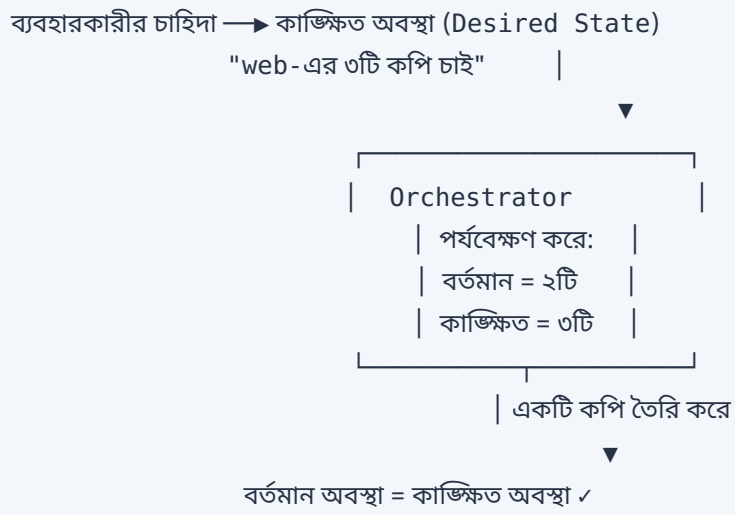
এই প্রশ্নগুলোর উত্তর মানুষ হাতে দিলে তা ধীর, ভুলপ্রবণ ও রাত তিনটায় ঘুম ভাঙানো কাজ হয়। অর্কেস্ট্রেশনের মূল ধারণা হলো: এই সিদ্ধান্তগুলোকে নিয়ম হিসেবে লিখে রাখা, আর একটি নিয়ন্ত্রণ ব্যবস্থাকে সার্বক্ষণিক সেই নিয়ম প্রয়োগ করতে দেওয়া। Docker Compose নিয়ম লিখতে পারে, কিন্তু তা একটি মেশিনের ভেতরে সীমাবদ্ধ। Kubernetes সেই নিয়মকে একটি ক্লাস্টার (cluster — একসাথে কাজ করা অনেকগুলো মেশিনের সমষ্টি) জুড়ে প্রয়োগ করে।

একটি উপমা: Compose হলো একজন দক্ষ রাঁধুনি যিনি একটি রান্নাঘরে সব সামলান। Kubernetes হলো একটি রেস্তোরাঁ চেইনের কেন্দ্রীয় ব্যবস্থাপনা — কোন শাখায় কতজন রাঁধুনি লাগবে, কোনো শাখা বন্ধ হলে অর্ডার কোথায় যাবে, নতুন শাখা কীভাবে খুলবে — সবই নিয়ম অনুযায়ী স্বয়ংক্রিয়।

১.৫ আর্কিটেকচার / ধারণাগত চিত্র



চিত্র ১.১: একক হোস্ট বনাম ক্লাস্টার — ব্যর্থতার ক্ষেত্র (failure domain) কীভাবে বদলায়।



চিত্র ১.২: Reconciliation loop — অর্কেস্ট্রেটর সার্বক্ষণিক বর্তমান অবস্থাকে কাঙ্ক্ষিত অবস্থার দিকে টানে।

১.৬ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

১.৬.১ একক হোস্টের সীমা

একটি হোস্টে সম্পদ সীমিত — CPU, মেমোরি শেষ হলে আর কন্টেইনার যোগ করা যায় না। হোস্টটি একটি *single point of failure* (একক ব্যর্থতার বিন্দু): ডিস্ক নষ্ট হলে, কার্নেল ক্র্যাশ করলে বা ডেটা সেন্টারের বিদ্যুৎ গেলে সব সার্ভিস একসাথে বন্ধ। আপগ্রেড করতে হলে ডাউনটাইম লাগে। বড় অ্যাপে এক হোস্টে সব চালানো তাই ঝুঁকি ও স্কেল — দুই দিক থেকেই সীমাবদ্ধ।

১.৬.২ Scheduling — কোন কন্টেইনার কোথায় চলবে

ক্লাস্টারে অনেক নোড থাকলে সিদ্ধান্ত লাগে: নতুন কন্টেইনার কোন নোডে বসবে? যে নোডে যথেষ্ট CPU ও মেমোরি খালি আছে, যেখানে একই অ্যাপের অন্য কপি নেই (যাতে এক নোড গেলে সব কপি না যায়), অথবা যেখানে বিশেষ হার্ডওয়্যার (যেমন GPU) আছে — এসব বিবেচনা করে শিডিউলার সিদ্ধান্ত নেয়। মানুষ হাতে এটি করলে ভুল হবেই; অর্কেস্ট্রেটর প্রতিবার নিয়ম মেনে করে।

১.৬.৩ Self-Healing — ব্যর্থতা থেকে নিজে থেকে পুনরুদ্ধার

কন্টেইনার ক্র্যাশ করলে, নোড নষ্ট হলে, বা স্বাস্থ্য পরীক্ষায় (health check) ব্যর্থ হলে অর্কেস্ট্রেটর নিজে থেকে নতুন কপি চালু করে, ব্যর্থ কপি সরিয়ে দেয়, এবং ট্রাফিক কেবল সুস্থ কপির দিকেই পাঠায়। একে **Self-Healing** বলে। এর মানে এই নয় যে বাগ সেরে যায় — এর মানে, বাগের প্রভাব ব্যবহারকারীর কাছে পৌঁছানোর আগেই সিস্টেম ক্ষতি সীমিত করে।

১.৬.৪ Scaling ও Service Discovery

Scaling দুই ধরনের: *horizontal* — কপির সংখ্যা বাড়ানো; *vertical* — প্রতিটি কপিকে বেশি CPU/মেমোরি দেওয়া। Kubernetes মূলত horizontal scaling-এ শক্তিশালী, এবং লোডের ভিত্তিতে স্বয়ংক্রিয়ভাবে কপি বাড়াতে-কমাতে পারে। **Service Discovery** মানে: `api` সার্ভিসটি `db`-কে IP ঠিকানা মুখস্থ না করে নাম ধরে ডাকতে পারে। কন্টেইনারের IP বদলালেও নাম স্থির থাকে — DNS ও অভ্যন্তরীণ লোড ব্যালান্সিং সেই নামকে সুস্থ কপিগুলোর দিকে নির্দেশ করে।

১.৬.৫ Rolling Update ও Rollback

নতুন সংস্করণ (version) আনার সময় অর্কেস্ট্রেটর একবারে সব কপি বদলায় না — ধীরে ধীরে পুরোনো কপি সরিয়ে নতুন কপি আনে, প্রতিটি ধাপে স্বাস্থ্য পরীক্ষা করে। একে **Rolling Update** বলে। নতুন সংস্করণে সমস্যা দেখা দিলে আগের সংস্করণে ফেরা যায় — এটাই **Rollback**। হাতে করা ডেপ্লয়মেন্টে এই নিরাপত্তা পাওয়া কঠিন।

১.৭ কনফিগারেশন উদাহরণ

একই অ্যাপ — প্রথমে Compose-এ, তারপর Kubernetes-এর ধারণায়। Compose ফাইলটি একটি হোস্টের জন্য; Kubernetes ম্যানিফেস্টটি ক্লাস্টারের জন্য (পূর্ণ ব্যাখ্যা অধ্যায় ৪-এ)।

```
# docker-compose.yml — TechNova ডেমো অ্যাপ, একক হোস্ট
services:
  web:
    image: technova/web:1.0.0
    ports:
      - "8080:80"
    depends_on:
      - api
  api:
    image: technova/api:1.0.0
    environment:
      - DATABASE_URL=postgres://app:CHANGE-ME-DB-PASSWORD@db:5432/appdb
  db:
    image: postgres:16
    environment:
      - POSTGRES_PASSWORD=CHANGE-ME-DB-PASSWORD
    volumes:
      - db-data:/var/lib/postgresql/data
volumes:
  db-data:
```

```
# Kubernetes-এ একই web সার্ভিসের ধারণা (সংক্ষিপ্ত, বিস্তারিত অধ্যায় ৪-এ)
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web
  namespace: technova
spec:
  replicas: 3          # তিনটি কপি চাই — অর্কেস্ট্রেটর তা বজায় রাখবে
  selector:
    matchLabels:
      app: web
  template:
    metadata:
      labels:
        app: web
    spec:
      containers:
        - name: web
          image: technova/web:1.0.0
          ports:
            - containerPort: 80
```

বিষয়	Docker Compose	Kubernetes
চালানোর ক্ষেত্র	একটি হোস্ট	অনেক নোডের ক্লাস্টার
Scheduling	নেই — সব এক হোস্টে	স্বয়ংক্রিয়, নিয়মভিত্তিক
Self-Healing	সীমিত (restart নীতি)	পূর্ণ — কন্টেইনার ও নোড স্তরে
Scaling	হাতে করে (`--scale`)	হাতে ও স্বয়ংক্রিয়ভাবে
Service Discovery	এক হোস্টের ভেতরে DNS নাম	ক্লাস্টারজুড়ে DNS ও Service
Rolling Update / Rollback	হাতে, ঝুঁকিপূর্ণ	অন্তর্নির্মিত
জটিলতা	কম — শেখা সহজ	বেশি — শেখার বক্ররেখা খাড়া
উপযুক্ত ব্যবহার	লোকাল ডেভ, ছোট ডেমো	প্রোডাকশন, বড় সিস্টেম

১.৮ বাস্তব প্রকৌশল পরিস্থিতি

TechNova পরিস্থিতি: TechNova-র ফ্ল্যাগশিপ অ্যাপ `technova.example`-এ একটি প্রোমোশন চলাকালে ট্রাফিক হঠাৎ দশ গুণ বেড়ে যায়। তাদের পুরোনো সেটআপে একটি সার্ভারে Docker Compose চলত। সার্ভারের CPU ১০০% ছুঁয়ে web কন্টেইনার ধীর হয়ে যায়, তারপর OOM (Out of Memory — মেমোরি শেষ হয়ে যাওয়া) হয়ে ক্র্যাশ করে। Compose-এর `restart: always` কন্টেইনার আবার চালু করে, কিন্তু একই সার্ভারে একই চাপ — আবার ক্র্যাশ। দলটি হাতে করে দ্বিতীয় সার্ভার যোগ করতে পারেনি, কারণ Compose দ্বিতীয় হোস্ট জানে না।

Kubernetes-এ একই পরিস্থিতিতে যা হতো: লোড বাড়লে অটোস্কেলার কপির সংখ্যা বাড়াত, শিডিউলার নতুন কপি অন্য নোডে বসাত, অসুস্থ কপি থেকে ট্রাফিক সরিয়ে নেওয়া হতো, আর একটি নোড সম্পূর্ণ নষ্ট হলেও বাকি নোডে সার্ভিস চলত। শিক্ষা: সমস্যাটি অ্যাপের কোডে ছিল না — সমস্যাটি ছিল পরিচালনার স্তরে। অর্কেস্ট্রেশন সেই স্তরের সমাধান।

> **REAL-WORLD:** ট্রাফিকের ঢেউয়ে প্রথমে যা ভাঙে তা সাধারণত অ্যাপ নয় — ভাঙে সেই ব্যবস্থা যা অ্যাপকে সম্পদ দেয়, পুনরায় চালু করে ও ট্রাফিক বণ্টন করে। তাই প্রোডাকশন আলোচনায় অর্কেস্ট্রেশন প্রথমে আসে, অ্যাপের ফিচার পরে।

১.৯ ধাপে ধাপে ব্যবহারিক উদাহরণ

Compose দিয়ে সীমাটি নিজে অনুভব করুন, তারপর Kubernetes-এর ধারণার সঙ্গে মিলিয়ে দেখুন।

```
# ১. Compose অ্যাপ চালু করুন
docker compose up -d
docker compose ps

# ২. একটি কন্টেইনার জোর করে থামান — Compose কী করে দেখুন
docker compose stop web
docker compose ps        # web থেমে আছে; নিজে থেকে ফেরে না (restart নীতি ছাড়া)

# ৩. আবার চালু করুন — এটি হাতে করা পুনরুদ্ধার
docker compose start web
```

```
# ৪. স্কেল করার চেষ্টা করুন — সব কপি একই হোস্টেই থাকবে
docker compose up -d --scale web=3
docker compose ps        # তিনটি web, একটিই হোস্ট, একটিই ব্যর্থতার বিন্দু
```

এবার ভাবুন: উপরের ২ নম্বর ধাপে যদি কেউ না দেখত, web কতক্ষণ বন্ধ থাকত? Kubernetes-এ এই প্রশ্নের উত্তর "সেকেন্ডের মধ্যে স্বয়ংক্রিয়ভাবে" — কারণ নিয়ন্ত্রণ ব্যবস্থা সার্বক্ষণিক পর্যবেক্ষণ করে। এই পার্থক্যটিই এই অধ্যায়ের মূল কথা।

১.১০ সাধারণ ভুল ও সমস্যা সমাধান

ভুল ধারণা / সমস্যা	বাস্তবতা ও সমাধান
"Kubernetes মানেই Docker-এর বড় সংস্করণ"	না — Kubernetes কন্টেইনার চালায় না, অর্কেস্ট্রেট করে; কন্টেইনার চালায় containerd/CRI-O-র মতো রানটাইম (অধ্যায় ৩)।
ছোট প্রজেক্টে সরাসরি Kubernetes নেওয়া	একটি স্ট্যাটিক সাইট বা দুটি সার্ভিসের জন্য Compose যথেষ্ট; অর্কেস্ট্রেশনের জটিলতার খরচ তখন লাভের চেয়ে বেশি।
Compose ফাইলকেই Kubernetes ম্যানিফেস্ট ভাবা	Compose ঘোষণা করে "এই হোস্টে কী চালাব"; Kubernetes ঘোষণা করে "ক্লাস্টারে কাঙ্ক্ষিত অবস্থা কী" — মডেল আলাদা (অধ্যায় ৪)।
Self-healing-কে বাগ-ফিক্স ভাবা	Self-healing ক্র্যাশ হওয়া কপি বদলায়, কিন্তু যে বাগ ক্র্যাশ ঘটায় তা সারায় না — লগ ও মেট্রিক দেখে মূল কারণ ঠিক করতে হয়।

TROUBLESHOOTING: "আমার অ্যাপ Compose-এ চলে, Kubernetes-এ চলে না" — প্রথমে দেখুন অ্যাপটি হার্ডকোড করা হোস্টনাম বা `localhost` ধরে নিয়েছে কি না। ক্লাস্টারে প্রতিটি সার্ভিসের নিজস্ব DNS নাম থাকে; `localhost` কেবল একই Pod-এর ভেতরে কাজ করে।

১.১১ নিরাপত্তা বিবেচনা

অর্কেস্ট্রেশন নিরাপত্তার পরিধি বাড়ায়: এখন শুধু অ্যাপ নয়, ক্লাস্টার পরিচালনার স্তরটিও সুরক্ষিত করতে হয়। Compose-এ যে ব্যক্তি হোস্টে ঢুকতে পারে সে-ই সব কন্টেইনার নিয়ন্ত্রণ করতে পারে; Kubernetes-এ API Server-এর মাধ্যমে সূক্ষ্ম অনুমতি (RBAC — Role-Based Access Control, ভূমিকাভিত্তিক প্রবেশাধিকার নিয়ন্ত্রণ) দেওয়া যায় — কে কী দেখতে ও বদলাতে পারবে তা আলাদা করা যায়। তবে এর উল্টো দিকও আছে: API Server-এ প্রবেশাধিকার পেলে আক্রমণকারী পুরো ক্লাস্টার নিয়ন্ত্রণ করতে পারে। তাই API Server কখনও ইন্টারনেটে উন্মুক্ত রাখা যাবে না, শক্তিশালী প্রমাণীকরণ (authentication) বাধ্যতামূলক, এবং সিক্রেট (গোপন তথ্য, যেমন পাসওয়ার্ড) কখনও ইমেজ বা YAML-এ সরাসরি লেখা যাবে না — শুধু `CHANGE-ME-...` ধরনের প্লেসহোল্ডার ব্যবহার করে বাইরের সিক্রেট ব্যবস্থাপনা থেকে সরবরাহ করতে হয়।

১.১২ কর্মক্ষমতা ও খরচ বিবেচনা

Kubernetes নিজেই সম্পদ খায়: Control Plane-এর কম্পোনেন্টগুলো CPU ও মেমোরি ব্যবহার করে, etcd ডিস্ক I/O চায়, আর প্রতিটি নোডে এজেন্ট চলে। ছোট সিস্টেমে এই ওভারহেড (overhead — মূল কাজের বাইরে অতিরিক্ত খরচ) মোট সম্পদের বড় অংশ হতে পারে। আবার বড় সিস্টেমে অর্কেস্ট্রেশন সম্পদ ব্যবহারের দক্ষতা বাড়ায় — খালি পড়ে থাকা ক্ষমতা কাজে লাগে, প্রয়োজনে স্কেল-ডাউন করে খরচ কমে। সিদ্ধান্তের সূত্র: যদি আপনার সার্ভিসের সংখ্যা, ট্রাফিকের ওঠানামা বা

নির্ভরযোগ্যতার চাহিদা হাতে সামলানো যায়, তবে Compose সস্তা; যখন হাতে সামলানো যায় না, তখন অর্কেস্ট্রেশনের খরচ তার লাভের তুলনায় ছোট।

১.১৩ সেরা অনুশীলন ও অ্যান্টি-প্যাটার্ন

সেরা অনুশীলন: ছোট দিয়ে শুরু করুন — লোকাল ক্লাস্টারে (অধ্যায় ৫) পরীক্ষা করুন, তারপর প্রোডাকশন ভাবুন। কাঙ্ক্ষিত অবস্থা YAML-এ লিখে Git-এ রাখুন (GitOps-এর ভিত্তি)। প্রতিটি সার্ভিসের স্বাস্থ্য পরীক্ষা (health check) সংজ্ঞায়িত করুন। সম্পদের সীমা (limits) নির্ধারণ করুন যাতে একটি সার্ভিস সব মেমোরি খেয়ে অন্যদের মারতে না পারে।

অ্যান্টি-প্যাটার্ন: সবকিছু একটিই বিশাল YAML-এ লেখা; `:latest` ট্যাগ ব্যবহার করা (কোন সংস্করণ চলছে বোঝা যায় না); সিক্রেট সরাসরি YAML-এ লেখা; প্রোডাকশনে প্রথম দিনেই অটোস্কেলিং চালু করে পরীক্ষা না করা; এবং সবচেয়ে বড়টি — প্রয়োজন না থাকতেও "সবাই ব্যবহার করে" বলে Kubernetes নেওয়া।

> **PRO TIP:** সিদ্ধান্ত নেওয়ার আগে একটি প্রশ্ন করুন: "আমার কি একাধিক মেশিন, স্বয়ংক্রিয় পুনরুদ্ধার ও স্বয়ংক্রিয় স্কেলিং — এই তিনটির অন্তত দুটি লাগবে?" উত্তর হ্যাঁ হলে Kubernetes বিবেচনা করুন; না হলে Compose-ই সঠিক পছন্দ।

১.১৪ অধ্যায়ের সারাংশ

একটি কন্টেইনার চালানো সহজ; অনেক কন্টেইনারকে নির্ভরযোগ্যভাবে, অনেক মেশিনে, ট্রাফিকের ওঠানামার মধ্যে চালানো কঠিন। সেই কঠিন কাজের নাম Container Orchestration। Docker Compose একটিমাত্র হোস্টে একাধিক কন্টেইনার সাজাতে পারে — লোকাল ডেভেলপমেন্ট ও ছোট ডেমোর জন্য চমৎকার। Kubernetes একটি ক্লাস্টারজুড়ে Scheduling, Self-Healing, Scaling, Service Discovery, Rolling Update ও Rollback স্বয়ংক্রিয় করে — প্রোডাকশন ও বড় সিস্টেমের জন্য তৈরি। পছন্দটি "কোনটি ভালো" নয়, পছন্দটি "আমার সমস্যার আকার কত" — সেই অনুযায়ী।

১.১৫ মূল প্রযুক্তিগত পরিভাষা

Container Orchestration: অনেকগুলো কন্টেইনারকে একাধিক মেশিনে সমন্বিতভাবে চালানো, সাজানো, পর্যবেক্ষণ ও পুনরুদ্ধারের স্বয়ংক্রিয় ব্যবস্থা।

Cluster: একসাথে কাজ করা অনেকগুলো মেশিনের (নোডের) সমষ্টি, যা বাইরে থেকে একটি সিস্টেম হিসেবে আচরণ করে।

Desired State: কাঙ্ক্ষিত অবস্থা — সিস্টেম যেমন থাকা উচিত তার ঘোষণা (যেমন "web-এর ৩টি কপি")।

Self-Healing: ব্যর্থ কন্টেইনার বা নোড নিজে থেকে শনাক্ত করে নতুন কপি চালু করে সার্ভিস সচল রাখা।

Service Discovery: একটি সার্ভিসের অন্য সার্ভিসকে নাম ধরে খুঁজে পাওয়ার ব্যবস্থা, IP মুখস্থ না করে।

Rolling Update: ধীরে ধীরে, ধাপে ধাপে পুরোনো কপি বদলে নতুন সংস্করণ আনা, যাতে সার্ভিস বন্ধ না হয়।

১.১৬ জ্ঞান যাচাই

১. Docker Compose একটিমাত্র হোস্টে সীমাবদ্ধ থাকার ফলে প্রোডাকশনে কোন তিনটি ঝুঁকি তৈরি হয়?

২. Desired State ও Actual State (বর্তমান অবস্থা)-এর মধ্যে পার্থক্য কী? অর্কেস্ট্রেটর এই পার্থক্য কীভাবে দূর করে?

৩. একটি কন্টেইনার ক্র্যাশ করলে Compose-এর `restart: always` ও Kubernetes-এর Self-Healing-এর মধ্যে পার্থক্য কোথায়?

৪. Service Discovery না থাকলে মাইক্রোসার্ভিসগুলো একে অপরকে খুঁজতে কী সমস্যায় পড়বে? একটি উদাহরণ দিন।

৫. Rolling Update-এর সময় নতুন সংস্করণে বাগ ধরা পড়লে কী করা উচিত, এবং কেন একবারে সব কপি বদলানো ঝুঁকিপূর্ণ?

৬. আপনার একটি ব্যক্তিগত ব্লগ (একটি web + একটি db) আছে — Compose না Kubernetes, কোনটি বেছে নেবেন এবং কেন?

৭. Scheduling বলতে কী বোঝায়? শিডিউলার কোন কোন বিষয় বিবেচনা করে একটি কন্টেইনার কোথায় বসাবে?

১.১৭ ব্যবহারিক অ্যাসাইনমেন্ট

অ্যাসাইনমেন্ট: TechNova-র ডেমো অ্যাপটি Docker Compose দিয়ে চালু করুন (উপরের `docker-compose.yml` ব্যবহার করে, পাসওয়ার্ডের জায়গায় `CHANGE-ME-DB-PASSWORD`-এর বদলে নিজের একটি নিরাপদ মান দিন)। তারপর (ক) একটি কন্টেইনার থামিয়ে দেখুন কী হয়; (খ) `--scale web=3` দিয়ে স্কেল করুন; (গ) একটি এক পৃষ্ঠার প্রতিবেদন লিখুন: Compose-এ কোন কোন সমস্যা হাতে সমাধান করতে হলো, আর Kubernetes থাকলে কোনগুলো স্বয়ংক্রিয় হতো? প্রতিবেদনে TechNova-র প্রোমোশন-দিনের পরিস্থিতি (১.৮) উল্লেখ করুন।

অধ্যায় ২ • PART I — Kubernetes Foundations & Architecture

Chapter 2 — Kubernetes Architecture — Control Plane and Worker Nodes

২.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি Kubernetes ক্লাস্টারের দুটি প্রধান অংশ — **Control Plane** (কন্ট্রোল প্লেন — ক্লাস্টারের মস্তিষ্ক, যা সিদ্ধান্ত নেয়) ও **Worker Node** (ওয়ার্কার নোড — যে মেশিনে আসল অ্যাপ চলে) — এর ভূমিকা আলাদা করে ব্যাখ্যা করতে পারবেন। আপনি প্রতিটি Control Plane কম্পোনেন্টের নাম ও কাজ জানবেন, নোডে কী কী চলে তা বলতে পারবেন, High Availability (উচ্চ প্রাপ্যতা — কোনো অংশ নষ্ট হলেও সিস্টেম চালু থাকা) কীভাবে অর্জিত হয় তা বুঝবেন, এবং একটি ব্যবহারকারীর অনুরোধ ক্লাস্টারের ভেতর দিয়ে কীভাবে প্রবাহিত হয় তা ধাপে ধাপে অনুসরণ করতে পারবেন।

২.২ কেন এই বিষয়টি গুরুত্বপূর্ণ

Kubernetes শেখার সময় সবচেয়ে সাধারণ ভুল হলো সব কম্পোনেন্টকে একসাথে গুলিয়ে ফেলা — মনে হয় সবকিছুই সব জায়গায় চলে। বাস্তবে ক্লাস্টারটি দুটি স্পষ্ট দায়িত্বে ভাগ করা: সিদ্ধান্ত নেওয়া বনাম কাজ করা। Control Plane সিদ্ধান্ত নেয় — "কী চালাতে হবে, কোথায় চালাতে হবে, কেমন অবস্থা থাকা উচিত"। Worker Node সেই সিদ্ধান্ত কার্যকর করে — কন্টেইনার চালায়, নেটওয়ার্ক সংযোগ দেয়, স্বাস্থ্য জানায়।

এই বিভাজন না বুঝলে ট্রাবলশুটিং অসম্ভব হয়ে যায়: Pod চালু হচ্ছে না — সমস্যা কি শিডিউলারের, নাকি নোডের kubelet-এর, নাকি API Server-এর? প্রতিটি সমস্যার ঠিকানা আলাদা। TechNova-র প্রোডাকশন ক্লাস্টারে একবার API Server ধীর হয়ে গিয়েছিল — যিনি আর্কিটেকচার বুঝতেন তিনি জানতেন অ্যাপ তখনও চলছে (কারণ Worker Node আলাদা), শুধু নতুন পরিবর্তন করা যাচ্ছে না। যিনি বুঝতেন না তিনি ভেবেছিলেন পুরো সিস্টেম ভেঙে গেছে। আর্কিটেকচার জানা মানে আতঙ্কের বদলে সঠিক জায়গায় তাকানো।

> **CONCEPT:** Control Plane হলো মস্তিষ্ক, Worker Node হলো হাত-পা। মস্তিষ্ক থেমে গেলে নতুন নির্দেশ আসে না, কিন্তু হাত-পা তখনও আগের নির্দেশ অনুযায়ী কাজ চালিয়ে যায় — এই বিচ্ছিন্নতাই Kubernetes-কে স্থিতিস্থাপক (resilient) করে।

২.৩ পূর্বশর্ত জ্ঞান

অধ্যায় ১-এর ধারণাগুলো — Desired State, Scheduling, Self-Healing, Service Discovery — জানা থাকতে হবে। Linux সার্ভিস ও প্রক্রিয়া (process) সম্পর্কে সাধারণ ধারণা এবং নেটওয়ার্কের মৌলিক জ্ঞান (IP, পোর্ট, DNS, লোড ব্যালান্সার) সহায়ক। etcd, API Server ইত্যাদি নাম আগে না জানলেও চলবে — এই অধ্যায়ে পরিচয়, পরের অধ্যায়ে গভীর আলোচনা।

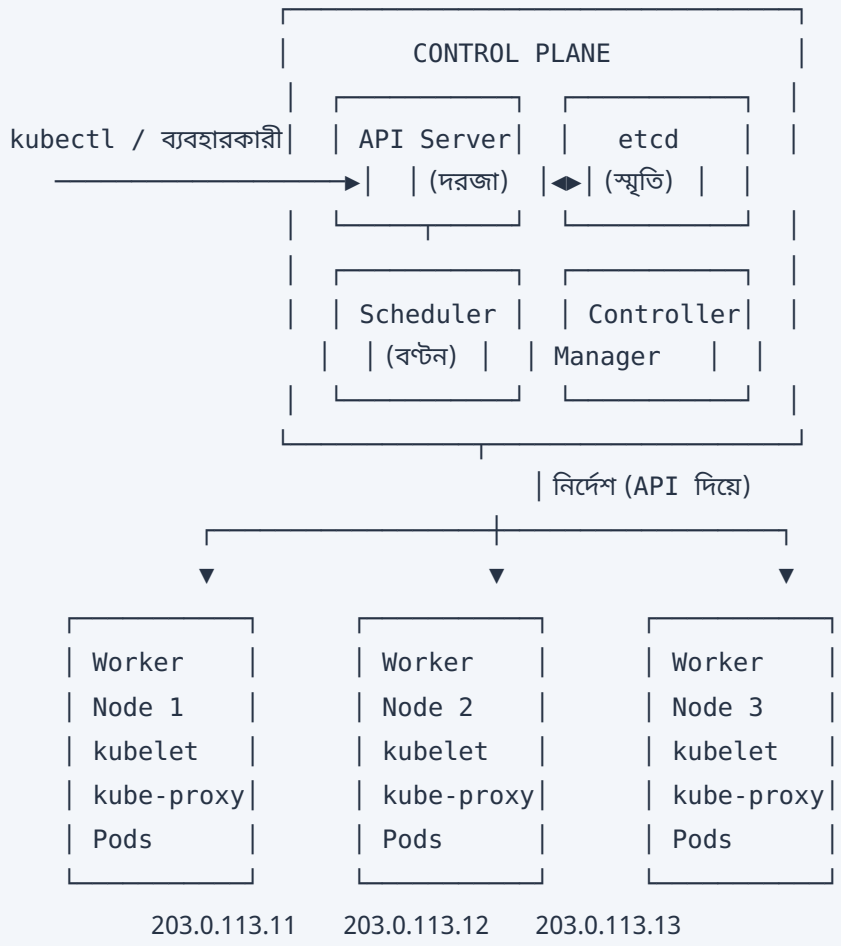
২.৪ প্রথম নীতি থেকে ব্যাখ্যা

প্রথম নীতি: কোনো বড় সিস্টেম পরিচালনা করতে হলে দুটি কাজ আলাদা করতে হয় — পরিকল্পনা ও সম্পাদন। একজন স্থপতি বাড়ির নকশা করেন, রাজমিস্ত্রি সেই নকশা অনুযায়ী ইট গাঁথেন। স্থপতি প্রতিটি ইট নিজে বসান না, রাজমিস্ত্রি নকশা বদলান না। Kubernetes-এ Control Plane স্থপতি, Worker Node রাজমিস্ত্রি।

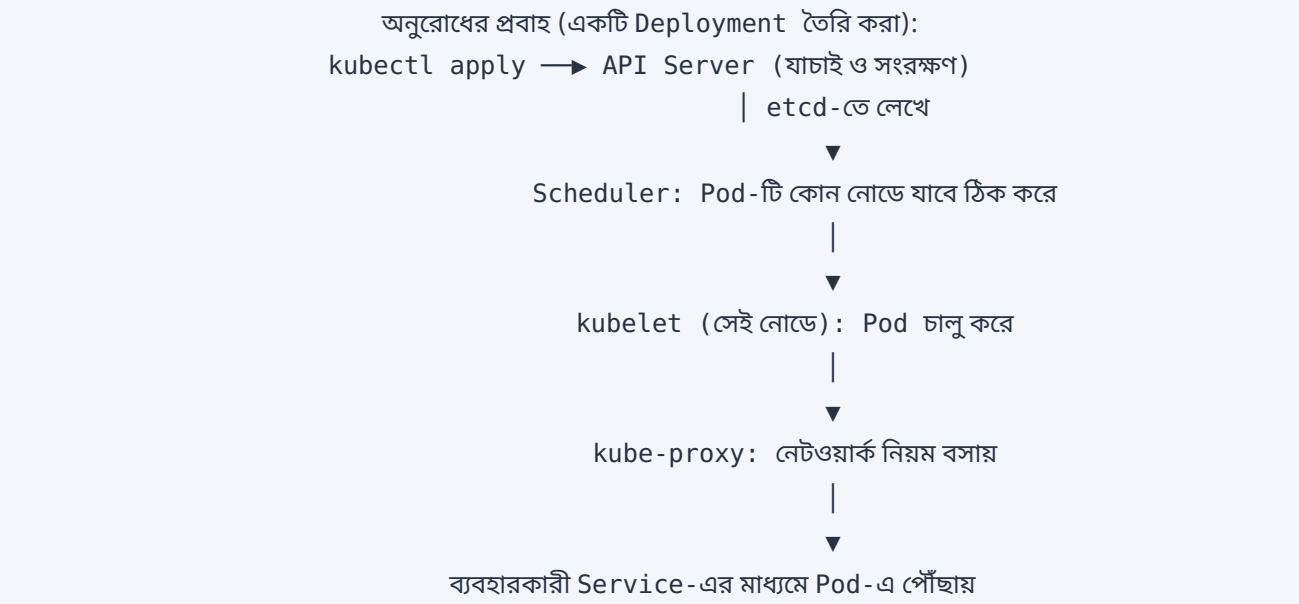
কেন এই বিভাজন? তিনটি কারণে। প্রথম, একক সত্যের উৎস (single source of truth): ক্লাস্টারের অবস্থা সম্পর্কে একটি মাত্র সিদ্ধান্ত-কেন্দ্র থাকলে দ্বিমত হয় না। দ্বিতীয়, স্কেল: কাজের চাপ বাড়লে Worker Node যোগ করা যায়, মস্তিষ্ক না বাড়িয়েই। তৃতীয়, ব্যর্থতার বিচ্ছিন্নতা (failure isolation): একটি Worker Node নষ্ট হলে মস্তিষ্ক অক্ষত থাকে এবং অন্য নোডে কাজ সরিয়ে নিতে পারে; আবার মস্তিষ্কের একটি অংশ থেমে গেলেও চলমান অ্যাপ থেমে যায় না।

এই বিভাজনের ভেতরে আরেকটি নীতি আছে: Control Plane-এর কম্পোনেন্টগুলো একে অপরের সঙ্গে সরাসরি কথা বলে না — তারা সবাই API Server-এর মাধ্যমে কথা বলে। API Server হলো ক্লাস্টারের একমাত্র দরজা। এই নকশা (যাকে *hub-and-spoke* বলা যায়) সিস্টেমকে বোঝা ও পরীক্ষা করা সহজ করে।

২.৫ আর্কিটেকচার / ধারণাগত চিত্র



চিত্র ২.১: Kubernetes ক্লাস্টারের সামগ্রিক আর্কিটেকচার — Control Plane ও Worker Node।



চিত্র ২.২: একটি অনুরোধ কীভাবে Control Plane থেকে Worker Node-এ পৌঁছায়।

২.৬ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

২.৬.১ Control Plane — ক্লাস্টারের মস্তিষ্ক

Control Plane চারটি প্রধান কম্পোনেন্ট নিয়ে গঠিত (বিস্তারিত অধ্যায় ৩-এ): **API Server** — ক্লাস্টারের একমাত্র প্রবেশদ্বার; সব অনুরোধ এখান দিয়ে যায়, এখানেই প্রমাণীকরণ ও অনুমতি যাচাই হয়। **etcd** — ক্লাস্টারের সব তথ্যের সংরক্ষণাগার; কাঙ্ক্ষিত ও বর্তমান অবস্থা এখানে থাকে। **Scheduler** — নতুন Pod কোন নোডে চলবে তা ঠিক করে। **Controller Manager** — বিভিন্ন কন্ট্রোলার চালায়, যারা সার্বক্ষণিক বর্তমান অবস্থাকে কাঙ্ক্ষিত অবস্থার দিকে টানে (যেমন Deployment কন্ট্রোলার কপির সংখ্যা বজায় রাখে)। ছোট ক্লাস্টারে এরা একটিমাত্র মেশিনে চলতে পারে; বড় ক্লাস্টারে আলাদা মেশিনে।

২.৬.২ Worker Node — যেখানে অ্যাপ চলে

প্রতিটি Worker Node-এ তিনটি জিনিস থাকে। **kubelet** — নোডের এজেন্ট; API Server থেকে নির্দেশ পেয়ে Pod চালু, বন্ধ ও পর্যবেক্ষণ করে। **kube-proxy** — নেটওয়ার্ক নিয়ম বজায় রাখে, যাতে Service-এর নামে আসা ট্রাফিক সঠিক Pod-এ পৌঁছায়। **Container Runtime** — আসলে কন্টেইনার চালায় (যেমন containerd বা CRI-O)। এ ছাড়া নোডে একটি **CNI Plugin** (Container Network Interface — কন্টেইনারের নেটওয়ার্ক সাজানোর মান) থাকে, যা Pod-গুলোকে IP দেয় ও তাদের মধ্যে সংযোগ করে।

২.৬.৩ Pod — সবচেয়ে ছোট স্থাপনার একক

Kubernetes সরাসরি কন্টেইনার চালায় না — চালায় **Pod**। Pod হলো এক বা একাধিক কন্টেইনারের একটি দল, যারা একই নেটওয়ার্ক ঠিকানা (IP) ও স্টোরেজ ভাগ করে এবং সবসময় একই নোডে একসাথে চলে। বেশিরভাগ ক্ষেত্রে একটি Pod-এ একটি কন্টেইনার থাকে। Pod অস্থায়ী (ephemeral): মারা গেলে আবার তৈরি হয়, নতুন IP পায় — তাই Pod-কে সরাসরি নির্ভর করা যায় না, Service-এর মাধ্যমে নির্ভর করতে হয়।

২.৬.৪ High Availability — Control Plane-কে কীভাবে বাঁচিয়ে রাখা যায়

Control Plane-এর একটি মাত্র কপি থাকলে সেটি নষ্ট হলে নতুন পরিবর্তন করা যাবে না (চলমান অ্যাপ তবু চলবে)। প্রোডাকশনে তাই Control Plane-এর একাধিক কপি (সাধারণত তিনটি) চালানো হয়, etcd-র তথ্য তাদের মধ্যে প্রতিলিপি (replicate) হয়, আর API Server-এর সামনে একটি লোড ব্যালান্সার বসে। তিনটি কপির মধ্যে একটি নষ্ট হলেও বাকি দুটি সংখ্যাগরিষ্ঠতা (quorum) ধরে রাখে — etcd-র Raft ঐকমত্য ব্যবস্থা এভাবেই কাজ করে। Worker Node-এর সংখ্যাও একাধিক রাখা হয়, যাতে একটি নোড গেলে তার Pod অন্য নোডে আবার চালু হয়।

২.৬.৫ অনুরোধের প্রবাহ — একটি উদাহরণ

ধরুন আপনি `kubectl apply -f web.yaml` চালালেন। প্রথমে kubectl আপনার kubeconfig থেকে ক্লাস্টারের ঠিকানা ও শংসাপত্র (credentials) নিয়ে API Server-কে অনুরোধ পাঠায়। API Server অনুরোধ যাচাই করে (আপনি কে, আপনার অনুমতি আছে কি না, YAML বৈধ কি না), তারপর etcd-তে লিখে রাখে। Scheduler লক্ষ করে একটি Pod এখনও কোনো নোড পায়নি, উপযুক্ত নোড বেছে নিয়ে সেই সিদ্ধান্ত etcd-তে লেখে। সেই নোডের kubelet সিদ্ধান্তটি দেখে Container Runtime-কে Pod চালু করতে বলে। kube-proxy নেটওয়ার্ক নিয়ম হালনাগাদ করে। Controller Manager সার্বক্ষণিক দেখে কপির সংখ্যা ঠিক আছে কি না। পুরো প্রবাহটি API Server-কে কেন্দ্র করে — কোনো কম্পোনেন্ট সরাসরি আরেকটির সঙ্গে কথা বলে না।

২.৭ কনফিগারেশন উদাহরণ

```
# ক্লাস্টারের নোডগুলো দেখুন — Control Plane ও Worker আলাদা করে চেনা যায়
kubectl get nodes
# চিত্রণমূলক নমুনা আউটপুট:
# NAME             STATUS   ROLES           AGE   VERSION
# control-1        Ready    control-plane   30d   v1.31.0
# worker-1         Ready    <none>          30d   v1.31.0
# worker-2         Ready    <none>          30d   v1.31.0

# Control Plane-এর Pod-গুলো দেখুন (kube-system নেমস্পেসে চলে)
kubectl get pods -n kube-system
# চিত্রণমূলক নমুনা আউটপুট:
# NAME                              READY   STATUS    RESTARTS   AGE
# etcd-control-1                    1/1     Running   0          30d
# kube-apiserver-control-1          1/1     Running   0          30d
# kube-controller-manager-control-1 1/1     Running   0          30d
# kube-scheduler-control-1          1/1     Running   0          30d
# kube-proxy-worker-1               1/1     Running   0          30d
# kube-proxy-worker-2               1/1     Running   0          30d
```

```
# একটি নোড সম্পর্কে বিস্তারিত দেখুন — ক্ষমতা, অবস্থা, লেবেল
kubectl describe node worker-1
# কোন নোডে কোন Pod চলছে দেখুন
kubectl get pods -A -o wide
```

> **WARNING:** `kubectl get nodes`-এ `ROLES` কলামে `<none>` দেখে ভাববেন না নোডটি অকেজো — এটিই সাধারণ Worker Node-এর চিহ্ন। Control Plane নোডে `control-plane` লেখা থাকে।

২.৮ বাস্তব প্রকৌশল পরিস্থিতি

TechNova পরিস্থিতি: TechNova-র ক্লাস্টারে তিনটি Worker Node ছিল: `203.0.113.11`, `203.0.113.12`, `203.0.113.13`। একদিন সকালে `203.0.113.12`-এর ডিস্ক পূর্ণ হয়ে নোডটি `NotReady` হয়ে যায়। ব্যবহারকারীরা ভাবলেন অ্যাপ বন্ধ হয়ে যাবে। বাস্তবে যা হলো: Control Plane লক্ষ করল নোডটি সাড়া দিচ্ছে না, তার Pod-গুলোকে অন্য দুটি নোডে আবার শিডিউল করল, kube-proxy নেটওয়ার্ক নিয়ম হালনাগাদ করল, আর Service-এর মাধ্যমে ট্রাফিক কেবল সুস্থ Pod-এই যেতে থাকল। মোট ডাউনটাইম: কয়েক মিনিটের আংশিক ধীরগতি। শিক্ষা: নোড একটি ব্যর্থতার একক (failure unit) — অ্যাপকে এমনভাবে সাজাতে হয় যাতে একটি নোডের ব্যর্থতা পুরো সার্ভিসকে না মারে। তাই গুরুত্বপূর্ণ সার্ভিসের একাধিক কপি একাধিক নোডে ছড়িয়ে রাখা হয়।

২.৯ ধাপে ধাপে ব্যবহারিক উদাহরণ

লোকাল ক্লাস্টারে (অধ্যায় ৫-এ সেটআপ শিখবেন) আর্কিটেকচারটি নিজে পরীক্ষা করুন। এখনই ক্লাস্টার না থাকলে ধাপগুলো পড়ে প্রবাহটি মুখস্থ করুন — পরের অধ্যায়ে কাজে লাগবে।

```
# ১. ক্লাস্টারের তথ্য দেখুন
kubectl cluster-info

# ২. নোডগুলোর অবস্থা ও ভূমিকা দেখুন
kubectl get nodes -o wide

# ৩. Control Plane কম্পোনেন্টগুলো কোথায় চলছে দেখুন
kubectl get pods -n kube-system -o wide

# ৪. একটি Pod কোন নোডে চলছে দেখুন
kubectl run demo --image=nginx:1.27 --port=80
kubectl get pods -o wide   # NODE কলামে নোডের নাম দেখুন

# ৫. পরিষ্কার করুন
kubectl delete pod demo
```

২.১০ সাধারণ ভুল ও সমস্যা সমাধান

সমস্যা / ভুল ধারণা	ব্যাখ্যা ও সমাধান
নোড `NotReady` দেখাচ্ছে	প্রথমে `kubectl describe node <name>` দেখুন — Conditions অংশে কারণ লেখা থাকে (ডিস্ক পূর্ণ, মেমোরি চাপ, kubelet বন্ধ)। নোডে ঢুকে `systemctl status kubelet` পরীক্ষা করুন।
"Control Plane নষ্ট মানেই অ্যাপ বন্ধ"	ভুল ধারণা। চলমান Pod চলতেই থাকে; শুধু নতুন পরিবর্তন, স্কেলিং ও পুনরুদ্ধার থেমে যায়। তাই Control Plane-এর সমস্যায় প্রথমে অ্যাপের অবস্থা দেখুন, আতঙ্কিত হবেন না।
Pod একটিই নোডে সব কপি বসে আছে	শিডিউলার ডিফল্টভাবে ছড়িয়ে দেওয়ার চেষ্টা করে, কিন্তু বাধ্য নয়। গুরুত্বপূর্ণ সার্ভিসে PodTopologySpreadConstraints বা Anti-Affinity ব্যবহার করুন (উন্নত অধ্যায়ে)।
`kubectl` সংযোগ দিতে পারছে না	kubeconfig ও context ঠিক আছে কি না দেখুন: `kubectl config get-contexts` ও `kubectl config current-context`। ভুল ক্লাস্টারে কমান্ড চালানো বিপজ্জনক ভুল।

২.১১ নিরাপত্তা বিবেচনা

Control Plane হলো ক্লাস্টারের সবচেয়ে সংবেদনশীল অংশ। API Server-এ প্রবেশ মানেই ক্লাস্টার নিয়ন্ত্রণ — তাই API Server শুধু বিশ্বস্ত নেটওয়ার্ক থেকে পৌঁছানো উচিত, শক্তিশালী প্রমাণীকরণ (যেমন ক্লায়েন্ট শংসাপত্র বা OIDC) বাধ্যতামূলক, এবং প্রতিটি ব্যবহারকারী ও সার্ভিস অ্যাকাউন্টকে কেবল প্রয়োজনীয় অনুমতি দিতে হয় (least privilege — সর্বনিম্ন অনুমতির নীতি)। etcd-তে ক্লাস্টারের সব তথ্য থাকে — সিক্রেটসহ — তাই etcd-তে সরাসরি প্রবেশ কেবল Control Plane-এর মধ্যেই সীমাবদ্ধ রাখতে হয় এবং etcd-র ডেটা এনক্রিপ্ট করা উচিত। Worker Node-এও kubelet-এর API সুরক্ষিত রাখতে হয়; নোডে ঢুকতে পারলেই যেন কেউ ক্লাস্টারের নিয়ন্ত্রণ না পায়।

২.১২ কর্মক্ষমতা ও খরচ বিবেচনা

Control Plane-এর আকার কাজের চাপের ওপর নির্ভর করে। ছোট লোকাল ক্লাস্টারে একটি Control Plane নোড যথেষ্ট; প্রোডাকশনে তিনটি Control Plane নোড আদর্শ। etcd কর্মক্ষমতার জন্য দ্রুত ডিস্ক (SSD) চায় — ধীর ডিস্কে API Server-এর প্রতিটি লেখা ধীর হয়, আর পুরো ক্লাস্টার ধীর মনে হয়। Worker Node-এর আকার নির্ভর করে Pod-গুলোর চাহিদার ওপর; অনেক ছোট নোডের বদলে কয়েকটি মাঝারি নোড সাধারণত পরিচালনা করা সহজ, তবে একটি নোড নষ্ট হলে ক্ষতির পরিমাণও বেশি — এটাই ভারসাম্য (trade-off)।

২.১৩ সেরা অনুশীলন ও অ্যান্টি-প্যাটার্ন

সেরা অনুশীলন: প্রোডাকশনে Control Plane-এর তিনটি কপি রাখুন; Worker Node কমপক্ষে দুটি রাখুন যাতে একটি নষ্ট হলে সার্ভিস বাঁচে; Control Plane নোডে ব্যবহারকারীর Pod চালাবেন না (taint দিয়ে আলাদা রাখুন); নিয়মিত etcd-র ব্যাকআপ নিন; ক্লাস্টারের সংস্করণ (version) সব নোডে সামঞ্জস্যপূর্ণ রাখুন।

অ্যান্টি-প্যাটার্ন: একটিমাত্র নোডে সবকিছু চালিয়ে তাকে "প্রোডাকশন ক্লাস্টার" বলা; Control Plane ও Worker একই মেশিনে চালিয়ে নিরাপত্তা ও স্থিতিস্থাপকতা হারানো; নোডের নামে গুরুত্বপূর্ণ তথ্য লিখে রাখা (নোড অস্থায়ী, নাম বদলায়); etcd-র ব্যাকআপ কখনও পরীক্ষা না করা।

২.১৪ অধ্যায়ের সারাংশ

Kubernetes ক্লাস্টার দুটি ভাগে বিভক্ত: Control Plane সিদ্ধান্ত নেয় (API Server, etcd, Scheduler, Controller Manager) আর Worker Node সেই সিদ্ধান্ত কার্যকর করে (kubelet, kube-proxy, Container Runtime, Pod)। সব যোগাযোগ API Server-এর মাধ্যমে হয়। Control Plane থেমে গেলেও চলমান অ্যাপ চলতে থাকে — তবে নতুন পরিবর্তন থেমে যায়। প্রোডাকশনে Control Plane-এর একাধিক কপি ও একাধিক Worker Node রাখা হয়, যাতে কোনো একক ব্যর্থতা পুরো সিস্টেমকে থামাতে না পারে।

২.১৫ মূল প্রযুক্তিগত পরিভাষা

Control Plane: ক্লাস্টারের সিদ্ধান্ত গ্রহণকারী অংশ — API Server, etcd, Scheduler ও Controller Manager নিয়ে গঠিত।

Worker Node: যে মেশিনে ব্যবহারকারীর অ্যাপ (Pod) আসলে চলে; এতে kubelet, kube-proxy ও Container Runtime থাকে।

Pod: Kubernetes-এ স্থাপনার সবচেয়ে ছোট একক — একই নেটওয়ার্ক ও স্টোরেজ ভাগ করা এক বা একাধিক কন্টেইনারের দল।

High Availability (HA): কোনো একটি অংশ নষ্ট হলেও সিস্টেম চালু থাকার ক্ষমতা, সাধারণত প্রতিলিপি ও সংখ্যাগরিষ্ঠতার মাধ্যমে।

kubelet: প্রতিটি নোডে চলা এজেন্ট, যা API Server-এর নির্দেশ অনুযায়ী Pod চালু ও পর্যবেক্ষণ করে।

kube-proxy: প্রতিটি নোডে চলা নেটওয়ার্ক এজেন্ট, যা Service-এর ট্রাফিক সঠিক Pod-এ পৌঁছে দেওয়ার নিয়ম বজায় রাখে।

২.১৬ জ্ঞান যাচাই

১. Control Plane ও Worker Node-এর দায়িত্বের পার্থক্য কী? প্রতিটিতে কোন কোন কম্পোনেন্ট থাকে?

২. API Server বন্ধ হয়ে গেলে চলমান অ্যাপ কেন চলতেই থাকে? তখন কী কী কাজ করা যায় না?

৩. Pod বলতে কী বোঝায়? Kubernetes সরাসরি কন্টেইনার না চালিয়ে Pod চালায় কেন?

৪. etcd-র তিনটি কপির মধ্যে একটি নষ্ট হলে ক্লাস্টার কেন তবু কাজ করে? Quorum বলতে কী বোঝায়?

৫. `kubectl apply` থেকে Pod চালু হওয়া পর্যন্ত অনুরোধটি কোন কোন কম্পোনেন্টের ভেতর দিয়ে যায়?

৬. একটি নোড `NotReady` হলে Control Plane কী করে? ব্যবহারকারীর ট্রাফিকের কী হয়?

৭. প্রোডাকশনে Control Plane নোডে ব্যবহারকারীর Pod চালানো কেন নিরুৎসাহিত করা হয়?

২.১৭ ব্যবহারিক অ্যাসাইনমেন্ট

অ্যাসাইনমেন্ট: আপনার লোকাল ক্লাস্টারে (অধ্যায় ৫ অনুযায়ী সেটআপ করুন, অথবা এখনও না থাকলে ধাপগুলো নোট করে রাখুন) `kubectl get nodes`, `kubectl get pods -n kube-system` ও `kubectl get pods -A -o wide` চালান। একটি চিত্র আঁকুন: আপনার ক্লাস্টারে কোন নোডে কোন Control Plane কম্পোনেন্ট ও কোন Pod চলছে। তারপর একটি Pod তৈরি করে দেখুন সেটি কোন নোডে গেল, এবং এক প্যারাগ্রাফে লিখুন — Pod-টি সেই নোডে যাওয়ার সিদ্ধান্ত কে নিয়েছিল এবং kubelet কী করেছিল।

অধ্যায় ৩ • PART I — Kubernetes Foundations & Architecture

Chapter 3 — API Server, etcd, Scheduler, Controller Manager, kubelet, kube-proxy, and Container Runtime

৩.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি Kubernetes-এর প্রতিটি মূল কম্পোনেন্টের ভেতরের কাজ ব্যাখ্যা করতে পারবেন: API Server-এ একটি অনুরোধ কী কী ধাপ পেরিয়ে etcd-তে পৌঁছায়; etcd কেন ক্লাস্টারের "সত্যের উৎস" এবং Raft ঐকমত্য (consensus — একাধিক কপি একই সিদ্ধান্তে পৌঁছানোর পদ্ধতি) কীভাবে কাজ করে; Scheduler কীভাবে নোড বেছে নেয়; Controller Manager-এর Reconciliation Loop (সমন্বয় চক্র — কাঙ্ক্ষিত ও বর্তমান অবস্থা মেলানোর পুনরাবৃত্ত প্রক্রিয়া) কী; kubelet কীভাবে Pod-এর জীবনচক্র পরিচালনা করে; kube-proxy কীভাবে নেটওয়ার্ক নিয়ম বসায়; এবং Container Runtime (containerd, CRI-O) কী করে — সেই সঙ্গে CRI বনাম OCI-র পার্থক্য এবং কেন Docker Engine বাধ্যতামূলক নয়।

৩.২ কেন এই বিষয়টি গুরুত্বপূর্ণ

অধ্যায় ২-এ আপনি মানচিত্র দেখেছেন; এই অধ্যায়ে আপনি প্রতিটি শহরের ভেতরে ঢুকবেন। পার্থক্যটি বাস্তব: Pod `Pending` অবস্থায় আটকে থাকলে কারণ হতে পারে Scheduler কোনো উপযুক্ত নোড খুঁজে পাচ্ছে না; Pod বারবার `CrashLoopBackOff` হলে কারণ kubelet-এর স্বাস্থ্য পরীক্ষা বা রানটাইমের সমস্যা; Service-এ ট্রাফিক না গেলে কারণ kube-proxy-র নিয়ম। কম্পোনেন্ট না জানলে সব সমস্যাই "Kubernetes ভেঙে গেছে" মনে হয় — জানলে প্রতিটি সমস্যার ঠিকানা ও লগের জায়গা জানা যায়। TechNova-র একজন প্রকৌশলী একবার দুই ঘণ্টা ধরে API Server-এর লগ খুঁজেছিলেন, অথচ সমস্যা ছিল kubelet-এ — এই অধ্যায় সেই ভুল থেকে বাঁচায়।

> **CONCEPT:** Kubernetes-এর কোনো কম্পোনেন্টই একে অপরকে সরাসরি নির্দেশ দেয় না। প্রত্যেকে etcd-তে (API Server-এর মাধ্যমে) অবস্থা পড়ে ও লেখে। এই শেয়ার্ড স্টেট নকশার কারণে যেকোনো কম্পোনেন্ট পুনরায় চালু হলেও সে হারানো তথ্য আবার পড়ে নিতে পারে।

৩.৩ পূর্বশর্ত জ্ঞান

অধ্যায় ২-এর আর্কিটেকচার, HTTP ও REST API-র সাধারণ ধারণা (GET/POST, JSON), এবং Linux প্রক্রিয়া ও নেটওয়ার্কের মৌলিক জ্ঞান প্রয়োজন। Raft, iptables/IPVS, CRI/OCI — এগুলো এই অধ্যায়েই প্রথম ব্যবহারে ব্যাখ্যা করা হয়েছে। YAML ও `kubectl`-এর প্রাথমিক পরিচয় থাকলে সুবিধা হবে।

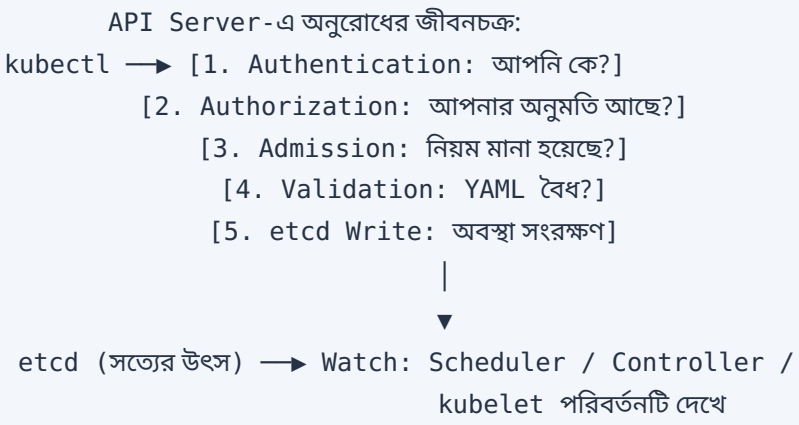
৩.৪ প্রথম নীতি থেকে ব্যাখ্যা

প্রথম নীতি: একটি বিতরণ করা সিস্টেমে (distributed system — অনেক মেশিনে ছড়ানো সিস্টেম) সবচেয়ে কঠিন কাজ হলো সবাই একই সত্য জানবে কীভাবে। দশটি মেশিনের প্রত্যেকে যদি নিজের মতো করে সিদ্ধান্ত নেয়, তবে বিশৃঙ্খলা অনিবার্য। সমাধান: একটি মাত্র জায়গায় সত্য লিখে রাখা, এবং সবাইকে সেই জায়গা থেকেই পড়তে বাধ্য করা। Kubernetes-এ সেই জায়গা etcd, আর তার দরজা API Server।

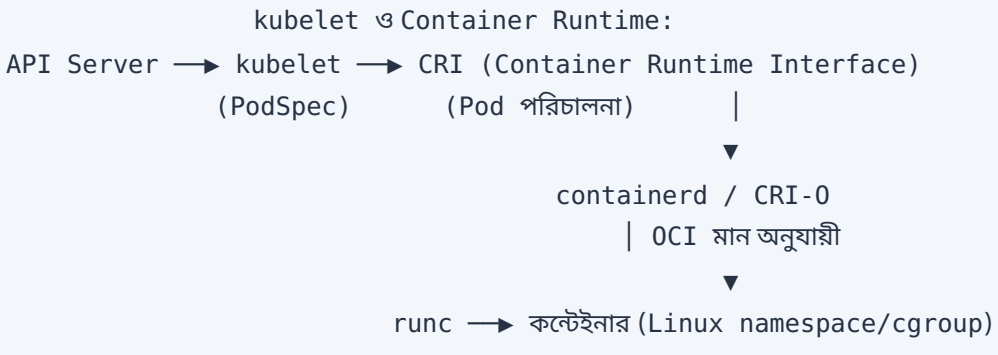
দ্বিতীয় নীতি: সিদ্ধান্ত ও কার্যকর করা আলাদা। API Server সিদ্ধান্ত নেয় না — সে শুধু অনুরোধ যাচাই করে ও সংরক্ষণ করে। Scheduler সিদ্ধান্ত নেয় কোথায় চলবে, Controller সিদ্ধান্ত নেয় কতগুলো চলবে, kubelet সিদ্ধান্ত কার্যকর করে। এই দায়িত্বের বিভাজন (separation of concerns) প্রতিটি কম্পোনেন্টকে ছোট, পরীক্ষাযোগ্য ও প্রতিস্থাপনযোগ্য রাখে।

তৃতীয় নীতি: প্রতিটি কম্পোনেন্ট স্তরভিত্তিক (level-based) ভাবে কাজ করে, ঘটনাভিত্তিক (event-based) নয়। অর্থাৎ কন্ট্রোলার "Pod মারা গেছে" এই ঘটনার অপেক্ষা করে না — সে বারবার জিজ্ঞেস করে "এখন কয়টি চলছে? কয়টি চলা উচিত?" এবং পার্থক্য দূর করে। ঘটনা হারিয়ে গেলেও এই পদ্ধতি নিজে থেকে সেরে ওঠে।

৩.৫ আর্কিটেকচার / ধারণাগত চিত্র



চিত্র ৩.১: API Server-এর অনুরোধ প্রক্রিয়াকরণের পাঁচটি ধাপ।



চিত্র ৩.২: kubelet থেকে কন্টেইনার পর্যন্ত স্তরগুলো — CRI হলো Kubernetes-এর ইন্টারফেস, OCI হলো রানটাইমের মান।

৩.৬ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

৩.৬.১ API Server — ক্লাস্টারের একমাত্র দরজা

API Server (`kube-apiserver`) হলো Kubernetes-এর সামনের দরজা ও কেন্দ্রীয় হাব। `kubectl`, Scheduler, Controller Manager, kubelet — সবাই এর সঙ্গে কথা বলে; কেউ etcd-র সঙ্গে সরাসরি কথা বলে না। একটি অনুরোধ পাঁচটি ধাপ পেরোয়: (১) **Authentication** (প্রমাণীকরণ) — আপনি কে? শংসাপত্র, টোকেন বা OIDC দিয়ে যাচাই। (২) **Authorization** (অনুমোদন) — আপনার কি এই কাজের অনুমতি আছে? RBAC এখানে সিদ্ধান্ত নেয়। (৩) **Admission**

Control — অতিরিক্ত নিয়ম: যেমন কোনো নেমস্পেসে নির্দিষ্ট লেবেল বাধ্যতামূলক কি না, বা সিক্রেটের ব্যবহার নিষিদ্ধ কি না। (৪) **Validation** — অবজেক্টের গঠন বৈধ কি না (যেমন `replicas` কি সংখ্যা?)। (৫) **etcd Write** — বৈধ অনুরোধ etcd-তে সংরক্ষিত হয়। কোনো ধাপে ব্যর্থ হলে অনুরোধ প্রত্যাখ্যাত হয় এবং etcd-তে কিছু লেখা হয় না। API Server নিজে কোনো অবস্থা মনে রাখে না (stateless) — তাই এর একাধিক কপি লোড ব্যালান্সারের পেছনে চালানো যায়।

৩.৬.২ etcd — ক্লাস্টারের সত্যের উৎস

etcd একটি বিতরণ করা কী-ভ্যালু স্টোর (distributed key-value store — তথ্য চাবি-মান জোড়ায় সংরক্ষণকারী ব্যবস্থা), যেখানে ক্লাস্টারের সব অবস্থা থাকে: কোন Pod কোথায় চলছে, কাঙ্ক্ষিত কপির সংখ্যা কত, সিক্রেট কী। etcd **Raft** নামের ঐকমত্য অ্যালগরিদম ব্যবহার করে — একাধিক কপির মধ্যে সংখ্যাগরিষ্ঠ (quorum) একমত না হলে কোনো লেখা গৃহীত হয় না। তিনটি কপির মধ্যে একটি নষ্ট হলেও বাকি দুটি কাজ চালায়; দুটি নষ্ট হলে লেখা বন্ধ হয়ে যায় (পড়া চলতে পারে) — এটি ইচ্ছাকৃত নিরাপত্তা, যাতে পরস্পরবিরোধী সত্য তৈরি না হয়। etcd-র আরেকটি গুরুত্বপূর্ণ ক্ষমতা **Watch**: কম্পোনেন্টগুলো etcd-র পরিবর্তন "দেখতে" (watch) পারে — নতুন Pod তৈরি হলে Scheduler সঙ্গে সঙ্গে জানতে পারে, অপেক্ষা করে পোলিং (polling — বারবার জিজ্ঞেস করা) করতে হয় না। etcd-র ডেটা হারানো মানে ক্লাস্টারের স্মৃতি হারানো — তাই নিয়মিত ব্যাকআপ অপরিহার্য।

৩.৬.৩ Scheduler — কোথায় চলবে

Scheduler (`kube-scheduler`) নতুন Pod-এর জন্য নোড বেছে নেয়। প্রথমে সে ফিল্টার করে: কোন নোডে যথেষ্ট CPU/মেমোরি আছে, Pod-এর চাহিদা (যেমন নির্দিষ্ট লেবেল বা হার্ডওয়্যার) মেলে, নোডটি `Ready` কি না। তারপর সে স্কোর দেয়: খালি সম্পদ বেশি এমন নোড, একই অ্যাপের কপি কম আছে এমন নোড — এগুলো বেশি নম্বর পায়। সর্বোচ্চ নম্বরের নোডে Pod-টি বাইন্ড (bind — সংযুক্ত) করা হয়, অর্থাৎ Pod-এর তথ্যে নোডের নাম লিখে দেওয়া হয়। Scheduler নিজে Pod চালায় না — সে শুধু সিদ্ধান্ত etcd-তে লেখে; সেই সিদ্ধান্ত দেখে সংশ্লিষ্ট নোডের kubelet কাজ শুরু করে।

৩.৬.৪ Controller Manager ও Reconciliation Loop

Controller Manager (`kube-controller-manager`) অনেকগুলো কন্ট্রোলারের সমষ্টি — Deployment কন্ট্রোলার, ReplicaSet কন্ট্রোলার, Node কন্ট্রোলার, Service কন্ট্রোলার ইত্যাদি। প্রতিটি কন্ট্রোলার একটি সরল চক্র চালায়: পর্যবেক্ষণ করো → কাঙ্ক্ষিত অবস্থার সঙ্গে তুলনা করো → পার্থক্য দূর করো। একে **Reconciliation Loop** বলে। যেমন ReplicaSet কন্ট্রোলার দেখে web-এর কাঙ্ক্ষিত কপি ৩টি, চলছে ২টি — সে একটি Pod তৈরির অনুরোধ API Server-কে পাঠায়। কন্ট্রোলার সরাসরি Pod চালায় না — সে API Server-কে অনুরোধ করে। এই চক্র বারবার চলতে থাকে, তাই যেকোনো বিচ্যুতি নিজে থেকে সংশোধিত হয়।

৩.৬.৫ kubelet — নোডের এজেন্ট ও Pod-এর জীবনচক্র

kubelet প্রতিটি নোডে চলা এজেন্ট। সে API Server-কে দেখে (watch) নিজের নোডের জন্য নির্ধারিত Pod-গুলো কী, তারপর Container Runtime Interface (CRI) দিয়ে রানটাইমকে Pod চালাতে বলে। Pod-এর জীবনচক্র: `Pending` (শিডিউল হয়েছে, চলছে না) → `Running` → `Succeeded` / `Failed`। kubelet নিয়মিত **Probe** (প্রোব — স্বাস্থ্য পরীক্ষা) চালায়: *Liveness Probe* — কন্টেইনার বেঁচে আছে কি না (ব্যর্থ হলে পুনরায় চালু); *Readiness Probe* — কন্টেইনার ট্রাফিক নেওয়ার জন্য প্রস্তুত কি না (প্রস্তুত না হলে Service থেকে সরিয়ে রাখা হয়); *Startup Probe* — ধীরে চালু হওয়া অ্যাপের জন্য। kubelet নোডের সম্পদও পর্যবেক্ষণ করে — ডিস্ক পূর্ণ হলে বা মেমোরি চাপে পড়লে সে Pod উচ্ছেদ (evict) করতে পারে।

৩.৬.৬ kube-proxy — Service-এর ট্রাফিক কীভাবে Pod-এ যায়

kube-proxy প্রতিটি নোডে চলা নেটওয়ার্ক এজেন্ট। সে Service ও Endpoint-এর পরিবর্তন দেখে নোডের নেটওয়ার্ক নিয়ম (সাধারণত **iptables** বা **IPVS** নিয়ম) হালনাগাদ করে, যাতে Service-এর ভার্চুয়াল IP-তে আসা ট্রাফিক সুস্থ Pod-গুলোর মধ্যে বণ্টিত হয়। গুরুত্বপূর্ণ: kube-proxy নিজে ট্রাফিক বহন করে না (বেশিরভাগ মোডে) — সে শুধু কার্নেলের নিয়ম লিখে দেয়, ট্রাফিক কার্নেলই পাঠায়। তবে আচরণ CNI ও নেটওয়ার্ক বাস্তবায়নের ওপর নির্ভর করে — কিছু CNI kube-proxy ছাড়াও Service পরিচালনা করতে পারে (যেমন eBPF-ভিত্তিক বাস্তবায়ন)। তাই "kube-proxy মানেই iptables" ধরে নেওয়া ভুল — বাস্তবায়ন ভিন্ন হতে পারে।

৩.৬.৭ Container Runtime, CRI বনাম OCI, এবং Docker প্রসঙ্গ

Container Runtime আসলে কন্টেইনার চালায় — ইমেজ টানা, namespace ও cgroup তৈরি করা, প্রক্রিয়া শুরু করা। Kubernetes-এ বর্তমানে দুটি প্রধান রানটাইম: **containerd** ও **CRI-O**। **CRI** (Container Runtime Interface) হলো Kubernetes (kubelet) ও রানটাইমের মধ্যের ইন্টারফেস — kubelet CRI দিয়ে বলে "এই Pod চালাও", রানটাইম যেকোনো হতে পারে যতক্ষণ CRI মানে। **OCI** (Open Container Initiative) হলো নিচের স্তরের মান — ইমেজের গঠন ও রানটাইমের আচরণ কেমন হবে তা নির্ধারণ করে; containerd ও CRI-O ভেতরে `runc` (একটি OCI রানটাইম) ব্যবহার করে। দুটি স্পষ্ট সত্য মনে রাখুন: Kubernetes ইমেজ তৈরি করে না — ইমেজ তৈরি হয় Docker বা Buildah-র মতো টুল দিয়ে, Kubernetes শুধু চালায়। এবং *Docker Engine* বাধ্যতামূলক নয় — আধুনিক Kubernetes সরাসরি containerd/CRI-O ব্যবহার করে। পুরোনো **Docker Shim** (dockershim — Docker Engine-কে CRI-এর সঙ্গে যুক্ত করার অস্থায়ী স্তর) আর বর্তমান অনুশীলন নয়; নতুন ক্লাস্টারে তা শেখানো বা ব্যবহার করা উচিত নয়।

৩.৭ কনফিগারেশন উদাহরণ

```
# API Server কীভাবে চলছে দেখুন (managed ক্লাস্টারে এটি দেখা নাও যেতে পারে)
kubectl get pods -n kube-system -l component=kube-apiserver
kubectl get pods -n kube-system -l component=etcd
kubectl get pods -n kube-system -l component=kube-scheduler
kubectl get pods -n kube-system -l component=kube-controller-manager

# kubelet ও kube-proxy-র অবস্থা নোডে দেখুন (নোডে সরাসরি প্রবেশ থাকলে)
# systemctl status kubelet
# kubectl get pods -n kube-system -l k8s-app=kube-proxy
```

```
# Probe-সহ একটি Pod — kubelet কীভাবে স্বাস্থ্য পরীক্ষা করে
apiVersion: v1
kind: Pod
metadata:
  name: web-probe
  namespace: technova
spec:
  containers:
    - name: web
      image: nginx:1.27
      ports:
        - containerPort: 80
      livenessProbe:
        httpGet:
          path: /
          port: 80
        initialDelaySeconds: 5
```

```
        periodSeconds: 10
      readinessProbe:
        httpGet:
          path: /
          port: 80
        initialDelaySeconds: 3
        periodSeconds: 5
```

```
# কোন রানটাইম ব্যবহার হচ্ছে দেখুন
kubectl get nodes -o wide
# অথবা বিস্তারিতভাবে:
kubectl describe node worker-1 | grep -i "Container Runtime"
```

কম্পোনেন্ট	কোথায় চলে	মূল কাজ	ব্যর্থ হলে কী হয়
API Server	Control Plane	একমাত্র দরজা; যাচাই ও সংরক্ষণ	নতুন পরিবর্তন বন্ধ; চলমান অ্যাপ চলে
etcd	Control Plane	সব অবস্থার সংরক্ষণ	ক্লাস্টারের স্মৃতি ঝুঁকিতে; ব্যাকআপ জরুরি
Scheduler	Control Plane	Pod-এর জন্য নোড নির্বাচন	নতুন Pod `Pending`-এ আটকায়
Controller Manager	Control Plane	কাঙ্ক্ষিত অবস্থা বজায় রাখা	কপির সংখ্যা সংশোধন থেমে যায়
kubelet	প্রতিটি নোড	Pod চালু ও পর্যবেক্ষণ	সেই নোডের Pod পরিচালনা থেমে যায়
kube-proxy	প্রতিটি নোড	Service নেটওয়ার্ক নিয়ম	সেই নোডে Service ট্রাফিক ব্যাহত
containerd / CRI-O	প্রতিটি নোড	কন্টেইনার চালানো	নতুন কন্টেইনার চালু হয় না

৩.৮ বাস্তব প্রকৌশল পরিস্থিতি

TechNova পরিস্থিতি: TechNova-র `api` সার্ভিসের Pod বারবার `CrashLoopBackOff` হচ্ছিল। প্রথম অনুমান ছিল "কোডে বাগ"। কিন্তু লগ দেখে বোঝা গেল অ্যাপটি চালু হওয়ার ৩০ সেকেন্ড পর মারা যাচ্ছে — ডেটাবেস সংযোগের জন্য অপেক্ষা না করে। kubelet-এর Liveness Probe ব্যর্থ হওয়ায় সে কন্টেইনার পুনরায় চালু করছিল, আর প্রতিবার একই ব্যর্থতা। সমাধান ছিল দুটি: অ্যাপে পুনঃচেষ্টা (retry) ও Startup Probe যোগ করা, যাতে ধীরে চালু হওয়া অ্যাপকে সময় দেওয়া হয়। শিক্ষা: kubelet সমস্যা লুকায় না — সে সমস্যাটি বারবার প্রকাশ করে। `kubectl describe pod`-এর Events অংশে Probe-এর ব্যর্থতা ও পুনরায় চালুর ইতিহাস লেখা থাকে — সেটাই প্রথম দেখার জায়গা।

> **REAL-WORLD:** প্রোডাকশন ডিবাগিং-এর ৮০% সময় যায় সঠিক কম্পোনেন্টটি চিহ্নিত করতে। "Pod চলছে না" বললে প্রথম প্রশ্ন: Pod-টি কি শিডিউলই হয়নি (Scheduler), শিডিউল হয়ে চালু হচ্ছে না (kubelet/রানটাইম), নাকি চালু হয়ে মারা যাচ্ছে (অ্যাপ/Probe)? প্রতিটি প্রশ্নের লগ আলাদা জায়গায়।

৩.৯ ধাপে ধাপে ব্যবহারিক উদাহরণ

```
# ১. একটি Pod তৈরি করে তার অবস্থা ও Events দেখুন
kubectl run demo --image=nginx:1.27 --port=80 -n technova
kubectl describe pod demo -n technova
# Events অংশে দেখুন: Scheduled → Pulled → Created → Started
```

```
# ২. Pod কোন নোডে গেল ও কেন, তা দেখুন
kubectl get pod demo -n technova -o wide

# ৩. Liveness Probe ব্যর্থ হলে কী হয় — একটি ভুল পথের Probe দিয়ে পরীক্ষা
kubectl apply -f - <<EOF
apiVersion: v1
kind: Pod
metadata:
  name: bad-probe
  namespace: technova
spec:
  containers:
    - name: web
      image: nginx:1.27
      livenessProbe:
        httpGet:
          path: /does-not-exist
          port: 80
        periodSeconds: 5
EOF
kubectl get pod bad-probe -n technova -w
# কিছুক্ষণ পর RESTARTS বাড়তে দেখবেন — kubelet বারবার পুনরায় চালু করছে

# ৪. পরিষ্কার করুন
kubectl delete pod demo bad-probe -n technova
```

উপরের পরীক্ষায় লক্ষ করুন: Probe-এর পথটি ভুল হলেও nginx নিজে ঠিকঠাক চলছিল — kubelet তবু কন্টেইনার পুনরায় চালু করেছে, কারণ Probe-ই স্বাস্থ্যের সংজ্ঞা। ভুল Probe লেখা তাই অ্যাপের বাগের মতোই ক্ষতিকর।

৩.১০ সাধারণ ভুল ও সমস্যা সমাধান

সমস্যা	সম্ভাব্য কারণ ও পরীক্ষা
Pod `Pending` -এ আটকে আছে	`kubectl describe pod` -এ Scheduler-এর বার্তা দেখুন: "Insufficient cpu/memory" বা নোডে মেলেনি। `kubectl describe node` দিয়ে খালি সম্পদ দেখুন।
Pod `ImagePullBackOff`	ইমেজের নাম/ট্যাগ ভুল, প্রাইভেট রেজিস্ট্রির শংসাপত্র নেই, বা নেটওয়ার্ক সমস্যা। `kubectl describe pod` -এ টানার (pull) ত্রুটি দেখুন।
Pod `CrashLoopBackOff`	`kubectl logs --previous` দিয়ে আগের চালুর লগ দেখুন; Probe-এর সেটিং পরীক্ষা করুন।
Service-এ ট্রাফিক যাচ্ছে না	Service-এর Selector Pod-এর লেবেলের সঙ্গে মেলে কি না দেখুন (`kubectl get endpoints`); kube-proxy ও CNI-র অবস্থা পরীক্ষা করুন।
API Server ধীর	etcd-র ডিস্ক ধীর কি না দেখুন — etcd ধীর হলে পুরো ক্লাস্টার ধীর হয়। `etcd` -র লগ ও ডিস্ক I/O পরীক্ষা করুন।

TROUBLESHOOTING: `kubectl describe`-এর Events হলো প্রথম সাক্ষী, `kubectl logs` দ্বিতীয়। Events বলে কী ঘটেছে (শিডিউল, টানা, চালু, ব্যর্থতা), logs বলে কেন (অ্যাপের ভেতরের ত্রুটি)। দুটো না দেখে সিদ্ধান্ত নেবেন না।

৩.১১ নিরাপত্তা বিবেচনা

প্রতিটি কম্পোনেন্টের নিজস্ব নিরাপত্তা সীমা আছে। API Server-এ শক্তিশালী প্রমাণীকরণ ও RBAC বাধ্যতামূলক — অজ্ঞাত কাউকে ক্লাস্টারে ঢুকতে দেওয়া যাবে না। etcd-তে সিক্রেটসহ সব তথ্য থাকে — etcd-র যোগাযোগ TLS দিয়ে এনক্রিপ্ট করতে হয় এবং সরাসরি প্রবেশ কেবল Control Plane-এর মধ্যে সীমাবদ্ধ রাখতে হয়। kubelet-এর API-তেও প্রমাণীকরণ প্রয়োজন — নইলে নোডে ঢুকে কেউ Pod-এর তথ্য নিতে বা নির্দেশ দিতে পারে। Container Runtime-এর ক্ষেত্রে: কন্টেইনার ডিফল্টভাবে root হিসেবে চালাবেন না, ইমেজ শুধু বিশ্বস্ত রেজিস্ট্রি থেকে নিন, এবং ইমেজের দুর্বলতা (vulnerability) নিয়মিত স্ক্যান করুন।

৩.১২ কর্মক্ষমতা ও খরচ বিবেচনা

etcd কর্মক্ষমতা পুরো ক্লাস্টারের কর্মক্ষমতা নির্ধারণ করে — etcd-র জন্য দ্রুত SSD, পর্যাপ্ত IOPS ও কম লেটেন্সি প্রয়োজন; ধীর ডিস্কে API-র প্রতিটি লেখা ধীর হয়। API Server-এর একাধিক কপি পড়ার (read) চাপ ভাগ করে নেয়, কিন্তু লেখা (write) সবসময় etcd-র মধ্য দিয়ে যায়। Scheduler-এর সিদ্ধান্ত দ্রুত হয়, তবে হাজার হাজার Pod-এর ক্লাস্টারে শিডিউলিং নীতি জটিল হলে বিলম্ব বাড়ে। kubelet-এর Probe খুব ঘন ঘন চালালে অ্যাপের ওপর চাপ বাড়ে — `periodSeconds` ও `timeoutSeconds` অ্যাপের স্বভাব অনুযায়ী নির্ধারণ করুন।

৩.১৩ সেরা অনুশীলন ও অ্যান্টি-প্যাটার্ন

সেরা অনুশীলন: প্রতিটি কন্টেইনারে Liveness ও Readiness Probe দিন — তবে Probe-এর পথটি অ্যাপের আসল স্বাস্থ্য প্রতিফলিত করে এমন রাখুন। etcd-র নিয়মিত ব্যাকআপ নিন ও পুনরুদ্ধার পরীক্ষা করুন। API Server-এর অডিট লগ (audit log) চালু রাখুন। রিসোর্স `requests` ও `limits` নির্ধারণ করুন যাতে Scheduler সঠিক সিদ্ধান্ত নিতে পারে।

অ্যান্টি-প্যাটার্ন: Probe ছাড়া প্রোডাকশনে যাওয়া; Liveness Probe-কে Readiness Probe-এর মতো ব্যবহার করা (Liveness ব্যর্থ হলে কন্টেইনার পুনরায় চালু হয় — ডেটাবেস সংযোগের মতো অস্থায়ী সমস্যায় এটি ক্ষতিকর); Docker Shim-এর ওপর নির্ভরশীল টুলিং নতুন ক্লাস্টারে ব্যবহার করা; etcd-কে হাতে সম্পাদনা করা; kubelet-এর লগ না দেখে Pod মুছে আবার তৈরি করা (তাতে ইতিহাস হারায়)।

PRO TIP: Pod-এর সমস্যায় একটি নির্দিষ্ট ক্রম অনুসরণ করুন: `kubectl get pod` → `kubectl describe pod` (Events) → `kubectl logs` → নোডের kubelet লগ। এই ক্রমটি Scheduler থেকে kubelet হয়ে অ্যাপ পর্যন্ত পৌঁছায় — এলোমেলো খোঁজার চেয়ে দশ গুণ দ্রুত।

৩.১৪ অধ্যায়ের সারাংশ

API Server ক্লাস্টারের দরজা — প্রমাণীকরণ, অনুমোদন, Admission ও যাচাই পেরিয়ে অনুরোধ etcd-তে যায়। etcd Raft ঐকমত্যে ক্লাস্টারের সত্য সংরক্ষণ করে ও Watch-এর মাধ্যমে পরিবর্তন জানায়। Scheduler Pod-এর জন্য নোড বেছে নেয়; Controller Manager Reconciliation Loop-এ কাঙ্ক্ষিত অবস্থা বজায় রাখে; kubelet Pod চালায় ও Probe দিয়ে স্বাস্থ্য দেখে; kube-proxy Service-এর নেটওয়ার্ক নিয়ম বসায় (আচরণ CNI-ভেদে ভিন্ন হতে পারে); আর containerd/CRI-O

CRI-এর মাধ্যমে আসলে কন্টেইনার চালায়। Kubernetes ইমেজ তৈরি করে না, Docker Engine বাধ্যতামূলক নয়, আর Docker Shim বর্তমান অনুশীলন নয়।

৩.১৫ মূল প্রযুক্তিগত পরিভাষা

API Server: Kubernetes ক্লাস্টারের কেন্দ্রীয় দরজা; সব অনুরোধ এখান দিয়ে যায় এবং যাচাই হয়ে etcd-তে সংরক্ষিত হয়।

etcd: বিতরণ করা কী-ভ্যালু স্টোর; ক্লাস্টারের সব অবস্থার সত্যের উৎস, Raft ঐকমত্য ব্যবহার করে।

Raft: একাধিক কপির মধ্যে সংখ্যাগরিষ্ঠতার ভিত্তিতে একই সিদ্ধান্তে পৌঁছানোর অ্যালগরিদম।

Reconciliation Loop: কন্ট্রোলারের পুনরাবৃত্ত চক্র — বর্তমান অবস্থা দেখা, কাঙ্ক্ষিত অবস্থার সঙ্গে তুলনা করা, পার্থক্য দূর করা।

CRI (Container Runtime Interface): kubelet ও Container Runtime-এর মধ্যের ইন্টারফেস।

OCI (Open Container Initiative): কন্টেইনার ইমেজ ও রানটাইমের উন্মুক্ত মান।

Liveness / Readiness Probe: kubelet-এর স্বাস্থ্য পরীক্ষা — Liveness বলে কন্টেইনার বেঁচে আছে কি না, Readiness বলে ট্রাফিক নেওয়ার জন্য প্রস্তুত কি না।

৩.১৬ জ্ঞান যাচাই

১. API Server-এ একটি অনুরোধের পাঁচটি ধাপ কী কী? কোন ধাপে ব্যর্থ হলে etcd-তে কিছু লেখা হয় না?

২. etcd-র তিনটি কপির মধ্যে দুটি নষ্ট হলে লেখা কেন বন্ধ হয়ে যায়? এটি কীভাবে ডেটার সুরক্ষা দেয়?

৩. Scheduler কীভাবে নোড বেছে নেয় — ফিল্টার ও স্কোর ধাপ দুটি ব্যাখ্যা করুন।

৪. Reconciliation Loop বলতে কী বোঝায়? ReplicaSet কন্ট্রোলার একটি Pod মারা গেলে কী করে?

৫. Liveness Probe ও Readiness Probe-এর পার্থক্য কী? ভুলভাবে ব্যবহার করলে কী ক্ষতি হয়?

৬. CRI ও OCI-র পার্থক্য কী? containerd কোন স্তরে কাজ করে?

৭. Kubernetes ইমেজ তৈরি করে না এবং Docker Engine বাধ্যতামূলক নয় — এই দুটি সত্যের ব্যবহারিক অর্থ কী?

৮. Service-এ ট্রাফিক না গেলে আপনি কোন কোন জায়গা পরীক্ষা করবেন, কোন ক্রমে?

৩.১৭ ব্যবহারিক অ্যাসাইনমেন্ট

অ্যাসাইনমেন্ট: TechNova-র `web-probe` Pod-টি (৩.৭) আপনার লোকাল ক্লাস্টারে চালু করুন। তারপর (ক) `kubectl describe pod` দিয়ে Events পড়ে Pod-এর জীবনচক্রের ধাপগুলো লিখুন; (খ) ইচ্ছাকৃতভাবে Readiness Probe-এর পথ ভুল করে দেখুন Pod-টি `Running` থাকলেও Service থেকে ট্রাফিক পায় কি না (`kubectl get endpoints` দিয়ে যাচাই করুন); (গ) একটি সংক্ষিপ্ত প্রতিবেদন লিখুন: Liveness ও Readiness Probe-এর পার্থক্য আপনি নিজের পরীক্ষায় কীভাবে দেখলেন।

অধ্যায় ৪ • PART I — Kubernetes Foundations & Architecture

Chapter 4 — Kubernetes Objects, Declarative Model, YAML Manifests, and API Resources

৪.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি **Declarative Model** (ঘোষণামূলক মডেল — "কী চাই" বলা, "কীভাবে করবে" না বলা) ও **Imperative Model** (আদেশমূলক মডেল — ধাপে ধাপে নির্দেশ দেওয়া)-এর পার্থক্য ব্যাখ্যা করতে পারবেন। আপনি Kubernetes **Object** (অবজেক্ট — API-তে সংরক্ষিত একটি সত্তা, যেমন Pod বা Deployment)-এর গঠন — `apiVersion`, `kind`, `metadata`, `spec`, `status` — পড়তে ও লিখতে পারবেন, YAML ম্যানিফেস্ট লাইন ধরে ব্যাখ্যা করতে পারবেন, `kubectl apply` বনাম `create`-এর পার্থক্য জানবেন, এবং API Resources, API Groups ও Versions (স্থিতিশীল বনাম অবচিত বনাম পরীক্ষামূলক) সম্পর্কে সচেতনভাবে কাজ করতে পারবেন।

৪.২ কেন এই বিষয়টি গুরুত্বপূর্ণ

Kubernetes-এর সঙ্গে আপনার প্রায় সব কথোপকথন YAML-এর মাধ্যমে হয়। YAML ভুল হলে Pod চালু হয় না, ভুল `apiVersion` দিলে API Server অনুরোধ ফিরিয়ে দেয়, `spec` ও `status` গুলিয়ে ফেললে আপনি এমন কিছু বদলাতে চান যা বদলানো যায় না। আরও গভীরে: Declarative মডেল না বুঝলে Kubernetes-কে আপনি `docker run`-এর বড় সংস্করণ ভাববেন — আর তখন প্রতিবার কিছু বদলাতে হাতে করে মুছে আবার তৈরি করবেন, যা প্রোডাকশনে বিপজ্জনক। এই অধ্যায় Kubernetes-এর ভাষা শেখায় — বাকি বই এই ভাষায় লেখা।

> **CONCEPT:** Imperative বলে "এই কাজগুলো করো" — Declarative বলে "এই অবস্থায় থাকো"। প্রথমটিতে আপনি চালক, দ্বিতীয়টিতে আপনি গন্তব্য বলেন আর সিস্টেম পথ খুঁজে নেয়। Kubernetes দ্বিতীয়টি বেছে নিয়েছে।

৪.৩ পূর্বশর্ত জ্ঞান

YAML-এর মৌলিক গঠন (ইন্ডেন্টেশন, কী-ভ্যালু, তালিকা) জানা থাকতে হবে — না জানলে এই অধ্যায়ের উদাহরণগুলো পড়েই শিখে নেওয়া যাবে। অধ্যায় ১-এর Desired State ধারণা ও অধ্যায় ৩-এর API Server/etcd আলোচনা এই অধ্যায়ের ভিত্তি। `kubectl` কমান্ড লাইন ব্যবহারের প্রাথমিক অভিজ্ঞতা সহায়ক।

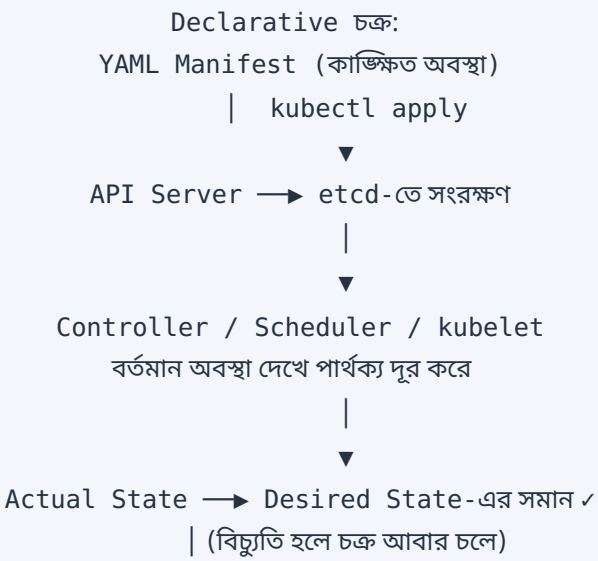
৪.৪ প্রথম নীতি থেকে ব্যাখ্যা

প্রথম নীতি: একটি সিস্টেমকে নির্দেশ দেওয়ার দুটি উপায় আছে। Imperative উপায়ে আপনি প্রতিটি ধাপ বলেন: "প্রথমে একটি কন্টেইনার চালাও, তারপর পোর্ট খোলো, তারপর স্বাস্থ্য পরীক্ষা করো।" মাঝখানে কিছু ব্যর্থ হলে আপনি কোথায় আছেন তা বোঝা কঠিন — অর্ধেক কাজ হয়েছে, বাকি হয়নি। Declarative উপায়ে আপনি শেষ অবস্থাটি বলেন: "তিনটি সুস্থ কপি চলতে থাকুক।" সিস্টেম বর্তমান অবস্থা দেখে নিজে ঠিক করে কী করতে হবে — একটি কপি কম থাকলে একটি তৈরি করে, বেশি থাকলে একটি সরায়।

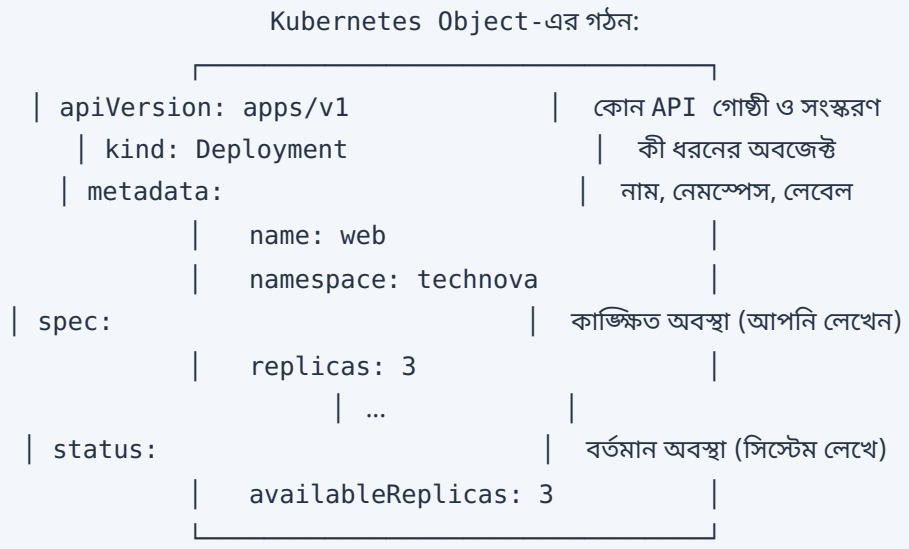
Declarative মডেলের তিনটি সুবিধা। প্রথম, পুনরাবৃত্তিযোগ্যতা (repeatability): একই ঘোষণা দশবার দিলে একই ফল হয়। দ্বিতীয়, স্বয়ংক্রিয় পুনরুদ্ধার: ঘোষণা সংরক্ষিত থাকে, তাই বিচ্যুতি হলেই সিস্টেম তা মেলায়। তৃতীয়, পর্যালোচনাযোগ্যতা: ঘোষণাটি একটি ফাইল — Git-এ রাখা যায়, পর্যালোচনা (review) করা যায়, আগের সংস্করণে ফেরা যায়। এই ফাইলকেই

Manifest (ম্যানিফেস্ট — কাঙ্ক্ষিত অবস্থার লিখিত ঘোষণা) বলে। Kubernetes-এর প্রায় সবকিছুই এই মডেলে চলে — এটাই এর সবচেয়ে গুরুত্বপূর্ণ ধারণা।

৪.৫ আর্কিটেকচার / ধারণাগত চিত্র



চিত্র ৪.১: Declarative মডেলের চক্র — ঘোষণা একবার, সমন্বয় সার্বক্ষণিক।



চিত্র ৪.২: একটি Kubernetes অবজেক্টের পাঁচটি অংশ।

৪.৬ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

৪.৬.১ Declarative বনাম Imperative — বাস্তব পার্থক্য

Imperative কমান্ডের উদাহরণ: `kubectl run web --image=nginx:1.27 --replicas=3` — আপনি সরাসরি নির্দেশ দিলেন। Declarative উদাহরণ: একটি YAML ফাইলে `replicas: 3` লিখে `kubectl apply -f web.yaml` চালানো। পার্থক্যটি সূক্ষ্ম কিন্তু গভীর: Imperative কমান্ডের কোনো সংরক্ষিত "সত্য" থাকে না — পরে কেউ জানতে পারবে না আপনি কী চেয়েছিলেন। Declarative ফাইলটি নিজেই সত্যের দলিল — Git-এ রাখা যায়, দলের সবাই দেখতে পারে, পরিবর্তনের ইতিহাস থাকে। প্রোডাকশনে Declarative-ই মান — Imperative শুধু দ্রুত পরীক্ষার জন্য।

৪.৬.২ Object-এর গঠন: apiVersion, kind, metadata, spec, status

প্রতিটি Kubernetes অবজেক্টের পাঁচটি অংশ থাকে। `apiVersion` বলে অবজেক্টটি কোন API গোষ্ঠী ও সংস্করণের (যেমন `v1`, `apps/v1`, `networking.k8s.io/v1`)। `kind` বলে অবজেক্টের ধরন (Pod, Deployment, Service)। `metadata`-তে থাকে `name` (নেমস্পেসের ভেতরে অনন্য), `namespace`, `labels` (লেবেল — বাছাই ও গোষ্ঠীভুক্ত করার জন্য কী-ভ্যালু চিহ্ন) ও `annotations` (টীকা — মানুষ বা টুলের জন্য অতিরিক্ত তথ্য)। `spec` হলো কাঙ্ক্ষিত অবস্থা — আপনি যা চান তা এখানে লেখেন। `status` হলো বর্তমান অবস্থা — এটি সিস্টেম লেখে, আপনি লেখেন না; ম্যানিফেস্টে `status` লেখার প্রয়োজন নেই, এমনকি লিখলেও API Server তা উপেক্ষা করে। `spec` ও `status`-এর পার্থক্যই Declarative মডেলের হৃদয়: আপনি চাওয়া বলেন, সিস্টেম বাস্তবতা জানায়।

৪.৬.৩ kubectl apply বনাম create

`kubectl create -f file.yaml` অবজেক্টটি একবার তৈরি করে; অবজেক্টটি আগে থেকে থাকলে ত্রুটি দেয়। `kubectl apply -f file.yaml` ঘোষণামূলকভাবে কাজ করে: অবজেক্ট না থাকলে তৈরি করে, থাকলে ফাইলের সঙ্গে মিলিয়ে হালনাগাদ করে। `apply` আগের অবস্থা মনে রাখে (একটি বিশেষ annotation-এ) — তাই পরিবর্তনগুলো একত্রিত (merge) হয়। প্রোডাকশনে `apply`-ই সঠিক পছন্দ, কারণ আপনি একই ফাইল বারবার চালাতে পারেন এবং ফল একই থাকে (idempotent — পুনরাবৃত্তিতে একই ফল)। `create` মূলত একবারের কাজের জন্য।

৪.৬.৪ API Resources, Groups ও Versions

Kubernetes API অনেকগুলো **Group**-এ ভাগ করা: মূল (core) গোষ্ঠী (`v1` — Pod, Service, Namespace, ConfigMap), `apps` গোষ্ঠী (`apps/v1` — Deployment, StatefulSet), `networking.k8s.io` গোষ্ঠী (`networking.k8s.io/v1` — Ingress, NetworkPolicy) ইত্যাদি। প্রতিটি গোষ্ঠীর একাধিক **Version** থাকতে পারে: `v1alpha1` (পরীক্ষামূলক — যেকোনো সময় বদলাতে পারে), `v1beta1` (বিটা — মোটামুটি স্থিতিশীল কিন্তু পরিবর্তন সম্ভব), `v1` (স্থিতিশীল — দীর্ঘমেয়াদি সমর্থন)। পুরোনো সংস্করণ **Deprecated** (অবচিত — ভবিষ্যতে সরিয়ে দেওয়া হবে) ঘোষিত হলে নতুন সংস্করণে যেতে হয়। নিয়ম: সবসময় বর্তমান স্থিতিশীল `apiVersion` ব্যবহার করুন; কোনো টিউটোরিয়ালে পুরোনো সংস্করণ দেখলে অফিসিয়াল ডকুমেন্টেশন (kubernetes.io) দিয়ে যাচাই করুন। `kubectl api-resources` দিয়ে ক্লাস্টারে কোন রিসোর্স আছে তা দেখা যায়।

৪.৬.৫ Namespace — API প্রসঙ্গে

Namespace হলো একটি ক্লাস্টারের ভেতরে নামের আলাদা ক্ষেত্র (scope)। একই নামের Pod দুটি ভিন্ন নেমস্পেসে থাকতে পারে, কিন্তু একই নেমস্পেসে নয়। নেমস্পেস দল, পরিবেশ (dev/staging/prod) বা অ্যাপ আলাদা করতে ব্যবহৃত হয়। কিছু অবজেক্ট নেমস্পেসের ভেতরে থাকে (Pod, Deployment, Service), কিছু ক্লাস্টারজুড়ে (Node, ClusterRole, PersistentVolume)। `kubectl get pods -n technova`-এ `-n` হলো নেমস্পেস নির্দেশ করা। ভুল নেমস্পেসে কাজ করা নতুনদের সবচেয়ে সাধারণ ভুল — সবসময় যাচাই করুন আপনি কোন নেমস্পেসে আছেন।

৪.৭ কনফিগারেশন উদাহরণ

নিচের Deployment ম্যানিফেস্টটি লাইন ধরে পড়ুন — এটিই এই অধ্যায়ের মূল অনুশীলন।

```
apiVersion: apps/v1          # API গোষ্ঠী apps, স্থিতিশীল সংস্করণ v1
kind: Deployment             # অবজেক্টের ধরন: Deployment
metadata:                    # পরিচয়
  name: web                  # নাম — নেমস্পেসের ভেতরে অনন্য
  namespace: technova        # কোন নেমস্পেসে থাকবে
  labels:                    # লেবেল — বাছাইয়ের চিহ্ন
    app: web
    team: platform
spec:                        # কাঙ্ক্ষিত অবস্থা — আপনি যা চান
  replicas: 3                # তিনটি কপি চলতে থাকুক
  selector:                  # কোন Pod এই Deployment-এর, লেবেল দিয়ে চেনা
    matchLabels:
      app: web
  template:                  # Pod-এর ছাঁচ (template)
    metadata:
      labels:
        app: web             # selector-এর সঙ্গে মিলতে হবে
    spec:
      containers:
        - name: web
          image: technova/web:1.0.0   # নির্দিষ্ট ট্যাগ — :latest নয়
          ports:
            - containerPort: 80
          resources:
            requests:                # Scheduler-এর জন্য ন্যূনতম চাহিদা
              cpu: "100m"
              memory: "128Mi"
            limits:                  # সর্বোচ্চ সীমা
              cpu: "500m"
              memory: "256Mi"
```

```
# একই অ্যাপের Service — Pod-গুলোর সামনে স্থির নাম ও ঠিকানা
apiVersion: v1
kind: Service
metadata:
  name: web
  namespace: technova
spec:
  selector:
    app: web
  ports:
    - port: 80
      targetPort: 80
  type: ClusterIP    # শুধু ক্লাস্টারের ভেতর থেকে পৌঁছানো যায়
```

```
# API রিসোর্স দেখুন ও যাচাই করুন
kubectl api-resources | head -n 30
kubectl explain deployment.spec.replicas
kubectl apply -f web-deployment.yaml --dry-run=client    # না চালিয়ে যাচাই
kubectl apply -f web-deployment.yaml
```

```
kubectl get deployment web -n technova
kubectl get pods -n technova -l app=web
```

অংশ	কে লেখে	উদ্দেশ্য
`apiVersion + kind`	আপনি	অবজেক্টের ধরন ও API সংস্করণ চিহ্নিত করা
`metadata`	আপনি (+ সিস্টেম কিছু যোগ করে)	নাম, নেমস্পেস, লেবেল, টীকা
`spec`	আপনি	কাঙ্ক্ষিত অবস্থা
`status`	সিস্টেম (কন্ট্রোলার)	বর্তমান অবস্থা — পড়ার জন্য, লেখার জন্য নয়

৪.৮ বাস্তব প্রকৌশল পরিস্থিতি

TechNova পরিস্থিতি: TechNova-র একজন নতুন প্রকৌশলী `kubectl create -f web.yaml` দিয়ে Deployment তৈরি করলেন, পরে `replicas` ৩ থেকে ৫ করে আবার `create` চালালেন — ত্রুটি এলো: "already exists"। তিনি তখন Deployment মুছে আবার তৈরি করলেন — তাতে কয়েক মিনিট সার্ভিস বন্ধ থাকল। সঠিক পদ্ধতি ছিল `kubectl apply -f web.yaml` — তাতে Kubernetes নিজেই পার্থক্য বের করে Rolling Update করত, ডাউনটাইম ছাড়া। শিক্ষা: Declarative টুলকে Imperative ভঙ্গিতে ব্যবহার করলে তার সুবিধা হারায়। Git-এ ম্যানিফেস্ট রাখা, পরিবর্তন পর্যালোচনা করে `apply` করা — এটাই প্রোডাকশন পদ্ধতি।

> **REAL-WORLD:** প্রোডাকশন ঘটনার পর্যালোচনায় (post-mortem) প্রায়ই দেখা যায় সমস্যার মূলে ছিল "কে কী বদলেছিল তা জানা যায়নি"। Git-এ সংরক্ষিত ম্যানিফেস্ট সেই প্রশ্নের উত্তর দেয় — কে, কখন, কী বদলেছিল। Imperative কমান্ডের কোনো ইতিহাস থাকে না।

৪.৯ ধাপে ধাপে ব্যবহারিক উদাহরণ

```
# ১. নেমস্পেস তৈরি করুন
kubectl create namespace technova

# ২. ম্যানিফেস্টটি ফাইলে লিখুন (উপরের Deployment) — web-deployment.yaml
# ৩. না চালিয়ে যাচাই করুন
kubectl apply -f web-deployment.yaml --dry-run=client

# ৪. প্রয়োগ করুন
kubectl apply -f web-deployment.yaml

# ৫. অবস্থা দেখুন — spec বনাম status
kubectl get deployment web -n technova
kubectl get deployment web -n technova -o yaml
# status অংশে availableReplicas দেখুন — এটি সিস্টেম লিখেছে

# ৬. স্কেল করুন — ফাইলে replicas: 5 করে আবার apply করুন
kubectl apply -f web-deployment.yaml
kubectl get pods -n technova -l app=web

# ৭. পরিষ্কার করুন
```

```
kubectl delete -f web-deployment.yaml
kubectl delete namespace technova
```

৪.১০ সাধারণ ভুল ও সমস্যা সমাধান

ভুল / ত্রুটি	কারণ ও সমাধান
YAML ইন্ডেন্টেশন ভুল	YAML-এ ট্যাব নয়, স্পেস ব্যবহার করুন (সাধারণত ২ স্পেস)। ত্রুটির বার্তায় লাইন নম্বর দেখুন।
`selector` Pod-এর লেবেলের সঙ্গে মেলে না	Deployment কোনো Pod-কে নিজের বলে চিনতে পারে না — `kubectl get pods -l app=web` দিয়ে যাচাই করুন; selector ও template-এর লেবেল এক হতে হবে।
ভুল `apiVersion` — "no matches for kind"	API সংস্করণটি ক্লাস্টারে নেই বা অবচিত হয়েছে। `kubectl api-resources` ও অফিসিয়াল ডকুমেন্টেশন (kubernetes.io) দিয়ে সঠিক সংস্করণ নিন।
`status` হাতে বদলানোর চেষ্টা	`status` সিস্টেমের লেখা — হাতে বদলালে তা পরের Reconciliation-এ আবার ফিরে যাবে। কাঙ্ক্ষিত পরিবর্তন `spec` -এ করুন।
ভুল নেমস্পেসে অবজেক্ট তৈরি	`kubectl get pods -n technova` খালি দেখালে `-A` দিয়ে সব নেমস্পেসে খুঁজুন; `kubectl config set-context --current --namespace=technova` দিয়ে ডিফল্ট ঠিক করুন।
`:latest` ট্যাগ ব্যবহার	কোন সংস্করণ চলছে বোঝা যায় না, Rollback অনিশ্চিত হয়। সবসময় নির্দিষ্ট সংস্করণ ট্যাগ (`1.0.0`) ব্যবহার করুন।

৪.১১ নিরাপত্তা বিবেচনা

ম্যানিফেস্টে কখনও গোপন তথ্য (পাসওয়ার্ড, টোকেন, API কী) সরাসরি লিখবেন না — Git-এ গেলে তা চিরকাল থেকে যায়। সিক্রেটের জন্য `Secret` অবজেক্ট বা বাইরের সিক্রেট ব্যবস্থাপনা ব্যবহার করুন, আর উদাহরণে শুধু `CHANGE-ME-...` প্লেসহোল্ডার লিখুন। RBAC দিয়ে ঠিক করুন কে কোন নেমস্পেসে কী বদলাতে পারে — একজন ডেভেলপার `technova` নেমস্পেসে Deployment বদলাতে পারলেও `kube-system`-এ পারবে না। ম্যানিফেস্টে `imagePullPolicy` ও ইমেজের উৎস যাচাই করুন — অজানা রেজিস্ট্রির ইমেজ চালানো বিপজ্জনক।

৪.১২ কর্মক্ষমতা ও খরচ বিবেচনা

`resources.requests` ঠিকমতো নির্ধারণ করুন — Scheduler এই মান দেখে নোড বেছে নেয়; requests খুব কম দিলে নোডে অতিরিক্ত Pod বসে ও প্রতিযোগিতা বাড়ে, খুব বেশি দিলে সম্পদ খালি পড়ে থাকে। `limits` না দিলে একটি Pod সব মেমোরি খেয়ে অন্য Pod-কে মারতে পারে। বড় ম্যানিফেস্ট ফাইলকে ছোট ছোট ফাইলে ভাগ করুন (প্রতি অবজেক্ট একটি ফাইল বা Kustomize/Helm — পরের খণ্ডে) — পর্যালোচনা ও Rollback সহজ হয়।

৪.১৩ সেরা অনুশীলন ও অ্যান্টি-প্যাটার্ন

সেরা অনুশীলন: সব ম্যানিফেস্ট Git-এ রাখুন; `apply` ব্যবহার করুন, `create` নয়; প্রতিটি অবজেক্টে অর্থবহ `labels` দিন (`app`, `team`, `env`); নির্দিষ্ট ইমেজ ট্যাগ ব্যবহার করুন; `--dry-run=client` দিয়ে প্রয়োগের আগে যাচাই করুন; `kubectl explain` দিয়ে ক্ষেত্রের অর্থ দেখুন।

অ্যান্টি-প্যাটার্ন: হাতে `kubectl run` দিয়ে প্রোডাকশন Pod তৈরি করা; ম্যানিফেস্ট ছাড়া `kubectl scale` বা `kubectl edit`-এর পরিবর্তন রেখে দেওয়া (পরের `apply`-এ তা হারিয়ে যেতে পারে বা দ্বন্দ্ব তৈরি হয়); `status` হাতে সম্পাদনা করা; অবচিত (deprecated) `apiVersion` নতুন ফাইলে ব্যবহার করা; সিক্রেট YAML-এ লেখা।

> **PRO TIP:** কোনো ক্ষেত্রের অর্থ না জানলে `kubectl explain` ব্যবহার করুন — যেমন `kubectl explain deployment.spec.template.spec.containers`। এটি অফিসিয়াল API ডকুমেন্টেশন থেকেই তথ্য দেয়, অনুমানের প্রয়োজন হয় না।

৪.১৪ অধ্যায়ের সারাংশ

Kubernetes Declarative — আপনি কাঙ্ক্ষিত অবস্থা YAML ম্যানিফেস্টে লেখেন, সিস্টেম তা বজায় রাখে। প্রতিটি অবজেক্টের গঠন `apiVersion`, `kind`, `metadata`, `spec` (আপনার চাওয়া) ও `status` (সিস্টেমের জানানো বাস্তবতা)। `kubectl apply` ঘোষণামূলকভাবে তৈরি ও হালনাগাদ করে; `create` শুধু একবার তৈরি করে। API Resources গোষ্ঠী ও সংস্করণে ভাগ করা — স্থিতিশীল সংস্করণ ব্যবহার করুন, অবচিত সংস্করণ এড়িয়ে চলুন। Namespace অবজেক্টের নামের ক্ষেত্র আলাদা করে।

৪.১৫ মূল প্রযুক্তিগত পরিভাষা

Declarative Model: কাঙ্ক্ষিত শেষ অবস্থা ঘোষণা করার পদ্ধতি; সিস্টেম নিজে বর্তমান অবস্থাকে সেই দিকে নিয়ে যায়।

Manifest: কাঙ্ক্ষিত অবস্থা লেখা YAML (বা JSON) ফাইল, যা Git-এ রাখা ও পর্যালোচনা করা যায়।

Object: Kubernetes API-তে সংরক্ষিত সত্তা (Pod, Deployment, Service ইত্যাদি), যার spec ও status থাকে।

spec / status: spec হলো কাঙ্ক্ষিত অবস্থা (ব্যবহারকারী লেখে), status হলো বর্তমান অবস্থা (সিস্টেম লেখে)।

API Group / Version: API-র গোষ্ঠী (যেমন apps) ও সংস্করণ (যেমন v1); স্থিতিশীল সংস্করণ দীর্ঘমেয়াদি সমর্থন পায়।

Namespace: ক্লাস্টারের ভেতরে নামের আলাদা ক্ষেত্র; দল বা পরিবেশ আলাদা করতে ব্যবহৃত হয়।

Label: অবজেক্টে লাগানো কী-ভ্যালু চিহ্ন, যা দিয়ে অবজেক্ট বাছাই ও গোষ্ঠীভুক্ত করা যায়।

৪.১৬ জ্ঞান যাচাই

১. Declarative ও Imperative মডেলের পার্থক্য কী? প্রোডাকশনে Declarative কেন পছন্দ করা হয়?

২. একটি Kubernetes অবজেক্টের পাঁচটি অংশ কী কী? `spec` ও `status`-এর মধ্যে পার্থক্য কোথায়?

৩. `kubectl apply` ও `kubectl create`-এর মধ্যে পার্থক্য কী? একটি উদাহরণ দিয়ে বোঝান কখন `create` বিপজ্জনক হতে পারে।

৪. `apps/v1` বলতে কী বোঝায়? `v1alpha1`, `v1beta1` ও `v1`-এর মধ্যে পার্থক্য কী?

৫. Deployment-এর `selector` Pod-এর লেবেলের সঙ্গে না মিললে কী হয়? কীভাবে বুঝবেন?

৬. Namespace কী? একই নামের Pod দুটি ভিন্ন নেমস্পেসে থাকতে পারে কেন?

৭. ম্যানিফেস্টে সিক্রেট সরাসরি লেখা কেন বিপজ্জনক? এর বদলে কী করা উচিত?

৪.১৭ ব্যবহারিক অ্যাসাইনমেন্ট

অ্যাসাইনমেন্ট: TechNova-র `web` Deployment-এর ম্যানিফেস্টটি (৪.৭) নিজের লোকাল ক্লাস্টারে লিখে `apply` করুন। তারপর (ক) `replicas` ৩ থেকে ৫ করে আবার `apply` করুন ও Pod-এর সংখ্যা বাড়তে দেখুন; (খ) `kubectl get deployment -o yaml` দিয়ে `status` অংশটি পড়ে লিখুন সিস্টেম কী কী জানাচ্ছে; (গ) ইচ্ছাকৃতভাবে `selector`-এর লেবেল বদলে `apply` করুন, ত্রুটি বা আচরণটি লিপিবদ্ধ করুন, তারপর ঠিক করুন। এক পৃষ্ঠার প্রতিবেদনে তিনটি পরীক্ষার ফলাফল ও আপনার শেখা বিষয় লিখুন।

অধ্যায় ৫ • PART I — Kubernetes Foundations & Architecture

Chapter 5 — Local and Managed Cluster Setup — kind, Minikube, k3d, and Managed Kubernetes

৫.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি নিজের ল্যাপটপে একটি Kubernetes ক্লাস্টার চালু করতে পারবেন। আপনি **kind** (Kubernetes IN Docker — Docker কন্টেইনারের ভেতরে Kubernetes নোড চালানোর টুল), **Minikube** (লোকাল এক-নোড ক্লাস্টার চালানোর জনপ্রিয় টুল) ও **k3d** (k3s-কে Docker-এ চালানোর টুল, হালকা Kubernetes) — এই তিনটির পার্থক্য, সুবিধা ও অসুবিধা তুলনা করে নিজের প্রয়োজনে সঠিকটি বেছে নিতে পারবেন। আপনি `kubectl` ইনস্টল করতে, **kubeconfig** (ক্লাস্টারের ঠিকানা ও শংসাপত্রের ফাইল) ও **Context** (কোন ক্লাস্টারে কাজ করছেন তার নির্বাচন) পরিচালনা করতে, ক্লাস্টার যাচাই করতে পারবেন, এবং Managed Kubernetes (EKS/AKS/GKE/IBM Cloud — ক্লাউড সরবরাহকারী পরিচালিত Kubernetes) সম্পর্কে উচ্চ-স্তরের ধারণা পাবেন।

৫.২ কেন এই বিষয়টি গুরুত্বপূর্ণ

Kubernetes পড়ে শেখা যায় না — চালিয়ে শেখা যায়। কিন্তু প্রথম ক্লাস্টার চালু করতেই অনেকে হোঁচট খান: কোন টুল নেবেন, ল্যাপটপে চলবে কি না, ভুল ক্লাস্টারে কমান্ড চালিয়ে ফেলবেন কি না। ভুল টুল বেছে নিলে হয় ল্যাপটপ ধীর হয়ে যায়, নয়তো শেখার জন্য প্রয়োজনীয় বৈশিষ্ট্য (যেমন একাধিক নোড) পাওয়া যায় না। আরেকটি বাস্তব ঝুঁকি: kubeconfig-এ একাধিক ক্লাস্টার থাকলে ভুল context-এ `kubectl delete` চালানো — প্রোডাকশন মুছে যাওয়ার মতো দুর্ঘটনা এই ভুল থেকেই ঘটে। এই অধ্যায় আপনাকে নিরাপদে শুরু করার পথ দেখায়: লোকালে শিখুন, যাচাই করুন, তারপর Managed সেবার দিকে তাকান।

> **CONCEPT:** লোকাল ক্লাস্টার হলো আপনার পরীক্ষাগার (lab) — এখানে ভাঙা নিরাপদ। প্রোডাকশন ক্লাস্টার হলো বাস্তব সড়ক। পরীক্ষাগারে যা শিখবেন তা সড়কে কাজে লাগবে, কিন্তু পরীক্ষাগারের সেটআপকে প্রোডাকশন ভাববেন না।

৫.৩ পূর্বশর্ত জ্ঞান

Docker ইনস্টল করা ও চলমান থাকতে হবে (kind ও k3d Docker ব্যবহার করে; Minikube Docker ড্রাইভারও ব্যবহার করতে পারে)। কমান্ড লাইনে কাজের অভিজ্ঞতা, YAML ও `kubectl`-এর প্রাথমিক ধারণা (অধ্যায় ৪) প্রয়োজন। ল্যাপটপে অন্তত ৪ GB ফাঁকা মেমোরি ও ২টি CPU কোর থাকলে লোকাল ক্লাস্টার স্বচ্ছন্দে চলবে। ইন্টারনেট সংযোগ প্রয়োজন — ইমেজ ও টুল ডাউনলোডের জন্য।

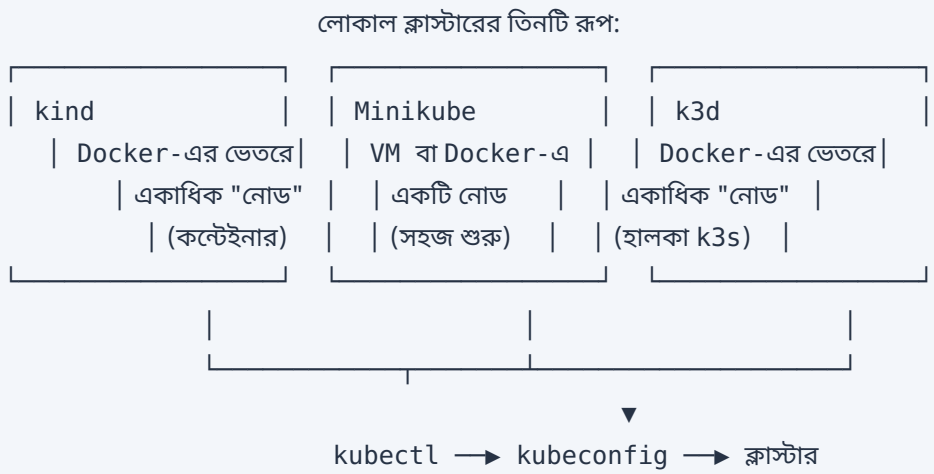
৫.৪ প্রথম নীতি থেকে ব্যাখ্যা

প্রথম নীতি: Kubernetes নিজে একটি বিতরণ করা সিস্টেম — তা চালাতে একাধিক মেশিন লাগে। ল্যাপটপে একাধিক মেশিন নেই — তাই লোকাল টুলগুলো ভান করে: Docker কন্টেইনারের ভেতরে "নোড" সাজায় (kind, k3d), অথবা ভার্চুয়াল মেশিনে একটি নোড চালায় (Minikube)। ভানটি যথেষ্ট ভালো — API, Scheduling, Pod, Service — সব একইভাবে কাজ করে। শুধু মনে রাখতে হবে: লোকাল ক্লাস্টারের কর্মক্ষমতা ও নির্ভরযোগ্যতা আসল প্রোডাকশনের মতো নয়।

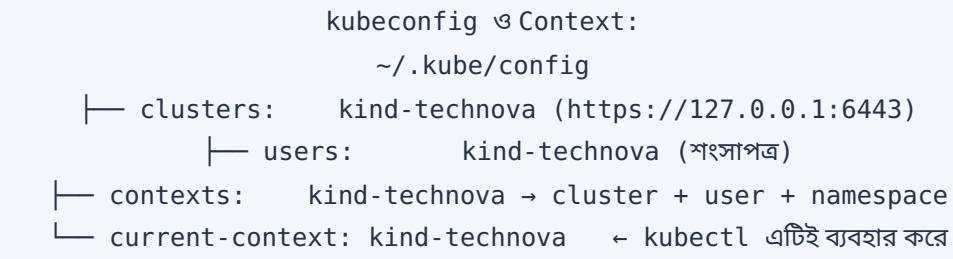
দ্বিতীয় নীতি: ক্লাস্টারের সঙ্গে কথা বলার জন্য `kubectl`-এর তিনটি জিনিস লাগে — ক্লাস্টারের ঠিকানা, আপনার পরিচয় (শংসাপত্র), এবং আপনি কোন নেমস্পেসে কাজ করছেন। এই তিনটি kubeconfig ফাইলে থাকে। একাধিক ক্লাস্টার থাকলে context বদলে আপনি কোন ক্লাস্টারে কাজ করছেন তা ঠিক করেন। Context না বুঝলে আপনি অজান্তে ভুল ক্লাস্টারে পরিবর্তন করতে পারেন — তাই context যাচাই করা প্রতিটি সেশনের প্রথম অভ্যাস হওয়া উচিত।

তৃতীয় নীতি: Managed Kubernetes-এ ক্লাউড সরবরাহকারী Control Plane পরিচালনা করে — আপনি শুধু Worker Node ও অ্যাপ নিয়ে ভাবেন। এর বিনিময়ে আপনি নিয়ন্ত্রণ হারান: Control Plane-এর ভেতরে ঢুকে দেখতে পারবেন না, কিছু সিদ্ধান্ত সরবরাহকারীর হাতে থাকে। লোকালে শেখা ও Managed-এ চালানো — দুটি ভিন্ন দক্ষতা, দুটিই প্রয়োজন।

৫.৫ আর্কিটেকচার / ধারণাগত চিত্র



চিত্র ৫.১: তিনটি লোকাল টুল — একই kubectl, ভিন্ন ভান।



চিত্র ৫.২: kubeconfig-এর গঠন — context হলো cluster + user + namespace-এর সমন্বয়।

৫.৬ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

৫.৬.১ kind — Kubernetes IN Docker

kind Docker কন্টেইনারের ভেতরে Kubernetes নোড চালায় — প্রতিটি নোড একটি Docker কন্টেইনার। এর সবচেয়ে বড় সুবিধা: একাধিক নোডের ক্লাস্টার কয়েক মিনিটে তৈরি হয়, ল্যাপটপেই মাল্টি-নোড আচরণ (যেমন একটি নোড নষ্ট হলে Pod সরানো) পরীক্ষা করা যায়। Kubernetes প্রকল্পের নিজস্ব পরীক্ষার টুল হিসেবে এটি অফিসিয়ালভাবে ব্যবহৃত হয়। সীমা: এটি

ল্যাপটপের Docker-এর ওপর নির্ভরশীল, নেটওয়ার্ক কিছুটা কৃত্রিম, আর ভারী লোডের জন্য নয়। এই বইয়ের ল্যাবগুলোর জন্য kind প্রথম পছন্দ। অফিসিয়াল ডকুমেন্টেশন: kind.sigs.k8s.io।

৫.৬.২ Minikube — সবচেয়ে পরিচিত শুরু

Minikube লোকাল এক-নোড ক্লাস্টার চালায় — Docker ড্রাইভার, ভার্চুয়াল মেশিন, এমনকি সরাসরি হোস্টেও চলতে পারে। এর সুবিধা: শুরু করা সবচেয়ে সহজ, অনেক অ্যাড-অন (addon) আছে — যেমন ড্যাশবোর্ড, Ingress, Metrics Server — এক কমান্ডে চালু করা যায়। সীমা: ডিফল্টভাবে একটিই নোড (একাধিক নোডও সম্ভব, তবে kind/k3d-এর চেয়ে ভারী), আর কিছু আচরণ (যেমন LoadBalancer) লোকালভাবে অনুকরণ করতে হয়। প্রথমবার Kubernetes ছোঁয়ার জন্য Minikube চমৎকার। অফিসিয়াল ডকুমেন্টেশন: minikube.sigs.k8s.io।

৫.৬.৩ k3d — হালকা ও দ্রুত

k3d হলো k3s (Rancher-এর হালকা Kubernetes বিতরণ) কে Docker-এ চালানোর টুল। k3s নিজেই ছোট — কম মেমোরি খায়, দ্রুত চালু হয় — তাই k3d পুরোনো বা কম সম্পদের ল্যাপটপেও ভালো চলে। একাধিক নোডের ক্লাস্টারও দ্রুত তৈরি হয়। সীমা: k3s কিছু ক্ষেত্রে আদর্শ Kubernetes থেকে সামান্য ভিন্ন (যেমন ডিফল্ট CNI ও কিছু কম্পোনেন্ট), তবে শেখার জন্য পার্থক্যটি নগণ্য। দ্রুত পরীক্ষা ও CI পরিবেশে k3d জনপ্রিয়।

৫.৬.৪ kubectl ইনস্টল ও kubeconfig পরিচালনা

`kubectl` হলো Kubernetes-এর কমান্ড-লাইন টুল — এটি ক্লাস্টারের সঙ্গে কথা বলে, নিজে ক্লাস্টার নয়। ইনস্টল করার পর `kubectl version --client` দিয়ে সংস্করণ যাচাই করুন — ক্লায়েন্ট ও সার্ভারের সংস্করণ খুব বেশি আলাদা হওয়া উচিত নয়। kubeconfig সাধারণত `~/.kube/config`-এ থাকে। kind/Minikube/k3d ক্লাস্টার তৈরি করার সময় নিজেরাই kubeconfig হালনাগাদ করে। একাধিক ক্লাস্টার থাকলে `kubectl config get-contexts` দিয়ে তালিকা দেখুন, `kubectl config use-context <name>` দিয়ে বদলান, আর `kubectl config current-context` দিয়ে যাচাই করুন। গুরুত্বপূর্ণ: kubeconfig-এ শংসাপত্র থাকে — ফাইলটি কখনও শেয়ার করবেন না, Git-এ রাখবেন না।

৫.৬.৫ Managed Kubernetes — উচ্চ-স্তরের ধারণা

Managed Kubernetes মানে ক্লাউড সরবরাহকারী Control Plane পরিচালনা করে: Amazon EKS, Microsoft AKS, Google GKE, IBM Cloud Kubernetes Service — এরা Control Plane-এর স্থাপন, হালনাগাদ ও ব্যাকআপের দায়িত্ব নেয়। আপনি Worker Node-এর আকার ও সংখ্যা ঠিক করেন, অ্যাপ স্থাপন করেন। সুবিধা: Control Plane-এর পরিচালনার ভার কমে, High Availability সরবরাহকারী নিশ্চিত করে। সীমা: Control Plane-এর ভেতরে সরাসরি প্রবেশ থাকে না, কিছু সিদ্ধান্ত সরবরাহকারীর নিয়ন্ত্রণে, আর খরচ নির্ভর করে নোডের আকার ও সংখ্যার ওপর (এই বইয়ে কোনো মূল্য দেওয়া হয়নি — মূল্য সময়ের সঙ্গে বদলায়, সরবরাহকারীর অফিসিয়াল পৃষ্ঠা দেখুন)। শেখার জন্য Managed প্রয়োজন নেই — লোকাল ক্লাস্টারই যথেষ্ট; প্রোডাকশনের সিদ্ধান্তে Managed বিবেচনা করা হয়।

৫.৭ কনফিগারেশন উদাহরণ

```
# kind দিয়ে মাল্টি-নোড ক্লাস্টার তৈরি (kind-config.yaml)
# kind: Cluster
# apiVersion: kind.x-k8s.io/v1alpha4
# nodes:
#   - role: control-plane
#   - role: worker
#   - role: worker
kind create cluster --name technova --config kind-config.yaml
```

```
kubectl cluster-info --context kind-technova
kubectl get nodes
```

```
# Minikube দিয়ে ক্লাস্টার তৈরি
minikube start --driver=docker --cpus=2 --memory=4096
minikube status
kubectl get nodes

# k3d দিয়ে ক্লাস্টার তৈরি
k3d cluster create technova --servers 1 --agents 2
kubectl get nodes
k3d cluster list
```

```
# kubectl যাচাই ও context পরিচালনা
kubectl version --client
kubectl config get-contexts
kubectl config current-context
kubectl config use-context kind-technova
kubectl cluster-info

# ক্লাস্টার কাজ করছে কি না — ধোঁয়া পরীক্ষা (smoke test)
kubectl run smoke --image=nginx:1.27 --port=80 -n default
kubectl get pods -o wide
kubectl port-forward pod/smoke 8080:80
# ব্রাউজারে http://localhost:8080 খুলুন — nginx পৃষ্ঠা দেখা যাবে
kubectl delete pod smoke
```

টুল	নোড	সেরা ব্যবহার	সুবিধা	অসুবিধা
kind	একাধিক (Docker কন্টেইনার)	মাল্টি-নোড ল্যাব, CI পরীক্ষা	দ্রুত, অফিসিয়াল পরীক্ষার টুল	Docker-নির্ভর, ভারী লোডের জন্য নয়
Minikube	ডিফল্ট এক (বহু সম্ভব)	প্রথম শেখা, অ্যাড-অন পরীক্ষা	সহজ শুরু, সমৃদ্ধ অ্যাড-অন	এক নোডে মাল্টি-নোড আচরণ শেখা যায় না
k3d	একাধিক (Docker কন্টেইনার)	হালকা ল্যাব, দ্রুত পরীক্ষা	খুব হালকা ও দ্রুত	k3s-এর কিছু ডিফল্ট ভিন্ন
Managed (EKS/AKS/GKE/IBM)	অনেক	প্রোডাকশন	Control Plane পরিচালিত, HA	নিয়ন্ত্রণ কম, ক্লাউড খরচ

৫.৮ বাস্তব প্রকৌশল পরিস্থিতি

TechNova পরিস্থিতি: TechNova-র একজন জুনিয়র প্রকৌশলী ল্যাপটপে Minikube চালিয়ে Deployment পরীক্ষা করছিলেন — সব ঠিকঠাক চলছিল। কিন্তু প্রোডাকশনে (তিন নোডের Managed ক্লাস্টার) একই Deployment-এর Pod-গুলো একটি নোডেই জমা হলো, আর সেই নোড রক্ষণাবেক্ষণের জন্য বন্ধ হলে সার্ভিস থেমে গেল। কারণ: এক-নোড ল্যাবে তিনি কখনও নোড-ব্যর্থতা দেখেননি, তাই Pod ছড়িয়ে রাখার (topology spread) কথা ভাবেননি। শিক্ষা: লোকাল ল্যাব শেখার জন্য চমৎকার, কিন্তু প্রোডাকশনের আচরণ অনুকরণ করতে মাল্টি-নোড ল্যাব (kind বা k3d-তে দুটি Worker) ব্যবহার করুন — এক নোডের ল্যাবে যা দেখা যায় না, তা প্রোডাকশনে প্রথমবার দেখা বিপজ্জনক।

> **WARNING:** কোনো কমান্ড চালানোর আগে সবসময় `kubectl config current-context` দেখুন। লোকাল ল্যাবের context-এ থাকা অবস্থায় প্রোডাকশনের কমান্ড চালানো — আর প্রোডাকশন context-এ থাকা অবস্থায় পরীক্ষামূলক `delete` চালানো — দুটোই বাস্তবে ঘটে যাওয়া দুর্ঘটনা।

৫.৯ ধাপে ধাপে ব্যবহারিক উদাহরণ

এই বইয়ের ল্যাবগুলোর জন্য প্রস্তাবিত পথ: kind দিয়ে শুরু করুন।

```
# ১. Docker চলছে কি না যাচাই করুন
docker version

# ২. kind ইনস্টল করুন (অফিসিয়াল নির্দেশনা: kind.sigs.k8s.io)
# ৩. ক্লাস্টার তৈরি করুন — একটি control-plane + দুটি worker
kind create cluster --name technova

# ৪. যাচাই করুন
kubectl cluster-info --context kind-technova
kubectl get nodes
kubectl get pods -n kube-system

# ৫. TechNova নেমস্পেস তৈরি ও একটি Pod চালু করুন
kubectl create namespace technova
kubectl run web --image=nginx:1.27 --port=80 -n technova
kubectl get pods -n technova -o wide

# ৬. ক্লাস্টার মুছে ফেলুন (পরীক্ষা শেষে)
kind delete cluster --name technova
# Minikube হলে: minikube delete
# k3d হলে: k3d cluster delete technova
```

৫.১০ সাধারণ ভুল ও সমস্যা সমাধান

সমস্যা	কারণ ও সমাধান
`kubectl`: "connection refused"	ক্লাস্টার চলছে না, বা context ভুল। `kubectl config current-context` দেখুন; kind হলে `kind get clusters`, Minikube হলে `minikube status` দিয়ে ক্লাস্টার চালু আছে কি না দেখুন।
ল্যাপটপ ধীর / ক্লাস্টার চালু হচ্ছে না	মেমোরি বা CPU কম। Minikube-এ `--memory` কমিয়ে দিন, বা k3d-এর মতো হালকা টুল ব্যবহার করুন; অপ্রয়োজনীয় অ্যাপ বন্ধ করুন।
ভুল context-এ কমান্ড চলে গেছে	সঙ্গে সঙ্গে থামুন, `kubectl config get-contexts` দেখুন, সঠিক context-এ ফিরুন। প্রতিরোধে: প্রম্পটে context দেখানোর টুল (যেমন kubectx) ব্যবহার করুন।
Pod `Pending` — লোকাল ক্লাস্টারে	ল্যাপটপের সম্পদ শেষ। `kubectl describe node` দিয়ে খালি CPU/মেমোরি দেখুন; অপ্রয়োজনীয় Pod মুছুন।
kubeconfig ফাইল নষ্ট হয়ে গেছে	ব্যাকআপ না থাকলে ক্লাস্টার আবার তৈরি করতে হয় — লোকাল ক্লাস্টার পুনর্নির্মাণ সস্তা, তাই আতঙ্কিত হবেন না; তবে প্রোডাকশন kubeconfig-এর ব্যাকআপ ও সুরক্ষা অপরিহার্য।

৫.১১ নিরাপত্তা বিবেচনা

লোকাল ক্লাস্টারেও নিরাপত্তা অভ্যাস গড়ে তুলুন। kubeconfig ফাইলটি আপনার পরিচয়পত্র — কখনও শেয়ার করবেন না, Git-এ কমিট করবেন না, স্ক্রিনশটে দেখাবেন না। লোকাল ক্লাস্টারে সিক্রেটের জন্য শুধু `CHANGE-ME-...` ধরনের প্লেসহোল্ডার ব্যবহার করুন — আসল পাসওয়ার্ড কখনও ল্যাবে লিখবেন না, অভ্যাসটি প্রোডাকশনে বাঁচাবে। Managed ক্লাস্টারে API Server সাধারণত ইন্টারনেট থেকে পৌঁছানো যায় — সেখানে শক্তিশালী প্রমাণীকরণ, RBAC ও নেটওয়ার্ক সীমাবদ্ধতা বাধ্যতামূলক। লোকাল ক্লাস্টার মুছে ফেলা সহজ, কিন্তু তার kubeconfig-এর শংসাপত্র ফাঁস হলে তা Managed ক্লাস্টারের মতোই বিপজ্জনক হতে পারে — যদি একই শংসাপত্র কোথাও পুনর্ব্যবহৃত হয়।

৫.১২ কর্মক্ষমতা ও খরচ বিবেচনা

লোকাল ক্লাস্টারের খরচ আপনার ল্যাপটপের সম্পদ: kind-এ প্রতিটি নোড একটি কন্টেইনার — নোড বাড়ালে মেমোরি বাড়ে। দুটি Worker-এর kind ক্লাস্টার সাধারণত ২–৩ GB মেমোরি নেয়। Minikube এক নোডে কম খায়, তবে অ্যাড-অন চালু করলে খরচ বাড়ে। ব্যবহার না করার সময় ক্লাস্টার বন্ধ করুন (`minikube stop` বা `kind delete cluster`) — ল্যাপটপের ব্যাটারি ও গতি বাঁচবে। Managed ক্লাস্টারে খরচের হিসাব আলাদা — Control Plane ও Worker Node-এর জন্য ক্লাউড বিল আসে; এই বইয়ে কোনো মূল্য দেওয়া হয়নি, কারণ মূল্য অঞ্চল ও সময়ভেদে বদলায় — সিদ্ধান্তের আগে সরবরাহকারীর অফিসিয়াল পৃষ্ঠা দেখুন।

৫.১৩ সেরা অনুশীলন ও অ্যান্টি-প্যাটার্ন

সেরা অনুশীলন: এই বইয়ের ল্যাবের জন্য kind দিয়ে শুরু করুন (মাল্টি-নোড আচরণ শেখা যায়); প্রতিটি সেশনের শুরুতে context যাচাই করুন; ক্লাস্টার তৈরি ও মুছে ফেলার কমান্ড লিখে রাখুন; ল্যাবের ম্যানিফেস্ট Git-এ রাখুন; Managed-এ যাওয়ার আগে লোকালে সব পরীক্ষা করুন।

অ্যান্টি-প্যাটার্ন: লোকাল ক্লাস্টারকে প্রোডাকশন ভাবা; একটিমাত্র নোডে সব শিখে প্রোডাকশনের মাল্টি-নোড আচরণ অনুমান করা; kubeconfig শেয়ার করা বা Git-এ রাখা; context না দেখে `delete` চালানো; ল্যাবে আসল সিক্রেট ব্যবহার করা; ক্লাস্টার চালু রেখে ল্যাপটপের সম্পদ নষ্ট করা।

> **PRO TIP:** আপনার শেল প্রম্পটে বর্তমান kubectl context ও নেমস্পেস দেখানোর ব্যবস্থা করুন (kubectx/kubens বা শেল প্লাগইন দিয়ে)। ভুল ক্লাস্টারে কমান্ড চালানোর দুর্ঘটনা এতে প্রায় শূন্যে নেমে আসে।

> **LAB:** এই অধ্যায়ের ধাপগুলো অনুসরণ করে kind ক্লাস্টার তৈরি করুন, `kubectl get nodes` ও `kubectl get pods -n kube-system`-এর আউটপুট সংরক্ষণ করুন — পরের অধ্যায়গুলোর সব ল্যাব এই ক্লাস্টারেই চলবে।

৫.১৪ অধ্যায়ের সারাংশ

লোকাল Kubernetes শেখার তিনটি প্রধান টুল: kind (Docker-এ মাল্টি-নোড, এই বইয়ের ল্যাবের প্রথম পছন্দ), Minikube (সহজ শুরু, সমৃদ্ধ অ্যাড-অন) ও k3d (হালকা ও দ্রুত)। `kubectl` ক্লাস্টারের সঙ্গে কথা বলার টুল; kubeconfig-এ ক্লাস্টার, ব্যবহারকারী ও context থাকে — প্রতিটি সেশনে context যাচাই করা অপরিহার্য। Managed Kubernetes (EKS/AKS/GKE/IBM Cloud)-এ সরবরাহকারী Control Plane পরিচালনা করে — প্রোডাকশনের জন্য উপযুক্ত, শেখার জন্য লোকালই যথেষ্ট। ক্লাস্টার যাচাই করুন `cluster-info`, `get nodes` ও একটি smoke-test Pod দিয়ে।

৫.১৫ মূল প্রযুক্তিগত পরিভাষা

kind: Kubernetes IN Docker — Docker কন্টেইনারের ভেতরে Kubernetes নোড চালিয়ে লোকাল মাল্টি-নোড ক্লাস্টার তৈরির টুল।

Minikube: ল্যাপটপে এক-নোড (বা বহু-নোড) Kubernetes ক্লাস্টার চালানোর জনপ্রিয় টুল।

k3d: হালকা Kubernetes (k3s) কে Docker-এ চালানোর টুল; দ্রুত ও কম সম্পদে চলে।

kubeconfig: ক্লাস্টারের ঠিকানা, ব্যবহারকারীর শংসাপত্র ও context সংরক্ষণকারী ফাইল (সাধারণত ~/.kube/config)।

Context: kubeconfig-এ cluster + user + namespace-এর সমন্বয় — kubectl বর্তমানে কোন ক্লাস্টারে কাজ করছে তা নির্ধারণ করে।

Managed Kubernetes: ক্লাউড সরবরাহকারী পরিচালিত Kubernetes, যেখানে Control Plane-এর দায়িত্ব সরবরাহকারীর (যেমন EKS, AKS, GKE)।

৫.১৬ জ্ঞান যাচাই

১. kind, Minikube ও k3d-এর মধ্যে পার্থক্য কী? মাল্টি-নোড আচরণ শিখতে কোনটি সবচেয়ে উপযুক্ত এবং কেন?

২. kubeconfig-এ কী কী থাকে? Context বলতে কী বোঝায়, এবং ভুল context-এ কমান্ড চালালে কী বিপদ হতে পারে?

৩. `kubectl` নিজে কি ক্লাস্টার? না হলে এটি কী করে?

৪. Managed Kubernetes-এ Control Plane-এর দায়িত্ব কার? এর সুবিধা ও সীমা কী?

৫. ক্লাস্টার ঠিকমতো কাজ করছে কি না তা যাচাই করতে কোন কোন কমান্ড চালাবেন, কোন ক্রমে?

৬. এক-নোড লোকাল ল্যাবে শেখা আচরণ প্রোডাকশনে কেন সবসময় একইভাবে কাজ নাও করতে পারে? একটি উদাহরণ দিন।

৭. kubeconfig ফাইলটি কেন গোপন রাখতে হয়? এতে কী থাকে?

৫.১৭ ব্যবহারিক অ্যাসাইনমেন্ট

অ্যাসাইনমেন্ট: আপনার ল্যাপটপে kind দিয়ে `technova` নামে একটি ক্লাস্টার তৈরি করুন (একটি control-plane + দুটি worker)। তারপর (ক) `kubectl get nodes` ও `kubectl get pods -n kube-system`-এর আউটপুট সংরক্ষণ করুন; (খ) `technova` নেমস্পেসে একটি nginx Pod চালু করে `port-forward` দিয়ে ব্রাউজারে দেখুন; (গ) একটি তুলনা-নোট

লিখুন: আপনার ল্যাপটপে kind চালানোর সময় কতটা মেমোরি/CPU লেগেছে, এবং Minikube বা k3d কখন বেছে নিতেন — কারণসহ। ক্লাস্টারটি রেখে দিন — পরের অধ্যায়গুলোর ল্যাব এখানেই চলবে।

PART II • DEVOPS ENGINEERING MASTERY

Workloads & Application Management

Pod, Deployment, StatefulSet, DaemonSet, Job, CronJob, Namespace, Label, ConfigMap, Secret, Resource & Autoscaling

অধ্যায় ৬ • PART II — Workloads & Application Management

Chapter 6 — Pods, Lifecycle, Probes, Init Containers, and Sidecars

৬.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি Pod কী, কেন Kubernetes-এ সবচেয়ে ছোট স্থাপনযোগ্য একক (smallest deployable unit) হিসেবে Pod-কেই বেছে নেওয়া হয়েছে, এবং একটি Pod-এর ভেতরে একাধিক কনটেইনার কীভাবে একসাথে চলে — তা পরিষ্কারভাবে ব্যাখ্যা করতে পারবেন। আপনি Pod-এর জীবনচক্রের (lifecycle) প্রতিটি ধাপ — Pending, Running, Succeeded, Failed, Unknown — এবং কনটেইনারের অবস্থা — Waiting, Running, Terminated — চিনতে পারবেন। আপনি liveness, readiness ও startup probe-এর পার্থক্য, প্রতিটি probe ব্যর্থ হলে Kubernetes ঠিক কী করে, এবং ভুল probe কনফিগারেশন কীভাবে একটি সুস্থ অ্যাপকেও বারবার পুনরায় চালু (restart) করে দেয়, তা বুঝবেন। এছাড়া init container ও sidecar pattern কখন ব্যবহার করতে হয়, কখন নয়, এবং Pod IP কেন ক্ষণস্থায়ী (ephemeral) — তার ব্যবহারিক পরিণতি কী, তা শিখবেন।

ব্যবহারিক দিক থেকে আপনি একটি বহু-কনটেইনার Pod-এর YAML লিখতে পারবেন, তিন ধরনের probe কনফিগার করতে পারবেন, init container দিয়ে ডেটাবেস প্রস্তুত হওয়ার জন্য অপেক্ষা করতে পারবেন, এবং `kubectl describe pod` ও `kubectl logs` দিয়ে Pod-এর সমস্যা নির্ণয় করতে পারবেন।

৬.২ কেন এই বিষয়টি গুরুত্বপূর্ণ

Kubernetes-এর প্রায় সবকিছুই Pod-কে কেন্দ্র করে ঘোরে। Deployment, StatefulSet, DaemonSet, Job — যে workload-ই ব্যবহার করুন না কেন, শেষ পর্যন্ত Kubernetes একটি Pod তৈরি করে, সেটিকে কোনো Node-এ সময়সূচি (schedule) করে, তার কনটেইনারগুলো চালায়, এবং সেগুলোর স্বাস্থ্য পর্যবেক্ষণ করে। Pod না বুঝলে Deployment-এর rollout, Service-এর endpoint, কিংবা autoscaling — কোনোটাই গভীরে বোঝা যায় না। বাস্তব প্রকল্পে অধিকাংশ প্রোডাকশন সমস্যার মূলেও থাকে Pod: CrashLoopBackOff, ImagePullBackOff, probe ব্যর্থতা, OOMKilled, কিংবা Pod IP বদলে যাওয়ার কারণে হারিয়ে যাওয়া সংযোগ।

TechNova-র কথাই ধরুন। তাদের MERN অ্যাপ — `technova-web` ফ্রন্টএন্ড, `technova-api` ব্যাকএন্ড, MongoDB ডেটাবেস — Kubernetes-এ চলে। একদিন সকালে দেখা গেল API মাঝে মাঝে ৫০২ দিচ্ছে, অথচ Pod-গুলো Running অবস্থায় আছে। তদন্তে বের হলো readiness probe ভুল পথে (path) পাঠানো হচ্ছিল; Pod চলছে কিন্তু Service থেকে ট্রাফিক পাচ্ছে না, আবার কিছু Pod লাইভনেস probe-এর ভুল সেটিংয়ের কারণে অকারণে পুনরায় চালু হচ্ছিল। Pod, lifecycle ও probe ঠিকভাবে বোঝা থাকলে এই সমস্যা কয়েক মিনিটে ধরা পড়ত। এই অধ্যায় সেই বোঝাপড়াই গড়ে তুলবে।

৬.৩ পূর্বশর্ত জ্ঞান

এই অধ্যায় পড়ার আগে আপনার Docker কনটেইনারের মৌলিক ধারণা থাকা দরকার — image, container, port, environment variable, volume কী। আগের অধ্যায়গুলো থেকে Kubernetes cluster-এর গঠন — control plane, kubelet, kube-scheduler, etcd — এবং `kubectl` দিয়ে cluster-এ সংযোগ করার অভিজ্ঞতা থাকলে ভালো হয়। YAML-এর মৌলিক গঠন (indentation, key-value, list) জানা আবশ্যক, কারণ সব উদাহরণ YAML-এ দেওয়া। HTTP status code (যেমন ২০০, ৫০০) এবং TCP port-এর প্রাথমিক ধারণা probe বোঝার কাজে লাগবে।

৬.৪ প্রথম নীতি থেকে ব্যাখ্যা

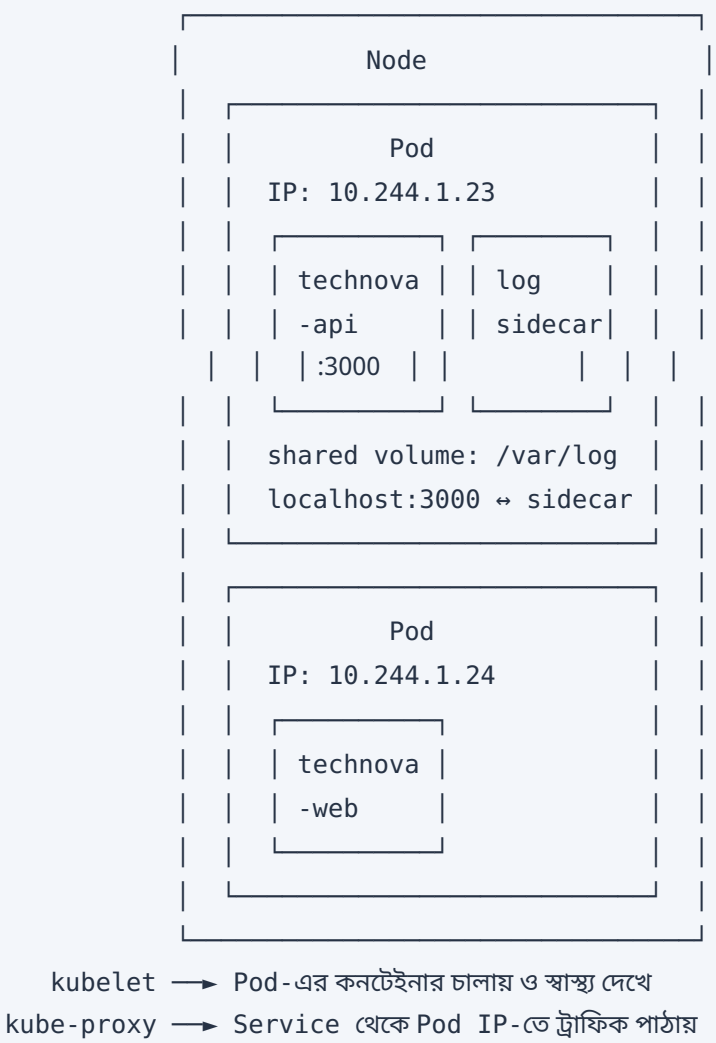
প্রথম নীতি থেকে ভাবুন: একটি অ্যাপ্লিকেশন চালাতে হলে কী লাগে? একটি প্রক্রিয়া (process), তার প্রয়োজনীয় ফাইল, নেটওয়ার্ক ঠিকানা, এবং কিছু কনফিগারেশন। Docker সেই প্রক্রিয়াকে একটি কনটেইনারে বন্দি করে — আলাদা ফাইলসিস্টেম, আলাদা প্রক্রিয়া-স্থান। কিন্তু বাস্তব অ্যাপ একা চলে না। একটি ওয়েব সার্ভারের পাশে অনেক সময় একটি লগ সংগ্রাহক (log collector), একটি প্রক্সি, কিংবা একটি কনফিগারেশন পুনঃলোডকারী (config reloader) দরকার হয়। এগুলো আলাদা প্রক্রিয়া, কিন্তু তাদের জীবন একসাথে বাঁধা: তারা একই মেশিনে, একই নেটওয়ার্কে, একই ফাইল ভাগ করে চলতে চায়; একটি মারা গেলে অন্যগুলোও অর্থহীন হয়ে যায়; একসাথে তৈরি ও একসাথে মুছে ফেলা উচিত।

এই সমস্যার সমাধানই Pod। Pod হলো একগুচ্ছ কনটেইনারের একটি আবরণ (wrapper) — যারা একই Node-এ, একই নেটওয়ার্ক নেমস্পেসে, একই IP ঠিকানায়, এবং প্রয়োজনে একই ভলিউম ভাগ করে চলে। কনটেইনারগুলো `localhost` দিয়ে একে অপরের সাথে কথা বলতে পারে, যেন তারা একই মেশিনের ভিন্ন প্রক্রিয়া। Kubernetes Pod-কে একক একক হিসেবে সময়সূচি করে, তৈরি করে, মুছে ফেলে এবং পুনরায় চালু করে — আলাদা কনটেইনারকে নয়। তাই বলা হয়, Pod হলো Kubernetes-এর সবচেয়ে ছোট স্থাপনযোগ্য একক, কনটেইনার নয়। আপনি কখনো একটিমাত্র কনটেইনার সরাসরি deploy করেন না; আপনি একটি Pod deploy করেন, যার ভেতরে এক বা একাধিক কনটেইনার থাকে।

> **CONCEPT:** Pod হলো কনটেইনারের জন্য একটি “অণু” — কনটেইনারগুলো পরমাণুর মতো, Pod হলো সেই পরমাণুগুলোকে বেঁধে রাখা অণু। একই Pod-এর কনটেইনারগুলো একই ভাগ্য ভাগ করে নেয়: একসাথে তৈরি হয়, একসাথে চলে, একসাথে মরে।

আরেকটি প্রথম নীতি: কম্পিউটারে কোনো কিছুই চিরস্থায়ী নয়। প্রক্রিয়া ক্র্যাশ করে, মেশিন বন্ধ হয়, নেটওয়ার্ক ছিঁড়ে যায়। Kubernetes এই বাস্তবতা মেনে নিয়ে Pod-কে ক্ষণস্থায়ী (ephemeral) হিসেবে ডিজাইন করেছে। Pod মারা গেলে Kubernetes সেটিকে “সারিয়ে” তোলে না; বরং নতুন একটি Pod তৈরি করে, নতুন IP দিয়ে। এই নীতিটি বুঝলে বাকি সবকিছু — Service, Deployment, StatefulSet — স্বাভাবিকভাবে বোঝা যায়।

৬.৫ আর্কিটেকচার / ধারণাগত চিত্র



চিত্র ৬.১: একটি Node-এ দুটি Pod; একই Pod-এর কনটেইনারগুলো একই IP ও localhost ভাগ করে নেয়।

উপরের চিত্রে লক্ষ করুন: Pod-এর নিজস্ব IP আছে, কনটেইনারের আলাদা IP নেই। `technova-api` কনটেইনার `localhost:3000`-এ শুনছে, আর একই Pod-এর log sidecar `localhost:3000` দিয়েই তার সাথে যোগাযোগ করতে পারে। বাইরে থেকে কেউ Pod IP `10.244.1.23`-তে পৌঁছায়। Pod মুছে গেলে এই IP হারিয়ে যায়; নতুন Pod পায় নতুন IP — তাই সরাসরি Pod IP-তে নির্ভর করা যায় না, Service ব্যবহার করতে হয় (পরের অধ্যায়গুলোতে বিস্তারিত আসবে)।

৬.৬ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

৬.৬.১ Pod বনাম কনটেইনার

Docker-এ আপনি একটি কনটেইনার চালান; Kubernetes-এ আপনি একটি Pod চালান। পার্থক্যটি শুধু নামের নয়। Pod একাধিক কনটেইনারকে একই নেটওয়ার্ক ও স্টোরেজ প্রসঙ্গ (context) দেয়। Pod-এর সব কনটেইনার একই Linux নেমস্পেস ভাগ করে: একই IP, একই port স্থান (তাই দুটি কনটেইনার একই Pod-এ একই port ব্যবহার করতে পারে না), একই IPC। ভলিউমও ভাগ করা যায়: একটি কনটেইনার ফাইল লিখলে অন্যটি তা পড়তে পারে। এই ঘনিষ্ঠতা (tight coupling) Pod-এর উদ্দেশ্য — যেসব কনটেইনার সবসময় একসাথে থাকা উচিত, তাদেরই এক Pod-এ রাখা হয়।

বিষয়	কনটেইনার	Pod
সবচেয়ে ছোট একক	Docker-এর একক	Kubernetes-এর সবচেয়ে ছোট স্থাপনযোগ্য একক
IP ঠিকানা	নিজস্ব নয় (Pod-এর IP ব্যবহার করে)	নিজস্ব IP আছে
জীবনচক্র	আলাদাভাবে শুরু/বন্ধ হতে পারে	একসাথে তৈরি ও মুছে ফেলা হয়
পুনরায় চালু	Docker restart করে	kubelet Pod-এর ভেতরে কনটেইনার পুনরায় চালু করে
সময়সূচি	—	scheduler Pod-কে Node বেছে দেয়

৬.৬.২ Pod জীবনচক্র: Phase ও Container State

Pod-এর `status.phase` পাঁচটি মান নিতে পারে। **Pending** মানে Pod তৈরি হয়েছে কিন্তু এখনো সব কনটেইনার চলছে না — হয়তো image ডাউনলোড হচ্ছে, কিংবা scheduler এখনো Node বাছাই করেনি। **Running** মানে অন্তত একটি কনটেইনার চলছে (সব কনটেইনার চলছে, এমন নয়)। **Succeeded** মানে সব কনটেইনার সফলভাবে শেষ হয়েছে এবং আর পুনরায় চালু হবে না — Job-এর ক্ষেত্রে স্বাভাবিক। **Failed** মানে অন্তত একটি কনটেইনার ব্যর্থ হয়ে শেষ হয়েছে। **Unknown** মানে kubelet-এর সাথে যোগাযোগ বিচ্ছিন্ন — Node-এর অবস্থা জানা যাচ্ছে না।

কনটেইনারের নিজের অবস্থা তিনটি: **Waiting** (শুরু হওয়ার অপেক্ষায় — যেমন image টানা হচ্ছে), **Running** (চলছে), **Terminated** (শেষ হয়েছে — সফল বা ব্যর্থ, exit code সহ)। `kubectl get pods`-এ দেখা `RESTARTS` সংখ্যা বলে দেয় কনটেইনার কতবার পুনরায় চালু হয়েছে। বারবার restart হওয়া Pod-এর নামের পাশে `CrashLoopBackOff` দেখা যায় — এটি কোনো phase নয়, বরং kubelet-এর backoff অবস্থা: প্রতিবার ব্যর্থতার পর সে অপেক্ষার সময় বাড়িয়ে দেয় (১০ সেকেন্ড, ২০ সেকেন্ড, ৪০ সেকেন্ড... সর্বোচ্চ ৫ মিনিট) যাতে ব্যর্থ প্রক্রিয়া CPU নষ্ট না করে।

৬.৬.৩ Probes: liveness, readiness ও startup

Probe হলো kubelet-এর করা নিয়মিত স্বাস্থ্য পরীক্ষা। তিন ধরনের probe আছে, প্রত্যেকের কাজ আলাদা। **Liveness probe** জিজ্ঞেস করে: "এই কনটেইনার কি বেঁচে আছে, নাকি আটকে গেছে?" ব্যর্থ হলে kubelet কনটেইনারটিকে মেরে পুনরায় চালু করে। এটি deadlock বা আটকে যাওয়া প্রক্রিয়া সারানোর উপায়। **Readiness probe** জিজ্ঞেস করে: "এই কনটেইনার কি ট্রাফিক নেওয়ার জন্য প্রস্তুত?" ব্যর্থ হলে Pod-কে Service-এর endpoint তালিকা থেকে সরিয়ে দেওয়া হয় — ট্রাফিক আসা বন্ধ হয়, কিন্তু কনটেইনার পুনরায় চালু হয় না। **Startup probe** ধীরগতিতে শুরু হওয়া অ্যাপের জন্য: অ্যাপ চালু হতে ৬০ সেকেন্ড লাগলে liveness probe শুরুতেই ব্যর্থ হয়ে তাকে মেরে ফেলত; startup probe সফল না হওয়া পর্যন্ত liveness ও readiness probe-কে থামিয়ে রাখে।

প্রতিটি probe তিনভাবে করা যায়: **HTTP GET** (কোনো পথে অনুরোধ পাঠিয়ে ২০০–৩৯৯ status আশা করা), **TCP socket** (port খোলা আছে কি না দেখা), এবং **exec** (কনটেইনারের ভেতরে একটি কমান্ড চালিয়ে exit code ০ আশা করা)। গুরুত্বপূর্ণ সেটিংগুলো হলো `initialDelaySeconds` (প্রথম পরীক্ষার আগে অপেক্ষা), `periodSeconds` (কতক্ষণ পর পর), `timeoutSeconds`, `failureThreshold` (কতবার ব্যর্থ হলে ব্যবস্থা নেওয়া হবে) এবং `successThreshold`।

> **WARNING:** Liveness probe-এ কখনো ডেটাবেস বা বাইরের সেবার উপর নির্ভরশীল পরীক্ষা দেবেন না। ডেটাবেস সাময়িকভাবে বন্ধ হলে liveness probe ব্যর্থ হবে, kubelet সুস্থ কনটেইনারটিকেও মেরে ফেলবে, আর restart-এর ঝড় (restart storm) শুরু হবে। Liveness probe হালকা রাখুন — শুধু প্রক্রিয়াটি সাড়া দিচ্ছে কি না দেখুন। ভারী পরীক্ষা readiness probe-এ রাখুন।

৬.৬.৪ Init Containers

Init container হলো এমন কনটেইনার যা মূল (app) কনটেইনার শুরু হওয়ার আগে চলে এবং শেষ হয়ে যায়। একাধিক init container থাকলে তারা একের পর এক ক্রমানুসারে (sequentially) চলে; একটি ব্যর্থ হলে Pod পুনরায় শুরু হয় যতক্ষণ না সেটি সফল হয়। ব্যবহার: ডেটাবেস প্রস্তুত হওয়ার জন্য অপেক্ষা করা, কনফিগারেশন ফাইল তৈরি করা, ডেটাবেস migration চালানো, কিংবা ভলিউমে প্রয়োজনীয় ফাইল নামিয়ে রাখা। Init container-এর নিজস্ব image, resource, ভলিউম মাউন্ট থাকতে পারে, কিন্তু তাদের readiness probe থাকে না — তারা শেষ হলেই কাজ শেষ।

৬.৬.৫ Sidecar Pattern ও বহু-কনটেইনার Pod ডিজাইন

Sidecar pattern-এ মূল কনটেইনারের পাশে একটি সহায়ক কনটেইনার চলে — যেমন লগ সংগ্রাহক, প্রক্সি, কিংবা কনফিগারেশন পুনঃলোডকারী। Sidecar মূল অ্যাপের কোড বদলায় না; সে পাশে থেকে কাজ করে। তবে সাবধান: প্রতিটি সমস্যা sidecar দিয়ে সমাধান করা উচিত নয়। দুটি কনটেইনারকে আলাদাভাবে স্কেল করতে হলে, কিংবা তাদের জীবনচক্র আলাদা হলে, তাদের আলাদা Pod-এ রাখুন। এক Pod-এ রাখার যুক্তি হতে হবে — তারা একই ডেটা ভাগ করে, একই নেটওয়ার্কে `localhost`-এ কথা বলে, এবং একসাথে স্থাপন ও মুছে ফেলা যুক্তিসংগত। TechNova-র API Pod-এ একটি log-shipping sidecar যুক্তিসংগত; কিন্তু `technova-web` ও `technova-api` আলাদা Pod-এ থাকা উচিত, কারণ তাদের স্কেল ও deploy চক্র আলাদা।

৬.৬.৬ Pod IP-এর ক্ষণস্থায়িত্ব

Pod IP কখনো স্থায়ী ঠিকানা নয়। Pod মুছে গেলে, Node বদলালে, কিংবা Deployment নতুন revision তৈরি করলে নতুন Pod নতুন IP পায়। তাই অ্যাপের কোডে কখনো Pod IP হার্ডকোড করবেন না, ডেটাবেসে Pod IP সংরক্ষণ করবেন না, এবং Pod IP-তে সরাসরি সংযোগের উপর নির্ভর করবেন না। স্থায়ী ঠিকানার জন্য Service ব্যবহার করুন, যা Pod-গুলোর সামনে একটি স্থির নাম ও IP দেয় এবং Pod বদলে গেলেও endpoint তালিকা হালনাগাদ করে।

৬.৭ কনফিগারেশন উদাহরণ

নিচের উদাহরণে TechNova-র API Pod: একটি init container MongoDB প্রস্তুত হওয়ার জন্য অপেক্ষা করে, মূল API কনটেইনারে তিনটি probe আছে, এবং একটি log sidecar `/var/log` ভলিউম ভাগ করে।

```
apiVersion: v1
kind: Pod
metadata:
  name: technova-api
  namespace: technova-dev
  labels:
    app: technova-api
    tier: backend
spec:
  initContainers:
  - name: wait-for-mongo
    image: busybox:1.36
    command: ['sh', '-c', 'until nc -z mongo 27017; do echo waiting for mongo; sleep 2; done']
  containers:
  - name: technova-api
    image: technova/api:1.4.0
    ports:
    - containerPort: 3000
    env:
    - name: MONGO_URI
      value: "mongodb://mongo:27017/technova"
```

```
    volumeMounts:
    - name: applogs
      mountPath: /var/log/app
    livenessProbe:
      httpGet:
        path: /healthz
        port: 3000
      initialDelaySeconds: 10
      periodSeconds: 10
      failureThreshold: 3
    readinessProbe:
      httpGet:
        path: /ready
        port: 3000
      initialDelaySeconds: 5
      periodSeconds: 5
      failureThreshold: 3
    startupProbe:
      httpGet:
        path: /healthz
        port: 3000
      failureThreshold: 30
      periodSeconds: 5
  - name: log-shipper
    image: technova/log-shipper:1.0.0
    volumeMounts:
    - name: applogs
      mountPath: /var/log/app
  volumes:
  - name: applogs
    emptyDir: {}
```

```
# Pod দেখা ও পরীক্ষা করা
kubectl get pods -n technova-dev
kubectl describe pod technova-api -n technova-dev
kubectl logs technova-api -c technova-api -n technova-dev
kubectl logs technova-api -c log-shipper -n technova-dev
kubectl exec -it technova-api -c technova-api -n technova-dev -- sh
kubectl delete pod technova-api -n technova-dev
```

লক্ষ করুন: `startupProbe`-এ `failureThreshold: 30` ও `periodSeconds: 5` মানে অ্যাপের চালু হতে ১৫০ সেকেন্ড পর্যন্ত সময় দেওয়া হয়েছে। `emptyDir` ভলিউম Pod-এর জীবনকাল পর্যন্ত থাকে — Pod মুছলে এর ডেটাও মুছে যায়, তাই এটি স্থায়ী স্টোরেজ নয়।

৬.৮ বাস্তব প্রকৌশল পরিস্থিতি

TechNova পরিস্থিতি: technova.example ডোমেইনে চলা MERN অ্যাপের API Pod-গুলো সকালে হঠাৎ CrashLoopBackOff-এ পড়ল। লগে দেখা গেল অ্যাপটি MongoDB-র সাথে সংযোগ করতে পারছে না, কারণ MongoDB Pod তখনো চালু হচ্ছিল। API কনটেইনার শুরু হয়েই সংযোগ ব্যর্থ হয়ে ক্র্যাশ করছিল, kubelet তাকে পুনরায় চালু করছিল, আবার ক্র্যাশ — এই চক্র চলছিল। সমাধান ছিল দুই স্তরে: প্রথমে একটি init container যোগ করা হলো যা MongoDB-র ২৭০১৭ port খোলা না হওয়া পর্যন্ত অপেক্ষা করে; দ্বিতীয়ত, API কোডে সংযোগের পুনঃচেষ্টা (retry with backoff) যোগ করা হলো, যাতে ক্ষণস্থায়ী ব্যর্থতায় অ্যাপ সাথে সাথে মারা না যায়। এর পাশাপাশি readiness probe-এর পথ `/ready`

এমনভাবে বদলানো হলো যাতে সেটি সত্যিই ডেটাবেস সংযোগ পরীক্ষা করে — ফলে প্রস্তুত নয় এমন Pod Service থেকে ট্রাফিক পায় না, কিন্তু liveness probe হালকা `/healthz` রাখা হলো যাতে ডেটাবেসের সাময়িক সমস্যায় Pod মারা না যায়।

এই ঘটনা থেকে শিক্ষা: Pod-এর নির্ভরতা (dependency) শুরু করার ক্রম, probe-এর ভূমিকা, এবং অ্যাপের নিজের retry আচরণ — তিনটি একসাথে ভাবতে হয়। শুধু YAML ঠিক করলেই হয় না; অ্যাপকেও ক্ষণস্থায়ী ব্যর্থতা সামলাতে শিখতে হয়।

৬.৯ ধাপে ধাপে ব্যবহারিক উদাহরণ

লক্ষ্য: একটি probe-সহ Pod তৈরি করে তার জীবনচক্র পর্যবেক্ষণ করা।

ধাপ ১: namespace তৈরি করুন: `kubectl create namespace technova-dev`। **ধাপ ২:** উপরের Pod YAML-টি `api-pod.yaml` নামে সংরক্ষণ করুন এবং `kubectl apply -f api-pod.yaml` চালান। **ধাপ ৩:** `kubectl get pods -n technova-dev -w` দিয়ে দেখুন Pod কীভাবে Pending থেকে Running-এ যায়; init container চলার সময় Pod-এর অবস্থা লক্ষ করুন। **ধাপ ৪:** `kubectl describe pod technova-api -n technova-dev` দিয়ে Events অংশে probe-এর সফলতা, image টানার ঘটনা, ও container শুরুর সময় দেখুন। **ধাপ ৫:** ইচ্ছাকৃতভাবে readiness probe-এর পথ ভুল করে (`/ready` বদলে `/nope`) apply করুন — দেখুন Pod Running থাকে কিন্তু `READY` কলামে `0/2` দেখায়। **ধাপ ৬:** পথটি ঠিক করে আবার apply করুন এবং `READY 2/2` হতে দেখুন। **ধাপ ৭:** `kubectl logs` দিয়ে দুটি কনটেইনারের লগ আলাদাভাবে দেখুন।

```
# চিত্রণমূলক নমুনা আউটপুট
$ kubectl get pods -n technova-dev
NAME            READY   STATUS    RESTARTS   AGE
technova-api    2/2     Running   0          3m
```

এই পরীক্ষাটি probe-এর পার্থক্য হাতে-কলমে শেখায়: readiness ব্যর্থতা Pod-কে ট্রাফিক থেকে সরায়, liveness ব্যর্থতা Pod-কে পুনরায় চালু করে।

৬.১০ সাধারণ ভুল ও সমস্যা সমাধান

সমস্যা / লক্ষণ	সম্ভাব্য কারণ	সমাধান
CrashLoopBackOff	অ্যাপ শুরু হয়েই ক্র্যাশ করছে; ভুল env, ভুল image, নির্ভরতা অনুপস্থিত	`kubectl logs --previous` দিয়ে আগের লগ দেখুন; `describe` -এ Events দেখুন
ImagePullBackOff	ভুল image নাম/tag, private registry-তে প্রমাণীকরণ নেই	image নাম যাচাই করুন; imagePullSecrets যোগ করুন
Pod Pending-এ আটকে আছে	Node-এ যথেষ্ট CPU/memory নেই, কিংবা volume মাউন্ট হচ্ছে না	`kubectl describe pod` -এ scheduler বার্তা দেখুন; resource request কমান
Running কিন্তু READY 0/2	readiness probe ব্যর্থ	probe-এর পথ/port যাচাই করুন; অ্যাপের ভেতরে endpoint পরীক্ষা করুন
বারবার restart, কিন্তু অ্যাপ সুস্থ	liveness probe খুব কঠোর বা ডেটাবেস-নির্ভর	liveness probe হালকা করুন; initialDelay ও failureThreshold বাড়ান
Pod IP বদলে যাওয়ায় সংযোগ হারানো	সরাসরি Pod IP ব্যবহার করা হয়েছে	Service ব্যবহার করুন; DNS নামে সংযোগ করুন

TROUBLESHOOTING: সমস্যা নির্ণয়ের ক্রম মনে রাখুন — প্রথমে `kubectl get pods`, তারপর `kubectl describe pod` (Events), তারপর `kubectl logs` ও `kubectl logs --previous`। বেশিরভাগ উত্তর Events ও লগেই থাকে।

৬.১১ নিরাপত্তা বিবেচনা

Pod নিরাপত্তার প্রথম নিয়ম: কনটেইনার root হিসেবে চালাবেন না, যদি না একান্ত প্রয়োজন হয়। `securityContext`-এ `runAsNonRoot: true` ও `allowPrivilegeEscalation: false` দিন। দ্বিতীয়ত, Secret কখনো Pod-এর YAML-এ সরাসরি লিখবেন না; ConfigMap ও Secret-এর ব্যবহার অধ্যায় ১০-এ বিস্তারিত আছে। তৃতীয়ত, Pod-এর ভেতরে `kubectl exec` দিয়ে ঢোকা ডিবাগিংয়ের জন্য ঠিক আছে, কিন্তু প্রোডাকশনে কার কখন exec করল তার নিরীক্ষা (audit) চালু রাখুন। চতুর্থত, image-এর উৎস যাচাই করুন — শুধু বিশ্বস্ত registry থেকে image টানুন, এবং নিয়মিত image স্ক্যান করুন। সবশেষে, Pod-এর নেটওয়ার্ক নীতি (NetworkPolicy) ছাড়া একই namespace-এর সব Pod একে অপরের সাথে কথা বলতে পারে — সংবেদনশীল Pod-কে আলাদা namespace বা NetworkPolicy দিয়ে সুরক্ষিত করুন।

৬.১২ কর্মক্ষমতা ও খরচ বিবেচনা

প্রতিটি Pod Node-এর সম্পদ (CPU, memory) খরচ করে, এমনকি `emptyDir` ভলিউম ও sidecar-ও। অপ্রয়োজনীয় sidecar প্রতিটি Pod-এ যোগ করলে খরচ গুণিতক হারে বাড়ে — এক হাজার Pod-এ একটি ভারী sidecar মানে এক হাজার অতিরিক্ত কনটেইনার। Probe-এর ঘনত্বও খরচ: প্রতি ২ সেকেন্ডে HTTP probe মানে অপ্রয়োজনীয় লোড; সাধারণত ৫–১০ সেকেন্ডের ব্যবধান যথেষ্ট। Init container শুধু শুরুতেই চলে, তাই তার খরচ এককালীন — কিন্তু ভারী init কাজ (বড় ফাইল ডাউনলোড) Pod শুরুর সময় বাড়িয়ে দেয়, যা rollout-কে ধীর করে। Resource request/limit-এর বিস্তারিত অধ্যায় ১১-এ আছে; এখানে মনে রাখুন — Pod-এর request ঠিক না করলে scheduler ভুল Node বেছে নিতে পারে।

৬.১৩ সেরা অনুশীলন ও অ্যান্টি-প্যাটার্ন

সেরা অনুশীলন: এক Pod-এ একটিমাত্র মূল দায়িত্ব রাখুন; readiness ও liveness probe আলাদা endpoint-এ রাখুন; startup probe ধীর অ্যাপে ব্যবহার করুন; Pod-কে অপরিবর্তনীয় (immutable) ভাবুন — সমস্যা হলে সারানোর চেষ্টা না করে নতুন Pod তৈরি করুন; Pod-এর নামে নয়, label দিয়ে চিনুন।

অ্যান্টি-প্যাটার্ন: সরাসরি Pod তৈরি করে প্রোডাকশন চালানো (এর বদলে Deployment ব্যবহার করুন — অধ্যায় ৭ দেখুন); এক Pod-এ দুটি স্বাধীন অ্যাপ রাখা; liveness probe-এ বাইরের নির্ভরতা পরীক্ষা করা; Pod IP হার্ডকোড করা; `latest` tag ব্যবহার করে অনিশ্চিত image চালানো।

PRO TIP: Pod-এর YAML-এ সবসময় `imagePullPolicy: IfNotPresent` বা নির্দিষ্ট tag ব্যবহার করুন। `latest` tag দিয়ে কোন version চলছে তা বোঝা যায় না, আর সমস্যা হলে আগের version-এ ফেরা কঠিন হয়।

৬.১৪ অধ্যায়ের সারাংশ

Pod হলো Kubernetes-এর সবচেয়ে ছোট স্থাপনযোগ্য একক — একগুচ্ছ কনটেইনার যারা একই Node, একই IP, একই নেটওয়ার্ক ও প্রয়োজনে একই ভলিউম ভাগ করে। Pod-এর জীবনচক্রের phase (Pending, Running, Succeeded, Failed, Unknown) ও কনটেইনারের অবস্থা (Waiting, Running, Terminated) আলাদা ধারণা। Liveness probe আটকে যাওয়া কনটেইনারকে পুনরায় চালু করে, readiness probe প্রস্তুত নয় এমন Pod-কে ট্রাফিক থেকে সরায়, startup probe

ধীর অ্যাপকে শুরুর সময় দেয়। Init container মূল কনটেইনারের আগে ক্রমানুসারে চলে; sidecar মূল অ্যাপের পাশে সহায়ক কাজ করে। Pod IP ক্ষণস্থায়ী — স্থায়ী ঠিকানার জন্য Service দরকার।

৬.১৫ মূল প্রযুক্তিগত পরিভাষা

Pod: Kubernetes-এর সবচেয়ে ছোট স্থাপনযোগ্য একক; এক বা একাধিক কনটেইনারের দল যারা একই Node, IP ও নেটওয়ার্ক ভাগ করে।

Lifecycle (জীবনচক্র): Pod তৈরির পর থেকে মুছে যাওয়া পর্যন্ত ধাপগুলো — Pending থেকে Running, Succeeded বা Failed।

Liveness Probe: কনটেইনার বেঁচে আছে কি না তা পরীক্ষা; ব্যর্থ হলে kubelet কনটেইনার পুনরায় চালু করে।

Readiness Probe: কনটেইনার ট্রাফিক নেওয়ার জন্য প্রস্তুত কি না তা পরীক্ষা; ব্যর্থ হলে Pod Service থেকে সরিয়ে দেওয়া হয়।

Startup Probe: ধীরগতিতে চালু হওয়া অ্যাপের জন্য প্রাথমিক পরীক্ষা; সফল না হওয়া পর্যন্ত অন্য probe থামিয়ে রাখে।

Init Container: মূল কনটেইনার শুরু হওয়ার আগে ক্রমানুসারে চলা কনটেইনার; প্রস্তুতিমূলক কাজ করে।

Sidecar: মূল কনটেইনারের পাশে চলা সহায়ক কনটেইনার; যেমন লগ সংগ্রাহক বা প্রক্সি।

CrashLoopBackOff: বারবার ক্র্যাশ করা কনটেইনারকে kubelet ক্রমবর্ধমান বিরতিতে পুনরায় চালু করার অবস্থা।

৬.১৬ জ্ঞান যাচাই

১. Pod ও কনটেইনারের পার্থক্য কী, এবং কেন Kubernetes সরাসরি কনটেইনার deploy করে না?
২. Pod-এর পাঁচটি phase-এর নাম লিখুন এবং প্রতিটি কখন দেখা যায় ব্যাখ্যা করুন।
৩. Liveness probe ব্যর্থ হলে কী হয়, আর readiness probe ব্যর্থ হলে কী হয় — পার্থক্যটি লিখুন।
৪. Startup probe কেন দরকার? একটি ধীরগতিতে চালু হওয়া অ্যাপের উদাহরণ দিন।
৫. Init container কখন ব্যবহার করবেন? দুটি বাস্তব ব্যবহার লিখুন।
৬. একই Pod-এর দুটি কনটেইনার কীভাবে একে অপরের সাথে যোগাযোগ করে?
৭. Pod IP কেন স্থায়ী ঠিকানা হিসেবে ব্যবহার করা যায় না?
৮. CrashLoopBackOff দেখলে আপনি কোন তিনটি কমান্ড দিয়ে তদন্ত শুরু করবেন?

৬.১৭ ব্যবহারিক অ্যাসাইনমেন্ট

একটি Pod YAML লিখুন যাতে থাকবে: (ক) `technova-api` মূল কনটেইনার, HTTP liveness probe `/healthz`-এ ও readiness probe `/ready`-এ; (খ) একটি init container যা একটি কাল্পনিক ডেটাবেস হোস্ট `mongo`-এর ২৭০১৭ port খোলা না হওয়া পর্যন্ত অপেক্ষা করে; (গ) একটি log sidecar যা মূল কনটেইনারের সাথে `emptyDir` ভলিউম ভাগ করে। Pod-টি আপনার স্থানীয় বা পরীক্ষামূলক cluster-এ চালিয়ে `kubectl describe pod`-এর Events অংশের স্ক্রিনশট বা অনুলিপি সংরক্ষণ করুন, এবং readiness probe-এর পথ ইচ্ছাকৃতভাবে ভুল করে দেখুন `READY` কলাম কীভাবে বদলায়। ফলাফল এক পৃষ্ঠায় লিখে জমা দিন।

অধ্যায় ৭ • PART II — Workloads & Application Management

Chapter 7 — ReplicaSets and Deployments

৭.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি ReplicaSet কী, কীভাবে এটি নির্দিষ্ট সংখ্যক Pod সবসময় চলমান রাখে, এবং Deployment কীভাবে ReplicaSet-এর উপরে বসে rollout, rollback ও revision পরিচালনা করে — তা ব্যাখ্যা করতে পারবেন। আপনি selector ও template-এর ভূমিকা, reconciliation loop (পুনর্মিলন চক্র) কীভাবে কাঙ্ক্ষিত অবস্থা (desired state) ও বাস্তব অবস্থার (actual state) পার্থক্য মেটায়, এবং RollingUpdate বনাম Recreate কৌশলের পার্থক্য বুঝবেন। ব্যবহারিকভাবে আপনি `maxSurge` ও `maxUnavailable` সেট করতে পারবেন, `kubectl rollout` দিয়ে ইতিহাস দেখে আগের version-এ ফিরতে পারবেন, rollout থামিয়ে ও আবার চালু করতে পারবেন, এবং কখন Deployment ব্যবহার করা উচিত নয় তা বলতে পারবেন।

৭.২ কেন এই বিষয়টি গুরুত্বপূর্ণ

একটি Pod একা চালালে সেটি মারা গেলে আর ফিরে আসে না — Kubernetes তাকে পুনরায় তৈরি করে না। বাস্তব অ্যাপে আমরা চাই তিনটি, পাঁচটি বা দশটি কপি সবসময় চলুক, একটি মারা গেলে নতুন একটি আসুক, নতুন version এলে পুরোনো কপিগুলো ধীরে ধীরে বদলে যাক, আর ভুল version এলে আগেরটিতে ফেরা যাক। এই সবকিছুই Deployment দেয়। প্রোডাকশনে প্রায় সব stateless অ্যাপ — ওয়েব সার্ভার, API, ফ্রন্টএন্ড — Deployment দিয়েই চলে। Deployment না বুঝলে আপনি Kubernetes-এ অ্যাপ deploy করতেই পারবেন না বললে ভুল হবে না।

TechNova-র `technova-api` প্রথমে একটি Pod হিসেবে চালানো হয়েছিল। একদিন Node রক্ষণাবেক্ষণের জন্য বন্ধ হলে API সম্পূর্ণ বন্ধ হয়ে গেল — কোনো কপি বাকি ছিল না। Deployment-এ তিনটি replica রাখার পর থেকে একটি Pod মারা গেলেও বাকি দুটি ট্রাফিক সামলায়, আর নতুন version rollout-এর সময়ও সেবা বন্ধ হয় না। এই অধ্যায় সেই নির্ভরযোগ্যতা গড়ার উপায় শেখায়।

৭.৩ পূর্বশর্ত জ্ঞান

অধ্যায় ৬-এর Pod, lifecycle ও probe-এর ধারণা এখানে সরাসরি কাজে লাগবে। Label ও selector-এর প্রাথমিক ধারণা থাকলে ভালো হয় (বিস্তারিত অধ্যায় ৯-এ আছে)। YAML-এ Pod template লেখার অভিজ্ঞতা এবং `kubectl apply`, `kubectl get` ও `kubectl describe` ব্যবহার জানা থাকা দরকার।

৭.৪ প্রথম নীতি থেকে ব্যাখ্যা

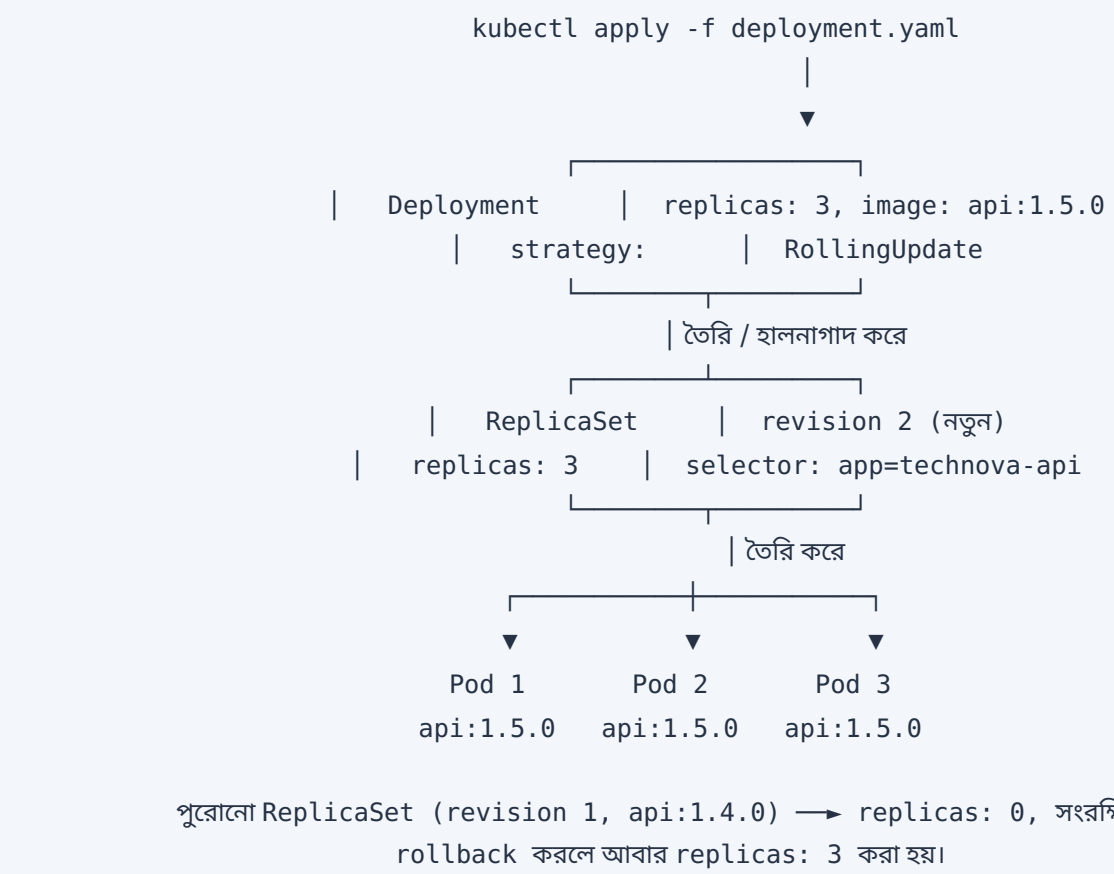
প্রথম নীতি থেকে ভাবুন: আপনি চান “সবসময় তিনটি API কপি চলুক”। এই ইচ্ছাটিকে একটি ঘোষণা (declaration) হিসেবে লিখে রাখা যায় — “replicas: 3”। কিন্তু কেউ না কেউ সেই ঘোষণা সত্যি রাখার দায়িত্ব নিতে হবে। কেউ একজন বারবার গুনবে: এখন কতটি Pod চলছে? তিনটির কম হলে নতুন তৈরি করবে, বেশি হলে অতিরিক্ত মুছে ফেলবে। এই “বারবার গুনে মেলানো” কাজটিই reconciliation loop। ReplicaSet সেই পর্যবেক্ষক ও সংশোধনকারী: সে selector দিয়ে চিনে নেয় কোন Pod-গুলো তার, template দিয়ে জানে নতুন Pod কেমন হবে, আর সংখ্যা মিলিয়ে নেয়।

Deployment আরেক ধাপ এগিয়ে: শুধু সংখ্যা নয়, version বদলের প্রক্রিয়াও সে পরিচালনা করে। নতুন image এলে সে পুরোনো ReplicaSet-কে একবারে মুছে ফেলে না; বরং নতুন ReplicaSet তৈরি করে, ধীরে ধীরে Pod বদলায়, প্রতিটি ধাপে

পরীক্ষা করে। ভুল হলে সে আগের ReplicaSet-কে ফিরিয়ে আনতে পারে, কারণ প্রতিটি version-এর ReplicaSet সংরক্ষিত থাকে — এটাই revision ইতিহাস। তাই Deployment = ReplicaSet + rollout কৌশল + ইতিহাস ব্যবস্থাপনা।

> **CONCEPT:** Kubernetes-এর মূল নীতি হলো ঘোষণামূলক (declarative) ব্যবস্থাপনা: আপনি কী চান তা ঘোষণা করেন, controller ক্রমাগত বাস্তব অবস্থাকে সেই ঘোষণার সাথে মিলিয়ে নেয়। ReplicaSet ও Deployment এই নীতির সবচেয়ে পরিষ্কার উদাহরণ।

৭.৫ আর্কিটেকচার / ধারণাগত চিত্র



চিত্র ৭.১: Deployment → ReplicaSet → Pod স্তরবিন্যাস; rollback মানে পুরোনো ReplicaSet-কে আবার সক্রিয় করা।

৭.৬ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

৭.৬.১ ReplicaSet: selector, template ও reconciliation

ReplicaSet-এর তিনটি মূল অংশ: **selector** (কোন Pod-গুলো তার — label মিলিয়ে চেনে), **template** (নতুন Pod কেমন হবে — Pod-এর সম্পূর্ণ বিবরণ), এবং **replicas** (কতটি চাই)। Controller প্রতি কয়েক সেকেন্ডে দেখে: selector মেলা Pod কতটি চলছে? কম হলে template থেকে নতুন তৈরি করে; বেশি হলে অতিরিক্ত মুছে ফেলে। লক্ষ করুন — ReplicaSet Pod-এর ভেতরের কনটেইনারের অবস্থা দেখে না, শুধু Pod-এর সংখ্যা দেখে। Pod-এর ভেতরে কনটেইনার ক্র্যাশ করলে kubelet সেটিকে পুনরায় চালু করে; Pod মুছে গেলে ReplicaSet নতুন Pod তৈরি করে।

সরাসরি ReplicaSet তৈরি করা যায়, কিন্তু প্রায় কেউ তা করে না — Deployment-ই ReplicaSet তৈরি ও পরিচালনা করে। সরাসরি ReplicaSet ব্যবহার করলে rollout ও rollback সুবিধা পাওয়া যায় না।

৭.৬.২ Deployment: ReplicaSet-এর উপরে ব্যবস্থাপনা স্তর

Deployment-এর spec-এ থাকে replicas, selector, template (Pod template), strategy, এবং revision ইতিহাসের সীমা (`revisionHistoryLimit`, ডিফল্ট ১০)। আপনি image বদলে `kubectl apply` করলে Deployment নতুন একটি ReplicaSet তৈরি করে এবং পুরোনোটির replicas ধীরে ধীরে কমায়। প্রতিটি apply একটি নতুন revision তৈরি করে, যা `kubectl rollout history`-তে দেখা যায়। Deployment-এর `status`-এ দেখা যায় কতটি replica হালনাগাদ হয়েছে, কতটি প্রস্তুত, কতটি উপলব্ধ (available)।

৭.৬.৩ Rollout কৌশল: Recreate ও RollingUpdate

Recreate কৌশলে প্রথমে সব পুরোনো Pod মুছে ফেলা হয়, তারপর নতুন Pod তৈরি হয়। মাঝখানে কিছুক্ষণ সেবা বন্ধ থাকে (downtime)। এটি তখনই ব্যবহার করুন যখন পুরোনো ও নতুন version একসাথে চলতে পারে না — যেমন একটিমাত্র ডেটাবেস সংযোগ অনুমোদিত, কিংবা একই port বা ফাইল লক নিয়ে সংঘাত হয়। **RollingUpdate** (ডিফল্ট) কৌশলে নতুন Pod একে একে তৈরি হয় ও পুরোনো Pod মুছে যায়, ফলে সেবা বন্ধ হয় না। এর দুটি নিয়ন্ত্রক: `maxSurge` (কাঙ্ক্ষিত সংখ্যার চেয়ে কতটি বেশি Pod একসাথে থাকতে পারে — যেমন ১ বা ২৫%) এবং `maxUnavailable` (rollout চলাকালে কতটি Pod অনুপলব্ধ থাকতে পারে — যেমন ১ বা ২৫%)। `maxSurge: 1, maxUnavailable: 0` মানে কখনো সেবা কমবে না, তবে rollout-এর সময় অতিরিক্ত সম্পদ লাগবে। `maxSurge: 0, maxUnavailable: 1` মানে অতিরিক্ত সম্পদ লাগবে না, কিন্তু একবারে একটি Pod অনুপলব্ধ থাকবে।

কৌশল	Downtime	অতিরিক্ত সম্পদ	কখন ব্যবহার করবেন
Recreate	আছে	নেই	পুরোনো ও নতুন একসাথে চলতে পারে না
RollingUpdate	নেই (সঠিক probe থাকলে)	maxSurge অনুযায়ী	সাধারণ stateless অ্যাপ — ডিফল্ট পছন্দ

৭.৬.৪ Rollout পরিচালনা: history, undo, pause, resume

`kubectl rollout status deployment/technova-api` দিয়ে rollout-এর অগ্রগতি দেখা যায়। `kubectl rollout history deployment/technova-api` প্রতিটি revision ও তার কারণ (change-cause) দেখায়। ভুল version deploy হলে `kubectl rollout undo deployment/technova-api` আগের revision-এ ফেরায়; নির্দিষ্ট revision-এ ফিরতে `--to-revision=2` ব্যবহার করুন। Rollout থামাতে `kubectl rollout pause`, আবার চালু করতে `kubectl rollout resume`। Pause করে নতুন image পরীক্ষা করা যায় — যেমন একটি Pod নতুন version-এ চালিয়ে দেখে তারপর বাকিগুলো বদলানো। লক্ষ করুন: rollback আসলে নতুন revision তৈরি করে না, পুরোনো ReplicaSet-কে আবার সক্রিয় করে।

৭.৬.৫ Revision ব্যবস্থাপনা ও অগ্রগতির সময়সীমা

`progressDeadlineSeconds` (ডিফল্ট ৬০০ সেকেন্ড) বলে দেয় rollout কতক্ষণে শেষ না হলে তাকে ব্যর্থ ধরা হবে। নতুন Pod-গুলো readiness probe-এ ব্যর্থ হলে Deployment আটকে যায়, পুরোনো Pod-গুলো চালু থাকে — এটাই RollingUpdate-এর নিরাপত্তা। `revisionHistoryLimit` কমিয়ে (যেমন ৩) পুরোনো ReplicaSet-এর সংখ্যা সীমিত রাখা যায়, তবে খুব কম রাখলে দূরের revision-এ ফেরা যাবে না।

৭.৬.৬ কখন Deployment ব্যবহার করবেন না

Deployment stateless অ্যাপের জন্য। ডেটাবেসের মতো stateful অ্যাপের জন্য StatefulSet ব্যবহার করুন (অধ্যায় ৮)। প্রতিটি Node-এ একটি করে agent চালাতে DaemonSet, একবারের কাজে Job, নির্ধারিত সময়ে CronJob ব্যবহার করুন। একটিমাত্র Pod, যার কোনো কপি বা rollout দরকার নেই, সরাসরি Pod হিসেবে চালানো যায় — তবে প্রোডাকশনে সেটিও বিরল।

৭.৭ কনফিগারেশন উদাহরণ

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: technova-api
  namespace: technova-prod
  labels:
    app: technova-api
spec:
  replicas: 3
  revisionHistoryLimit: 5
  progressDeadlineSeconds: 600
  selector:
    matchLabels:
      app: technova-api
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 0
  template:
    metadata:
      labels:
        app: technova-api
        tier: backend
    spec:
      containers:
      - name: technova-api
        image: technova/api:1.5.0
        ports:
        - containerPort: 3000
        readinessProbe:
          httpGet:
            path: /ready
            port: 3000
          periodSeconds: 5
        livenessProbe:
          httpGet:
            path: /healthz
            port: 3000
          periodSeconds: 10
```

```
# Rollout পরিচালনা
kubectl apply -f api-deployment.yaml
kubectl rollout status deployment/technova-api -n technova-prod
kubectl rollout history deployment/technova-api -n technova-prod
kubectl set image deployment/technova-api technova-api=technova/api:1.6.0 -n technova-prod
```

```
kubectl rollout undo deployment/technova-api -n technova-prod
kubectl rollout undo deployment/technova-api --to-revision=2 -n technova-prod
kubectl rollout pause deployment/technova-api -n technova-prod
kubectl rollout resume deployment/technova-api -n technova-prod
kubectl get replicasets -n technova-prod
```

WARNING: Deployment-এর `selector.matchLabels` একবার তৈরি হয়ে গেলে বদলানো যায় না। Selector বদলাতে চাইলে Deployment মুছে নতুন করে তৈরি করতে হয়। তাই শুরুতেই selector সাবধানে বাছুন — সাধারণত `app: technova-api`-এর মতো স্থির label ব্যবহার করুন।

৭.৮ বাস্তব প্রকৌশল পরিস্থিতি

TechNova পরিস্থিতি: technova.example-এ নতুন API version 1.6.0 deploy করা হলো। Rollout শুরু হওয়ার পর দেখা গেল নতুন Pod-গুলো readiness probe-এ ব্যর্থ হচ্ছে — নতুন version-এ `/ready` endpoint-এর পথ বদলে গেছে, কিন্তু Deployment-এর probe আগের পথেই ডাকছে। RollingUpdate-এর কারণে পুরোনো Pod-গুলো তখনো ট্রাফিক সামলাচ্ছিল, তাই ব্যবহারকারীরা কিছু টের পেলেন না। কিন্তু rollout আটকে গেল, `progressDeadlineSeconds` পার হওয়ার পর Deployment ব্যর্থ হিসেবে চিহ্নিত হলো। দল `kubectl rollout undo` দিয়ে 1.5.0-তে ফিরল, probe-এর পথ ঠিক করে, আবার rollout করল — এবার সফলভাবে। শিক্ষা: probe-এর পথ কোডের সাথে বদলায়; rollout-এর নিরাপত্তা তখনই কাজ করে যখন readiness probe সত্যিই অ্যাপের প্রস্তুতি যাচাই করে।

৭.৯ ধাপে ধাপে ব্যবহারিক উদাহরণ

লক্ষ্য: RollingUpdate দেখা ও rollback অনুশীলন করা। **ধাপ ১:** উপরের Deployment 1.5.0 দিয়ে apply করুন, `kubectl get pods -w` দিয়ে তিনটি Pod চলতে দেখুন। **ধাপ ২:** `kubectl set image` দিয়ে 1.6.0-তে বদলান এবং `kubectl rollout status` দিয়ে দেখুন নতুন Pod একে একে আসছে। **ধাপ ৩:** rollout চলাকালে `kubectl get pods` বারবার চালিয়ে দেখুন — একসময় ৪টি Pod (৩ পুরোনো + ১ নতুন, maxSurge ১) দেখা যাবে। **ধাপ ৪:** `kubectl rollout history` দিয়ে revision দেখুন। **ধাপ ৫:** `kubectl rollout undo` দিয়ে আগের version-এ ফিরুন এবং আবার Pod-গুলো বদলাতে দেখুন। **ধাপ ৬:** `kubectl rollout pause` করে image বদলান, দেখুন rollout থেমে আছে; তারপর `resume` করুন।

```
# চিত্রণমূলক নমুনা আউটপুট
$ kubectl rollout history deployment/technova-api -n technova-prod
REVISION  CHANGE-CAUSE
1         kubectl apply --filename=api-deployment.yaml
2         kubectl set image deployment/technova-api technova-api=technova/api:1.6.0
```

৭.১০ সাধারণ ভুল ও সমস্যা সমাধান

সমস্যা	কারণ	সমাধান
Rollout আটকে আছে	নতুন Pod readiness probe-এ ব্যর্থ, কিংবা image টানা যাচ্ছে না	`describe pod` ও `logs` দেখুন; probe/image ঠিক করুন বা undo করুন
replicas সংখ্যা মিলছে না	selector ভুল label মেলাচ্ছে, কিংবা Pod-গুলো অন্য ReplicaSet-এর	selector ও Pod label মিলিয়ে দেখুন

Rollback-এর পরও ভুল version	revision ইতিহাসে সীমা পেরিয়ে গেছে	`revisionHistoryLimit` বাড়ান; প্রয়োজনে image সরাসরি set করুন
Deployment apply করলে Pod বদলায় না	image tag একই (`latest` বা অপরিবর্তিত tag)	নির্দিষ্ট নতুন tag ব্যবহার করুন; `imagePullPolicy: Always` বিবেচনা করুন
selector বদলাতে গিয়ে ত্রুটি	selector অপরিবর্তনীয় (immutable)	Deployment মুছে নতুন করে তৈরি করুন

৭.১১ নিরাপত্তা বিবেচনা

Deployment-এর মাধ্যমে নতুন image আসে — তাই image-এর উৎস নিয়ন্ত্রণ করুন। শুধু বিশ্বস্ত registry ও নির্দিষ্ট tag (কখনো `latest` নয়) ব্যবহার করুন। Rollback করার অধিকার সীমিত রাখুন — যে কেউ rollback করতে পারলে সে পুরোনো, দুর্বল version-ও ফিরিয়ে আনতে পারে। Revision ইতিহাসে পুরোনো image-এর তথ্য থাকে, তাই `kubectl rollout history` সবার জন্য উন্মুক্ত রাখবেন না। প্রোডাকশন namespace-এ Deployment বদলানোর জন্য আলাদা অনুমোদন (approval) প্রক্রিয়া রাখা ভালো অভ্যাস।

৭.১২ কর্মক্ষমতা ও খরচ বিবেচনা

`maxSurge` বেশি রাখলে rollout দ্রুত হয়, কিন্তু rollout-এর সময় অতিরিক্ত Pod-এর জন্য অতিরিক্ত CPU ও memory লাগে — খরচ বাড়ে। `maxUnavailable` বেশি রাখলে খরচ কমে, কিন্তু rollout চলাকালে সক্ষমতা (capacity) কমে যায়, ফলে বাকি Pod-গুলোর উপর চাপ বাড়ে। ব্যস্ত সময়ে rollout না করে কম ট্রাফিকের সময়ে করা বুদ্ধিমানের কাজ। Replica সংখ্যা বাড়ানো মানে খরচ বাড়ানো — অপ্রয়োজনে ১০টি replica রাখবেন না; Horizontal Pod Autoscaler (অধ্যায় ১১) দিয়ে চাহিদা অনুযায়ী সংখ্যা বদলান।

৭.১৩ সেরা অনুশীলন ও অ্যান্টি-প্যাটার্ন

সেরা অনুশীলন: সবসময় নির্দিষ্ট image tag ব্যবহার করুন; readiness probe সঠিক রাখুন; `maxUnavailable: 0` দিয়ে প্রোডাকশনে downtime এড়ান; প্রতিটি deploy-এর কারণ `--record` বা annotation দিয়ে লিখে রাখুন; ছোট ছোট rollout করুন, বড় বদল একবারে নয়।

অ্যান্টি-প্যাটার্ন: Recreate কৌশল প্রোডাকশনে ডিফল্ট হিসেবে ব্যবহার করা; `latest` tag দিয়ে deploy করা; probe ছাড়া Deployment চালানো — তাতে প্রস্তুত নয় এমন Pod-ও ট্রাফিক পাবে; rollout আটকে গেলে না বুঝে বারবার apply করা।

> **REAL WORLD:** TechNova এখন প্রতিটি deploy-এর আগে staging-এ rollout পরীক্ষা করে, এবং প্রোডাকশন deploy করে কম ট্রাফিকের সময়ে। একটি ভুল deploy-এর পর rollback-এর অভ্যাস তাদের গড়ে ৫ মিনিটে পুনরুদ্ধার (recovery) করতে শিখিয়েছে।

৭.১৪ অধ্যায়ের সারাংশ

ReplicaSet selector দিয়ে Pod চিনে template থেকে নতুন Pod তৈরি করে নির্দিষ্ট সংখ্যা বজায় রাখে — এটাই reconciliation। Deployment ReplicaSet-এর উপরে বসে rollout কৌশল, revision ইতিহাস ও rollback পরিচালনা করে। RollingUpdate সেবা বন্ধ না করে ধীরে ধীরে Pod বদলায়, Recreate প্রথমে সব মুছে তারপর নতুন তৈরি করে। `maxSurge`

ও `maxUnavailable` rollout-এর গতি ও ঝুঁকি নিয়ন্ত্রণ করে। `kubectl rollout` দিয়ে status, history, undo, pause ও resume করা যায়। Deployment stateless অ্যাপের জন্য; stateful কাজে StatefulSet দরকার।

৭.১৫ মূল প্রযুক্তিগত পরিভাষা

ReplicaSet: নির্দিষ্ট সংখ্যক Pod সবসময় চলমান রাখার দায়িত্বে থাকা controller; selector, template ও replicas দিয়ে কাজ করে।

Deployment: ReplicaSet-এর উপরে বসা controller যা rollout, rollback ও revision ইতিহাস পরিচালনা করে।

Reconciliation Loop: কাঙ্ক্ষিত অবস্থা ও বাস্তব অবস্থার পার্থক্য বারবার মিলিয়ে নেওয়ার চক্র।

RollingUpdate: পুরোনো Pod ধীরে ধীরে নতুন Pod দিয়ে বদলানোর কৌশল; সেবা বন্ধ হয় না।

maxSurge: Rollout চলাকালে কাঙ্ক্ষিত সংখ্যার চেয়ে কতটি বেশি Pod থাকতে পারে।

maxUnavailable: Rollout চলাকালে কতটি Pod অনুপলব্ধ থাকতে পারে।

Revision: Deployment-এর প্রতিটি বদলের সংরক্ষিত সংস্করণ; rollback-এর ভিত্তি।

৭.১৬ জ্ঞান যাচাই

১. ReplicaSet কীভাবে জানে কতটি Pod তৈরি করতে হবে?
২. Deployment সরাসরি Pod তৈরি করে, নাকি ReplicaSet-এর মাধ্যমে? ব্যাখ্যা করুন।
৩. RollingUpdate ও Recreate-এর পার্থক্য লিখুন; প্রতিটি কখন ব্যবহার করবেন?
৪. `maxSurge: 1, maxUnavailable: 0` মানে কী?
৫. ভুল version deploy করলে কীভাবে আগের version-এ ফিরবেন? কমান্ডসহ লিখুন।
৬. `progressDeadlineSeconds`-এর কাজ কী?
৭. কেন Deployment-এর selector বদলানো যায় না?
৮. কোন ধরনের অ্যাপে Deployment ব্যবহার করা উচিত নয়, এবং কেন?

৭.১৭ ব্যবহারিক অ্যাসাইনমেন্ট

TechNova-র `technova-web` ফ্রন্টএন্ডের জন্য একটি Deployment লিখুন: replicas ৩, RollingUpdate কৌশলে `maxSurge: 1` ও `maxUnavailable: 1`, readiness probe `/` পথে। এটি apply করে rollout status দেখুন, তারপর image tag বদলে rollout করুন, ইতিহাস দেখুন, এবং rollback করে আগের revision-এ ফিরুন। প্রতিটি ধাপের কমান্ড ও আউটপুট এক পৃষ্ঠায় লিখুন, এবং ব্যাখ্যা করুন rollout চলাকালে Pod-এর সংখ্যা কীভাবে বদলেছে।

অধ্যায় ৮ • PART II — Workloads & Application Management

Chapter 8 — StatefulSets, DaemonSets, Jobs, and CronJobs

৮.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি ব্যাখ্যা করতে পারবেন কেন সব workload Deployment দিয়ে চলে না — কখন StatefulSet, কখন DaemonSet, কখন Job, আর কখন CronJob দরকার। আপনি StatefulSet-এর স্থিতিশীল পরিচয় (stable identity), ক্রমিক সংখ্যা (ordinal), headless Service ও `volumeClaimTemplates` বুঝবেন; DaemonSet কীভাবে প্রতিটি Node-এ একটি করে Pod নিশ্চিত করে তা জানবেন; Job-এর completions, parallelism ও backoffLimit-এর অর্থ বুঝবেন; এবং CronJob-এর schedule, concurrency নীতি ও history সীমা কনফিগার করতে পারবেন। ব্যবহারিকভাবে আপনি MongoDB-র মতো ডেটাবেসের জন্য StatefulSet, লগ সংগ্রাহকের জন্য DaemonSet, একবারের migration-এর জন্য Job, এবং রাতের ব্যাকআপের জন্য CronJob লিখতে পারবেন।

৮.২ কেন এই বিষয়টি গুরুত্বপূর্ণ

Deployment ধরে নেয় Pod-গুলো একে অপরের বদলে চলতে পারে — যেকোনো Pod মারা গেলে যেকোনো নতুন Pod তার জায়গা নিতে পারে, এবং তাদের কোনো স্থায়ী পরিচয় বা ডেটা নেই। ডেটাবেস এই ধারণার ঠিক উল্টো: MongoDB-র প্রতিটি সদস্যের নিজস্ব পরিচয়, নিজস্ব ডেটা, এবং নিজস্ব ভূমিকা (primary বা secondary) থাকে। একটি Pod মুছে নতুন Pod তৈরি হলে তার আগের ডেটা হারিয়ে গেলে চলবে না। আবার কিছু কাজ প্রতিটি Node-এই চলতে হয় — যেমন লগ সংগ্রহ বা মনিটরিং agent — সেখানে replica সংখ্যা গুনে লাভ নেই, Node সংখ্যাই হিসাব। আর কিছু কাজ একবার শেষ হলেই শেষ — যেমন ডেটাবেস migration বা রিপোর্ট তৈরি — সেগুলোকে সবসময় চলমান রাখা ভুল। ভুল workload বেছে নিলে ডেটা হারানো, সেবা বন্ধ হওয়া, কিংবা বারবার একই কাজ চলার মতো গুরুতর সমস্যা হয়।

৮.৩ পূর্বশর্ত জ্ঞান

অধ্যায় ৬ ও ৭-এর Pod, ReplicaSet ও Deployment-এর ধারণা এখানে ভিত্তি। PersistentVolume ও PersistentVolumeClaim-এর প্রাথমিক ধারণা StatefulSet বুঝতে সাহায্য করবে (বিস্তারিত Storage অধ্যায়ে আসবে, তবে এখানে প্রয়োজনীয়টুকু ব্যাখ্যা করা হয়েছে)। Cron সময়সূচির লেখার ধরন (যেমন `0 2 * * *`) জানা থাকলে CronJob সহজ হবে।

৮.৪ প্রথম নীতি থেকে ব্যাখ্যা

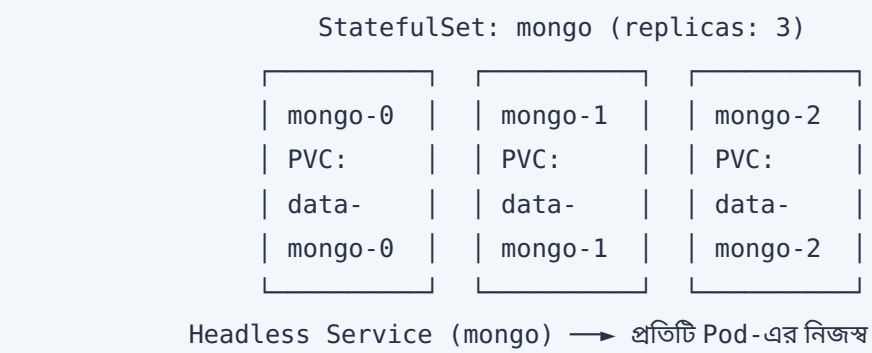
প্রথম নীতি থেকে ভাবুন: workload-গুলো আসলে চারটি ভিন্ন প্রশ্নের উত্তর। প্রথম প্রশ্ন — “এই Pod-টির কি নিজস্ব পরিচয় ও ডেটা আছে?” থাকলে StatefulSet, না থাকলে Deployment। দ্বিতীয় প্রশ্ন — “এই কাজটি কি প্রতিটি মেশিনে চলতে হবে?” হ্যাঁ হলে DaemonSet। তৃতীয় প্রশ্ন — “এই কাজটি কি একবার শেষ হলেই শেষ?” হ্যাঁ হলে Job। চতুর্থ প্রশ্ন — “এই কাজটি কি নিয়মিত সময়সূচি অনুযায়ী চলবে?” হ্যাঁ হলে CronJob।

ডেটাবেসের উদাহরণে ফিরুন। MongoDB replica set-এর তিন সদস্যের নাম — mongo-0, mongo-1, mongo-2 — তাদের পরিচয়। mongo-0 মুছে গেলে নতুন Pod-ও mongo-0 নামেই ফিরবে, একই স্টোরেজ পাবে, এবং replica set-এ নিজের জায়গা চিনতে পারবে। এই স্থিতিশীলতা না থাকলে প্রতিবার Pod বদলে গেলে ডেটাবেস নিজেকে নতুন সদস্য ভাবত,

ডেটা সিঙ্ক করতে গিয়ে বিভ্রান্ত হতো। StatefulSet এই স্থিতিশীল পরিচয় ও স্টোরেজের নিশ্চয়তা দেয় — আর এটাই Deployment থেকে তার মৌলিক পার্থক্য।

> **CONCEPT:** Deployment বলে "যেকোনো তিনটি Pod হলেই চলবে"; StatefulSet বলে "ঠিক এই তিনটি — mongo-0, mongo-1, mongo-2 — প্রত্যেকের নিজস্ব ডেটা সহ"। এই পার্থক্যই stateful বনাম stateless-এর মূল।

৮.৫ আর্কিটেকচার / ধারণাগত চিত্র



চিত্র ৮.১: StatefulSet-এর স্থিতিশীল পরিচয় ও স্টোরেজ, DaemonSet-এর প্রতি-Node Pod, Job ও CronJob-এর সম্পর্ক।

৮.৬ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

৮.৬.১ StatefulSet: স্থিতিশীল পরিচয়, ক্রম ও স্টোরেজ

StatefulSet-এর Pod-গুলোর নামে ক্রমিক সংখ্যা থাকে — `mongo-0`, `mongo-1`, `mongo-2` — এবং এই নাম ও ক্রম স্থিতিশীল। Pod মুছে গেলে একই নামে, একই ক্রমে নতুন Pod ফিরে আসে। Pod-গুলো ক্রমানুসারে তৈরি হয় (০, তারপর ১, তারপর ২) এবং মুছে ফেলার সময় উল্টো ক্রমে মুছে যায় — ডেটাবেসের মতো সিস্টেমে এটা গুরুত্বপূর্ণ, কারণ primary আগে তৈরি হয়ে প্রস্তুত না হলে secondary-রা যোগ দিতে পারে না। আপডেটের সময়ও একই ক্রম মেনে চলা হয়।

সবচেয়ে গুরুত্বপূর্ণ বৈশিষ্ট্য হলো `volumeClaimTemplates`: প্রতিটি Pod-এর জন্য আলাদা PersistentVolumeClaim স্বয়ংক্রিয়ভাবে তৈরি হয় — যেমন `data-mongo-0`, `data-mongo-1`। Pod মুছে গেলেও তার PVC ও ডেটা থেকে যায়, নতুন Pod একই PVC আবার মাউন্ট করে। তাই StatefulSet মুছে ফেললেও PVC স্বয়ংক্রিয়ভাবে মোছে না — ডেটা হারানো এড়াতে ইচ্ছাকৃতভাবেই এমন করা হয়েছে। Headless Service (`clusterIP: None`) StatefulSet-এর সাথে ব্যবহার করা হয়, যাতে প্রতিটি Pod-এর নিজস্ব DNS নাম থাকে (`mongo-0.mongo`) — Deployment-এর Service যেখানে সব Pod-কে একটি ঠিকানার পেছনে লুকায়, headless Service সেখানে প্রতিটি Pod-কে আলাদাভাবে খুঁজে পাওয়ার উপায় দেয়।

৮.৬.২ DaemonSet: প্রতিটি Node-এ একজন করে

DaemonSet নিশ্চিত করে প্রতিটি (বা নির্বাচিত) Node-এ ঠিক একটি করে Pod চলে। নতুন Node cluster-এ যোগ হলে সেখানে স্বয়ংক্রিয়ভাবে Pod তৈরি হয়; Node মুছে গেলে তার Pod-ও মুছে যায়। ব্যবহার: লগ সংগ্রাহক (Fluent Bit, Filebeat), মনিটরিং agent (node exporter), নেটওয়ার্ক প্লাগইন, স্টোরেজ driver। DaemonSet-এর Pod-গুলো সাধারণত Node-এর ফাইলসিস্টেম (যেমন `/var/log`) hostPath ভলিউম দিয়ে পড়ে। সতর্কতা: DaemonSet-এর Pod-ও সম্পদ খরচ করে — প্রতিটি Node-এ একটি করে ভারী agent মানে cluster জুড়ে উল্লেখযোগ্য খরচ।

৮.৬.৩ Job: একবারের কাজ, সফল হলেই শেষ

Job এমন Pod তৈরি করে যা একটি কাজ শেষ করে থেমে যায়। `completions` বলে কতবার সফলভাবে শেষ হতে হবে (ডিফল্ট ১), `parallelism` বলে একসাথে কতটি Pod চলতে পারে। কাজ ব্যর্থ হলে Job সেটিকে `backoffLimit` (ডিফল্ট ৬) বার পর্যন্ত পুনরায় চেষ্টা করে — প্রতিবার বিরতি বাড়িয়ে। `restartPolicy` Job-এ `Never` বা `OnFailure` হতে হয়, `Always` নয়। ব্যবহার: ডেটাবেস migration, ডেটা আমদানি, ব্যাচ রিপোর্ট, এককালীন সেটআপ। Job শেষ হলে তার Pod-গুলো `Completed` অবস্থায় থাকে — লগ দেখার জন্য সেগুলো কিছুক্ষণ রেখে দেওয়া হয়, তারপর মুছে ফেলা যায়।

৮.৬.৪ CronJob: সময়সূচি অনুযায়ী Job

CronJob নির্দিষ্ট সময়সূচিতে (cron expression — মিনিট, ঘণ্টা, দিন, মাস, সপ্তাহের দিন) Job তৈরি করে। উদাহরণ: `0 2 * * *` মানে প্রতিদিন রাত ২টা। `concurrencyPolicy` নির্ধারণ করে আগের Job শেষ না হলে নতুনটি কী করবে: `Allow` (একসাথে চলতে দেবে), `Forbid` (আগেরটি শেষ না হওয়া পর্যন্ত নতুনটি বাদ দেবে), `Replace` (আগেরটি বন্ধ করে নতুনটি চালাবে)। `successfulJobsHistoryLimit` ও `failedJobsHistoryLimit` কতটি পুরোনো Job ইতিহাস হিসেবে রাখা হবে তা নিয়ন্ত্রণ করে — না হলে Job জমে cluster ভারী হয়ে যায়। `startingDeadlineSeconds` বলে দেয় নির্ধারিত সময় পেরিয়ে কতক্ষণ পর্যন্ত Job শুরু করা যাবে — control plane ব্যস্ত থাকলে দেরিতে শুরু হওয়া Job এড়াতে এটি কাজে লাগে।

৮.৬.৫ Batch বনাম দীর্ঘস্থায়ী (long-running) workload

ধরন	উদাহরণ	Workload
দীর্ঘস্থায়ী, stateless	API, ফ্রন্টএন্ড	Deployment
দীর্ঘস্থায়ী, stateful	MongoDB, Kafka	StatefulSet
প্রতিটি Node-এ	লগ, মনিটরিং agent	DaemonSet
একবারের কাজ	migration, রিপোর্ট	Job
নিয়মিত কাজ	ব্যাকআপ, পরিষ্কার	CronJob

৮.৭ কনফিগারেশন উদাহরণ

```
apiVersion: v1
kind: Service
metadata:
  name: mongo
  namespace: technova-prod
spec:
  clusterIP: None   # headless service
```

```
  selector:
    app: mongo
  ports:
  - port: 27017
    name: mongo
---
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: mongo
  namespace: technova-prod
spec:
  serviceName: mongo
  replicas: 3
  selector:
    matchLabels:
      app: mongo
  template:
    metadata:
      labels:
        app: mongo
    spec:
      containers:
      - name: mongo
        image: mongo:7.0
        ports:
        - containerPort: 27017
        volumeMounts:
        - name: data
          mountPath: /data/db
  volumeClaimTemplates:
  - metadata:
      name: data
    spec:
      accessModes: ["ReadWriteOnce"]
      resources:
        requests:
          storage: 10Gi
---
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: log-collector
  namespace: technova-prod
spec:
  selector:
    matchLabels:
      app: log-collector
  template:
    metadata:
      labels:
        app: log-collector
    spec:
      containers:
      - name: fluent-bit
        image: fluent/fluent-bit:2.2
        volumeMounts:
        - name: varlog
          mountPath: /var/log
```

```
      volumes:
      - name: varlog
        hostPath:
          path: /var/log
---
apiVersion: batch/v1
kind: Job
metadata:
  name: db-migration
  namespace: technova-prod
spec:
  completions: 1
  parallelism: 1
  backoffLimit: 3
  template:
    spec:
      restartPolicy: Never
      containers:
      - name: migrate
        image: technova/api:1.6.0
        command: ["npm", "run", "migrate"]
---
apiVersion: batch/v1
kind: CronJob
metadata:
  name: nightly-backup
  namespace: technova-prod
spec:
  schedule: "0 2 * * *"
  concurrencyPolicy: Forbid
  successfulJobsHistoryLimit: 3
  failedJobsHistoryLimit: 1
  jobTemplate:
    spec:
      template:
        spec:
          restartPolicy: OnFailure
          containers:
          - name: backup
            image: technova/backup:1.0.0
            command: ["sh", "-c", "mongodump --uri=mongodb://mongo-0.mongo:27017/technova --
out=/backup/$(date +%F)"]
```

```
kubectl get statefulsets -n technova-prod
kubectl get pods -n technova-prod -l app=mongo
kubectl get daemonsets -n technova-prod
kubectl get jobs -n technova-prod
kubectl get cronjobs -n technova-prod
kubectl logs job/db-migration -n technova-prod
```

৮.৮ বাস্তব প্রকৌশল পরিস্থিতি

TechNova পরিস্থিতি: প্রথমে MongoDB Deployment দিয়ে চালানো হয়েছিল — তিনটি replica, কিন্তু প্রতিটির নিজস্ব খালি স্টোরেজ। একদিন একটি Pod মুছে গেলে নতুন Pod খালি ডেটাবেস নিয়ে ফিরল, আর অ্যাপের কিছু ব্যবহারকারীর ডেটা “হারিয়ে” গেল (আসলে অন্য Pod-এর স্টোরেজে আটকে ছিল)। দল MongoDB-কে StatefulSet-এ সরিয়ে নিল:

প্রতিটি Pod এখন নিজস্ব PVC পায়, Pod বদলে গেলেও ডেটা ফিরে আসে, আর headless Service দিয়ে `mongo-0.mongo` নামে প্রতিটি সদস্যকে আলাদাভাবে ডাকা যায়। একই সময়ে রাতের ব্যাকআপের জন্য একটি CronJob যোগ করা হলো — প্রতিদিন রাত ২টায় `mongodump` চালায়, `concurrencyPolicy: Forbid` দিয়ে নিশ্চিত করা হলো আগের ব্যাকআপ শেষ না হলে নতুনটি শুরু হবে না। আর সব Node-এর লগ সংগ্রহের জন্য DaemonSet-এ Fluent Bit চালানো হলো, যাতে নতুন Node যোগ হলেই লগ সংগ্রহ শুরু হয়।

৮.৯ ধাপে ধাপে ব্যবহারিক উদাহরণ

লক্ষ্য: একটি Job চালিয়ে তার সমাপ্তি দেখা, এবং একটি CronJob তৈরি করা। **ধাপ ১:** উপরের Job YAML apply করুন: `kubectl apply -f db-migration.yaml`। **ধাপ ২:** `kubectl get jobs -w` দিয়ে দেখুন Job কীভাবে `1/1` completions-এ পৌঁছায়। **ধাপ ৩:** `kubectl logs job/db-migration` দিয়ে কাজের লগ দেখুন। **ধাপ ৪:** CronJob apply করুন, তারপর `kubectl get cronjobs` দিয়ে পরবর্তী সময়সূচি (schedule) দেখুন। **ধাপ ৫:** পরীক্ষার জন্য একটি CronJob তৈরি করুন যার schedule প্রতি মিনিটে (`* * * * *`) — দেখুন প্রতি মিনিটে নতুন Job তৈরি হচ্ছে। পরীক্ষা শেষে সেটি মুছে ফেলুন, নইলে Job জমতে থাকবে। **ধাপ ৬:** StatefulSet-এর ক্ষেত্রে `kubectl get pvc` দিয়ে দেখুন প্রতিটি Pod-এর আলাদা PVC তৈরি হয়েছে।

৮.১০ সাধারণ ভুল ও সমস্যা সমাধান

সমস্যা	কারণ	সমাধান
StatefulSet-এর Pod-এ পুরোনো ডেটা নেই	Deployment ব্যবহার করা হয়েছে, কিংবা PVC মুছে ফেলা হয়েছে	StatefulSet + volumeClaimTemplates ব্যবহার করুন; PVC আলাদাভাবে মুছবেন না
Job বারবার চলছে, শেষ হচ্ছে না	কনটেইনার exit code ০ দিচ্ছে না; backoffLimit শেষ হয়নি	লগ দেখে ব্যর্থতার কারণ ঠিক করুন; `restartPolicy: Never` যাচাই করুন
CronJob দুইবার চলছে	concurrencyPolicy Allow, আগের Job শেষ হয়নি	Forbid বা Replace ব্যবহার করুন
CronJob-এর Job জমে যাচ্ছে	history limit নির্ধারণ করা হয়নি	successful/failedJobsHistoryLimit সেট করুন
DaemonSet কিছু Node-এ Pod তৈরি করছে না	Node-এ taint আছে, কিংবা nodeSelector মিলছে না	`describe node` ও toleration যাচাই করুন
StatefulSet আপডেট খুব ধীর	ক্রমানুসারে একে একে Pod বদলানো হচ্ছে — এটাই স্বাভাবিক	প্রয়োজনে `podManagementPolicy: Parallel` বিবেচনা করুন (সতর্কতার সাথে)

TROUBLESHOOTING: StatefulSet মুছে ফেললেও PVC মোছে না — এটা বৈশিষ্ট্য, বাগ নয়। ডেটা সত্যিই মুছতে চাইলে PVC আলাদাভাবে মুছতে হয়। মুছার আগে দুবার ভাবুন।

৮.১১ নিরাপত্তা বিবেচনা

StatefulSet-এর ডেটা সবচেয়ে সংবেদনশীল সম্পদ — PVC-র ডেটা এনক্রিপ্ট করা আছে কি না যাচাই করুন, এবং ব্যাকআপ কোথায় যাচ্ছে তা নিয়ন্ত্রণ করুন। DaemonSet-এর Pod-গুলো প্রায়ই hostPath দিয়ে Node-এর ফাইল পড়ে —

এই ক্ষমতা অপব্যবহার হলে Node-এর সব লগ পড়া যায়, তাই DaemonSet-এর image ও অনুমতি কঠোরভাবে নিয়ন্ত্রণ করুন। Job ও CronJob-এ দেওয়া credential (যেমন ডেটাবেস পাসওয়ার্ড) Secret থেকে আসা উচিত, YAML-এ সরাসরি লেখা নয়। CronJob-এর ব্যাকআপে ডেটাবেসের সম্পূর্ণ ডেটা থাকে — ব্যাকআপ ফাইলের সংরক্ষণ ও মুছে ফেলার নীতি নির্ধারণ করুন।

৮.১২ কর্মক্ষমতা ও খরচ বিবেচনা

StatefulSet-এর প্রতিটি replica নিজস্ব স্টোরেজ খরচ করে — ৩টি replica মানে ৩ গুণ স্টোরেজ খরচ। DaemonSet-এর খরচ Node সংখ্যার সাথে বাড়ে — Node দ্বিগুণ হলে agent খরচও দ্বিগুণ। CronJob-এর ব্যাকআপ Job চলার সময় CPU ও নেটওয়ার্ক চাপ বাড়ায় — ব্যস্ত সময় এড়িয়ে রাতের বেলা চালান। Job-এর parallelism বেশি রাখলে কাজ দ্রুত শেষ হয়, কিন্তু একসাথে অনেক Pod সম্পদ দখল করে — ডেটাবেসের উপর চাপের কথা ভাবুন।

৮.১৩ সেরা অনুশীলন ও অ্যান্টি-প্যাটার্ন

সেরা অনুশীলন: ডেটাবেস ও যেকোনো stateful অ্যাপে StatefulSet ব্যবহার করুন; headless Service দিয়ে প্রতিটি Pod-কে নামে ডাকুন; CronJob-এ সবসময় concurrencyPolicy ও history limit সেট করুন; Job-এর লগ সংরক্ষণ করুন; DaemonSet-এ resource limit দিন যাতে agent Node-এর সব সম্পদ খেয়ে না ফেলে।

অ্যান্টি-প্যাটার্ন: ডেটাবেস Deployment দিয়ে চালানো; CronJob-এ schedule না বুঝে `* * * * *` রেখে দেওয়া; Job-এর ব্যর্থতা না দেখে বারবার চালানো; StatefulSet-এর PVC না বুঝে মুছে ফেলা; DaemonSet দিয়ে অ্যাপ চালানো (DaemonSet অ্যাপের workload নয়, agent-এর জন্য)।

> **PRO TIP:** CronJob পরীক্ষা করার সময় schedule সাময়িকভাবে প্রতি মিনিটে সেট করে দেখুন কাজটি ঠিকভাবে চলে কি না, তারপর আসল সময়সূচিতে ফিরুন। কিন্তু পরীক্ষা শেষে মনে করে schedule ফেরাতে ভুলবেন না।

৮.১৪ অধ্যায়ের সারাংশ

StatefulSet স্থিতিশীল পরিচয়, ক্রমিক সৃষ্টি ও `volumeClaimTemplates`-এর মাধ্যমে প্রতিটি Pod-কে নিজস্ব ডেটা দেয় — ডেটাবেসের জন্য অপরিহার্য, সাথে headless Service প্রতিটি Pod-কে DNS নাম দেয়। DaemonSet প্রতিটি Node-এ একটি করে Pod নিশ্চিত করে — লগ ও মনিটরিং agent-এর জন্য। Job একবারের কাজ সফলভাবে শেষ করে, completions ও parallelism দিয়ে নিয়ন্ত্রিত হয়। CronJob সময়সূচি অনুযায়ী Job তৈরি করে, concurrency নীতি ও history সীমা দিয়ে পরিচালিত হয়। সঠিক workload বাছাই করা মানে কাজের প্রকৃতি বোঝা — stateless, stateful, প্রতি-Node, একবারের, না নিয়মিত।

৮.১৫ মূল প্রযুক্তিগত পরিভাষা

StatefulSet: স্থিতিশীল পরিচয় ও স্টোরেজসহ Pod পরিচালনা করা workload; ডেটাবেসের মতো stateful অ্যাপের জন্য।

Headless Service: `clusterIP: None` দিয়ে তৈরি Service; প্রতিটি Pod-এর নিজস্ব DNS নাম দেয়, লোড ব্যালান্স করে না।

volumeClaimTemplates: StatefulSet-এর প্রতিটি Pod-এর জন্য আলাদা PersistentVolumeClaim তৈরির ছাঁচ।

DaemonSet: প্রতিটি (বা নির্বাচিত) Node-এ ঠিক একটি করে Pod চালানো নিশ্চিত করা workload।

Job: একটি কাজ সফলভাবে শেষ করা পর্যন্ত Pod চালানো workload; শেষ হলে আর চলে না।

CronJob: সময়সূচি অনুযায়ী নিয়মিত Job তৈরি করা workload।

Ordinal: StatefulSet Pod-এর নামের ক্রমিক সংখ্যা (০, ১, ২) যা তার স্থিতিশীল পরিচয়ের অংশ।

৮.১৬ জ্ঞান যাচাই

১. ডেটাবেসের জন্য Deployment নয়, StatefulSet কেন দরকার?
২. Headless Service সাধারণ Service থেকে কীভাবে আলাদা?
৩. `volumeClaimTemplates`-এর কাজ কী? Pod মুছলে PVC-র কী হয়?
৪. DaemonSet কখন ব্যবহার করবেন? দুটি উদাহরণ দিন।
৫. Job-এ `completions` ও `parallelism`-এর পার্থক্য লিখুন।
৬. CronJob-এর `concurrencyPolicy: Forbid` মানে কী? কখন এটি দরকার?
৭. `0 2 * * *` schedule-টি কী বোঝায়?
৮. একটি ব্যাচ রিপোর্টের কাজে Deployment ব্যবহার করলে কী ভুল হবে?

৮.১৭ ব্যবহারিক অ্যাসাইনমেন্ট

TechNova-র জন্য দুটি YAML লিখুন: (ক) একটি CronJob যা প্রতিদিন ভোর ৩টায় (`0 3 * * *`) একটি কাল্পনিক ব্যাকআপ image চালায়, concurrencyPolicy Forbid, history limit সফল ৩টি ও ব্যর্থ ১টি; (খ) একটি Job যা `technova/api:1.6.0` image দিয়ে একটি migration কমান্ড চালায়, backoffLimit ৩। দুটিই পরীক্ষামূলক cluster-এ apply করে `kubectl get jobs` ও `kubectl get cronjobs`-এর আউটপুট সংরক্ষণ করুন এবং ব্যাখ্যা করুন Job শেষ হওয়ার পর তার Pod-এর অবস্থা কী হয়।

অধ্যায় ৯ • PART II — Workloads & Application Management

Chapter 9 — Namespaces, Labels, Selectors, and Annotations

৯.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি Namespace, Label, Selector ও Annotation — এই চারটি ধারণার পার্থক্য ও প্রত্যেকের কাজ পরিষ্কারভাবে ব্যাখ্যা করতে পারবেন। আপনি জানবেন Namespace কীভাবে cluster-কে ভাগ করে, কেন এটি শক্ত নিরাপত্তা সীমানা (strong security boundary) নয়, এবং Label কীভাবে Pod, Deployment ও Service-কে একে অপরের সাথে যুক্ত করে। আপনি equality-based ও set-based selector লিখতে পারবেন, Service ও Deployment-এ selector ব্যবহার করতে পারবেন, এবং Annotation কখন ব্যবহার করতে হয় তা বলতে পারবেন। ব্যবহারিকভাবে আপনি TechNova-র dev, staging ও prod পরিবেশের জন্য Namespace সাজাতে পারবেন এবং একটি সামঞ্জস্যপূর্ণ labelling নীতি তৈরি করতে পারবেন।

৯.২ কেন এই বিষয়টি গুরুত্বপূর্ণ

Kubernetes cluster-এ শত শত Pod, Service ও Deployment থাকলে নাম দিয়ে সবকিছু খুঁজে পাওয়া অসম্ভব হয়ে পড়ে। Label হলো সেই খোঁজার ব্যবস্থা — “সব backend Pod দেখাও”, “শুধু prod-এর API দেখাও” — এই প্রশ্নগুলোর উত্তর Label ছাড়া দেওয়া যায় না। Namespace ছাড়া dev-এর পরীক্ষামূলক Pod prod-এর Pod-এর সাথে মিশে যেত, নামের সংঘাত হতো, আর ভুল করে dev-এর Deployment prod-এর ডেটা মুছে ফেলত। আর Selector ভুল হলে Service ভুল Pod-এ ট্রাফিক পাঠায়, কিংবা Deployment ভুল Pod গুনে — এই ভুলগুলো প্রোডাকশনে খুবই সাধারণ এবং খুবই ব্যয়বহুল। এই চারটি ধারণা Kubernetes-এর “সংগঠন ব্যবস্থা” — এগুলো না বুঝলে বড় cluster পরিচালনা করা যায় না।

৯.৩ পূর্বশর্ত জ্ঞান

অধ্যায় ৬–৮-এর Pod, Deployment, StatefulSet ও Service-এর ধারণা এখানে ভিত্তি। বিশেষ করে Deployment-এর selector ও Pod template-এর label-এর সম্পর্কটি মনে রাখুন — এই অধ্যায়ে সেটি গভীরে ব্যাখ্যা করা হবে। `kubectl get -l` দিয়ে label অনুযায়ী খোঁজার অভিজ্ঞতা থাকলে ভালো হয়।

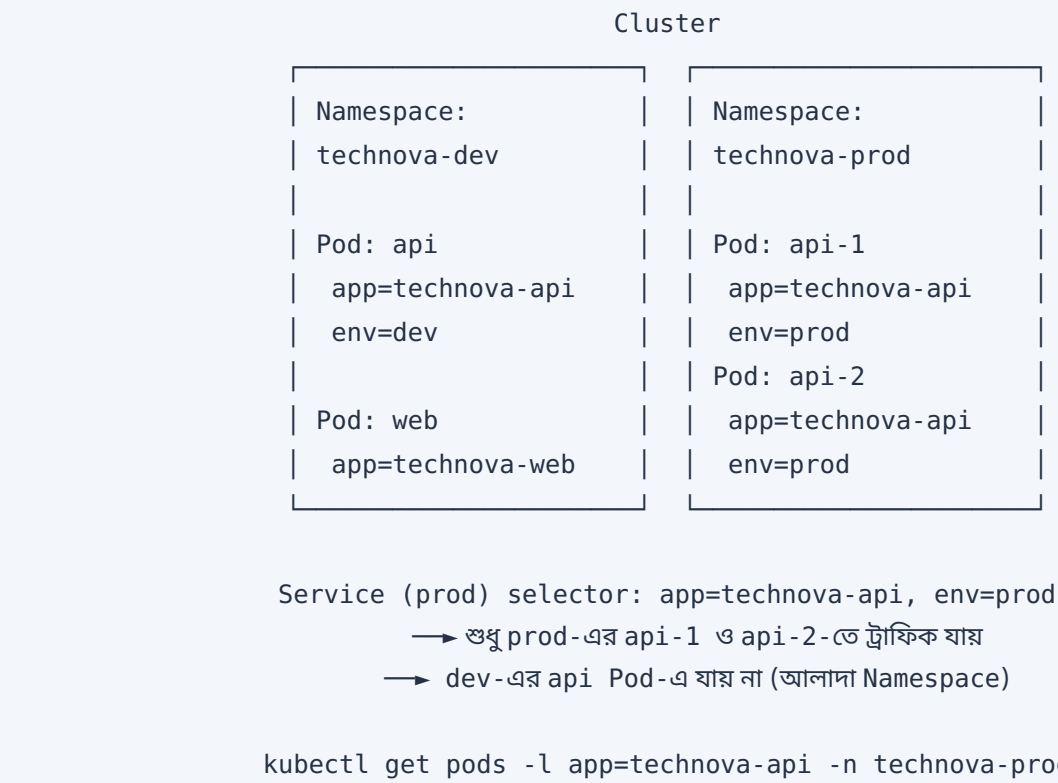
৯.৪ প্রথম নীতি থেকে ব্যাখ্যা

প্রথম নীতি থেকে ভাবুন: যেকোনো বড় ব্যবস্থায় জিনিসগুলোকে ভাগ করা ও চিহ্নিত করা দরকার। একটি অফিসে যেমন বিভাগ (department) থাকে আর প্রতিটি ফাইলে ট্যাগ (tag) লাগানো থাকে, Kubernetes-এও তেমনি Namespace হলো বিভাগ, Label হলো ট্যাগ। Namespace বলে “এই জিনিসটি কোন পরিবেশের বা কোন দলের” — যেমন `technova-dev`, `technova-prod`। Label বলে “এই জিনিসটি কী” — যেমন `app: technova-api`, `tier: backend`, `env: prod`। Selector হলো সেই ট্যাগ দিয়ে খোঁজার প্রশ্ন — “`app=technova-api` এমন সব Pod দাও”। আর Annotation হলো ফাইলের গায়ে লেখা মন্তব্য — মানুষ বা টুলের জন্য বাড়তি তথ্য, যা Kubernetes নিজে খোঁজার কাজে ব্যবহার করে না।

এই চারটির মধ্যে একটি মৌলিক পার্থক্য আছে: Namespace ও Label হলো কাঠামো (structure), Selector হলো প্রশ্ন (query), Annotation হলো মন্তব্য (note)। কাঠামো ভুল হলে প্রশ্নের উত্তরও ভুল আসে। তাই labelling-এর শৃঙ্খলা Kubernetes পরিচালনার শৃঙ্খলা।

> **CONCEPT:** Label হলো পরিচয়পত্র, Selector হলো সেই পরিচয়পত্র দিয়ে খোঁজা, Namespace হলো ঠিকানা (কোন এলাকায়), আর Annotation হলো পরিচয়পত্রের পেছনে লেখা নোট। চারটি মিলেই Kubernetes-এর সংগঠন তৈরি হয়।

৯.৫ আর্কিটেকচার / ধারণাগত চিত্র



চিত্র ৯.১: Namespace দিয়ে পরিবেশ আলাদা করা, Label ও Selector দিয়ে Pod বাছাই করা।

৯.৬ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

৯.৬.১ Namespace: ভাগ করা, কিন্তু শক্ত সীমানা নয়

Namespace হলো cluster-এর ভেতরে নামের পরিসর (scope) ভাগ করার উপায়। একই নামের দুটি Pod দুটি Namespace-এ থাকতে পারে — যেমন dev ও prod-এ দুটোই `technova-api` নামে Pod থাকতে পারে, কোনো সংঘাত হয় না। বেশিরভাগ resource (Pod, Service, Deployment, ConfigMap, Secret) Namespace-এর ভেতরে থাকে; কিছু resource (Node, PersistentVolume, Namespace নিজে) cluster-ব্যাপী (cluster-scoped)। Kubernetes শুরুতে চারটি Namespace দেয়: `default`, `kube-system` (control plane-এর জন্য), `kube-public`, ও `kube-node-lease`।

গুরুত্বপূর্ণ সতর্কতা: **Namespace শক্ত নিরাপত্তা সীমানা নয়।** ডিফল্টভাবে এক Namespace-এর Pod অন্য Namespace-এর Pod-এর সাথে নেটওয়ার্কে কথা বলতে পারে, Service-এর DNS নামে (`service.namespace.svc.cluster.local`) পৌঁছাতে পারে। সত্যিকারের বিচ্ছিন্নতা (isolation) চাইলে

NetworkPolicy, ResourceQuota ও RBAC (ভূমিকা-ভিত্তিক প্রবেশ নিয়ন্ত্রণ) একসাথে ব্যবহার করতে হয়। Namespace মূলত সংগঠন, নামের সংঘাত এড়ানো, এবং নীতি প্রয়োগের একক — নিরাপত্তার একমাত্র স্তর নয়।

৯.৬.২ Label: key-value ট্যাগ

Label হলো resource-এর সাথে যুক্ত key-value জোড়া — যেমন `app: technova-api`, `version: "1.6.0"`, `team: platform`। Label-এর key ও value-র নিয়ম আছে: ৬৩ অক্ষরের মধ্যে, অক্ষর, সংখ্যা, `-`, `_`, `.` দিয়ে শুরু ও শেষ হতে হয়। Label দিয়ে resource-কে শ্রেণিবদ্ধ করা হয়, খোঁজা হয়, এবং Selector দিয়ে বাছাই করা হয়। প্রস্তাবিত (recommended) label-গুলোর মধ্যে আছে `app.kubernetes.io/name`, `app.kubernetes.io/instance`, `app.kubernetes.io/version`, `app.kubernetes.io/component` — এগুলো ব্যবহার করলে টুল ও ড্যাশবোর্ড একইভাবে সব অ্যাপকে চিনতে পারে।

৯.৬.৩ Selector: Label দিয়ে বাছাই

Selector হলো Label-এর উপর প্রশ্ন। দুই ধরনের: **equality-based** — `app=technova-api`, `env!=dev` (সমান বা সমান নয়); এবং **set-based** — `app in (technova-api, technova-web)`, `env notin (dev)`, `team` (এই key আছে কি না)। Deployment ও ReplicaSet-এর selector-এ `matchLabels` (equality) ও `matchExpressions` (set-based) ব্যবহার করা যায়। Service-এর selector শুধু equality-based। একটি সতর্কতা: Deployment-এর selector তার Pod template-এর label-এর সাথে মিলতে হবে, নইলে Deployment Pod তৈরি করবে কিন্তু নিজের Pod হিসেবে চিনবে না — Pod তৈরি হবে কিন্তু replica গোনায় আসবে না।

৯.৬.৪ Annotation: মন্তব্য, খোঁজার উপায় নয়

Annotation-ও key-value, কিন্তু এর উদ্দেশ্য আলাদা: এটি মানুষ বা বাইরের টুলের জন্য বাড়তি তথ্য — যেমন deploy-এর কারণ, যোগাযোগের ঠিকানা, বাইরের টুলের কনফিগারেশন। Annotation-এর value যেকোনো লেখা হতে পারে, Label-এর মতো কঠোর নিয়ম নেই, এবং Selector দিয়ে Annotation খোঁজা যায় না। উদাহরণ: `description: "TechNova API, MERN stack"`, `contact: "platform@technova.example"`, `kubernetes.io/change-cause: "upgrade to 1.6.0"`। একটি সাধারণ ভুল হলো খোঁজার জন্য দরকারি তথ্য Annotation-এ রাখা — সেটি Label-এ রাখা উচিত।

বিষয়	Label	Annotation
উদ্দেশ্য	চিহ্নিত করা ও বাছাই করা	বাড়তি তথ্য রাখা
Selector দিয়ে খোঁজা যায়	হ্যাঁ	না
নিয়ম	কঠোর (৬৩ অক্ষর, সীমিত অক্ষর)	শিথিল (যেকোনো লেখা)
উদাহরণ	`app: technova-api`	`contact: platform@technova.example`

৯.৬.৫ নামকরণের নিয়ম

Resource-এর নাম ছোট হাতের অক্ষর, সংখ্যা ও `-` দিয়ে লেখা হয় (DNS subdomain নিয়ম)। Namespace-এর নামও একই নিয়মে — যেমন `technova-prod`, `technova-dev`। নামে পরিবেশ, অ্যাপ ও উদ্দেশ্য বোঝা গেলে ভালো — `api`-র বদলে `technova-api` লেখা ভালো, কারণ একাধিক দলের `api` এক cluster-এ থাকতে পারে। Label-এর key-তে দলের নামের prefix দেওয়া যেতে পারে — যেমন `technova.example/team: platform` — এতে অন্য দলের label-এর সাথে সংঘাত হয় না।

৯.৭ কনফিগারেশন উদাহরণ

```
apiVersion: v1
kind: Namespace
metadata:
  name: technova-prod
  labels:
    env: prod
    team: platform
---
apiVersion: v1
kind: Service
metadata:
  name: technova-api
  namespace: technova-prod
  annotations:
    description: "TechNova API service for technova.example"
    contact: "platform@technova.example"
  labels:
    app: technova-api
spec:
  selector:
    app: technova-api
    tier: backend
  ports:
  - port: 80
    targetPort: 3000
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: technova-api
  namespace: technova-prod
spec:
  replicas: 3
  selector:
    matchLabels:
      app: technova-api
      tier: backend
  template:
    metadata:
      labels:
        app: technova-api
        tier: backend
        version: "1.6.0"
    spec:
      containers:
      - name: technova-api
        image: technova/api:1.6.0
```

```
# Label দিয়ে খোঁজা
kubectl get pods -l app=technova-api -n technova-prod
kubectl get pods -l 'app in (technova-api, technova-web)' -n technova-prod
kubectl get pods -l env=prod,tier=backend -n technova-prod
kubectl get pods --show-labels -n technova-prod
kubectl label pod technova-api-abc version=1.6.0 -n technova-prod
```

```
kubectl annotate deployment technova-api description="API 1.6.0" -n technova-prod
kubectl get namespaces --show-labels
```

৯.৮ বাস্তব প্রকৌশল পরিস্থিতি

TechNova পরিস্থিতি: প্রথমে সবকিছু `default` Namespace-এ চলত। একদিন একজন developer পরীক্ষার জন্য একটি Deployment apply করলেন — সেটির selector ভুল করে prod-এর Pod-গুলোকেও মেলাল, কারণ Label একই ছিল (`app: technova-api`) আর Namespace-এর কোনো ভাগ ছিল না। ফলে dev-এর Deployment prod-এর Pod গুনতে শুরু করল, replica সংখ্যা এলোমেলো হয়ে গেল। সমাধান: তিনটি Namespace তৈরি করা হলো — `technova-dev`, `technova-staging`, `technova-prod` — এবং প্রতিটি Namespace-এ `env` label বাধ্যতামূলক করা হলো। Service-এর selector-এ এখন `env` সহ Label দেওয়া হয়, আর প্রতিটি পরিবেশের Deployment শুধু নিজের Namespace-এর Pod দেখে। এরপর থেকে ভুল Namespace-এ apply করলে Pod তৈরি হলেও prod-এর ট্রাফিক তাতে যায় না।

৯.৯ ধাপে ধাপে ব্যবহারিক উদাহরণ

লক্ষ্য: Namespace ও Label দিয়ে পরিবেশ আলাদা করা। **ধাপ ১:** দুটি Namespace তৈরি করুন: `kubectl create namespace technova-dev` ও `kubectl create namespace technova-prod`। **ধাপ ২:** একই Deployment YAML দুই Namespace-এ apply করুন — দেখুন একই নামের Pod দুই জায়গায় চলছে, কোনো সংঘাত নেই। **ধাপ ৩:** `kubectl get pods -l app=technova-api -n technova-dev` ও `-n technova-prod` দিয়ে আলাদাভাবে দেখুন। **ধাপ ৪:** একটি Pod-এ `kubectl label` দিয়ে নতুন label যোগ করুন, তারপর সেই label দিয়ে খুঁজুন। **ধাপ ৫:** Service-এর selector ইচ্ছাকৃতভাবে ভুল করুন (যেমন `app: wrong-name`) — দেখুন Service-এর endpoint খালি থাকে; তারপর ঠিক করুন।

৯.১০ সাধারণ ভুল ও সমস্যা সমাধান

সমস্যা	কারণ	সমাধান
Service-এর endpoint খালি	selector কোনো Pod-এর label-এর সাথে মিলছে না	`kubectl get endpoints` ও Pod label মিলিয়ে দেখুন
Deployment Pod গুনতে পারছে না	selector ও template label-এর অমিল	matchLabels ও template label এক করুন
দুটি Deployment একই Pod নিয়ে টানাটানি করছে	দুটি selector একই Pod মেলাচ্ছে	selector-এ অনন্য label (যেমন version বা instance) যোগ করুন
খোঁজার তথ্য Annotation-এ রাখা হয়েছে	Label ও Annotation-এর পার্থক্য না জানা	খোঁজার তথ্য Label-এ সরান
Namespace মুছতে গিয়ে আটকে আছে (Terminating)	ভেতরে finalizer-সহ resource আটকে আছে	ভেতরের resource দেখুন; জোর করে মুছবেন না, কারণ খুঁজুন
dev-এর কাজ prod-কে প্রভাবিত করছে	সবকিছু default Namespace-এ, NetworkPolicy নেই	আলাদা Namespace, ResourceQuota ও NetworkPolicy ব্যবহার করুন

৯.১১ নিরাপত্তা বিবেচনা

Namespace আলাদা করলেই নিরাপত্তা হয় না — এটি বারবার মনে রাখা দরকার। প্রতিটি Namespace-এ ResourceQuota দিয়ে সম্পদের সীমা দিন, যাতে একটি Namespace পুরো cluster-এর সম্পদ খেয়ে না ফেলে। RBAC দিয়ে নির্ধারণ করুন কোন দল কোন Namespace-এ কী করতে পারবে — developer-দের prod Namespace-এ লেখার অধিকার দেবেন না। NetworkPolicy দিয়ে নির্ধারণ করুন কোন Namespace থেকে কোন Namespace-এ ট্রাফিক যেতে পারবে — ডিফল্টভাবে সব খোলা থাকে। Secret Namespace-এর ভেতরে থাকে — এক Namespace-এর Secret অন্য Namespace থেকে সরাসরি ব্যবহার করা যায় না, এটা ভালো বৈশিষ্ট্য, কাজে লাগান।

৯.১২ কর্মক্ষমতা ও খরচ বিবেচনা

Namespace নিজে প্রায় কোনো খরচ করে না — এটি শুধু একটি সংগঠন। কিন্তু Namespace-ভিত্তিক ResourceQuota ও LimitRange দিয়ে খরচ নিয়ন্ত্রণ করা যায়: dev Namespace-এ কম সীমা, prod-এ বেশি। Label-এর সংখ্যা বেশি হলে etcd-তে সামান্য বাড়তি জায়গা লাগে, কিন্তু তা নগণ্য। বড় খরচের জায়গা হলো ভুল selector — Service ভুল Pod-এ ট্রাফিক পাঠালে সেটি শুধু ভুল নয়, অপ্রয়োজনীয় লোডও তৈরি করে।

৯.১৩ সেরা অনুশীলন ও অ্যান্টি-প্যাটার্ন

সেরা অনুশীলন: পরিবেশ অনুযায়ী Namespace ভাগ করুন (dev, staging, prod); প্রস্তাবিত label (`app.kubernetes.io/name` ইত্যাদি) ধারাবাহিকভাবে ব্যবহার করুন; প্রতিটি resource-এ `app`, `env` ও `team` label রাখুন; খোঁজার তথ্য Label-এ, মন্তব্য Annotation-এ রাখুন; selector তৈরির আগে Pod label যাচাই করুন।

অ্যান্টি-প্যাটার্ন: সবকিছু `default` Namespace-এ চালানো; Label ছাড়া resource তৈরি করা; দুটি Deployment-এ একই selector ব্যবহার করা; Annotation-এ খোঁজার তথ্য রাখা; Namespace-কে নিরাপত্তা সীমানা ভেবে NetworkPolicy না দেওয়া।

> **PRO TIP:** একটি labelling নীতি লিখে দলে ভাগ করে দিন — কোন label বাধ্যতামূলক, কোন মান অনুমোদিত। নীতি ছাড়া কিছুদিনেই প্রত্যেকে নিজের মতো label দিতে শুরু করবে, আর খোঁজার ব্যবস্থা ভেঙে পড়বে।

৯.১৪ অধ্যায়ের সারাংশ

Namespace cluster-কে পরিবেশ বা দল অনুযায়ী ভাগ করে — নামের সংঘাত এড়ায় ও নীতি প্রয়োগের একক দেয়, তবে এটি শক্ত নিরাপত্তা সীমানা নয়। Label হলো resource-এর key-value ট্যাগ, Selector সেই ট্যাগ দিয়ে বাছাইয়ের প্রশ্ন, আর Annotation হলো খোঁজা যায় না এমন বাড়তি মন্তব্য। Service ও Deployment-এর selector Pod-এর label-এর সাথে মিলতে হবে, নইলে ট্রাফিক বা replica গোনা ভুল হবে। সামঞ্জস্যপূর্ণ নামকরণ ও labelling নীতি বড় cluster পরিচালনার ভিত্তি।

৯.১৫ মূল প্রযুক্তিগত পরিভাষা

Namespace: Cluster-এর ভেতরে resource-কে ভাগ করার পরিসর; পরিবেশ বা দল অনুযায়ী সংগঠনের একক।

Label: Resource-এর সাথে যুক্ত key-value ট্যাগ; চিহ্নিত করা ও বাছাই করার কাজে ব্যবহৃত হয়।

Selector: Label-এর উপর প্রশ্ন; কোন resource-গুলো বাছাই করা হবে তা নির্ধারণ করে।

Annotation: Resource-এর সাথে যুক্ত বাড়তি মন্তব্য; Selector দিয়ে খোঁজা যায় না।

Cluster-scoped Resource: কোনো Namespace-এর ভেতরে নয়, পুরো cluster জুড়ে থাকা resource; যেমন Node।

৯.১৬ জ্ঞান যাচাই

১. Namespace কী? একই নামের Pod দুটি Namespace-এ থাকতে পারে কেন?
২. Namespace কেন শক্ত নিরাপত্তা সীমানা নয়? সত্যিকারের বিচ্ছিন্নতার জন্য আর কী দরকার?
৩. Label ও Annotation-এর পার্থক্য লিখুন।
৪. Equality-based ও set-based selector-এর উদাহরণ দিন।
৫. Service-এর endpoint খালি থাকলে আপনি কীভাবে তদন্ত করবেন?
৬. Deployment-এর selector ও Pod template-এর label-এর সম্পর্ক কী?
৭. দুটি Deployment একই Pod মেলালে কী সমস্যা হয়?
৮. প্রস্তাবিত label (`app.kubernetes.io/name` ইত্যাদি) ব্যবহারের সুবিধা কী?

৯.১৭ ব্যবহারিক অ্যাসাইনমেন্ট

TechNova-র জন্য তিনটি Namespace তৈরি করুন — `technova-dev`, `technova-staging`, `technova-prod` — প্রতিটিতে `env` label দিন। প্রতিটি Namespace-এ `technova-api` Deployment-এর একটি কপি চালান, প্রতিটিতে `app`, `env`, `team` ও `version` label দিন। তারপর `kubectl get pods -l` দিয়ে দেখান: (ক) শুধু prod-এর Pod, (খ) সব Namespace মিলিয়ে `app=technova-api` Pod, (গ) `env` label-এর মান অনুযায়ী ভাগ। ফলাফল ও আপনার labelling নীতির এক পৃষ্ঠার বিবরণ জমা দিন।

অধ্যায় ১০ • PART II — Workloads & Application Management

Chapter 10 — ConfigMaps, Secrets, and Application Configuration

১০.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি ব্যাখ্যা করতে পারবেন কেন অ্যাপের কনফিগারেশন কোড বা image থেকে আলাদা রাখা হয়, ConfigMap ও Secret-এর পার্থক্য কী, এবং প্রতিটি কখন ব্যবহার করতে হয়। আপনি ConfigMap-কে environment variable হিসেবে ও ভলিউম হিসেবে মাউন্ট করতে পারবেন, Secret-এর ধরনগুলো (Opaque, TLS, docker-registry) চিনতে পারবেন, এবং immutable ConfigMap-এর সুবিধা বুঝবেন। সবচেয়ে গুরুত্বপূর্ণভাবে, আপনি জানবেন Secret ডিফল্টভাবে etcd-তে এনক্রিপ্ট করা থাকে না — cluster কনফিগারেশন ছাড়া Secret স্বয়ংক্রিয়ভাবে সুরক্ষিত নয় — এবং Secret কখনো Git-এ commit করা যায় না। ব্যবহারিকভাবে আপনি TechNova-র API-র জন্য নিরাপদ কনফিগারেশন লিখতে পারবেন।

১০.২ কেন এই বিষয়টি গুরুত্বপূর্ণ

একই অ্যাপ dev, staging ও prod-এ চলে, কিন্তু প্রতিটি পরিবেশের কনফিগারেশন আলাদা: ডেটাবেসের ঠিকানা, API-র URL, লগের মাত্রা, ফিচার ফ্ল্যাগ। এই মানগুলো কোডে লিখে রাখলে প্রতিটি পরিবেশের জন্য আলাদা image বানাতে হয় — ভুল হওয়ার সম্ভাবনা বাড়ে, আর একই image সব পরিবেশে চলে না। আরও খারাপ: পাসওয়ার্ড বা API key কোডে লিখলে সেটি Git-এ চলে যায়, সেখান থেকে সবার কাছে পৌঁছে যায়, আর মুছে ফেললেও Git ইতিহাসে থেকে যায়। ConfigMap ও Secret এই সমস্যার সমাধান: কনফিগারেশন image থেকে আলাদা থাকে, পরিবেশ অনুযায়ী বদলানো যায়, আর সংবেদনশীল মান আলাদাভাবে সুরক্ষিত থাকে। প্রোডাকশনে সবচেয়ে বড় নিরাপত্তা দুর্ঘটনাগুলোর অনেকগুলোই শুরু হয় Git-এ ফাঁস হওয়া Secret থেকে।

১০.৩ পূর্বশর্ত জ্ঞান

অধ্যায় ৬-এর Pod ও environment variable, অধ্যায় ৯-এর Namespace (Secret ও ConfigMap Namespace-এর ভেতরে থাকে), এবং YAML-এর ধারণা এখানে ভিত্তি। Base64 এনকোডিংয়ের প্রাথমিক ধারণা Secret বুঝতে সাহায্য করবে — তবে মনে রাখবেন, Base64 এনকোডিং এনক্রিপশন নয়, শুধু উপস্থাপনের ধরন।

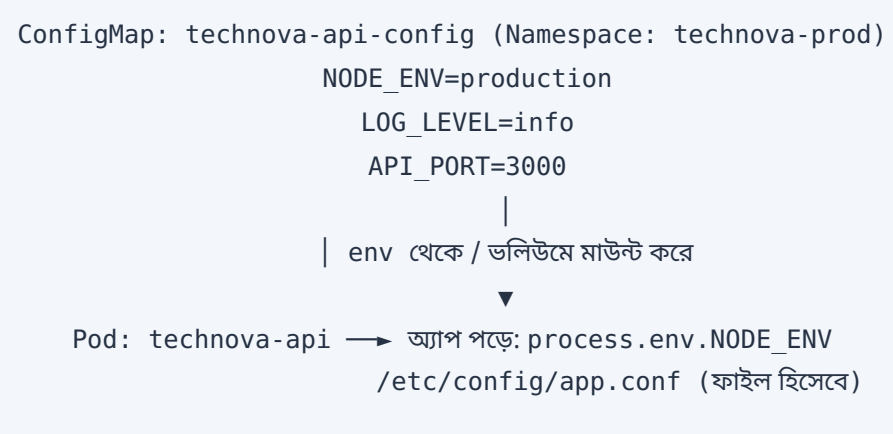
১০.৪ প্রথম নীতি থেকে ব্যাখ্যা

প্রথম নীতি থেকে ভাবুন: একটি অ্যাপের দুটি অংশ আছে — যা বদলায় না (কোড, লজিক) এবং যা পরিবেশ অনুযায়ী বদলায় (ঠিকানা, পাসওয়ার্ড, সীমা)। কোডকে image-এ বন্দি করা যায়, কিন্তু পরিবেশের মান image-এ বন্দি করলে image আর সর্বজনীন থাকে না। তাই নীতিটি হলো: **কোড ও কনফিগারেশন আলাদা রাখুন।** একই image dev-এ এক কনফিগারেশন নিয়ে চলে, prod-এ আরেক কনফিগারেশন নিয়ে — image বদলায় না, শুধু কনফিগারেশন বদলায়।

এরপর দ্বিতীয় নীতি: সব কনফিগারেশন সমান সংবেদনশীল নয়। ডেটাবেসের হোস্টনাম জানলে ক্ষতি কম; ডেটাবেসের পাসওয়ার্ড জানলে পুরো ডেটা হাতছাড়া হয়। তাই দুটি আলাদা ধারক দরকার: সাধারণ কনফিগারেশনের জন্য ConfigMap, সংবেদনশীল মানের জন্য Secret। Secret-কে আলাদা রাখার উদ্দেশ্য হলো তাকে আলাদা নিয়মে সুরক্ষিত করা — কে দেখতে পারবে, কোথায় সংরক্ষিত হবে, কীভাবে ঘোরানো (rotate) হবে।

> **CONCEPT:** Twelve-Factor App নীতি বলে — কনফিগারেশন পরিবেশ (environment) থেকে আসবে, কোড থেকে নয়। ConfigMap ও Secret হলো Kubernetes-এ সেই নীতির বাস্তবায়ন।

১০.৫ আর্কিটেকচার / ধারণাগত চিত্র



চিত্র ১০.১: ConfigMap ও Secret থেকে Pod-এ কনফিগারেশন প্রবাহ; Secret কখনো Git-এ যায় না।

১০.৬ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

১০.৬.১ ConfigMap: সাধারণ কনফিগারেশন

ConfigMap হলো key-value জোড়া বা সম্পূর্ণ ফাইলের সংগ্রহ — যেমন `NODE_ENV=production`, কিংবা একটি `nginx.conf` ফাইলের সম্পূর্ণ বিষয়বস্তু। ConfigMap Pod-এ দুইভাবে দেওয়া যায়: **environment variable হিসেবে** (একটি key বা সব key একসাথে `envFrom` দিয়ে) এবং **ভলিউম হিসেবে মাউন্ট করে** (প্রতিটি key একটি ফাইল হয়)। Env variable সহজ, কিন্তু Pod চালু হওয়ার পর ConfigMap বদলালে Pod-এর env বদলায় না — Pod পুনরায় চালু করতে হয়। ভলিউম হিসেবে মাউন্ট করলে ConfigMap বদলালে ফাইলটি স্বয়ংক্রিয়ভাবে হালনাগাদ হয় (kubelet নিয়মিত সিঙ্ক করে, সাধারণত কয়েক মিনিটের মধ্যে), তবে অ্যাপকে সেই ফাইল আবার পড়তে জানতে হয়।

ConfigMap-এ `immutable: true` দিলে সেটি তৈরি হওয়ার পর আর বদলানো যায় না। সুবিধা: ভুল করে বদলে ফেলার ঝুঁকি নেই, আর kubelet-কে বারবার দেখতে হয় না বলে কর্মক্ষমতা ভালো হয়। অসুবিধা: বদলাতে চাইলে নতুন ConfigMap

তৈরি করে Pod-কে সেটির দিকে নির্দেশ করতে হয়। ঘন ঘন বদলানো কনফিগারেশনে immutable ব্যবহার করবেন না; স্থিতিশীল কনফিগারেশনে এটি ভালো।

১০.৬.২ Secret: সংবেদনশীল মান

Secret-ও key-value, কিন্তু সংবেদনশীল মানের জন্য — পাসওয়ার্ড, API key, টোকেন, সার্টিফিকেট। Secret-এর ধরন (`type`) আছে: `Opaque` (সাধারণ Secret, ডিফল্ট), `kubernetes.io/tls` (TLS সার্টিফিকেট ও key-র জন্য — `tls.crt` ও `tls.key`), `kubernetes.io/dockerconfigjson` (private registry-তে লগইনের জন্য, `imagePullSecrets`-এ ব্যবহৃত হয়)। Secret-এর মান etcd-তে Base64 এনকোড করা অবস্থায় সংরক্ষিত হয় — **এটি এনক্রিপশন নয়।** Base64 যে কেউ ডিকোড করতে পারে; এটি শুধু বাইনারি ডেটা লেখায় রাখার উপায়।

> **WARNING:** Secret ডিফল্টভাবে etcd-তে এনক্রিপ্ট করা থাকে না। Cluster-এ encryption at rest কনফিগার না করা থাকলে etcd-তে প্রবেশ করতে পারা যে কেউ Secret পড়তে পারে। প্রোডাকশন cluster-এ অবশ্যই encryption at rest চালু করুন, etcd-তে প্রবেশ সীমিত করুন, এবং Secret দেখার RBAC অনুমতি কঠোরভাবে নিয়ন্ত্রণ করুন। Secret-কে “নিরাপদ” ভাবা সবচেয়ে বিপজ্জনক ভুল ধারণাগুলোর একটি।

১০.৬.৩ Env injection বনাম ভলিউম মাউন্ট

বিষয়	Env Variable	ভলিউম মাউন্ট
ব্যবহারের সহজতা	খুব সহজ; অ্যাপ `process.env` থেকে পড়ে	ফাইল পড়তে হয়
হালনাগাদ	Pod পুনরায় চালু না করলে বদলায় না	ফাইল স্বয়ংক্রিয়ভাবে হালনাগাদ হয়
Secret-এর ঝুঁকি	ভুল করে লগে ছাপা হতে পারে; `/proc` থেকে দেখা যায়	ফাইলের অনুমতি নিয়ন্ত্রণ করা যায়
বড় ফাইল	অনুপযুক্ত	উপযুক্ত (যেমন কনফিগ ফাইল)

সংবেদনশীল মানের জন্য ভলিউম মাউন্ট সাধারণত নিরাপদ, কারণ env variable অ্যাপের লগে বা ত্রুটির বার্তায় ভুল করে ছাপা হয়ে যেতে পারে। তবে বেশিরভাগ অ্যাপ env variable-ই আশা করে — তাই বাস্তবে দুটোর মিশ্রণ দেখা যায়: সাধারণ কনফিগ env-তে, সংবেদনশীল মান Secret থেকে env-তে (সতর্কতার সাথে) বা ভলিউমে।

১০.৬.৪ Secret ব্যবস্থাপনার সেরা পথ

Secret কখনো Git-এ commit করবেন না — এমনকি private repository-তেও নয়, কারণ Git ইতিহাস থেকে মুছে ফেলা কঠিন এবং repository-তে প্রবেশ পাওয়া মাত্রই সব Secret ফাঁস হয়। বাস্তব পদ্ধতি: Secret-এর মান বাইরে সংরক্ষণ করুন (যেমন HashiCorp Vault, AWS Secrets Manager, কিংবা cloud-এর Secret ব্যবস্থাপনা) এবং deploy-এর সময় সেখান থেকে cluster-এ তৈরি করুন; অথবা External Secrets Operator-এর মতো টুল দিয়ে বাইরের ভান্ডার থেকে Secret স্বয়ংক্রিয়ভাবে সিঙ্ক করুন। Secret নিয়মিত ঘোরান (rotate) — পাসওয়ার্ড বদলে নতুন Secret তৈরি করুন, Pod পুনরায় চালু করুন। Sealed Secrets-এর মতো পদ্ধতিতে Secret এনক্রিপ্ট করা অবস্থায় Git-এ রাখা যায়, কিন্তু সেটি বিশেষায়িত পদ্ধতি — সাধারণ Secret সরাসরি Git-এ রাখা কখনোই নিরাপদ নয়।

১০.৭ কনফিগারেশন উদাহরণ

```
apiVersion: v1
kind: ConfigMap
```

```
metadata:
  name: technova-api-config
  namespace: technova-prod
data:
  NODE_ENV: "production"
  LOG_LEVEL: "info"
  API_PORT: "3000"
---
apiVersion: v1
kind: Secret
metadata:
  name: technova-api-secret
  namespace: technova-prod
type: Opaque
stringData:
  MONGO_PASSWORD: "CHANGE-ME-mongo-password"
  JWT_SECRET: "CHANGE-ME-jwt-secret"
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: technova-api
  namespace: technova-prod
spec:
  replicas: 3
  selector:
    matchLabels:
      app: technova-api
  template:
    metadata:
      labels:
        app: technova-api
    spec:
      containers:
      - name: technova-api
        image: technova/api:1.6.0
        envFrom:
        - configMapRef:
            name: technova-api-config
        env:
        - name: MONGO_PASSWORD
          valueFrom:
            secretKeyRef:
              name: technova-api-secret
              key: MONGO_PASSWORD
        - name: JWT_SECRET
          valueFrom:
            secretKeyRef:
              name: technova-api-secret
              key: JWT_SECRET
        volumeMounts:
        - name: app-config
          mountPath: /etc/config
          readOnly: true
      volumes:
      - name: app-config
        configMap:
          name: technova-api-config
```

```
# ConfigMap ও Secret তৈরি ও দেখা
kubectl create configmap technova-api-config --from-literal=NODE_ENV=production -n technova-prod
kubectl get configmaps -n technova-prod
kubectl get secrets -n technova-prod
kubectl describe secret technova-api-secret -n technova-prod
# Secret-এর মান দেখতে (শুধু অনুমোদিত ব্যবহারকারী):
kubectl get secret technova-api-secret -n technova-prod -o jsonpath='{.data.MONGO_PASSWORD}' | base64 -d
```

লক্ষ করুন: YAML-এ `stringData` ব্যবহার করলে মান সরাসরি লেখা যায় — Kubernetes নিজে Base64 করে নেয়। `data` ব্যবহার করলে মান নিজে Base64 করে দিতে হয়। উদাহরণের `CHANGE-ME-...` মানগুলো placeholder — আসল মান কখনো এই ফাইলে লিখবেন না।

১০.৮ বাস্তব প্রকৌশল পরিস্থিতি

TechNova পরিস্থিতি: একজন developer ভুল করে MongoDB-র পাসওয়ার্ডসহ একটি Secret YAML Git repository-তে commit করে ফেললেন। কয়েক ঘণ্টা পর সেটি ধরা পড়ল, ফাইলটি মুছে ফেলা হলো — কিন্তু Git ইতিহাসে সেটি রয়ে গেল। দলকে তৎক্ষণাৎ পাসওয়ার্ড বদলাতে হলো, সব পরিবেশে নতুন Secret তৈরি করতে হলো, এবং repository-এর ইতিহাস পরিষ্কার করতে হলো। এরপর থেকে নিয়ম করা হলো: Secret YAML কখনো Git-এ যাবে না; Secret তৈরি হয় deploy পাইপলাইনে বাইরের ভান্ডার থেকে, আর প্রতিটি commit-এ একটি স্ক্যানার পরীক্ষা করে কোনো Secret-এর ধরন (যেমন `password`, `BEGIN PRIVATE KEY`) ফাইলে আছে কি না। এছাড়া ConfigMap-এর একটি মান — `LOG_LEVEL` — ভুল করে `debug` রেখে দেওয়ায় prod-এ বিপুল লগ তৈরি হচ্ছিল; ConfigMap আলাদা থাকায় শুধু সেটি বদলে Pod পুনরায় চালু করেই সমস্যা মিটল, image বদলাতে হয়নি।

১০.৯ ধাপে ধাপে ব্যবহারিক উদাহরণ

লক্ষ্য: ConfigMap বদলে Pod-এর আচরণ বদলানো। **ধাপ ১:** উপরের ConfigMap ও Deployment apply করুন। **ধাপ ২:** Pod-এর ভেতরে ঢুকে `printenv NODE_ENV` ও `ls /etc/config` দিয়ে দেখুন env ও ফাইল দুটোই এসেছে। **ধাপ ৩:** ConfigMap-এর `LOG_LEVEL` বদলে `debug` করুন — দেখুন চলমান Pod-এর env বদলায়নি, কিন্তু `/etc/config`-এর ফাইল কিছুক্ষণ পর হালনাগাদ হয়েছে। **ধাপ ৪:** Pod মুছে নতুন Pod তৈরি হতে দিন — নতুন Pod-এর env-এ নতুন মান দেখা যাবে। **ধাপ ৫:** একটি ভুল Secret YAML Git-এ commit করার চেষ্টা না করে, বরং `kubectl create secret generic` দিয়ে হাতে Secret তৈরি করার অভ্যাস করুন।

১০.১০ সাধারণ ভুল ও সমস্যা সমাধান

সমস্যা	কারণ	সমাধান
Pod-এ env variable আসছে না	ConfigMap/Secret-এর নাম ভুল, কিংবা অন্য Namespace-এ আছে	নাম ও Namespace মিলিয়ে দেখুন; `describe pod` -এ Events দেখুন
ConfigMap বদলালেও অ্যাপ বদলায়নি	env variable Pod চালুর সময় একবারই পড়া হয়	Pod পুনরায় চালু করুন (rollout restart)
Secret Git-এ ফাঁস হয়েছে	Secret YAML commit করা হয়েছে	তৎক্ষণাৎ Secret ঘোরান (rotate); Git ইতিহাস পরিষ্কার করুন; স্ক্যানার যোগ করুন

Secret-এর মান পড়তে পারছি না	Base64 ডিকোড করতে ভুলে গেছেন	`base64 -d` দিয়ে ডিকোড করুন
ভলিউমে মাউন্ট করা ফাইল হালনাগাদ হচ্ছে না	subPath ব্যবহার করা হয়েছে (subPath-এ হালনাগাদ হয় না)	subPath ছাড়া পুরো ConfigMap মাউন্ট করুন

TROUBLESHOOTING: Pod তৈরি হতে না পারলে প্রথমে `kubectl describe pod`-এর Events দেখুন — “ConfigMap not found” বা “Secret not found” বার্তা সেখানেই থাকে। বেশিরভাগ ক্ষেত্রে নাম বা Namespace-এর ভুলই কারণ।

১০.১১ নিরাপত্তা বিবেচনা

এই অধ্যায়ের সবচেয়ে গুরুত্বপূর্ণ নিরাপত্তা বার্তাগুলো একসাথে: (১) Secret ডিফল্টভাবে etcd-তে এনক্রিপ্ট করা থাকে না — encryption at rest চালু করুন। (২) Secret কখনো Git-এ commit করবেন না, এমনকি private repo-তেও নয়। (৩) Secret দেখার RBAC অনুমতি শুধু তাদেরই দিন যাদের সত্যিই দরকার — `kubectl get secrets` করার অধিকার অনেকেরই থাকা উচিত নয়। (৪) Secret-এর মান লগে ছাপাবেন না — অ্যাপের কোডে পাসওয়ার্ড লগ করা বন্ধ করুন। (৫) Secret নিয়মিত ঘোরান, বিশেষ করে কেউ দল ছাড়লে বা ফাঁসের সন্দেহ হলে। (৬) ভলিউমে মাউন্ট করা Secret ফাইলের অনুমতি (`defaultMode`) সীমিত রাখুন।

১০.১২ কর্মক্ষমতা ও খরচ বিবেচনা

ConfigMap ও Secret নিজে প্রায় কোনো খরচ করে না — তবে খুব বড় ConfigMap (যেমন মেগাবাইটের ফাইল) প্রতিটি Pod-এ মাউন্ট হলে kubelet ও etcd-এর উপর চাপ বাড়ে। ConfigMap-এর আকার ১ মেগাবাইটের মধ্যে রাখা ভালো; বড় ফাইল image-এ বা object storage-এ রাখুন। Immutable ConfigMap ব্যবহার করলে kubelet-কে বারবার পরিবর্তন দেখতে হয় না — বড় cluster-এ এটি লক্ষণীয় সাশ্রয় দেয়। Env variable হিসেবে দেওয়া Secret Pod-এর সংখ্যা বাড়লে etcd থেকে বারবার পড়া হয় — এটিও বড় cluster-এ বিবেচনার বিষয়।

১০.১৩ সেরা অনুশীলন ও অ্যান্টি-প্যাটার্ন

সেরা অনুশীলন: কনফিগারেশন image থেকে আলাদা রাখুন; সাধারণ মান ConfigMap-এ, সংবেদনশীল মান Secret-এ রাখুন; Secret বাইরের ভান্ডার থেকে তৈরি করুন; Secret নিয়মিত ঘোরান; ভলিউম মাউন্টে Secret-এর ফাইল অনুমতি সীমিত রাখুন; immutable ConfigMap স্থিতিশীল কনফিগারেশনে ব্যবহার করুন।

অ্যান্টি-প্যাটার্ন: পাসওয়ার্ড কোডে বা image-এ লিখে রাখা; Secret Git-এ commit করা; Secret-কে “এনক্রিপ্ট করা” ভেবে নিশ্চিন্ত থাকা; ConfigMap বদলে Pod পুনরায় চালু না করে অপেক্ষা করা; একই Secret সব পরিবেশে ব্যবহার করা।

REAL WORLD: TechNova এখন প্রতিটি পরিবেশে আলাদা Secret ব্যবহার করে — dev-এর পাসওয়ার্ড prod-এ চলে না। ফলে dev-এর একটি Secret ফাঁস হলেও prod নিরাপদ থাকে। এই আলাদা করাটা শুরুতে বাড়তি কাজ মনে হলেও একবার ফাঁসের ঘটনার পর তার মূল্য সবাই বুঝেছে।

১০.১৪ অধ্যায়ের সারাংশ

ConfigMap সাধারণ কনফিগারেশন (key-value বা ফাইল) ধারণ করে, Secret সংবেদনশীল মান ধারণ করে — দুটোই Pod-এ env variable বা ভলিউম হিসেবে দেওয়া যায়। Env variable Pod চালুর সময় একবারই পড়া হয়; ভলিউমে মাউন্ট করা ফাইল হালনাগাদ হয়। Secret-এর মান Base64-এ সংরক্ষিত হয়, যা এনক্রিপশন নয় — ডিফল্টভাবে etcd-তে Secret এনক্রিপ্ট করা থাকে না, encryption at rest আলাদাভাবে চালু করতে হয়। Secret কখনো Git-এ commit করা যায় না; বাইরের Secret ব্যবস্থাপনা থেকে তৈরি ও নিয়মিত ঘোরানো উচিত।

১০.১৫ মূল প্রযুক্তিগত পরিভাষা

ConfigMap: সাধারণ কনফিগারেশন (key-value বা ফাইল) সংরক্ষণের Kubernetes resource।

Secret: সংবেদনশীল মান (পাসওয়ার্ড, key, টোকেন) সংরক্ষণের Kubernetes resource; Base64-এ সংরক্ষিত হয়।

Opaque Secret: সাধারণ Secret-এর ধরন; যেকোনো key-value মান রাখা যায়।

TLS Secret: TLS সার্টিফিকেট ও private key রাখার Secret ধরন।

Immutable ConfigMap: তৈরি হওয়ার পর আর বদলানো যায় না এমন ConfigMap; ভুল বদল ও কর্মক্ষমতা সমস্যা এড়ায়।

Encryption at Rest: etcd-তে সংরক্ষিত ডেটা এনক্রিপ্ট করে রাখার cluster কনফিগারেশন; ডিফল্টভাবে চালু থাকে না।

Secret Rotation: নিয়মিত Secret-এর মান বদলে নতুন মান ব্যবহার করা; ফাঁসের ঝুঁকি কমায়।

১০.১৬ জ্ঞান যাচাই

১. কনফিগারেশন কোড থেকে আলাদা রাখা কেন জরুরি?
২. ConfigMap ও Secret-এর পার্থক্য লিখুন।
৩. ConfigMap Pod-এ দেওয়ার দুটি উপায় কী? প্রতিটির সুবিধা ও অসুবিধা লিখুন।
৪. Base64 এনকোডিং কেন এনক্রিপশন নয়?
৫. “Secret ডিফল্টভাবে এনক্রিপ্ট করা থাকে না” — এর অর্থ কী, এবং এর সমাধান কী?
৬. Secret Git-এ commit করলে কী কী ঝুঁকি হয়?
৭. TLS Secret ও docker-registry Secret কখন ব্যবহার করবেন?
৮. Immutable ConfigMap-এর সুবিধা কী?

১০.১৭ ব্যবহারিক অ্যাসাইনমেন্ট

TechNova-র `technova-api`-র জন্য একটি ConfigMap ও একটি Secret তৈরি করুন: ConfigMap-এ `NODE_ENV=production` ও `LOG_LEVEL=info`; Secret-এ `MONGO_PASSWORD` ও `JWT_SECRET` — মান হিসেবে শুধু `CHANGE-ME-...` placeholder ব্যবহার করুন। একটি Deployment লিখুন যা ConfigMap-কে `envFrom` দিয়ে ও Secret-কে `secretKeyRef` দিয়ে ব্যবহার করে। তারপর ConfigMap-এর একটি মান বদলে দেখান Pod পুনরায় চালু না করলে env বদলায় না। আপনার YAML ও পরীক্ষার ফলাফল এক পৃষ্ঠায় জমা দিন, সাথে লিখুন আসল Secret আপনি Git-এ না রেখে কীভাবে দিতেন।

অধ্যায় ১১ • PART II — Workloads & Application Management

Chapter 11 — Resource Requests, Limits, QoS Classes, and Horizontal Pod Autoscaler

১১.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি resource request ও limit-এর পার্থক্য, CPU-র millicore ও memory-র byte একক, এবং QoS (Quality of Service) শ্রেণি — Guaranteed, Burstable, BestEffort — ব্যাখ্যা করতে পারবেন। আপনি জানবেন Node-এ চাপ পড়লে কোন Pod আগে বাদ (evict) পড়ে, LimitRange ও ResourceQuota কীভাবে সীমা নির্ধারণ করে, এবং Horizontal Pod Autoscaler (HPA) কীভাবে চাহিদা অনুযায়ী Pod-এর সংখ্যা বাড়ায়-কমায়। ব্যবহারিকভাবে আপনি request/limit সেট করতে পারবেন, HPA কনফিগার করতে পারবেন, OOMKilled ও CPU throttling নির্ণয় করতে পারবেন, এবং request/limit বনাম প্রকৃত ব্যবহারের পার্থক্য ব্যাখ্যা করতে পারবেন।

১১.২ কেন এই বিষয়টি গুরুত্বপূর্ণ

একটি Node-এর সম্পদ সীমিত — ধরুন ৪টি CPU ও ৮ GB memory। সেখানে দশটি Pod চললে প্রত্যেকে কতটুকু পাবে? কেউ যদি সব memory খেয়ে ফেলে, বাকিরা মারা যাবে। কেউ যদি CPU বেশি ব্যবহার করে, বাকিরা ধীর হয়ে যাবে। Request ও limit ছাড়া Kubernetes জানে না কোন Pod-কে কোন Node-এ বসাবে, আর চাপ পড়লে কোন Pod-কে বাদ দেবে। ফলে একটি মাত্র ভুল কনফিগার করা Pod পুরো Node-কে অচল করে দিতে পারে। অন্যদিকে, চাহিদা বাড়লে Pod-এর সংখ্যা না বাড়ালে সেবা ধীর হয়ে যায়; চাহিদা কমলে Pod কমিয়ে খরচ বাঁচানো যায় — এই কাজটিই HPA করে। সম্পদ ব্যবস্থাপনা না জানলে প্রোডাকশন হয় অস্থির, নয়তো অপ্রয়োজনীয় ব্যয়বহুল।

TechNova-র API একদিন হঠাৎ ধীর হয়ে গেল। তদন্তে দেখা গেল একটি Pod memory limit ছাড়িয়ে OOMKilled হচ্ছে — বারবার মারা যাচ্ছে আর তৈরি হচ্ছে। আরেকটি Pod-এর CPU request এত বেশি দেওয়া ছিল যে scheduler নতুন Pod-কে Node-এ বসাতেই পারছিল না। Request, limit ও QoS বোঝা থাকলে এই দুটি সমস্যাই আগে ধরা পড়ত।

১১.৩ পূর্বশর্ত জ্ঞান

অধ্যায় ৬–৭-এর Pod ও Deployment, অধ্যায় ৯-এর Namespace (ResourceQuota Namespace-ভিত্তিক), এবং CPU ও memory-র মৌলিক ধারণা এখানে ভিত্তি। Metrics Server-এর ধারণা (HPA-র জন্য metrics সংগ্রহকারী) এই অধ্যায়ে ব্যাখ্যা করা হয়েছে। শতাংশ ও গড় হিসাবের প্রাথমিক ধারণা HPA-র লক্ষ্য (target) বুঝতে লাগবে।

১১.৪ প্রথম নীতি থেকে ব্যাখ্যা

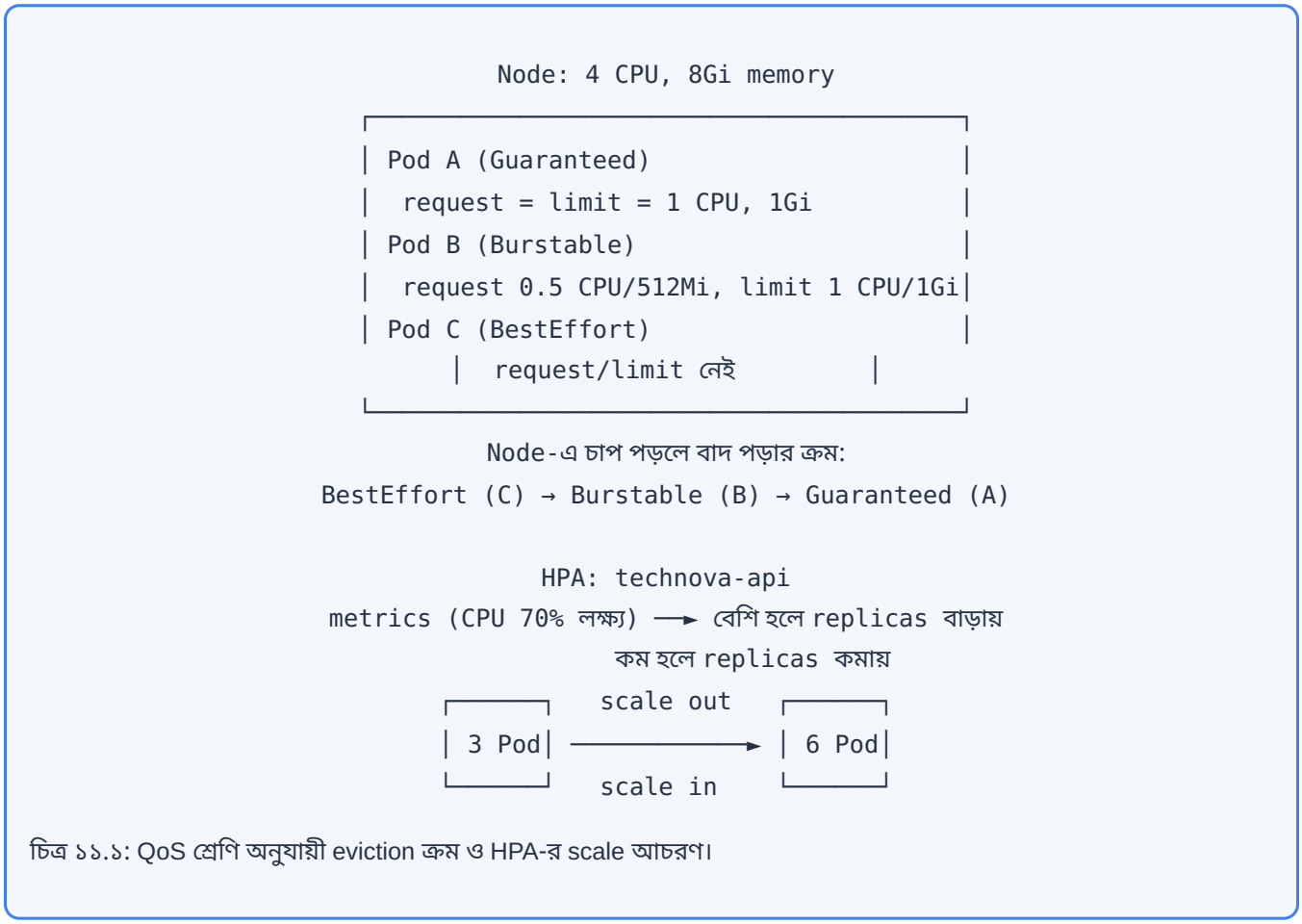
প্রথম নীতি থেকে ভাবুন: একটি Node হলো একটি ভাগ করা অ্যাপার্টমেন্ট — CPU হলো রান্নাঘরের চুলা, memory হলো ঘরের জায়গা। প্রতিটি Pod একজন বাসিন্দা। কেউ যদি বলে “আমার অন্তত ১টি চুলা ও ২৫৬ MB জায়গা চাই” — সেটাই request: scheduler সেই নিশ্চয়তা দেখে তাকে Node-এ বসায়। কেউ যদি বলে “আমি সর্বোচ্চ ২টি চুলা ও ৫১২ MB-এর বেশি ব্যবহার করব না” — সেটাই limit: এর বেশি ব্যবহার করলে তাকে থামানো হয়। Request হলো ন্যূনতম নিশ্চয়তা, limit হলো সর্বোচ্চ সীমা। দুটোর মধ্যে ফাঁকা জায়গা হলো “ধার করা” সম্পদ — Node-এ ফাঁকা থাকলে Pod সেটি ব্যবহার করতে পারে, কিন্তু চাপ পড়লে সেই বাড়তি অংশটুকুই প্রথমে কেড়ে নেওয়া হয়।

এই ধারণা থেকেই QoS শ্রেণি আসে। যে Pod-এর প্রতিটি কনটেইনারে request ও limit সমান — সে Guaranteed: তার সম্পদ নিশ্চিত। যে Pod-এর request ও limit আলাদা, কিংবা শুধু একটিতে দেওয়া আছে — সে Burstable: প্রয়োজনে

বাড়তি নিতে পারে, চাপে ছাড়তে হয়। যে Pod-এর কোনো request/limit নেই — সে BestEffort: চাপ পড়লে সবার আগে বাদ পড়ে। HPA এই ব্যবস্থার উপরে বসে: সে দেখে Pod-গুলো গড়ে কতটুকু CPU ব্যবহার করছে, লক্ষ্যের বেশি হলে Pod বাড়ায়, কম হলে কমায়।

> **CONCEPT:** Request হলো "আমার এইটুকু চাই-ই চাই" (scheduler-এর প্রতিশ্রুতি), limit হলো "এর বেশি আমি নেব না" (সুরক্ষার সীমা)। Request দিয়ে scheduling হয়, limit দিয়ে সুরক্ষা হয় — দুটো আলাদা কাজ।

১১.৫ আর্কিটেকচার / ধারণাগত চিত্র



চিত্র ১১.১: QoS শ্রেণি অনুযায়ী eviction ক্রম ও HPA-র scale আচরণ।

১১.৬ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

১১.৬.১ Requests বনাম Limits: CPU ও Memory

CPU-র একক হলো millicore: ১ CPU = ১০০০m। `500m` মানে অর্ধেক CPU। CPU হলো সংকোচনযোগ্য (compressible) সম্পদ — limit ছাড়িয়ে গেলে Pod-কে থামানো হয় না, শুধু ধীর করা হয় (throttling)। Memory-র একক byte: `Mi` (mebibyte, ১০২৪×১০২৪) ও `Gi` ব্যবহার করা হয়। Memory অসংকোচনযোগ্য — limit ছাড়িয়ে গেলে Pod-কে মেরে ফেলা হয়, যার নাম **OOMKilled** (Out Of Memory Killed)। এই পার্থক্যটি মনে রাখা জরুরি: CPU limit বেশি হলে অ্যাপ ধীর হয়, memory limit বেশি হলে অ্যাপ মারা যায়।

Scheduler শুধু request দেখে সিদ্ধান্ত নেয়: Node-এ যতটুকু request-এর জায়গা ফাঁকা আছে, ততটুকুই বরাদ্দ দেওয়া যায়। Limit scheduler দেখে না — তাই সব Pod-এর request-এর যোগফল Node-এর ধারণক্ষমতার বেশি হলে নতুন Pod

Pending-এ আটকে যায়, এমনকি Node-এ আসলে ফাঁকা CPU থাকলেও। এটাই overcommit ও undercommit-এর মূল: request কম দিলে Node-এ বেশি Pod বসে (overcommit — ঝুঁকি বেশি), request বেশি দিলে Node ফাঁকা থাকে (undercommit — খরচ বেশি)।

১১.৬.২ QoS Classes ও Eviction ক্রম

Kubernetes প্রতিটি Pod-কে তিনটি শ্রেণির একটিতে ফেলে। **Guaranteed:** প্রতিটি কনটেইনারে CPU ও memory-র request ও limit সমান এবং দুটোই দেওয়া আছে। **Burstable:** অন্তত একটি কনটেইনারে request বা limit দেওয়া আছে, কিন্তু Guaranteed-এর শর্ত মেলে না। **BestEffort:** কোনো কনটেইনারে কোনো request বা limit নেই। Node-এ memory বা disk-এর চাপ (pressure) পড়লে kubelet প্রথমে BestEffort Pod বাদ দেয়, তারপর Burstable, সবশেষে Guaranteed। তাই গুরুত্বপূর্ণ Pod-কে Guaranteed করা উচিত — তবে সব Pod Guaranteed করলে নমনীয়তা (flexibility) কমে যায়, কারণ বাড়তি সম্পদ ধার নেওয়ার সুযোগ থাকে না।

QoS	শর্ত	Eviction-এ অগ্রাধিকার
Guaranteed	সব কনটেইনারে request = limit (CPU ও memory)	সবার শেষে বাদ পড়ে
Burstable	কিছু request/limit আছে, কিন্তু সমান নয়	মাঝামাঝি
BestEffort	কোনো request/limit নেই	সবার আগে বাদ পড়ে

১১.৬.৩ LimitRange ও ResourceQuota

LimitRange একটি Namespace-এর ভেতরে প্রতিটি Pod বা কনটেইনারের ডিফল্ট ও সর্বোচ্চ সীমা নির্ধারণ করে — যেমন, request না দিলে ডিফল্ট ১০০m CPU দেওয়া হবে, limit ১ CPU-র বেশি দেওয়া যাবে না। এতে কেউ ভুল করে request/limit না দিলেও Pod BestEffort হয়ে যায় না। **ResourceQuota** পুরো Namespace-এর মোট সীমা নির্ধারণ করে — যেমন, `technova-dev` Namespace মোট ৪ CPU ও ৮Gi-র বেশি ব্যবহার করতে পারবে না। Quota পেরিয়ে গেলে নতুন Pod তৈরি হতে দেওয়া হয় না। দুটো একসাথে ব্যবহার করলে এক Namespace অন্য Namespace-এর সম্পদ খেয়ে ফেলতে পারে না।

১১.৬.৪ Horizontal Pod Autoscaler (HPA)

HPA Deployment বা StatefulSet-এর replica সংখ্যা স্বয়ংক্রিয়ভাবে বাড়ায়-কমায়। `autoscaling/v2` API-তে HPA CPU, memory, কিংবা নিজস্ব (custom) metric-এর উপর ভিত্তি করে কাজ করতে পারে। সাধারণ ব্যবহার: “CPU ব্যবহার গড়ে ৭০% রাখো” — ব্যবহার ৭০%-এর বেশি হলে replica বাড়ে, কম হলে কমে। HPA প্রতি ১৫ সেকেন্ডে (ডিফল্ট) metric দেখে, তবে scale-down ধীরে করে — হঠাৎ চাহিদা কমলেই Pod কমিয়ে দেয় না, কিছুক্ষণ অপেক্ষা করে (stabilization window) যাতে ওঠানামা (flapping) না হয়। গুরুত্বপূর্ণ: HPA CPU-র শতাংশ গণনা করে request-এর সাপেক্ষে — তাই request না দিলে HPA ঠিকভাবে কাজ করে না। HPA-র জন্য cluster-এ Metrics Server চালু থাকতে হয়; না থাকলে HPA metric পায় না।

HPA-র সীমা: `minReplicas` ও `maxReplicas` — Pod-এর সংখ্যা এই সীমার বাইরে যাবে না। HPA শুধু Pod-এর সংখ্যা বদলায়, Pod-এর আকার (request/limit) বদলায় না — সেটি Vertical Pod Autoscaler-এর কাজ। আর HPA দ্রুত বাড়তে পারে, কিন্তু Pod তৈরি হতে সময় লাগে (image টানা, probe) — তাই হঠাৎ ট্রাফিকের ঝড়ে HPA একা যথেষ্ট নয়, আগে থেকে কিছু বাড়তি সক্ষমতা রাখা ভালো।

১১.৬.৫ Request/Limit বনাম প্রকৃত ব্যবহার

Request ও limit হলো ঘোষণা — প্রকৃত ব্যবহার আলাদা জিনিস। একটি Pod request ৫০০m CPU দিয়েও আসলে ১০০m ব্যবহার করতে পারে (অপচয়), কিংবা limit ১ CPU দিয়েও বারবার throttling-এর শিকার হতে পারে। সঠিক মান বের করতে `kubectl top pods` দিয়ে প্রকৃত ব্যবহার দেখুন, কয়েকদিন পর্যবেক্ষণ করুন, তারপর request প্রকৃত ব্যবহারের কাছাকাছি (সামান্য বাড়তি সহ) সেট করুন। Memory-র ক্ষেত্রে limit প্রকৃত সর্বোচ্চ ব্যবহারের চেয়ে সামান্য বেশি রাখুন — খুব কম রাখলে OOMKilled, খুব বেশি রাখলে অপচয়।

১১.৭ কনফিগারেশন উদাহরণ

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: technova-api
  namespace: technova-prod
spec:
  replicas: 3
  selector:
    matchLabels:
      app: technova-api
  template:
    metadata:
      labels:
        app: technova-api
    spec:
      containers:
      - name: technova-api
        image: technova/api:1.6.0
        resources:
          requests:
            cpu: "500m"
            memory: "512Mi"
          limits:
            cpu: "1000m"
            memory: "1Gi"
---
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: technova-api-hpa
  namespace: technova-prod
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: technova-api
  minReplicas: 3
  maxReplicas: 10
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 70
  behavior:
```

```
    scaleDown:
      stabilizationWindowSeconds: 300
---
apiVersion: v1
kind: LimitRange
metadata:
  name: technova-limits
  namespace: technova-prod
spec:
  limits:
  - default:
      cpu: "500m"
      memory: "512Mi"
    defaultRequest:
      cpu: "250m"
      memory: "256Mi"
    max:
      cpu: "2"
      memory: "2Gi"
    type: Container
---
apiVersion: v1
kind: ResourceQuota
metadata:
  name: technova-quota
  namespace: technova-prod
spec:
  hard:
    requests.cpu: "10"
    requests.memory: 20Gi
    limits.cpu: "20"
    limits.memory: 40Gi
    pods: "30"
```

```
kubectl top pods -n technova-prod
kubectl top nodes
kubectl get hpa -n technova-prod
kubectl describe hpa technova-api-hpa -n technova-prod
kubectl get resourcequota -n technova-prod
```

১১.৮ বাস্তব প্রকৌশল পরিস্থিতি

TechNova পরিস্থিতি: technova.example-এ একটি প্রচার (campaign) চলাকালে ট্রাফিক হঠাৎ তিনগুণ বেড়ে গেল। HPA CPU ৭০% লক্ষ্য দেখে replica ৩ থেকে ৯-এ বাড়িয়ে দিল — সেবা চালু থাকল। কিন্তু একটি সমস্যা দেখা গেল: নতুন Pod-গুলো তৈরি হতে সময় লাগছিল, কারণ image টানতে দেরি হচ্ছিল। দল শিখল: HPA-র সাথে সাথে image টানার গতি, readiness probe-এর সময়, এবং `minReplicas`-কে ব্যস্ত সময়ের আগে বাড়িয়ে রাখা — সবই পরিকল্পনার অংশ। আরেকদিন একটি Pod OOMKilled হলো — memory limit ৫১২Mi দেওয়া ছিল, কিন্তু অ্যাপের একটি memory leak-এর কারণে ব্যবহার ৮০০Mi ছাড়িয়ে গিয়েছিল। Limit বাড়িয়ে সাময়িক সমাধান করা হলো, কিন্তু আসল সমাধান ছিল কোডের leak ঠিক করা — limit শুধু লক্ষণ ঢাকে, রোগ সারায় না।

১১.৯ ধাপে ধাপে ব্যবহারিক উদাহরণ

লক্ষ্য: HPA-র scale আচরণ পর্যবেক্ষণ করা। **ধাপ ১:** উপরের Deployment ও HPA apply করুন, `kubectl get hpa -w` দিয়ে দেখুন বর্তমান CPU শতাংশ ও replica সংখ্যা। **ধাপ ২:** একটি পরীক্ষামূলক Pod থেকে API-তে চাপ তৈরি করুন (যেমন বারবার অনুরোধ পাঠিয়ে) — দেখুন CPU শতাংশ বাড়ছে। **ধাপ ৩:** কিছুক্ষণ অপেক্ষা করুন — HPA replica বাড়াচ্ছে কি না দেখুন। **ধাপ ৪:** চাপ বন্ধ করুন — দেখুন scale-down ধীরে হয় (stabilization window-এর কারণে)। **ধাপ ৫:** `kubectl top pods` দিয়ে প্রকৃত CPU ও memory ব্যবহার request/limit-এর সাথে তুলনা করুন। **ধাপ ৬:** ইচ্ছাকৃতভাবে একটি Pod-এর memory limit খুব কম (যেমন ৬৪Mi) সেট করে OOMKilled দেখুন — `kubectl describe pod`-এ `Reason: OOMKilled` খুঁজুন।

১১.১০ সাধারণ ভুল ও সমস্যা সমাধান

সমস্যা	কারণ	সমাধান
OOMKilled (exit 137)	memory limit ছাড়িয়ে গেছে; memory leak বা limit খুব কম	`kubectl top` দিয়ে ব্যবহার দেখুন; limit বাড়ান বা leak ঠিক করুন
CPU throttling — অ্যাপ ধীর	CPU limit কম, কিংবা Node-এ চাপ	limit বাড়ান; `kubectl top` দিয়ে যাচাই করুন
Pod Pending-এ আটকে আছে	request-এর যোগফল Node-এর ধারণক্ষমতা ছাড়িয়েছে	request কমান বা Node বাড়ান
HPA কাজ করছে না	Metrics Server নেই, কিংবা request দেওয়া হয়নি	Metrics Server যাচাই করুন; সব কনটেইনারে CPU request দিন
HPA বারবার scale up/down করছে (flapping)	stabilization window ছোট, কিংবা লক্ষ্য খুব কঠোর	scaleDown stabilization বাড়ান; লক্ষ্য শিথিল করুন
গুরুত্বপূর্ণ Pod চাপে বাদ পড়ছে	Pod BestEffort বা Burstable	গুরুত্বপূর্ণ Pod Guaranteed করুন (request = limit)

TROUBLESHOOTING: OOMKilled দেখলে প্রথমে ভাববেন না limit-ই ভুল — অনেক সময় কোডে memory leak থাকে। Limit বাড়ানো সাময়িক স্বস্তি দেয়, leak ঠিক করাই স্থায়ী সমাধান। `kubectl describe pod`-এ `Last State: Terminated, Reason: OOMKilled` খুঁজুন।

১১.১১ নিরাপত্তা বিবেচনা

Resource limit নিরাপত্তারও অংশ: একটি Pod-কে সীমাহীন সম্পদ ব্যবহার করতে দিলে সে ইচ্ছাকৃত বা অনিচ্ছাকৃতভাবে পুরো Node-কে অচল করে দিতে পারে — এটাই resource exhaustion আক্রমণের ধরন। LimitRange দিয়ে প্রতিটি কনটেইনারে বাধ্যতামূলক সীমা দিন, ResourceQuota দিয়ে Namespace-এর মোট সীমা দিন। HPA-র `maxReplicas` সীমিত রাখুন — না হলে একটি আক্রমণ বা ভুল metric Pod-এর সংখ্যা বাড়িয়ে খরচ আকাশে তুলতে পারে। Metrics Server-এর তথ্য সংবেদনশীল নয়, তবে HPA বদলানোর অধিকার সীমিত রাখুন।

১১.১২ কর্মক্ষমতা ও খরচ বিবেচনা

Request ও limit-এর সবচেয়ে বড় খরচের প্রভাব: request বেশি দিলে Node-এ কম Pod বসে — Node ফাঁকা থাকে, খরচ বাড়ে। Request কম দিলে Node-এ বেশি Pod বসে — খরচ কমে, কিন্তু চাপে Pod বাদ পড়ার ঝুঁকি বাড়ে। সঠিক ভারসাম্য বের করতে প্রকৃত ব্যবহার মাপুন (`kubectl top`, মনিটরিং) এবং request সেট করুন প্রকৃত ব্যবহারের কাছাকাছি। HPA খরচ বাঁচায় — রাতে চাহিদা কমলে Pod কমে যায় — কিন্তু `minReplicas` খুব বেশি রাখলে সেই সাশ্রয় হয় না। CPU limit খুব কম রাখলে throttling-এর কারণে অ্যাপ ধীর হয়, ফলে ব্যবহারকারী অভিজ্ঞতা খারাপ হয় — খরচ বাঁচাতে গিয়ে সেবা নষ্ট করবেন না।

১১.১৩ সেরা অনুশীলন ও অ্যান্টি-প্যাটার্ন

সেরা অনুশীলন: প্রতিটি কনটেইনারে request ও limit দিন; request প্রকৃত ব্যবহার মেপে সেট করুন; গুরুত্বপূর্ণ Pod Guaranteed করুন; HPA-তে min/max সীমা ও stabilization window সেট করুন; LimitRange ও ResourceQuota ব্যবহার করুন; নিয়মিত `kubectl top` দিয়ে ব্যবহার পর্যালোচনা করুন।

অ্যান্টি-প্যাটার্ন: request/limit একদম না দেওয়া (BestEffort); memory limit না দিয়ে OOMKilled-এর ঝুঁকি নেওয়া; CPU limit খুব কম রেখে throttling করা; HPA-তে request ছাড়া CPU শতাংশ লক্ষ্য করা; `maxReplicas` সীমাহীন রাখা; শুধু অনুমান করে request সেট করা, না মেপে।

PRO TIP: প্রথমে request = limit দিয়ে শুরু করবেন না — প্রথমে request প্রকৃত ব্যবহারের কাছাকাছি রাখুন, limit তার প্রায় দ্বিগুণ রাখুন। কয়েক সপ্তাহ পর্যবেক্ষণের পর ধীরে ধীরে সীমা শক্ত করুন। একবারে কঠোর সীমা দিলে প্রোডাকশনে অপ্রত্যাশিত সমস্যা হয়।

REAL WORLD: TechNova এখন প্রতি মাসে resource ব্যবহার পর্যালোচনা করে — কোন Pod-এর request প্রকৃত ব্যবহারের চেয়ে অনেক বেশি, কোনটির limit বারবার ছুঁয়ে যাচ্ছে। এই পর্যালোচনায় তাদের cluster খরচ প্রায় ২০% কমেছে, আর OOMKilled-এর ঘটনা প্রায় শূন্যে নেমেছে।

১১.১৪ অধ্যায়ের সারাংশ

Request হলো scheduler-এর নিশ্চয়তা, limit হলো সর্বোচ্চ সীমা। CPU limit ছাড়ালে throttling হয়, memory limit ছাড়ালে OOMKilled হয়। QoS শ্রেণি — Guaranteed, Burstable, BestEffort — নির্ধারণ করে চাপ পড়লে কোন Pod আগে বাদ পড়বে। LimitRange কনটেইনারের ডিফল্ট ও সর্বোচ্চ সীমা দেয়, ResourceQuota Namespace-এর মোট সীমা দেয়। HPA (`autoscaling/v2`) metric-এর ভিত্তিতে replica সংখ্যা বাড়ায়-কমায়, তবে request ছাড়া CPU শতাংশ ঠিকভাবে কাজ করে না। সঠিক request/limit বের করতে প্রকৃত ব্যবহার মাপতে হয়, অনুমান করে নয়।

১১.১৫ মূল প্রযুক্তিগত পরিভাষা

Request: Pod-এর জন্য নিশ্চিতভাবে বরাদ্দ করা CPU/memory; scheduler এই মান দেখে Node বাছাই করে।

Limit: Pod সর্বোচ্চ কত CPU/memory ব্যবহার করতে পারবে তার সীমা; ছাড়ালে throttling বা OOMKilled হয়।

QoS Class: Pod-এর সেবার মানের শ্রেণি — Guaranteed, Burstable, BestEffort; eviction ক্রম নির্ধারণ করে।

OOMKilled: Memory limit ছাড়িয়ে যাওয়ায় Pod-কে মেরে ফেলা; exit code ১৩৭।

CPU Throttling: CPU limit ছাড়িয়ে গেলে Pod-কে ধীর করে দেওয়া; Pod মারা যায় না, শুধু ধীর হয়।

LimitRange: Namespace-এর ভেতরে কনটেইনারের ডিফল্ট ও সর্বোচ্চ resource সীমা।

ResourceQuota: Namespace-এর মোট resource ব্যবহারের সীমা।

Horizontal Pod Autoscaler (HPA): Metric-এর ভিত্তিতে Pod-এর replica সংখ্যা স্বয়ংক্রিয়ভাবে বাড়ানো-কমানো।

১১.১৬ জ্ঞান যাচাই

১. Request ও limit-এর পার্থক্য লিখুন; scheduler কোনটি দেখে?
২. CPU limit ছাড়ালে কী হয়, আর memory limit ছাড়ালে কী হয়?
৩. তিনটি QoS শ্রেণির শর্ত লিখুন; চাপ পড়লে কোনটি আগে বাদ পড়ে?
৪. `500m` CPU ও `512Mi` memory মানে কী?
৫. LimitRange ও ResourceQuota-র পার্থক্য লিখুন।
৬. HPA কীভাবে কাজ করে? CPU শতাংশ কিসের সাপেক্ষে গণনা করা হয়?
৭. OOMKilled দেখলে আপনি কীভাবে তদন্ত করবেন?
৮. HPA-র scale-down কেন scale-up-এর চেয়ে ধীরে করা হয়?

১১.১৭ ব্যবহারিক অ্যাসাইনমেন্ট

TechNova-র `technova-api` Deployment-এর জন্য request/limit সেট করুন: CPU request ৫০০m, limit ১০০০m; memory request ৫১২Mi, limit ১Gi। তারপর একটি HPA তৈরি করুন: minReplicas ৩, maxReplicas ১০, CPU লক্ষ্য ৭০%। `kubectl top pods` দিয়ে প্রকৃত ব্যবহার মাপুন এবং এক পৃষ্ঠায় লিখুন: (ক) আপনার Pod-এর QoS শ্রেণি কোনটি এবং কেন, (খ) প্রকৃত ব্যবহার request/limit-এর তুলনায় কেমন, (গ) HPA-র বর্তমান অবস্থা (`kubectl describe hpa`-এর সারাংশ)। এছাড়া একটি LimitRange ও একটি ResourceQuota YAML লিখুন `technova-prod` Namespace-এর জন্য।

PART III • DEVOPS ENGINEERING MASTERY

Networking, Storage & Traffic Management

Kubernetes নেটওয়ার্কিং মডেল ও CNI, Service, DNS, Ingress, Gateway API, PersistentVolume, StorageClass ও CSI।

অধ্যায় ১২ • PART III — Networking, Storage & Traffic Management

Chapter 12 — Kubernetes Networking Model and CNI

১২.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: Kubernetes-এর নেটওয়ার্কিং মডেলের চারটি মৌলিক নিয়ম ব্যাখ্যা করতে; প্রতিটি `Pod`-এর নিজস্ব আইপি (IP) থাকার অর্থ ও সুবিধা বুঝতে; `CNI` (Container Network Interface) কী, কেন দরকার এবং কীভাবে কাজ করে তা বলতে; `Calico`, `Flannel` ও `Cilium`-এর মধ্যে আচরণগত পার্থক্য তুলনা করতে; `kube-proxy`-এর `iptables` ও `IPVS` মোডের পার্থক্য ব্যাখ্যা করতে; `NetworkPolicy` দিয়ে Pod-স্তরের ফায়ারওয়াল তৈরি করতে; `overlay` বনাম `underlay` নেটওয়ার্কের সিদ্ধান্ত নিতে; এবং নেটওয়ার্ক সমস্যার ধাপে ধাপে সমস্যা সমাধান (troubleshooting) করতে।

এই অধ্যায়টি Part III-এর ভিত্তি। Service, DNS, Ingress — সবই এই নেটওয়ার্ক মডেলের ওপর দাঁড়িয়ে আছে। নেটওয়ার্কিং না বুঝে Service বোঝা যায় না, তাই এখানে সময় দিন।

১২.২ কেন এই বিষয়টি গুরুত্বপূর্ণ

Docker-এ একটি কন্টেইনারের নিজস্ব আইপি থাকলেও হোস্টের বাইরে থেকে তাকে সরাসরি ডাকা যেত না — পোর্ট ম্যাপিং, NAT (Network Address Translation) আর হোস্ট পোর্টের দ্বন্দ্ব ছিল নিত্যদিনের সমস্যা। Kubernetes সেই জটিলতা দূর করার জন্য একটি সরল কিন্তু কঠোর নিয়ম চালু করেছে: প্রতিটি Pod-এর নিজস্ব আইপি থাকবে, এবং ক্লাস্টারের ভেতরে সবাই সবাইকে সরাসরি ডাকতে পারবে, কোনো NAT ছাড়া।

TechNova-র MERN অ্যাপে এর অর্থ খুবই বাস্তব। ফ্রন্টএন্ড Pod সরাসরি ব্যাকএন্ড Pod-এর আইপিতে `http://10.244.1.5:5000` ডাকতে পারে; ব্যাকএন্ড সরাসরি MongoDB Pod-কে ডাকতে পারে। কোনো পোর্ট-ম্যাপিং টেবিল মুখস্থ করতে হয় না। কিন্তু এই সরলতার পেছনে রয়েছে CNI প্লাগইন, ভার্চুয়াল ইথারনেট ইন্টারফেস, রাউটিং টেবিল, `kube-proxy` নিয়ম আর নেটওয়ার্ক পলিসি — এগুলো না জানলে প্রোডাকশনে প্রথম দিনই `Connection refused` বা `timeout` দেখে আটকে যাবেন।

> **REAL WORLD:** TechNova-র প্রথম ক্লাস্টারে ডেভেলপাররা অভিযোগ করেছিলেন, “এক নোডের Pod অন্য নোডের Pod-কে পিং করতে পারে না।” কারণ ছিল CNI প্লাগইন ইনস্টল না করেই Pod চালানো হয়েছিল — তখন Pod-গুলো `Pending` বা `ContainerCreating` অবস্থায় আটকে ছিল। CNI ইনস্টল করতেই নেটওয়ার্ক চালু হয়। এই অধ্যায়ে আপনি সেই ভুল আর করবেন না।

১২.৩ পূর্বশর্ত জ্ঞান

এই অধ্যায়ের জন্য আপনার জানা থাকা দরকার: Pod, Deployment ও Namespace-এর ধারণা (Part I–II); Linux নেটওয়ার্কিংয়ের প্রাথমিক ধারণা — আইপি অ্যাড্রেস, সাবনেট, রাউটিং টেবিল, NAT; `kubectl get pods -o wide` ও `kubectl describe` চালানোর অভ্যাস; এবং `curl` ও `ping`-এর মৌলিক ব্যবহার। CIDR (যেমন `10.244.0.0/16`) কী তা না জানলে প্রথমে সেটি ঝালিয়ে নিন — Pod CIDR ছাড়া CNI বোঝা কঠিন।

পূর্বশর্ত	কেন দরকার	কোথায় শিখেছেন
Pod ও Deployment	নেটওয়ার্কের একক হলো Pod	Part I–II

IP, সাবনেট, রাউটিং	Pod CIDR ও রাউট বোঝার জন্য	Linux নেটওয়ার্কিং
kubectl মৌলিক কমান্ড	নেটওয়ার্ক যাচাইয়ের জন্য	Part II
iptables ধারণা	kube-proxy ও NetworkPolicy বোঝার জন্য	এই অধ্যায়ে ব্যাখ্যা করা হয়েছে

১২.৪ প্রথম নীতি থেকে ব্যাখ্যা

প্রথম নীতি থেকে ভাবুন: একটি ক্লাস্টারে শত শত Pod চলছে, বিভিন্ন নোডে। প্রতিটি Pod মরতে পারে, নতুন Pod অন্য আইপি নিয়ে জন্মাতে পারে। প্রশ্ন হলো — তারা একে অপরকে কীভাবে খুঁজে পাবে ও কথা বলবে, যেন মনে হয় তারা একই মেশিনে বসে আছে?

Kubernetes-এর উত্তর চারটি নিয়মে বাঁধা। প্রথম, প্রতিটি Pod-এর নিজস্ব ক্লাস্টার-ব্যাপী অনন্য আইপি থাকে — Pod-এর ভেতরের সব কন্টেইনার সেই আইপি ভাগ করে নেয়, `localhost` দিয়ে একে অপরকে ডাকে। দ্বিতীয়, Pod-গুলো NAT ছাড়াই একে অপরের সঙ্গে কথা বলে — কোনো ঠিকানা বদল হয় না, তাই লগে যে আইপি দেখেন সেটিই আসল প্রেরক। তৃতীয়, প্রতিটি নোডের প্রতিটি Pod-কে সব নোড থেকে পৌঁছানো যায়। চতুর্থ, Pod-এর ভেতরের কন্টেইনারগুলো `localhost` দিয়ে যোগাযোগ করে, যেন একই প্রক্রিয়া।

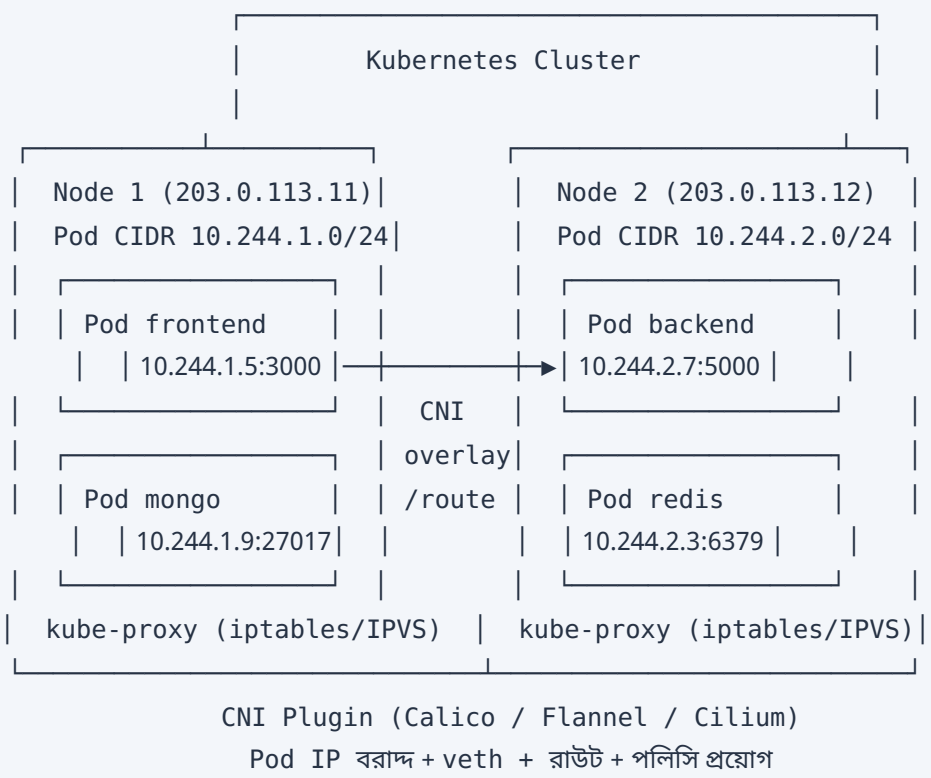
এই মডেলের সৌন্দর্য হলো, অ্যাপ্লিকেশন কোডকে জানতে হয় না সে Kubernetes-এ চলছে কি না — সে শুধু আইপি আর পোর্টে ডাকে, বাকিটা প্ল্যাটফর্মের দায়িত্ব। আর এই দায়িত্ব পালন করে CNI প্লাগইন ও `kube-proxy`।

> **CONCEPT:** Kubernetes নেটওয়ার্কিং মডেল = “সব Pod একটাই ফ্ল্যাট নেটওয়ার্কে আছে” — এই কল্পনাটি সত্যি করার দায়িত্ব CNI-এর। NAT না থাকায় ডিবাগিং সহজ: প্যাকেটের উৎস আইপি কখনো লুকানো হয় না।

CNI (Container Network Interface): কন্টেইনার রানটাইম (যেমন containerd) ও নেটওয়ার্ক প্লাগইনের মধ্যে একটি মানক ইন্টারফেস। Pod তৈরি হলে রানটাইম CNI প্লাগইনকে ডাকে — “এই Pod-কে একটি আইপি দাও, নেটওয়ার্কে যুক্ত করো।”

NAT (Network Address Translation): প্যাকেটের উৎস বা গন্তব্য আইপি বদলে দেওয়া। Kubernetes Pod-থেকে-Pod যোগাযোগে NAT হয় না, তাই পরিচয় অটুট থাকে।

১২.৫ আর্কিটেকচার / ধারণাগত চিত্র



চিত্র ১২.১ — Pod-থেকে-Pod যোগাযোগ: প্রতিটি Pod-এর নিজস্ব আইপি, CNI নোডের মধ্যে পথ তৈরি করে।

প্রতিটি নোডে একটি Pod CIDR বরাদ্দ থাকে (যেমন Node 1-এ `10.244.1.0/24`)। Pod তৈরি হলে CNI সেই পরিসর থেকে আইপি দেয়, একটি `veth` (virtual ethernet) জোড়া তৈরি করে — এক প্রান্ত Pod-এর ভেতরে (`eth0`), অন্য প্রান্ত নোডের ব্রিজ বা রাউটিং টেবিলে। অন্য নোডের Pod-এর প্যাকেট হয় `overlay` টানেল (VXLAN) দিয়ে, নয়তো সরাসরি রাউট দিয়ে গন্তব্যে পৌঁছায়।

১২.৬ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

১২.৬.১ CNI কীভাবে কাজ করে — ADD, DEL ও চেইন

Pod তৈরির সময় `kubelet` রানটাইমকে বলে Pod-এর নেটওয়ার্ক সাজাতে। রানটাইম CNI কনফিগারেশন (`/etc/cni/net.d/`) পড়ে প্লাগইন চালায়, প্যারামিটার দেয়: Pod-এর নাম, Namespace, নেটওয়ার্ক Namespace-এর পথ। প্লাগইন তখন আইপি বরাদ্দ করে (IPAM — IP Address Management), `veth` তৈরি করে, রাউট যোগ করে, প্রয়োজনে পলিসি প্রয়োগ করে। Pod মুছলে একইভাবে `DEL` অপারেশন চলে — আইপি ফেরত যায়। একাধিক প্লাগইন চেইনে চলতে পারে: যেমন Flannel আইপি ও রাউট, আর Calico পলিসি।

১২.৬.২ Calico, Flannel ও Cilium — আচরণের পার্থক্য

প্লাগইন	কীভাবে কাজ করে	শক্তি	দুর্বলতা / কখন এড়াবেন
Flannel			

	সহজ overlay (VXLAN); প্রতি নোডে সাবনেট; রাউটিং সহজ	সবচেয়ে সহজ, শেখার জন্য আদর্শ, হালকা	NetworkPolicy প্রয়োগ করে না; বড় ক্লাস্টারে সীমিত
Calico	BGP রাউটিং (underlay-সদৃশ) বা VXLAN; পূর্ণ NetworkPolicy সমর্থন	শক্তিশালী পলিসি, পারফরম্যান্স ভালো, বহুল ব্যবহৃত	সেটআপ তুলনামূলক জটিল; BGP-র জন্য নেটওয়ার্ক বোঝা লাগে
Cilium	eBPF-ভিত্তিক; কার্নেল-স্তরে দ্রুত প্যাকেট প্রক্রিয়া; L7 পলিসি, observability	অত্যন্ত দ্রুত, L7-সচেতন পলিসি, Hubble দিয়ে দৃশ্যমানতা	নতুন কার্নেল দরকার; শেখার বক্ররেখা খাড়া

নিয়ম: শেখার জন্য Flannel, প্রোডাকশনের সাধারণ পছন্দ Calico, আর আধুনিক, observability-সমৃদ্ধ ক্লাস্টারের জন্য Cilium। একবার CNI বেছে নিলে বদলানো কষ্টকর — ক্লাস্টার তৈরির আগেই সিদ্ধান্ত নিন।

১২.৬.৩ kube-proxy — iptables বনাম IPVS

Service-এর ভার্চুয়াল আইপি আসলে কোনো ইন্টারফেসে থাকে না — সেটিকে বাস্তব Pod আইপিতে বদলে দেওয়ার কাজ `kube-proxy`-এর। `iptables` মোডে এটি প্রতিটি Service-এর জন্য NAT নিয়ম লেখে; Pod সংখ্যা বাড়লে নিয়মের তালিকা লম্বা হয়, প্রতিটি প্যাকেট ক্রমানুসারে মিলিয়ে দেখতে হয় (O(n))। `IPVS` (IP Virtual Server) মোডে কার্নেলের হ্যাশ টেবিল ব্যবহার হয় — হাজার হাজার Service থাকলেও গতি প্রায় একই থাকে (O(1)), লোড ব্যালান্সিং অ্যালগরিদমও বেশি (round-robin, least-connection, source-hashing)। ছোট ক্লাস্টারে `iptables` যথেষ্ট; বড় ক্লাস্টারে IPVS বেছে নিন। আধুনিক Cilium `kube-proxy` ছাড়াই eBPF দিয়ে একই কাজ করতে পারে — একে `kube-proxy replacement` বলে।

১২.৬.৪ Overlay বনাম Underlay

`Overlay` মানে আসল প্যাকেটকে আরেকটি প্যাকেটের ভেতরে পুরে পাঠানো (VXLAN encapsulation) — হোস্ট নেটওয়ার্ক কিছু জানতে পারে না, সেটআপ সহজ, তবে সামান্য ওভারহেড থাকে। `Underlay` মানে Pod আইপি সরাসরি হোস্ট নেটওয়ার্কে রাউট করা হয় (BGP বা ক্লাউড রাউট টেবিল দিয়ে) — দ্রুত, তবে নেটওয়ার্ক টিমের সহযোগিতা লাগে। ক্লাউডে সাধারণত overlay বা ক্লাউড-নেটিভ CNI (যেমন VPC-তে Pod আইপি সরাসরি) ব্যবহৃত হয়।

১২.৬.৫ NetworkPolicy — CNI-দ্বারা প্রয়োগকৃত ফায়ারওয়াল

`NetworkPolicy` একটি Kubernetes অবজেক্ট, কিন্তু এর প্রয়োগ করে CNI প্লাগইন। Flannel-এর মতো প্লাগইন এটি উপেক্ষা করে — তাই পলিসি লিখেও কাজ না হলে প্রথমে দেখুন আপনার CNI পলিসি সমর্থন করে কি না। পলিসি ডিফল্টভাবে “সবকিছু অনুমোদিত” — একটি পলিসি Pod-কে বেছে নিলেই সেই Pod-এর জন্য অনুল্লেখিত ট্রাফিক বন্ধ হয়ে যায়। এটাই সবচেয়ে বড় ফাঁদ: একটি ingress পলিসি লাগালেন, egress ভুলে গেলেন, আর DNS-সহ সবকিছু বন্ধ হয়ে গেল।

১২.৭ কনফিগারেশন উদাহরণ

নিচে একটি NetworkPolicy দেখুন — TechNova-র `backend` Pod-কে শুধু `frontend` থেকে ডাকা যাবে, আর সে শুধু `mongo`-কে ডাকতে পারবে:

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: backend-policy
  namespace: technova
spec:
  podSelector:
    matchLabels:
```

```
      app: backend
  policyTypes:
  - Ingress
  - Egress
  ingress:
  - from:
    - podSelector:
        matchLabels:
          app: frontend
    ports:
    - protocol: TCP
      port: 5000
  egress:
  - to:
    - podSelector:
        matchLabels:
          app: mongo
    ports:
    - protocol: TCP
      port: 27017
  - to:
    - namespaceSelector:
        matchLabels:
          name: kube-system
    ports:
    - protocol: UDP
      port: 53
    - protocol: TCP
      port: 53
```

লক্ষ করুন: DNS-এর জন্য `kube-system`-এ UDP/TCP 53 খোলা রাখা হয়েছে — এটি ভুলে গেলে backend DNS রেজলভ করতে পারবে না। `kubectl get pods -o wide` দিয়ে Pod আইপি দেখুন, আর `kubectl get nodes` দিয়ে Pod CIDR যাচাই করুন।

```
kubectl get pods -n technova -o wide
kubectl get nodes -o jsonpath='{.items[*].spec.podCIDR}'
kubectl get svc -n kube-system
# kube-proxy মোড যাচাই (ক্লাস্টার অ্যাডমিনের জন্য)
kubectl -n kube-system get configmap kube-proxy -o yaml | grep -i mode
```

১২.৮ বাস্তব প্রকৌশল পরিস্থিতি

TechNova পরিস্থিতি: TechNova-র ক্লাস্টারে তিনটি নোড, CNI হিসেবে Calico। একদিন সকালে frontend Pod-গুলো backend-কে ডাকতে গিয়ে `timeout` পাচ্ছে, অথচ একই নোডের Pod-গুলো ঠিক আছে। তদন্তে দেখা গেল রাতে একজন ইঞ্জিনিয়ার একটি NetworkPolicy লাগিয়েছিলেন যাতে `podSelector`-এ ভুল লেবেল ছিল — backend-এর লেবেল `app: backend`, কিন্তু পলিসি বেছে নিয়েছিল `app: api`। ফলে পলিসি backend-কে নয়, অন্য Pod-কে বেছে নিয়েছিল, আর একটি পুরোনো `default-deny` পলিসি সব ingress বন্ধ করে দিয়েছিল। সমাধান: `kubectl get networkpolicy -n technova` দিয়ে সব পলিসি তালিকা করা, `describe` দিয়ে সিলেক্টর মেলানো, ভুল পলিসি মুছে সঠিক লেবেলে আবার লাগানো। শিক্ষা: পলিসি লাগানোর আগে `kubectl get pods --show-labels` দিয়ে লেবেল যাচাই করুন, আর প্রোডাকশনে প্রথমে audit/log মোডে পরীক্ষা করুন।

> **WARNING:** `default-deny` NetworkPolicy প্রোডাকশনে হঠাৎ লাগাবেন না — DNS, health check আর মনিটরিং ট্রাফিকসহ সবকিছু বন্ধ হয়ে যেতে পারে। প্রথমে একটি Namespace-এ পরীক্ষা করুন, তারপর ধাপে ধাপে ছড়ান।

১২.৯ ধাপে ধাপে ব্যবহারিক উদাহরণ

লক্ষ্য: দুটি Pod-এর মধ্যে যোগাযোগ যাচাই করা এবং একটি NetworkPolicy-র প্রভাব দেখা।

ধাপ ১: একটি ডিবাগ Pod চালান: `kubectl run nettest --image=busybox -n technova -- sleep 3600`। **ধাপ ২:** backend Pod-এর আইপি নিন: `kubectl get pods -n technova -o wide`। **ধাপ ৩:** ডিবাগ Pod থেকে ডাকুন: `kubectl exec -it nettest -n technova -- wget -qO- http://10.244.2.7:5000/health` (আইপি আপনার ক্লাস্টারেরটি বসান)। **ধাপ ৪:** ১২.৭-এর NetworkPolicy লাগান এবং আবার ডাকুন — এবার ব্যর্থ হওয়ার কথা, কারণ `nettest`-এর লেবেল `app: frontend` নয়। **ধাপ ৫:** পলিসি মুছুন (`kubectl delete networkpolicy backend-policy -n technova`) এবং আবার যাচাই করুন। এই পরীক্ষা আপনাকে শেখাবে পলিসি আসলে প্রয়োগ হচ্ছে কি না।

> **LAB:** আপনার ক্লাস্টারে Flannel নাকি Calico চলছে তা খুঁজে বের করুন (`kubectl -n kube-system get pods`)। তারপর উপরের পরীক্ষাটি করুন এবং ফলাফল নোট করুন।

১২.১০ সাধারণ ভুল ও সমস্যা সমাধান

লক্ষণ	সম্ভাব্য কারণ	সমাধান
Pod `ContainerCreating`-এ আটকে	CNI ইনস্টল হয়নি বা ভুল Pod CIDR	`kubectl describe pod`; CNI Pod লগ দেখুন; `kubeadm init --pod-network-cidr` মেলান
একই নোডে চলে, অন্য নোডে timeout	Overlay/রাউট ভাঙা; ফায়ারওয়াল VXLAN পোর্ট (4789) বন্ধ	নোডের মধ্যে পোর্ট খোলা কি না যাচাই; CNI লগ দেখুন
Service ডাকলে `Connection refused`	Pod-এর পোর্ট ভুল; `targetPort` মেলেনি	অধ্যায় ১৩ দেখুন; `kubectl get endpoints` যাচাই
DNS রেজলভ হয় না	NetworkPolicy পোর্ট 53 বন্ধ করেছে	পলিসিতে DNS egress অনুমতি দিন (১২.৭ দেখুন)
Pod আইপি শেষ	Pod CIDR ছোট; IPAM লিক	বড় CIDR দিয়ে ক্লাস্টার পুনর্নির্মাণের পরিকল্পনা; মুছে যাওয়া Pod-এর আইপি ফেরত যাচ্ছে কি না দেখুন

সবসময় এই ক্রমে যাচাই করুন: Pod চলছে কি না → আইপি আছে কি না → একই নোডে পিং হয় কি না → অন্য নোডে হয় কি না → Service আইপি হয় কি না → DNS হয় কি না। এতে সমস্যার স্তর দ্রুত চিহ্নিত হয়।

> **TROUBLE:** `kubectl exec` দিয়ে Pod-এর ভেতরে `ping` না থাকলে `wget` বা `curl` দিয়ে পোর্ট পরীক্ষা করুন। `busybox` ইমেজে অনেক টুল থাকে না — প্রয়োজনে `nicolaka/netshoot` ডিবাগ ইমেজ ব্যবহার করুন।

১২.১১ নিরাপত্তা বিবেচনা

ফ্ল্যাট নেটওয়ার্কের অর্থ: ডিফল্টভাবে যেকোনো Pod যেকোনো Pod-কে ডাকতে পারে — MongoDB-ও, এমনকি আপস হওয়া একটি Pod থেকেও। তাই প্রোডাকশনে `default-deny` দিয়ে শুরু করে শুধু প্রয়োজনীয় ট্রাফিক খুলুন (zero-trust পদ্ধতি)। Namespace-ভিত্তিক আলাদাকরণ করুন: `production` Namespace-এর ডাটাবেস যেন `staging` থেকে ডাকা না যায়। CNI-এর মাধ্যমে প্রয়োগ করা পলিসি নিয়মিত অডিট করুন — পলিসি আছে মানেই প্রয়োগ হচ্ছে নয়, CNI সমর্থন করছে কি না যাচাই করুন। আর Pod-এর নেটওয়ার্ক Namespace-এ হোস্ট নেটওয়ার্ক (`hostNetwork: true`) ব্যবহার এড়িয়ে চলুন, কারণ তাতে Pod হোস্টের সব পোর্ট দেখতে পায়।

১২.১২ কর্মক্ষমতা ও খরচ বিবেচনা

Overlay (VXLAN) প্রতি প্যাকেটে হেডার যোগ করে — ছোট প্যাকেটে ওভারহেড তুলনামূলক বেশি, MTU কমে যায়। তাই Pod নেটওয়ার্কের MTU সাধারণত 1450 (হোস্টের 1500 থেকে কম) রাখা হয়; ভুল MTU-তে বড় প্যাকেট ভেঙে যায় বা ড্রপ হয়। বড় ক্লাস্টারে `iptables` মোড ধীর হয় — হাজার Service-এ IPVS বা eBPF-ভিত্তিক Cilium বিবেচনা করুন। CNI-এর লগ ও মেট্রিক সংগ্রহে খরচ হয় — শুধু প্রয়োজনীয় observability চালু রাখুন। ক্লাউডে Pod আইপি সরাসরি VPC-তে বরাদ্দ হলে (যেমন AWS VPC CNI) আইপি ঠিকানা দ্রুত শেষ হতে পারে — সাবনেট পরিকল্পনায় এটি ধরুন।

১২.১৩ সেরা অনুশীলন ও অ্যান্টি-প্যাটার্ন

সেরা অনুশীলন	অ্যান্টি-প্যাটার্ন
ক্লাস্টার তৈরির আগেই CNI বেছে নিন	চলমান ক্লাস্টারে CNI বদলানোর চেষ্টা
default-deny দিয়ে শুরু, ধাপে ধাপে খুলুন	সব ট্রাফিক চিরকাল খোলা রাখা
পলিসি Git-এ সংস্করণ নিয়ন্ত্রণে রাখুন	হাতে `kubectl apply` করে ভুলে যাওয়া
Pod CIDR বড় রাখুন (প্রতি নোডে যথেষ্ট)	ছোট CIDR দিয়ে শুরু করে পরে আটকে যাওয়া
DNS egress সবসময় পলিসিতে রাখুন	পলিসি লাগিয়ে DNS বন্ধ করে ফেলা

PRO TIP: NetworkPolicy পরীক্ষার জন্য একটি আলাদা Namespace-এ প্রথমে প্রয়োগ করুন। সেখানে সবকিছু ঠিক থাকলে তবেই প্রোডাকশন Namespace-এ নিন।

১২.১৪ অধ্যায়ের সারাংশ

Kubernetes নেটওয়ার্কিং মডেলের মূল কথা: প্রতিটি Pod-এর নিজস্ব আইপি, NAT ছাড়া Pod-থেকে-Pod যোগাযোগ, সব নোড থেকে সব Pod-এ পৌঁছানো। এই মডেল বাস্তবায়ন করে CNI প্লাগইন — Flannel সহজ, Calico পলিসি-শক্তিশালী, Cilium eBPF-দ্রুত। `kube-proxy` Service আইপিকে Pod আইপিতে বদলায়; ছোট ক্লাস্টারে `iptables`, বড় ক্লাস্টারে IPVS। NetworkPolicy হলো CNI-দ্বারা প্রয়োগকৃত Pod-স্তরের ফায়ারওয়াল, তবে CNI সমর্থন না করলে তা কাজ করে না। Overlay সহজ, underlay দ্রুত — সিদ্ধান্ত নিন ক্লাস্টার তৈরির আগেই।

১২.১৫ মূল প্রযুক্তিগত পরিভাষা

CNI: Container Network Interface — Pod-কে নেটওয়ার্কে যুক্ত করার মানক ইন্টারফেস।

Pod CIDR: প্রতিটি নোডের জন্য বরাদ্দ আইপি পরিসর, যেখান থেকে Pod আইপি দেওয়া হয়।

IPAM: IP Address Management — Pod আইপি বরাদ্দ ও ফেরত নেওয়ার ব্যবস্থা।

veth: Virtual Ethernet — Pod-কে হোস্ট নেটওয়ার্কের সঙ্গে যুক্ত করা ভার্চুয়াল কেবল জোড়া।

Overlay নেটওয়ার্ক: আসল প্যাকেটকে আরেকটি প্যাকেটে পুরে পাঠানো (যেমন VXLAN); হোস্ট নেটওয়ার্কের পরিবর্তন ছাড়াই কাজ করে।

Underlay নেটওয়ার্ক: Pod আইপি সরাসরি হোস্ট/ফিজিক্যাল নেটওয়ার্কে রাউট করা হয়।

kube-proxy: প্রতি নোডে চলা কম্পোনেন্ট, যা Service-এর ভার্চুয়াল আইপিকে Pod আইপিতে ম্যাপ করে।

IPVS: IP Virtual Server — কার্নেল-ভিত্তিক দ্রুত লোড ব্যালান্সিং, বড় ক্লাস্টারের জন্য উপযোগী।

NetworkPolicy: কোন Pod কোন Pod/Service-কে ডাকতে পারবে তা নির্ধারণ করা Kubernetes অবজেক্ট; প্রয়োগ করে CNI।

eBPF: কার্নেলে নিরাপদে প্রোগ্রাম চালানোর প্রযুক্তি; Cilium এটি দিয়ে দ্রুত নেটওয়ার্কিং ও পর্যবেক্ষণ করে।

১২.১৬ জ্ঞান যাচাই

১. Kubernetes নেটওয়ার্কিং মডেলের চারটি মৌলিক নিয়ম কী কী?

২. প্রতিটি Pod-এর নিজস্ব আইপি থাকার সুবিধা কী? একই Pod-এর কন্টেইনারগুলো কীভাবে যোগাযোগ করে?

৩. CNI কী এবং Pod তৈরি হলে CNI কী কী কাজ করে?

৪. Flannel, Calico ও Cilium-এর মধ্যে মূল পার্থক্য কী? কোনটি কখন বেছে নেবেন?

৫. kube-proxy-এর iptables মোড ও IPVS মোডের পার্থক্য কী? বড় ক্লাস্টারে কোনটি ভালো?

৬. Overlay ও underlay নেটওয়ার্কের পার্থক্য ও ট্রেড-অফ কী?

৭. NetworkPolicy লাগানোর পর DNS বন্ধ হয়ে গেলে কারণ কী হতে পারে, সমাধান কী?

৮. একই নোডের Pod-গুলো যোগাযোগ করতে পারলেও অন্য নোডের Pod-এ timeout হলে আপনি কী কী যাচাই করবেন?

১২.১৭ ব্যবহারিক অ্যাসাইনমেন্ট

অ্যাসাইনমেন্ট: আপনার টেস্ট ক্লাস্টারে (বা kind/minikube-এ) TechNova-র তিনটি Pod চালান — `frontend`, `backend`, `mongo` — এবং তাদের আইপি নোট করুন। প্রথমে যাচাই করুন সব Pod একে অপরকে ডাকতে পারে। তারপর ১২.৭-এর NetworkPolicy লাগিয়ে প্রমাণ করুন শুধু `frontend` থেকেই `backend` ডাকা যায়, অন্য Pod থেকে নয়, এবং `backend` শুধু `mongo` ও DNS-কে ডাকতে পারে। প্রতিটি ধাপের কমান্ড ও ফলাফল (সফল/ব্যর্থ) একটি সংক্ষিপ্ত রিপোর্টে লিখুন, এবং ভুল লেবেল দিলে কী হয় তা পরীক্ষা করে দেখান।

অধ্যায় ১৩ • PART III — Networking, Storage & Traffic Management

Chapter 13 — Services — ClusterIP, NodePort, LoadBalancer, and ExternalName

১৩.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: Pod ক্ষণস্থায়ী হলেও Service কেন স্থায়ী ঠিকানা দেয় তা ব্যাখ্যা করতে; `ClusterIP`, `NodePort`, `LoadBalancer` ও `ExternalName` — চার ধরনের Service-এর পার্থক্য, ব্যবহার ও সীমা বলতে; `selector`, `Endpoints` ও `EndpointSlice` কীভাবে Pod-গুলোকে Service-এর সঙ্গে যুক্ত করে তা বোঝাতে; `port`, `targetPort` ও `nodePort`-এর পার্থক্য নির্ভুলভাবে বলতে; `sessionAffinity` ও `headless Service`-এর ব্যবহার জানতে; এবং Service বনাম Ingress-এর পার্থক্য স্পষ্ট করে সঠিকটি বেছে নিতে।

১৩.২ কেন এই বিষয়টি গুরুত্বপূর্ণ

অধ্যায় ১২-এ আমরা দেখেছি প্রতিটি Pod-এর নিজস্ব আইপি আছে। কিন্তু সেই আইপি স্থায়ী নয় — Pod মরলে নতুন Pod অন্য আইপি পায়, Deployment স্কেল করলে আইপির তালিকা বদলে যায়। ফ্রন্টএন্ড যদি ব্যাকএন্ডের আইপি হার্ডকোড করে রাখে, ব্যাকএন্ড Pod বদলালেই অ্যাপ ভেঙে যাবে।

Service এই সমস্যার সমাধান। Service হলো Pod-গুলোর সামনে দাঁড়ানো একটি স্থায়ী ভার্চুয়াল ঠিকানা — একটি স্থায়ী ক্লাস্টার আইপি ও একটি স্থায়ী DNS নাম। Pod আসুক-যাক, Service-এর ঠিকানা একই থাকে। TechNova-র ফ্রন্টএন্ড তাই `http://backend:5000` ডাকে — `backend` Service-এর নাম, কোনো Pod আইপি নয়। পেছনে কোন Pod আছে, কয়টি আছে, কোথায় আছে — সেসব Service নিজে সামলায়।

> **CONCEPT:** Service একটি বিমূর্ত স্তর (abstraction)। এটি নিজে কোনো প্রক্রিয়া চালায় না, কোনো পোর্ট শোনে না — এটি শুধু একটি নিয়ম: “এই নামে/আইপিতে আসা ট্রাফিক এই লেবেলের Pod-গুলোতে পাঠাও।” নিয়মটি বাস্তবায়ন করে kube-proxy।

১৩.৩ পূর্বশর্ত জ্ঞান

আপনার জানা থাকা দরকার: Pod, Deployment, ReplicaSet ও লেবেল/সিলেক্টরের ধারণা; অধ্যায় ১২-এর নেটওয়ার্কিং মডেল ও kube-proxy-এর ভূমিকা; DNS-এর প্রাথমিক ধারণা (অধ্যায় ১৪-এ বিস্তারিত আসছে); এবং `kubectl get svc`, `kubectl get endpoints` চালানোর অভ্যাস।

ধারণা	কেন দরকার
লেবেল ও সিলেক্টর	Service কোন Pod-কে বেছে নেবে তা নির্ধারণ করে
Pod আইপির ক্ষণস্থায়িত্ব	Service কেন দরকার তা বোঝার জন্য
kube-proxy	Service আইপি কীভাবে Pod-এ পৌঁছায় তা বোঝার জন্য

১৩.৪ প্রথম নীতি থেকে ব্যাখ্যা

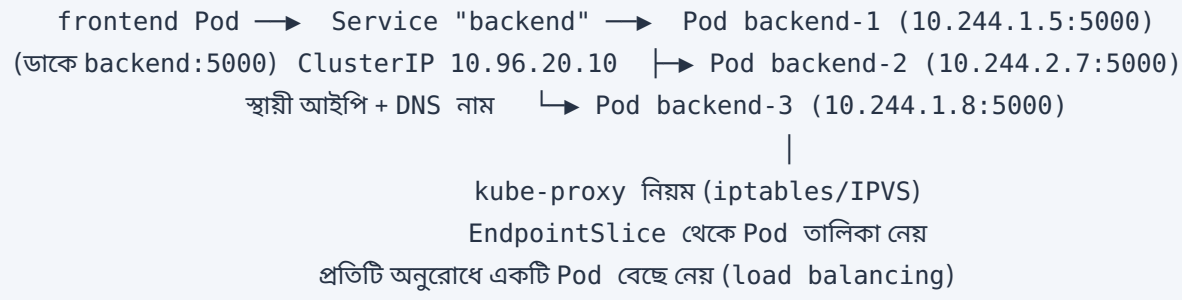
প্রথম নীতি থেকে ভাবুন: আপনার একটি সেবা দরকার — "ব্যাকএন্ড" — যার পেছনে যেকোনো সময় ২টি বা ৫টি Pod থাকতে পারে, যেকোনো নোডে। গ্রাহক (ফ্রন্টএন্ড) চায় একটি ঠিকানা, যা কখনো বদলাবে না। সমাধান: একটি স্থায়ী নাম ও আইপি তৈরি করুন, আর একটি নিয়ন্ত্রক (controller) রাখুন যে সবসময় দেখে কোন Pod-গুলো জীবিত ও প্রস্তুত, এবং সেই তালিকা নেটওয়ার্কের নিয়মে লিখে রাখে।

এটাই Service। Kubernetes-এ `Service` অবজেক্ট তৈরি করলে `kube-controller-manager`-এর Endpoint কন্ট্রোলার সিলেক্টরের সঙ্গে মেলা Pod-গুলোর আইপি সংগ্রহ করে `Endpoints` (বা আধুনিক `EndpointSlice`) অবজেক্টে লেখে। আর `kube-proxy` প্রতি নোডে সেই তালিকা পড়ে `iptables`/IPVS নিয়ম তৈরি করে। ফলে Service আইপিতে আসা প্যাকেট স্বয়ংক্রিয়ভাবে একটি জীবিত Pod-এ পৌঁছে যায়। Pod মরলে তালিকা থেকে বাদ যায়; নতুন Pod এলে যোগ হয় — সব স্বয়ংক্রিয়।

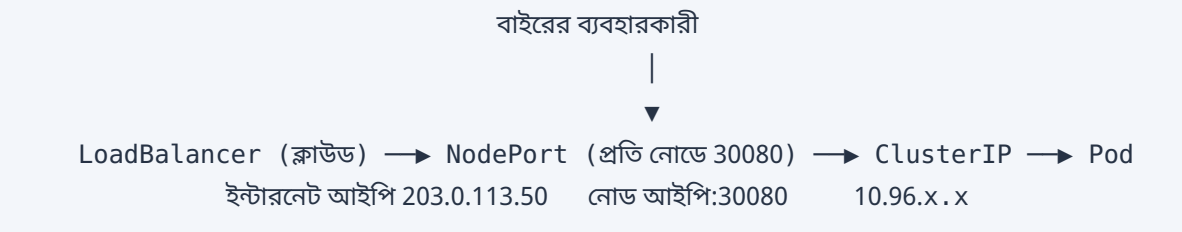
Service: Pod-গুলোর সামনে স্থায়ী ভার্চুয়াল আইপি ও DNS নাম দেওয়া Kubernetes অবজেক্ট; Pod-এর ক্ষণস্থায়িত্ব থেকে গ্রাহককে আড়াল করে।

Endpoints / EndpointSlice: Service-এর সিলেক্টরের সঙ্গে মেলা Pod-গুলোর আইপি ও পোর্টের তালিকা। EndpointSlice হলো বড় ক্লাস্টারের জন্য আরও কার্যকর নতুন সংস্করণ।

১৩.৫ আর্কিটেকচার / ধারণাগত চিত্র



চিত্র ১৩.১ — Service কীভাবে স্থায়ী ঠিকানার আড়ালে Pod-গুলোর মধ্যে ট্রাফিক ভাগ করে।



চিত্র ১৩.২ — বাইরে থেকে ভেতরে: LoadBalancer, NodePort ও ClusterIP-এর স্তর।

১৩.৬ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

১৩.৬.১ ClusterIP — ডিফল্ট, শুধু ক্লাস্টারের ভেতরে

`ClusterIP` হলো Service-এর ডিফল্ট ধরন। এটি ক্লাস্টারের Service CIDR (যেমন `10.96.0.0/12`) থেকে একটি ভার্চুয়াল আইপি পায়, যা শুধু ক্লাস্টারের ভেতর থেকে পৌঁছানো যায়। ক্লাস্টারের ভেতরের সেবাগুলোর মধ্যে যোগাযোগের জন্য এটাই সঠিক পছন্দ — ব্যাকএন্ড, ডাটাবেস, ক্যাশ — সবই ClusterIP হওয়া উচিত। বাইরে থেকে এটি ডাকা যায় না, এটাই এর নিরাপত্তা।

১৩.৬.২ NodePort — প্রতি নোডে একটি পোর্ট খোলা

`NodePort` ClusterIP-এর ওপর তৈরি: এটি প্রতিটি নোডে 30000–32767 পরিসরের একটি পোর্ট খোলে, সেই পোর্টে আসা ট্রাফিক Service-এ পাঠায়। অর্থাৎ `http://203.0.113.11:30080` ডাকলেই frontend পাওয়া যায় — যেকোনো নোডের আইপি দিয়েই। এটি সহজ, তবে পোর্ট পরিসর সীমিত, পোর্ট মনে রাখা কষ্টকর, আর সরাসরি নোড প্রকাশ করা নিরাপত্তার দিক থেকে ভালো নয়। ডেভেলপমেন্ট ও ডেমোতে উপযোগী; প্রোডাকশনে সাধারণত এর ওপর LoadBalancer বা Ingress বসানো হয়।

১৩.৬.৩ LoadBalancer — ক্লাউডের লোড ব্যালান্সার

`LoadBalancer` ধরনে Kubernetes ক্লাউড প্রোভাইডারকে (AWS, Azure, GCP বা অন-প্রিমিসে MetalLB) বলে একটি বাইরের লোড ব্যালান্সার তৈরি করতে, যার একটি পাবলিক আইপি থাকে। ব্যবহারকারী সেই আইপি ডাকে, লোড ব্যালান্সার নোডগুলোর NodePort-এ পাঠায়। সুবিধা: আসল পাবলিক ঠিকানা, ক্লাউডের স্বাস্থ্য পরীক্ষা, উচ্চ প্রাপ্যতা। অসুবিধা: প্রতিটি Service-এর জন্য একটি লোড ব্যালান্সার মানে খরচ বাড়ে, আর প্রতিটি সেবা আলাদা আইপি পায় — তাই অনেক সেবার জন্য Ingress (অধ্যায় ১৫) ভালো।

১৩.৬.৪ ExternalName — বাইরের নামের উপনাম

`ExternalName` কোনো Pod-এ ট্রাফিক পাঠায় না; এটি একটি DNS উপনাম (CNAME) তৈরি করে। যেমন `db.technova` Service-কে `mongo.technova.example`-এর দিকে নির্দেশ করা যায় — ক্লাস্টারের বাইরের ডাটাবেসকে ক্লাস্টারের ভেতরের নামে ডাকা যায়। কোড বদল না করে বাইরের সেবা বদলানোর জন্য উপযোগী। মনে রাখবেন: ExternalName-এর কোনো ClusterIP থাকে না, কোনো পোর্ট ফরওয়ার্ডিং হয় না — শুধু DNS-স্তরের নির্দেশনা।

ধরন	কে ডাকতে পারে	কখন ব্যবহার করবেন	খরচ/ঝুঁকি
ClusterIP	শুধু ক্লাস্টারের ভেতর	অভ্যন্তরীণ সেবা (backend, mongo)	কোনো খরচ নেই; সবচেয়ে নিরাপদ
NodePort	নোডে পৌঁছাতে পারে এমন যে কেউ	ডেভ/ডেমো, Ingress-এর নিচের স্তর	পোর্ট সীমিত; নোড প্রকাশ পায়
LoadBalancer	ইন্টারনেট (বা প্রাইভেট নেটওয়ার্ক)	প্রোডাকশনে একটি পাবলিক সেবা	প্রতিটি Service-এ ক্লাউড খরচ
ExternalName	ক্লাস্টারের ভেতর (DNS মাত্র)	বাইরের সেবাকে ভেতরের নাম দেওয়া	কোনো প্রক্সি নেই; DNS-নির্ভর

১৩.৬.৫ Selector, Endpoints ও EndpointSlice

Service-এর `selector` বলে কোন লেবেলের Pod-গুলো এর পেছনে থাকবে। যেমন `app: backend` সিলেক্টর মেলা সব Pod-ই Endpoint তালিকায় আসে। Pod-এর `readinessProbe` ব্যর্থ হলে Pod তালিকা থেকে সাময়িক বাদ যায় — তাই ট্রাফিক কখনো অপ্রস্তুত Pod-এ যায় না। বড় ক্লাস্টারে Endpoint অবজেক্ট একটাই বড় তালিকা হতো, যা প্রতি পরিবর্তনে

পুরো পাঠাতে হতো; `EndpointSlice` তালিকাকে ছোট ছোট টুকরোয় ভাগ করে — পরিবর্তন কম ডেটায় ছড়ায়, স্কেল ভালো হয়।

১৩.৬.৬ port, targetPort ও nodePort — তিনটি পোর্টের গল্প

এই তিনটি গুলিয়ে ফেলা সবচেয়ে সাধারণ ভুল। `port` হলো Service-এর নিজের পোর্ট — গ্রাহক এই পোর্টে Service-কে ডাকে। `targetPort` হলো Pod-এর ভেতরের পোর্ট — কন্টেইনার আসলে যে পোর্টে শুনছে। `nodePort` হলো প্রতি নোডে খোলা পোর্ট — শুধু NodePort/LoadBalancer ধরনে থাকে। উদাহরণ: Service `port: 80`, `targetPort: 3000` মানে গ্রাহক 80-এ ডাকে, Pod পায় 3000-এ।

১৩.৬.৭ Session Affinity ও Headless Service

ডিফল্টভাবে Service প্রতিটি অনুরোধ আলাদা Pod-এ পাঠাতে পারে (round-robin-সদৃশ)। `sessionAffinity: ClientIP` দিলে একই ক্লায়েন্ট আইপি থেকে আসা অনুরোধ একই Pod-এ যায় — সেশন-নির্ভর পুরোনো অ্যাপের জন্য দরকারি, তবে স্কেলিংয়ে ভারসাম্য নষ্ট হতে পারে। `headless Service` মানে `clusterIP: None` — কোনো ভার্চুয়াল আইপি নেই, DNS সরাসরি Pod-গুলোর আইপি ফেরত দেয়। StatefulSet-এর প্রতিটি Pod-কে আলাদাভাবে ডাকতে (যেমন MongoDB replica set) এটাই দরকার — বিস্তারিত অধ্যায় ১৪-এ।

১৩.৬.৮ Service বনাম Ingress

Service হলো L4-স্তরের (TCP/UDP) লোড ব্যালান্সিং — আইপি ও পোর্টে ট্রাফিক ভাগ করে, HTTP-এর ভেতরে কী আছে তা দেখে না। Ingress হলো L7-স্তরের (HTTP) রাউটিং — হোস্টনাম ও URL পাথ দেখে কোন Service-এ পাঠাবে তা ঠিক করে। তাই একটি পাবলিক ওয়েব অ্যাপে সাধারণত: Ingress (একটি পাবলিক প্রবেশপথ) → একাধিক ClusterIP Service → Pod। LoadBalancer দিয়ে প্রতিটি সেবা আলাদা করে প্রকাশ না করে Ingress দিয়ে একটি আইপিতেই অনেক সেবা চালানো যায়।

১৩.৬.৯ Service-এর ভেতরে লোড ব্যালান্সিং কীভাবে হয়

Service-এ আসা প্রতিটি সংযোগ kube-proxy-র নিয়ম অনুযায়ী একটি Pod-এ যায় — সাধারণত এলোমেলো বা round-robin-সদৃশভাবে, কোনো Pod-এর ওপর চাপ না দেখে। এর অর্থ হলো, একটি Pod ধীর হলেও সে সমান ট্রাফিক পেতে থাকে — Service নিজে Pod-এর স্বাস্থ্য বা গতি মাপে না, শুধু Pod প্রস্তুত (ready) কি না দেখে। তাই অ্যাপ-স্তরের লোড ব্যালান্সিং (যেমন ধীর Pod-কে কম ট্রাফিক দেওয়া) দরকার হলে Service যথেষ্ট নয় — তখন Ingress Controller বা service mesh-এর মতো L7 প্রক্সি দরকার। আরেকটি বিষয়: Service-এর মাধ্যমে যাওয়া সংযোগে ক্লায়েন্টের আসল আইপি হারিয়ে যেতে পারে — ClusterIP-এর ভেতরে NAT-এর কারণে Pod ক্লায়েন্টের বদলে নোডের আইপি দেখতে পারে। বাইরের ক্লায়েন্টের আসল আইপি দরকার হলে `externalTrafficPolicy: Local` ব্যবহার করুন, তবে মনে রাখবেন তাতে শুধু স্থানীয় Pod-এ ট্রাফিক যাবে।

১৩.৭ কনফিগারেশন উদাহরণ

```
apiVersion: v1
kind: Service
metadata:
  name: backend
  namespace: technova
spec:
  type: ClusterIP
  selector:
    app: backend
  ports:
```

```
  - name: http
    port: 5000
    targetPort: 5000
---
apiVersion: v1
kind: Service
metadata:
  name: frontend
  namespace: technova
spec:
  type: NodePort
  selector:
    app: frontend
  ports:
  - name: http
    port: 80
    targetPort: 3000
    nodePort: 30080
---
apiVersion: v1
kind: Service
metadata:
  name: frontend-public
  namespace: technova
spec:
  type: LoadBalancer
  selector:
    app: frontend
  ports:
  - port: 80
    targetPort: 3000
---
apiVersion: v1
kind: Service
metadata:
  name: external-db
  namespace: technova
spec:
  type: ExternalName
  externalName: mongo.technova.example
---
apiVersion: v1
kind: Service
metadata:
  name: mongo-headless
  namespace: technova
spec:
  clusterIP: None
  selector:
    app: mongo
  ports:
  - port: 27017
    targetPort: 27017
```

যাচাই করুন: `kubectl get svc -n technova`, `kubectl get endpoints -n technova`, `kubectl get endpointslices -n technova`। `Endpoints`-এ আইপি না থাকলে সিলেক্টর বা Pod-এর প্রস্তুতি (readiness) যাচাই করুন।

১৩.৮ বাস্তব প্রকৌশল পরিস্থিতি

TechNova পরিস্থিতি: TechNova-র ফ্রন্টএন্ড হঠাৎ `502 Bad Gateway` দিতে শুরু করল। Service-টি ঠিক আছে, Pod-গুলো `Running`। তদন্তে দেখা গেল নতুন Deployment-এ লেবেল বদলে `app: backend-v2` করা হয়েছে, কিন্তু Service-এর সিলেক্টর তখনো `app: backend`। ফলে `kubectl get endpoints backend -n technova`-এ কোনো আইপি নেই — Service-এর পেছনে কোনো Pod নেই। সমাধান: Deployment-এর লেবেল আগের মতো করা, অথবা Service-এর সিলেক্টর আপডেট করা। শিক্ষা: লেবেল বদলানোর আগে Service-এর সিলেক্টর মেলান; Deployment-এর `matchLabels` ও Pod টেমপ্লেটের লেবেল এক রাখুন। আরেকটি সাধারণ ঘটনা: `readinessProbe` ভুল পাথ দেওয়ায় Pod কখনো প্রস্তুত হয়নি — Endpoints খালি, Service কাজ করছে না।

> **REAL WORLD:** প্রোডাকশনে প্রতিটি Service-এর জন্য LoadBalancer তৈরি করবেন না — দশটি সেবা মানে দশটি ক্লাউড লোড ব্যালান্সার ও দশগুণ খরচ। একটি Ingress-এর পেছনে সব ClusterIP Service রাখুন (অধ্যায় ১৫)।

১৩.৯ ধাপে ধাপে ব্যবহারিক উদাহরণ

ধাপ ১: একটি সহজ Deployment চালান: `kubectl create deployment backend --image=nginx -n technova` (শেখার জন্য nginx-কে ব্যাকএন্ড হিসেবে ব্যবহার করা হচ্ছে)। **ধাপ ২:** লেবেল দেখুন: `kubectl get pods -n technova --show-labels`। **ধাপ ৩:** ClusterIP Service তৈরি করুন (১৩.৭-এর প্রথম YAML) এবং `kubectl get svc backend -n technova` দিয়ে ClusterIP নোট করুন। **ধাপ ৪:** একটি ডিবাগ Pod থেকে ডাকুন: `kubectl run curlpod --image=curlimages/curl -n technova -it --rm -- curl -s http://backend:5000` — মনে রাখবেন, এখানে পোর্ট মেলেনি বলে ব্যর্থ হতে পারে; `targetPort: 80` দিয়ে আবার চেষ্টা করুন এবং পার্থক্যটি বুঝুন। **ধাপ ৫:** NodePort Service লাগিয়ে নোড আইপি ও nodePort দিয়ে বাইরে থেকে ডাকুন। **ধাপ ৬:** `kubectl scale deployment backend --replicas=3 -n technova` করে দেখুন Endpoints কীভাবে বাড়ে।

১৩.১০ সাধারণ ভুল ও সমস্যা সমাধান

লক্ষণ	কারণ	সমাধান
Endpoints খালি	সিলেক্টর লেবেলের সঙ্গে মেলেনি	`kubectl get pods --show-labels`; সিলেক্টর ঠিক করুন
Endpoints আছে, তবু সংযোগ হয় না	targetPort ভুল; কন্টেইনার অন্য পোর্টে শুনছে	কন্টেইনারের পোর্ট যাচাই; `kubectl describe pod`
Service ডাকলে timeout	kube-proxy চলছে না; NetworkPolicy বাধা দিচ্ছে	`kubectl -n kube-system get pods`; পলিসি যাচাই
NodePort ডাকা যায় না	ফায়ারওয়াল পোর্ট বন্ধ; ভুল নোড আইপি	নোডের ফায়ারওয়াল ও `nodePort` পরিসর যাচাই
LoadBalancer-এ IP `<pending>`	ক্লাউড কন্ট্রোলার নেই; অন-প্রিমিসে MetalLB নেই	ক্লাউড প্রোভাইডার/ MetalLB যাচাই; কিছুক্ষণ অপেক্ষা
কখনো এক Pod, কখনো অন্য Pod	স্বাভাবিক লোড ব্যালান্সিং; সেশন দরকার হলে affinity দিন	`sessionAffinity: ClientIP` বিবেচনা করুন

TROUBLE: Service-এর সমস্যায় প্রথমে `kubectl get endpoints` দেখুন। Endpoints খালি মানে সমস্যা Pod/সিলেক্টরে; Endpoints ভরা থাকলে সমস্যা নেটওয়ার্ক বা kube-proxy-তে। এই একটি কমান্ড অর্ধেক সমস্যা আলাদা করে দেয়।

১৩.১১ নিরাপত্তা বিবেচনা

ডাটাবেস ও অভ্যন্তরীণ সেবা সবসময় ClusterIP রাখুন — NodePort বা LoadBalancer দিয়ে ডাটাবেস প্রকাশ করা গুরুতর ভুল। NodePort ব্যবহার করলে ফায়ারওয়ালে শুধু প্রয়োজনীয় উৎস আইপি থেকে অনুমতি দিন। LoadBalancer-এ অপ্রয়োজনীয় পোর্ট খুলবেন না; প্রয়োজনে প্রাইভেট (internal) লোড ব্যালান্সার ব্যবহার করুন। ExternalName ব্যবহারের সময় মনে রাখবেন, এটি DNS-মাত্র — বাইরের সেবার TLS ও প্রমাণীকরণ আপনার দায়িত্ব। আর Service-এর সিলেক্টর ভুল হলে ভুল Pod-এ ট্রাফিক যেতে পারে — লেবেল পরিবর্তন সতর্কতার সঙ্গে করুন।

১৩.১২ কর্মক্ষমতা ও খরচ বিবেচনা

Service-এর লোড ব্যালান্সিং কার্নেল-স্তরে হয়, তাই অতিরিক্ত হপ প্রায় নেই বললেই চলে — ClusterIP-এর ওভারহেড নগণ্য। তবে `externalTrafficPolicy: Local` বনাম `Cluster` জানুন: `Cluster` (ডিফল্ট) যেকোনো নোডের Pod-এ পাঠাতে পারে, এতে অতিরিক্ত নেটওয়ার্ক হপ হতে পারে; `Local` শুধু স্থানীয় Pod-এ পাঠায়, ফলে উৎস আইপি সংরক্ষিত থাকে, কিন্তু কোনো নোডে Pod না থাকলে ট্রাফিক ড্রপ হতে পারে। LoadBalancer-এর খরচ ক্লাউডভেদে ভিন্ন — প্রতিটি Service-এর জন্য আলাদা লোড ব্যালান্সার না বানিয়ে Ingress ব্যবহার করুন। EndpointSlice বড় ক্লাস্টারে API সার্ভারের চাপ কমায় — পুরোনো Endpoint অবজেক্টের বদলে এটিই আধুনিক পথ।

১৩.১৩ সেরা অনুশীলন ও অ্যান্টি-প্যাটার্ন

সেরা অনুশীলন	অ্যান্টি-প্যাটার্ন
অভ্যন্তরীণ সেবায় ClusterIP; পাবলিকে Ingress + একটি LoadBalancer	প্রতিটি সেবায় LoadBalancer
নামযুক্ত পোর্ট (`name: http`) ব্যবহার	শুধু সংখ্যা দিয়ে পোর্ট, পরে গুলিয়ে ফেলা
readinessProbe দিয়ে Endpoints নিয়ন্ত্রণ	প্রস্তুত না হওয়া Pod-এ ট্রাফিক পাঠানো
সিলেক্টর ও লেবেল Git-এ একসঙ্গে রাখা	হাতে লেবেল বদলে Service ভেঙে ফেলা
headless Service শুধু StatefulSet-এর জন্য	সাধারণ Deployment-এ অকারণে headless

PRO TIP: Service-এর পোর্টে নাম দিন (`name: http`)। তাহলে Deployment-এ `targetPort: http` নাম দিয়ে উল্লেখ করা যায় — পোর্ট সংখ্যা বদলালেও Service বদলাতে হয় না।

১৩.১৪ অধ্যায়ের সারাংশ

Service হলো ক্ষণস্থায়ী Pod-গুলোর সামনে স্থায়ী ঠিকানা। ClusterIP ক্লাস্টারের ভেতরের সেবার জন্য, NodePort প্রতি নোডে পোর্ট খোলে, LoadBalancer ক্লাউডের পাবলিক লোড ব্যালান্সার দেয়, ExternalName বাইরের নামের DNS উপনাম। সিলেক্টর Pod বেছে নেয়, Endpoints/EndpointSlice জীবিত Pod-এর তালিকা রাখে, kube-proxy সেই তালিকা

দিয়ে ট্রাফিক পাঠায়। `port` Service-এর, `targetPort` Pod-এর, `nodePort` নোডের। Headless Service DNS-কে সরাসরি Pod আইপি দেয়। Service L4, Ingress L7 — দুটির কাজ আলাদা।

১৩.১৫ মূল প্রযুক্তিগত পরিভাষা

ClusterIP: শুধু ক্লাস্টারের ভেতরে পৌঁছানো যায় এমন Service-এর ভার্চুয়াল আইপি; ডিফল্ট ধরন।

NodePort: প্রতিটি নোডে 30000–32767 পরিসরে খোলা পোর্ট দিয়ে Service প্রকাশের পদ্ধতি।

LoadBalancer: ক্লাউড প্রোভাইডারের বাইরের লোড ব্যালান্সার দিয়ে Service প্রকাশের ধরন।

ExternalName: বাইরের DNS নামের দিকে নির্দেশ করা Service; কোনো প্রক্সি করে না, শুধু CNAME দেয়।

Selector: Service কোন লেবেলের Pod-গুলোকে বেছে নেবে তা নির্ধারণ করা নিয়ম।

EndpointSlice: Service-এর পেছনের Pod আইপি ও পোর্টের তালিকা, ছোট টুকরোয় ভাগ করা আধুনিক সংস্করণ।

targetPort: কন্টেইনার আসলে যে পোর্টে শুনছে; Service-এর port থেকে আলাদা হতে পারে।

Headless Service: `clusterIP: None` — কোনো ভার্চুয়াল আইপি নেই; DNS সরাসরি Pod আইপি ফেরত দেয়।

Session Affinity: একই ক্লায়েন্টের অনুরোধ একই Pod-এ পাঠানোর সেটিং (`ClientIP`)।

externalTrafficPolicy: বাইরের ট্রাফিক স্থানীয় Pod-এই থাকবে (`Local`) নাকি যেকোনো Pod-এ যেতে পারবে (`Cluster`) তা নির্ধারণ করে।

১৩.১৬ জ্ঞান যাচাই

১. Pod আইপি ক্ষণস্থায়ী হলে Service কেন দরকার? Service কী সমস্যার সমাধান করে?

২. ClusterIP, NodePort, LoadBalancer ও ExternalName-এর মধ্যে পার্থক্য লিখুন এবং প্রতিটির একটি ব্যবহার লিখুন।

৩. port, targetPort ও nodePort-এর পার্থক্য কী? একটি উদাহরণ দিয়ে বোঝান।

৪. Service-এর Endpoints খালি থাকলে সম্ভাব্য কারণগুলো কী কী?

৫. Headless Service কী? এটি কখন ব্যবহার করবেন?

৬. sessionAffinity: ClientIP কখন দরকার, আর এর অসুবিধা কী?

৭. Service ও Ingress-এর পার্থক্য কী? কখন কোনটি ব্যবহার করবেন?

৮. LoadBalancer Service-এর IP `<pending>` দেখালে আপনি কী করবেন?

১৩.১৭ ব্যবহারিক অ্যাসাইনমেন্ট

অ্যাসাইনমেন্ট: TechNova-র জন্য একটি সম্পূর্ণ Service সেট তৈরি করুন: `backend` (ClusterIP, port 5000), `frontend` (NodePort 30080, port 80 → targetPort 3000), `external-db` (ExternalName → `mongo.technova.example`) এবং `mongo-headless` (clusterIP: None)। প্রতিটি Service-এর জন্য `kubectl get svc` ও `kubectl get endpoints`-এর আউটপুট সংগ্রহ করুন, একটি ডিবাগ Pod থেকে `backend`-কে নাম দিয়ে ডাকুন, এবং ইচ্ছাকৃতভাবে একটি Service-এর সিলেক্টর ভুল করে দেখান Endpoints খালি হয়ে যায় — তারপর ঠিক করুন। ফলাফল একটি সংক্ষিপ্ত রিপোর্টে লিখুন।

অধ্যায় ১৪ • PART III — Networking, Storage & Traffic Management

Chapter 14 — Kubernetes DNS and Service Discovery

১৪.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: Kubernetes-এ DNS কেন Service Discovery-র মূল ভিত্তি তা ব্যাখ্যা করতে; CoreDNS-এর আর্কিটেকচার ও কাজের ধারা বোঝাতে; Service-এর জন্য কোন DNS রেকর্ড (A, SRV) তৈরি হয় তা বলতে; একই Namespace-এর ভেতরে সংক্ষিপ্ত নাম ও অন্য Namespace থেকে FQDN ব্যবহার করতে; headless Service-এর DNS কীভাবে StatefulSet-এর Pod-গুলোকে আলাদাভাবে চেনায় তা বোঝাতে; এবং `nslookup` / `dig` দিয়ে ডিবাগ Pod থেকে DNS সমস্যা সমাধান করতে।

১৪.২ কেন এই বিষয়টি গুরুত্বপূর্ণ

অধ্যায় ১৩-এ আমরা দেখেছি Service স্থায়ী আইপি দেয়। কিন্তু আইপি মুখস্থ করা মানুষের কাজ নয় — মানুষ নাম মনে রাখে। Kubernetes-এ তাই প্রতিটি Service স্বয়ংক্রিয়ভাবে একটি DNS নাম পায়: একই Namespace-এ শুধু `backend`, অন্য Namespace থেকে `backend.technova.svc.cluster.local`। কোডে আইপি নয়, নাম লিখলেই চলে।

TechNova-র ফ্রন্টএন্ড কোডে `http://backend:5000` লেখা আছে — কোনো আইপি নেই, কোনো কনফিগ ফাইল বদলাতে হয় না, পরিবেশ (dev/staging/prod) বদলালেও নাম একই থাকে। কিন্তু DNS ভাঙলে সবকিছু ভাঙে: Pod চলছে, Service চলছে, তবু নাম রেজলভ হয় না বলে অ্যাপ “সার্ভার পাওয়া যাচ্ছে না” বলে। তাই DNS বোঝা মানে ক্লাস্টারের অর্ধেক সমস্যা সমাধান শেখা।

> **CONCEPT:** Kubernetes-এ DNS একটি সেবা, কোনো জাদু নয়। CoreDNS নামের Pod-গুলো ক্লাস্টারের ভেতরে DNS সার্ভার হিসেবে চলে; প্রতিটি Pod-এর `/etc/resolv.conf`-এ CoreDNS-এর ঠিকানা লেখা থাকে। নাম জিজ্ঞাসা করলে CoreDNS Kubernetes API থেকে উত্তর খুঁজে দেয়।

১৪.৩ পূর্বশর্ত জ্ঞান

জানা থাকা দরকার: DNS-এর প্রাথমিক ধারণা (নাম → আইপি, A রেকর্ড, FQDN); Service, Namespace ও headless Service-এর ধারণা (অধ্যায় ১৩); Pod-এর ভেতরে `kubectl exec` দিয়ে কমান্ড চালানো; এবং `/etc/resolv.conf` ও `/etc/hosts`-এর মৌলিক ধারণা।

ধারণা	কেন দরকার
DNS A ও SRV রেকর্ড	Service-এর জন্য কোন রেকর্ড তৈরি হয় তা বোঝার জন্য
Namespace	নাম কীভাবে Namespace-ভিত্তিক হয় তা বোঝার জন্য
Service ও Endpoints	DNS উত্তরের উৎস বোঝার জন্য

১৪.৪ প্রথম নীতি থেকে ব্যাখ্যা

প্রথম নীতি থেকে ভাবুন: Pod-এর আইপি বদলায়, Service-এর আইপি স্থায়ী হলেও তা মনে রাখা কঠিন। মানুষ ও কোড — দুজনেরই দরকার একটি স্থায়ী, অর্থবোধক নাম। সেই নামকে আইপিতে বদলে দেওয়ার জন্য দরকার একটি নাম-পরিষেবা, যা সবসময় ক্লাস্টারের বর্তমান অবস্থা জানে।

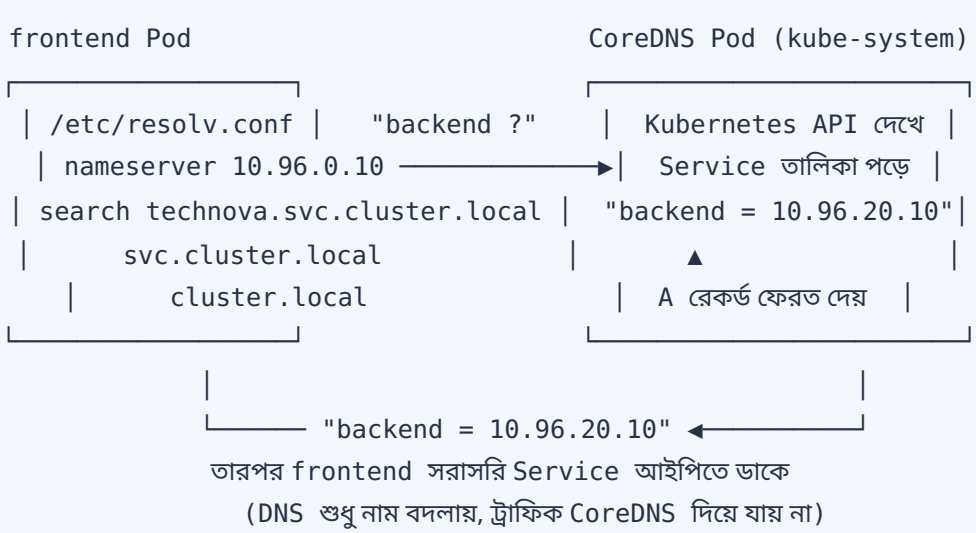
Kubernetes-এর সমাধান: ক্লাস্টারের ভেতরে একটি DNS সার্ভার (CoreDNS) চালানো, যা Kubernetes API-কে দেখে। Service তৈরি হলেই CoreDNS তার নাম জানে; Service মুছলে ভুলে যায়। Pod তৈরি হলে তার `resolv.conf`-এ CoreDNS-এর ঠিকানা ও `search` ডোমেইন লেখা হয় — তাই Pod শুধু `backend` লিখলেই CoreDNS সেটিকে `backend.technova.svc.cluster.local` হিসেবে খুঁজে নেয়। বাইরের নাম (যেমন `google.com`) হলে CoreDNS সেটিকে ক্লাস্টারের বাইরের DNS-এ পাঠিয়ে দেয় (forwarding)।

CoreDNS: Kubernetes-এর ডিফল্ট ক্লাস্টার DNS সার্ভার; Service ও Pod-এর নামকে আইপিতে বদলে দেয়। পুরোনো সিস্টেমে kube-dns ব্যবহৃত হতো, এখন CoreDNS মানক।

FQDN (Fully Qualified Domain Name): সম্পূর্ণ DNS নাম, যেমন `backend.technova.svc.cluster.local` — কোনো অনুমান ছাড়াই এককভাবে চিহ্নিত করে।

Service Discovery: একটি সেবা আরেকটি সেবাকে নাম বা রেজিস্ট্রি দিয়ে খুঁজে পাওয়ার ব্যবস্থা; Kubernetes-এ এর মূল উপায় DNS।

১৪.৫ আর্কিটেকচার / ধারণাগত চিত্র



চিত্র ১৪.১ — DNS জিজ্ঞাসার ধারা: নাম জিজ্ঞাসা CoreDNS-এ, আসল ট্রাফিক সরাসরি Service/Pod-এ।

নামের রূপ	কোথা থেকে কাজ করে
`backend`	শুধু `technova` Namespace থেকে
`backend.technova`	যেকোনো Namespace থেকে
`backend.technova.svc.cluster.local`	যেকোনো জায়গা থেকে (FQDN)

চিত্র ১৪.২ — সংক্ষিপ্ত নাম বনাম FQDN।

১৪.৬ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

১৪.৬.১ CoreDNS আর্কিটেকচার

CoreDNS `kube-system` Namespace-এ Deployment হিসেবে চলে, সাধারণত ২টি replica। এর Service-এর নাম `kube-dns`, ClusterIP সাধারণত `10.96.0.10` — এই আইপিই প্রতিটি Pod-এর `resolv.conf`-এ nameserver হিসেবে লেখা থাকে। CoreDNS-এর কনফিগারেশন `Corefile` নামের ConfigMap-এ থাকে — সেখানে `kubernetes` প্লাগইন API দেখে, `forward` প্লাগইন বাইরের নাম বাইরের DNS-এ পাঠায়, `cache` প্লাগইন উত্তর সাময়িকভাবে মনে রাখে। CoreDNS Pod মরলে নতুন Pod আসে, কিন্তু Service আইপি একই থাকে বলে Pod-গুলোর DNS ভাঙে না।

১৪.৬.২ Service-এর DNS রেকর্ড — A ও SRV

একটি সাধারণ (ClusterIP) Service `backend`-এর জন্য CoreDNS একটি A রেকর্ড দেয়: `backend.technova.svc.cluster.local → 10.96.20.10`। পোর্ট জানতে SRV রেকর্ডও তৈরি হয়: `_http._tcp.backend.technova.svc.cluster.local` — এতে পোর্ট নম্বর থাকে। ক্লায়েন্ট সাধারণত A রেকর্ডই ব্যবহার করে; SRV দরকার হয় যখন পোর্টও DNS থেকে জানতে হয় (যেমন কিছু ডাটাবেস ক্লায়েন্ট)।

১৪.৬.৩ একই Namespace বনাম অন্য Namespace

Pod-এর `resolv.conf`-এ `search` ডোমেইন থাকে: `technova.svc.cluster.local svc.cluster.local cluster.local`। তাই `technova` Namespace-এর Pod `backend` লিখলে রেজলভার প্রথমে `backend.technova.svc.cluster.local` চেষ্টা করে — মিলে যায়। অন্য Namespace (যেমন `staging`) থেকে `backend` লিখলে সেটি `backend.staging.svc.cluster.local` খুঁজবে — না পেলে ব্যর্থ হবে। তাই অন্য Namespace-এর Service ডাকতে পূর্ণ নাম (`backend.technova` বা FQDN) ব্যবহার করুন।

১৪.৬.৪ Headless Service-এর DNS — StatefulSet-এর জন্য

Headless Service (`clusterIP: None`)-এর কোনো A রেকর্ডে একটিমাত্র আইপি থাকে না। বরং CoreDNS Pod-গুলোর প্রত্যেকের আইপি ফেরত দেয় — একাধিক A রেকর্ড। StatefulSet-এর প্রতিটি Pod-এর আবার নিজস্ব স্থায়ী DNS নামও থাকে: `mongo-0.mongo-headless.technova.svc.cluster.local`। MongoDB replica set-এর সদস্যরা এভাবে একে অপরকে স্থায়ী নামে চেনে — Pod বদলালেও নাম একই থাকে। এটাই headless DNS-এর আসল শক্তি।

১৪.৬.৫ Pod-এর নিজস্ব DNS নাম ও ndots

সাধারণ Pod-এরও DNS নাম থাকতে পারে, তবে সেটি ডিফল্টভাবে তৈরি হয় না যদি না Pod-এর `hostname` ও `subdomain` দেওয়া হয়। আর `ndots` একটি গুরুত্বপূর্ণ সেটিং: `ndots:5` (ডিফল্ট) মানে নামে ৫টির কম ডট থাকলে রেজলভার প্রথমে search ডোমেইনগুলো যোগ করে চেষ্টা করে। তাই `backend` লিখলে কয়েকবার চেষ্টা হয় — প্রতিটি চেষ্টা

CoreDNS-এ যায়। বাইরের নাম (যেমন `api.example.com`) লিখলে ডট বেশি থাকায় সরাসরি চেষ্টা হয়। ndots না বুঝলে DNS-এর অতিরিক্ত জিজ্ঞাসা দেখে অবাক হবেন না — এটি স্বাভাবিক, তবে অপ্রয়োজনীয় DNS চাপ কমাতে বাইরের নাম সবসময় FQDN হিসেবে (শেষে ডট দিয়ে) লেখা ভালো অভ্যাস।

১৪.৬.৬ Pod-এর DNS নীতিমালা (dnsPolicy)

Pod-এর `dnsPolicy` নির্ধারণ করে Pod কোন DNS ব্যবহার করবে। `ClusterFirst` (ডিফল্ট) মানে ক্লাস্টারের নাম CoreDNS-এ, বাইরের নাম forward। `Default` মানে হোস্টের DNS ব্যবহার — ক্লাস্টারের নাম রেজলভ হবে না। `ClusterFirstWithHostNet` হলো `hostNetwork` Pod-এর জন্য। `None` মানে নিজে `dnsConfig` দিয়ে সম্পূর্ণ কনফিগারেশন দেওয়া। ভুল dnsPolicy দিলে Pod ক্লাস্টারের Service-কে নামে ডাকতে পারবে না — একটি সাধারণ কিন্তু ধরা কঠিন ভুল।

১৪.৬.৭ DNS-এর ভেতরের সময় — ক্যাশ, TTL ও বিলম্ব

CoreDNS উত্তরে একটি TTL (Time To Live) দেয় — সাধারণত ৫ সেকেন্ডের মতো কম। অর্থাৎ ক্লায়েন্ট বা ক্যাশ সেই উত্তর বেশিক্ষণ মনে রাখে না, ফলে Service-এর আইপি বদলালে নতুন উত্তর দ্রুত পাওয়া যায়। কিন্তু এর উল্টো দিকও আছে: প্রতিটি ডাকে আবার CoreDNS-এ যেতে হয়, চাপ বাড়ে। অনেক অ্যাপ্লিকেশন (যেমন Node.js-এর DNS লুকআপ) নিজে ক্যাশ করে না, তাই প্রতিটি নতুন সংযোগে আবার জিজ্ঞাসা হয়। প্রোডাকশনে এই আচরণ বুঝে নেওয়া জরুরি — DNS ধীর হলে মনে হতে পারে অ্যাপ ধীর, যদিও আসল সেবা ঠিক আছে। CoreDNS-এর `cache` প্লাগইন সার্ভার-দিকে উত্তর মনে রাখে, আর `node-local-dns-cache` প্রতি নোডে একটি স্থানীয় ক্যাশ রাখে, যাতে বেশিরভাগ জিজ্ঞাসা নোডের ভেতরেই মিটে যায়। বড় ক্লাস্টারে এই ক্যাশ প্রায় অপরিহার্য।

১৪.৬.৮ পরিবেশ-ভিত্তিক নামের কৌশল

TechNova-র মতো দলে একই অ্যাপ dev, staging ও production — তিন পরিবেশে চলে। কোডে যদি `backend.technova.svc.cluster.local` হার্ডকোড করা হয়, তাহলে staging-এও production-এর নাম ডাকার ঝুঁকি থাকে। ভালো কৌশল হলো: একই Namespace-এর ভেতরে সংক্ষিপ্ত নাম (`backend`) ব্যবহার করা, যাতে প্রতিটি পরিবেশে নামটি সেই পরিবেশের Service-কে নির্দেশ করে। আর ক্রস-Namespace ডাকার সময় পরিবেশের নাম পরিবেশ ভেরিয়েবল থেকে নেওয়া — যেমন `BACKEND_HOST=backend.technova.svc.cluster.local` ConfigMap-এ রাখা। এতে কোড বদল না করে পরিবেশ বদলানো যায়, আর ভুল পরিবেশে ডাকার ঝুঁকি কমে।

১৪.৭ কনফিগারেশন উদাহরণ

Pod-এর DNS সেটিং দেখুন ও নিজের মতো করে বদলান:

```
apiVersion: v1
kind: Pod
metadata:
  name: dns-test
  namespace: technova
spec:
  dnsPolicy: ClusterFirst
  dnsConfig:
    options:
    - name: ndots
      value: "2"
  containers:
  - name: test
    image: busybox
    command: ["sleep", "3600"]
```

```
---
# Headless Service + StatefulSet-এর জন্য Pod-এর স্থায়ী DNS নাম
# mongo-0.mongo-headless.technova.svc.cluster.local
# mongo-1.mongo-headless.technova.svc.cluster.local
apiVersion: v1
kind: Service
metadata:
  name: mongo-headless
  namespace: technova
spec:
  clusterIP: None
  selector:
    app: mongo
  ports:
  - port: 27017
```

ডিবাগ কমান্ড:

```
kubectl get pods -n kube-system -l k8s-app=kube-dns
kubectl -n kube-system get svc kube-dns
kubectl -n kube-system get configmap coredns -o yaml
kubectl exec -it dns-test -n technova -- cat /etc/resolv.conf
kubectl exec -it dns-test -n technova -- nslookup backend
kubectl exec -it dns-test -n technova -- nslookup backend.technova.svc.cluster.local
kubectl exec -it dns-test -n technova -- nslookup mongo-headless
```

১৪.৮ বাস্তব প্রকৌশল পরিস্থিতি

TechNova পরিস্থিতি: TechNova-র staging Namespace-এর ফ্রন্টএন্ড হঠাৎ ব্যাকএন্ডকে ডাকতে পারছে না — `getaddrinfo ENOTFOUND backend`। Production-এ একই কোড ঠিক চলছে। কারণ: কোডে শুধু `backend` লেখা, যা production Namespace-এ কাজ করে কারণ ব্যাকএন্ড সেখানেই আছে; staging-এ ব্যাকএন্ড আছে `technova` Namespace-এ, staging-এ নয়। staging-এর Pod `backend` খুঁজেছে `backend.staging.svc.cluster.local` হিসেবে — পায়নি। সমাধান: ক্রস-Namespace ডাকার সময় পূর্ণ নাম ব্যবহার করা — `backend.technova.svc.cluster.local`। শিক্ষা: সংক্ষিপ্ত নাম শুধু একই Namespace-এ নিরাপদ; কোডে সবসময় পূর্ণ নাম বা পরিবেশ-ভিত্তিক কনফিগ ব্যবহার করুন।

> **REAL WORLD:** MongoDB replica set-এর সদস্যরা headless DNS নামে একে অপরকে চেনে। একটি সদস্য Pod বদলালে তার আইপি বদলায়, কিন্তু `mongo-1.mongo-headless.technova.svc.cluster.local` নামটি একই থাকে — replica set ভাঙে না।

১৪.৯ ধাপে ধাপে ব্যবহারিক উদাহরণ

ধাপ ১: ডিবাগ Pod চালান: `kubectl run dns-test --image=busybox -n technova -- sleep 3600`। **ধাপ ২:** `kubectl exec -it dns-test -n technova -- cat /etc/resolv.conf` দিয়ে nameserver ও search ডোমেইন দেখুন। **ধাপ ৩:** `nslookup backend` ও `nslookup backend.technova.svc.cluster.local` — দুটির উত্তর একই কি না দেখুন। **ধাপ ৪:** `nslookup kubernetes.default` দিয়ে দেখুন ক্লাস্টারের নিজস্ব Service-ও DNS-এ আছে। **ধাপ ৫:** headless Service থাকলে `nslookup mongo-headless` চালান — একাধিক আইপি ফেরত আসার কথা। **ধাপ ৬:** ভুল

নাম `nslookup backend-xyz` চালিয়ে দেখুন কী ত্রুটি আসে, এবং CoreDNS লগ দেখুন: `kubectl -n kube-system logs -l k8s-app=kube-dns --tail=50`।

১৪.১০ সাধারণ ভুল ও সমস্যা সমাধান

লক্ষণ	কারণ	সমাধান
`ENOTFOUND` / নাম রেজলভ হয় না	ভুল Namespace; Service-এর নাম ভুল	FQDN দিয়ে চেষ্টা; Service আছে কি না দেখুন
সব DNS ব্যর্থ	CoreDNS Pod চলছে না; NetworkPolicy পোর্ট 53 বন্ধ	`kubectl -n kube-system get pods`; পলিসিতে DNS egress দিন
ক্লাস্টারের নাম হয়, বাইরের নাম হয় না	CoreDNS forward কনফিগ ভুল; বাইরের DNS বন্ধ	Corefile যাচাই; নোড থেকে বাইরের DNS পরীক্ষা
DNS ধীর, মাঝে মাঝে timeout	ndots-এর কারণে অতিরিক্ত জিজ্ঞাসা; CoreDNS ওভারলোড	বাইরের নাম FQDN-এ লিখুন; CoreDNS replica বাড়ান
নতুন Service-এর নাম সঙ্গে সঙ্গে পাওয়া যায় না	CoreDNS ক্যাশ; Pod-এর পুরোনো resolv	কিছুক্ষণ অপেক্ষা; Pod পুনরায় চালু করুন
headless Service একটিই আইপি দেয়	Pod ready নয়; Endpoints-এ একটি Pod	`kubectl get endpoints` যাচাই; readiness দেখুন

TROUBLE: DNS সমস্যায় প্রথমে `cat /etc/resolv.conf` দেখুন — nameserver কি CoreDNS-এর আইপি? তারপর `nslookup kubernetes.default` চালান — এটিও ব্যর্থ হলে সমস্যা CoreDNS-এ; এটিও সফল হলে সমস্যা আপনার Service-এর নামে।

১৪.১১ নিরাপত্তা বিবেচনা

DNS-এর মাধ্যমে যেকোনো Pod যেকোনো Service-এর নাম জানতে পারে — নাম জানা মানেই ডাকতে পারা নয়, সেটি NetworkPolicy নিয়ন্ত্রণ করে। তবে সংবেদনশীল Service-এর নাম অনুমানযোগ্য রাখবেন না; Namespace আলাদা করে সংবেদনশীল সেবা আলাদা করুন। CoreDNS-এর লগে কোন নাম জিজ্ঞাসা হচ্ছে তা দেখা যায় — লগ সংরক্ষণের নীতি ঠিক করুন। বাইরের DNS forward করার সময় নির্ভরযোগ্য আপস্ট্রিম ব্যবহার করুন; আপস হওয়া DNS সার্ভার ভুল আইপি দিতে পারে। আর `dnsPolicy: Default` দিয়ে Pod-কে হোস্ট DNS ব্যবহার করতে দিলে ক্লাস্টারের নাম রেজলভ হবে না — নিরাপত্তার নামে কার্যকারিতা হারাবেন না।

১৪.১২ কর্মক্ষমতা ও খরচ বিবেচনা

প্রতিটি নাম জিজ্ঞাসা CoreDNS-এ যায় — হাজার Pod-এর ক্লাস্টারে এটি বড় চাপ। CoreDNS-এর `cache` প্লাগইন উত্তর মনে রাখে, তবে TTL কম। ndots:5-এর কারণে একটি নামের জন্য ৩–৪টি জিজ্ঞাসা হতে পারে — বাইরের নামে এটি চাপ বাড়ায়। সমাধান: CoreDNS replica বাড়ানো, `node-local-dns-cache` (প্রতি নোডে স্থানীয় DNS ক্যাশ) ব্যবহার করা, এবং অ্যাপে DNS ফলাফল ক্যাশ করা। CoreDNS-এর CPU/মেমরি মনিটর করুন — DNS ধীর হলে পুরো অ্যাপ ধীর মনে হয়, যদিও আসল সেবা ঠিক আছে।

১৪.১৩ সেরা অনুশীলন ও অ্যান্টি-প্যাটার্ন

সেরা অনুশীলন	অ্যান্টি-প্যাটার্ন
কোডে Service নাম ব্যবহার, আইপি নয়	Pod/Service আইপি হার্ডকোড করা
ক্রস-Namespace ডাকে FQDN ব্যবহার	অন্য Namespace-এ সংক্ষিপ্ত নাম ধরে নেওয়া
CoreDNS-এর ২+ replica ও মনিটরিং	একটি CoreDNS Pod-এর ওপর নির্ভর করা
headless Service StatefulSet-এর জন্য	StatefulSet-এ সাধারণ Service দিয়ে সদস্য চেনার চেষ্টা
বাইরের নাম FQDN-এ লেখা	প্রতিটি বাইরের ডাকে ndots-এর অতিরিক্ত জিজ্ঞাসা

PRO TIP: একটি স্থায়ী ডিবাগ Pod (netshoot) টেমপ্লেট রাখুন — `nslookup`, `dig`, `curl` সবই থাকে। DNS সমস্যায় প্রথম পাঁচ মিনিটেই এটি কাজে লাগে।

১৪.১৪ অধ্যায়ের সারাংশ

Kubernetes DNS-এর কেন্দ্রে আছে CoreDNS — `kube-system`-এ চলা DNS সার্ভার, যা Kubernetes API থেকে Service-এর নাম জানে। প্রতিটি Service একটি A রেকর্ড (আইপি) ও SRV রেকর্ড (পোর্ট) পায়। একই Namespace-এ সংক্ষিপ্ত নাম চলে, অন্য Namespace থেকে FQDN দরকার। Headless Service DNS সরাসরি Pod আইপিগুলো দেয়, আর StatefulSet-এর প্রতিটি Pod-এর নিজস্ব স্থায়ী DNS নাম থাকে। Pod-এর `resolv.conf`-এর search ডোমেইন ও ndots বুঝলে DNS-এর আচরণ আর রহস্য থাকে না। DNS ভাঙলে প্রথমে resolv.conf, তারপর `kubernetes.default`, তারপর CoreDNS লগ — এই ক্রমে যাচাই করুন।

১৪.১৫ মূল প্রযুক্তিগত পরিভাষা

CoreDNS: Kubernetes-এর ডিফল্ট DNS সার্ভার; Service ও Pod-এর নামকে আইপিতে বদলায়।

A রেকর্ড: DNS রেকর্ড, যা একটি নামকে IPv4 ঠিকানায় ম্যাপ করে।

SRV রেকর্ড: DNS রেকর্ড, যা সেবার হোস্টনাম ও পোর্ট নম্বর জানায়।

FQDN: সম্পূর্ণ DNS নাম, যেমন `backend.technova.svc.cluster.local`।

Search ডোমেইন: resolv.conf-এর তালিকা; সংক্ষিপ্ত নামের সঙ্গে এগুলো যোগ করে পূর্ণ নাম তৈরি করা হয়।

ndots: নামে কতটি ডট থাকলে search ডোমেইন যোগ না করে সরাসরি চেষ্টা করা হবে তা নির্ধারণ করা সেটিং (ডিফল্ট ৫)।

Corefile: CoreDNS-এর কনফিগারেশন ফাইল (ConfigMap-এ থাকে)।

dnsPolicy: Pod কোন DNS ব্যবহার করবে তা নির্ধারণ করা নীতি (`ClusterFirst`, `Default` ইত্যাদি)।

Service Discovery: একটি সেবা আরেকটি সেবাকে নাম দিয়ে খুঁজে পাওয়ার ব্যবস্থা।

node-local-dns-cache: প্রতি নোডে চলা স্থানীয় DNS ক্যাশ; CoreDNS-এর চাপ কমায়।

১৪.১৬ জ্ঞান যাচাই

১. CoreDNS কী? প্রতিটি Pod-এর resolv.conf-এ nameserver হিসেবে কার ঠিকানা থাকে?

২. `backend` নামটি কখন কাজ করে, আর কখন `backend.technova.svc.cluster.local` লিখতে হয়?

৩. Service-এর জন্য কোন DNS রেকর্ড তৈরি হয়? A ও SRV রেকর্ডের পার্থক্য কী?

৪. Headless Service-এর DNS সাধারণ Service-এর DNS থেকে কীভাবে আলাদা? StatefulSet-এ এটি কেন দরকার?

৫. ndots:5-এর অর্থ কী? বাইরের নাম ডাকার সময় এটি কেন অতিরিক্ত জিজ্ঞাসা তৈরি করে?

৬. Pod থেকে `nslookup kubernetes.default` ব্যর্থ হলে আপনি কী কী যাচাই করবেন?

৭. dnsPolicy: ClusterFirst ও dnsPolicy: Default-এর পার্থক্য কী?

৮. ক্লাস্টারের নাম রেজলভ হয় কিন্তু বাইরের নাম হয় না — সম্ভাব্য কারণ কী?

১৪.১৭ ব্যবহারিক অ্যাসাইনমেন্ট

অ্যাসাইনমেন্ট: দুটি Namespace তৈরি করুন — `technova` ও `staging`। `technova`-তে একটি `backend` Service (nginx দিয়ে) ও একটি headless Service চালান। তারপর দুটি Namespace থেকে ডিবাগ Pod চালিয়ে পরীক্ষা করুন: (ক) `technova` থেকে সংক্ষিপ্ত নামে `backend` পাওয়া যায় কি না, (খ) `staging` থেকে সংক্ষিপ্ত নামে ব্যর্থ হয় কি না এবং FQDN-এ সফল হয় কি না, (গ) headless Service-এর `nslookup`-এ কয়টি আইপি আসে, (ঘ) ইচ্ছাকৃতভাবে একটি NetworkPolicy দিয়ে DNS পোর্ট 53 বন্ধ করে দেখান সব DNS ব্যর্থ হয় — তারপর পলিসি ঠিক করুন। প্রতিটি পরীক্ষার কমান্ড ও ফলাফল রিপোর্টে লিখুন।

অধ্যায় ১৫ • PART III — Networking, Storage & Traffic Management

Chapter 15 — Ingress, Ingress Controllers, TLS, and Gateway API

১৫.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: Ingress কেন দরকার এবং এটি Service থেকে কীভাবে আলাদা তা ব্যাখ্যা করতে; `Ingress` রিসোর্স বনাম `Ingress Controller`-এর পার্থক্য বলতে; NGINX Ingress Controller দিয়ে হোস্ট ও পাথ-ভিত্তিক রাউটিং তৈরি করতে; TLS termination ও cert-manager-এর ভূমিকা বোঝাতে; `IngressClass` ও সাধারণ annotation ব্যবহার করতে; Gateway API-এর `Gateway`, `HTTPRoute` ও `GatewayClass` ব্যাখ্যা করতে; এবং Ingress বনাম Gateway API-এর মধ্যে সিদ্ধান্ত নিতে।

১৫.২ কেন এই বিষয়টি গুরুত্বপূর্ণ

অধ্যায় ১৩-এ আমরা দেখেছি প্রতিটি Service-কে LoadBalancer দিয়ে প্রকাশ করলে খরচ বাড়ে, আর NodePort দিয়ে প্রকাশ করলে পোর্ট মনে রাখা যায় না। TechNova-র যদি দশটি সেবা থাকে — `app.technova.example`, `api.technova.example`, `admin.technova.example` — তাহলে কি দশটি পাবলিক আইপি কিনবেন? না। একটি প্রবেশপথ দরকার, যা HTTP অনুরোধের হোস্টনাম ও পাথ দেখে ঠিক করবে কোন Service-এ পাঠাবে। সেটাই Ingress।

Ingress হলো ক্লাস্টারের “প্রধান ফটক”। বাইরের ব্যবহারকারী একটি আইপি/ডোমেইনে আসে, Ingress Controller অনুরোধ পড়ে — হোস্ট `api.technova.example` হলে backend Service-এ, পাথ `/admin` হলে admin Service-এ — এবং প্রয়োজনে TLS ডিক্রিপ্ট করে ভেতরে সাধারণ HTTP-তে পাঠায়। একটি Ingress দিয়েই পুরো অ্যাপের পাবলিক ট্রাফিক সামলানো যায়।

> **CONCEPT:** Ingress রিসোর্স শুধু একটি নিয়মের কাগজ — “এই হোস্ট/পাথে আসা ট্রাফিক এই Service-এ যাবে।” সেই নিয়ম বাস্তবায়ন করে Ingress Controller নামের একটি প্রক্সি (যেমন NGINX)। রিসোর্স থাকলেও কন্ট্রোলার না থাকলে কিছুই হবে না — এটাই সবচেয়ে সাধারণ ভুল।

১৫.৩ পূর্বশর্ত জ্ঞান

জানা থাকা দরকার: Service-এর ধরন (ClusterIP, NodePort, LoadBalancer) ও port/targetPort; DNS ও FQDN-এর ধারণা; HTTP-এর প্রাথমিক ধারণা — হোস্ট হেডার, URL পাথ, TLS/HTTPS; এবং Secret-এর ধারণা (TLS সার্টিফিকেট রাখার জন্য)।

ধারণা	কেন দরকার
ClusterIP Service	Ingress-এর পেছনে এগুলোই থাকে
DNS ও ডোমেইন	হোস্ট-ভিত্তিক রাউটিং বোঝার জন্য
TLS/HTTPS	TLS termination বোঝার জন্য
Secret	সার্টিফিকেট সংরক্ষণের জন্য

১৫.৪ প্রথম নীতি থেকে ব্যাখ্যা

প্রথম নীতি থেকে ভাবুন: ইন্টারনেট থেকে আসা HTTP অনুরোধে দুটি তথ্য থাকে — কোন হোস্টে যাচ্ছে (`Host: api.technova.example`) এবং কোন পাথে যাচ্ছে (`/users`)। একটি স্মার্ট প্রক্সি যদি এই দুটি তথ্য পড়ে সিদ্ধান্ত নিতে পারে, তাহলে একটি আইপিতেই অসংখ্য সেবা চালানো যায়।

Kubernetes-এর Ingress সেই প্রক্সির জন্য একটি ঘোষণামূলক (declarative) নিয়ম তৈরি করে। আপনি YAML-এ লেখেন "`app.technova.example` → frontend Service, `api.technova.example` → backend Service"। Ingress Controller সেই নিয়ম দেখে নিজের প্রক্সি কনফিগারেশন (যেমন NGINX কনফিগ) তৈরি করে ও পুনরায় লোড করে। অনুরোধ এলে কন্ট্রোলার হোস্ট/পাথ মিলিয়ে সঠিক Service-এ পাঠায়। TLS-এর ক্ষেত্রে কন্ট্রোলার সার্টিফিকেট দিয়ে অনুরোধ ডিক্রিপ্ট করে (TLS termination), তারপর ভেতরে সাধারণ HTTP-তে পাঠায় — ভেতরের Pod-গুলোকে TLS সামলাতে হয় না।

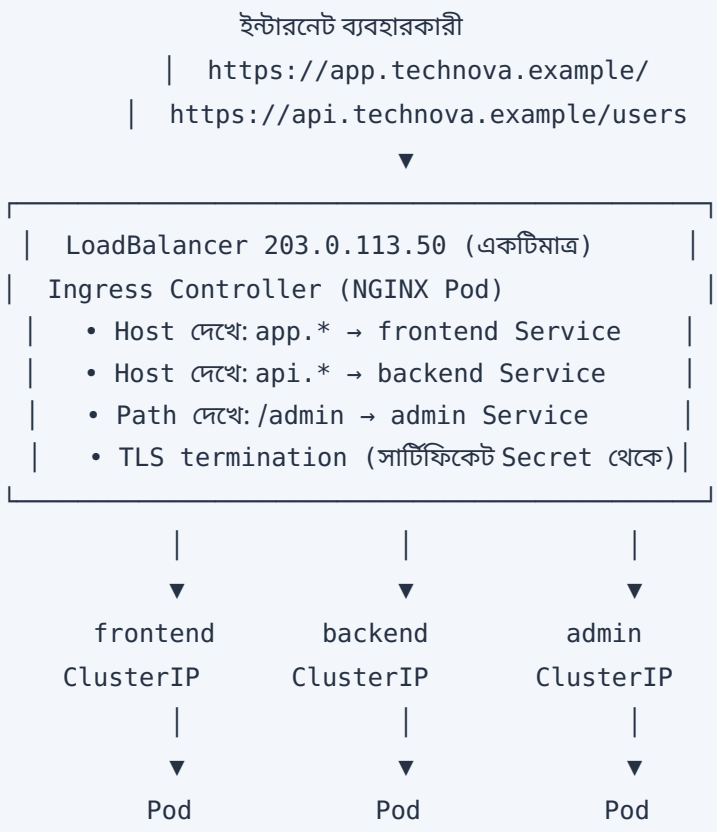
Ingress: HTTP/HTTPS ট্রাফিকের হোস্ট ও পাথ-ভিত্তিক রাউটিং নিয়ম নির্ধারণ করা Kubernetes অবজেক্ট (L7 রাউটিং)।

Ingress Controller: Ingress নিয়ম বাস্তবায়ন করা প্রক্সি (যেমন NGINX Ingress Controller); ক্লাস্টারে ইনস্টল করতে হয়।

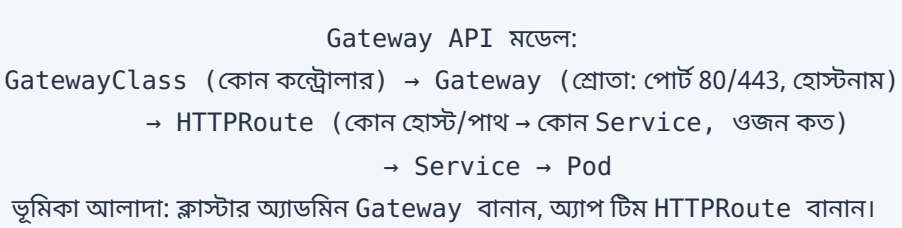
TLS Termination: Ingress Controller-এ HTTPS ডিক্রিপ্ট করে ভেতরে HTTP-তে পাঠানো; Pod-গুলো TLS-মুক্ত থাকে।

Gateway API: Ingress-এর নতুন, আরও প্রকাশভঙ্গিসম্পন্ন উত্তরসূরি — Gateway, HTTPRoute ইত্যাদি রিসোর্স নিয়ে গঠিত।

১৫.৫ আর্কিটেকচার / ধারণাগত চিত্র



চিত্র ১৫.১ — একটি Ingress Controller-এর পেছনে অনেক Service।



চিত্র ১৫.২ — Gateway API-তে দায়িত্বের বিভাজন।

১৫.৬ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

১৫.৬.১ Ingress রিসোর্স বনাম Ingress Controller

এই পার্থক্যটি মনে রাখা সবচেয়ে জরুরি। `Ingress` একটি API অবজেক্ট — শুধু নিয়ম। `Ingress Controller` একটি চলমান প্রক্সি — নিয়মের বাস্তবায়ন। Kubernetes নিজে কোনো কন্ট্রোলার দেয় না; আপনাকে NGINX, Traefik, HAProxy বা ক্লাউডের কন্ট্রোলার ইনস্টল করতে হয়। এক ক্লাস্টারে একাধিক কন্ট্রোলার থাকতে পারে — তখন `IngressClass` বলে কোন Ingress কোন কন্ট্রোলার ব্যবহার করবে। Ingress তৈরি করলেন কিন্তু কন্ট্রোলার ইনস্টল করলেন না — তাহলে Ingress-এর `ADDRESS` কলাম খালি থাকবে, কিছুই কাজ করবে না।

১৫.৬.২ হোস্ট ও পাথ-ভিত্তিক রাউটিং

Ingress নিয়মে দুটি মাত্রা: `host` (ডোমেইন) ও `path` (URL পাথ)। হোস্ট-ভিত্তিক রাউটিংয়ে `app.technova.example` ও `api.technova.example` আলাদা Service-এ যায়। পাথ-ভিত্তিক রাউটিংয়ে একই হোস্টের `/` frontend-এ, `/api` backend-এ যায়। `pathType` তিন ধরনের: `Prefix` (`/api` মানে `/api` ও `/api/users` — সবচেয়ে ব্যবহৃত), `Exact` (শুধু হুবহু পাথ), `ImplementationSpecific` (কন্ট্রোলার-নির্ভর)। পাথের ক্ষেত্রে একটি ফাঁদ: `/api` দিয়ে backend-এ পাঠালে backend `/api/users` পায় — backend যদি `/users` আশা করে, তাহলে `rewrite-target` annotation দিয়ে পাথ বদলাতে হয়।

১৫.৬.৩ TLS Termination ও cert-manager

HTTPS-এর জন্য Ingress-এ `tls` অংশে হোস্টনাম ও সার্টিফিকেট রাখা Secret-এর নাম লেখা হয়। কন্ট্রোলার সেই Secret থেকে সার্টিফিকেট নিয়ে TLS হ্যান্ডশেক করে, ভেতরে HTTP পাঠায়। সার্টিফিকেট হাতে তৈরি করে Secret-এ রাখা যায়, তবে প্রোডাকশনে সাধারণত `cert-manager` ব্যবহার করা হয় — এটি Let's Encrypt-এর মতো কর্তৃপক্ষ থেকে স্বয়ংক্রিয়ভাবে সার্টিফিকেট নেয়, নবায়ন করে, Secret-এ রাখে। Ingress-এ শুধু একটি annotation (`cert-manager.io/cluster-issuer: letsencrypt-prod`) দিলেই cert-manager কাজ শুরু করে। মনে রাখবেন: cert-manager একটি আলাদা টুল — Ingress-এর অংশ নয়, তবে প্রোডাকশনে প্রায় সবসময় একসঙ্গে ব্যবহৃত হয়।

১৫.৬.৪ IngressClass ও Annotation

`IngressClass` বলে কোন কন্ট্রোলার এই Ingress সামলাবে — যেমন `ingressClassName: nginx`। একাধিক কন্ট্রোলার থাকলে (যেমন অভ্যন্তরীণ ও বাহ্যিক) এটি অপরিহার্য। `Annotation` দিয়ে কন্ট্রোলার-নির্দিষ্ট আচরণ বদলানো যায় — যেমন NGINX-এ `nginx.ingress.kubernetes.io/rewrite-target: /$2` (পাথ পুনর্লেখন), `nginx.ingress.kubernetes.io/ssl-redirect: "true"` (HTTP-কে HTTPS-এ পাঠানো), বা rate-limit। সতর্কতা: annotation কন্ট্রোলার-নির্ভর — NGINX-এর annotation Traefik-এ কাজ করবে না। তাই annotation ব্যবহারের আগে আপনার কন্ট্রোলারের ডকুমেন্টেশন (kubernetes.io-তে তালিকাভুক্ত) দেখুন।

১৫.৬.৫ Gateway API — Ingress-এর উত্তরসূরি

Ingress-এর সীমা: একটি মাত্র রিসোর্সে সব নিয়ম, ভূমিকার বিভাজন নেই, ট্রাফিক বিভাজন (canary) বা হেডার-ভিত্তিক রাউটিং সীমিত, আর annotation-নির্ভর আচরণ পোর্টেবল নয়। Gateway API এই সীমা দূর করতে তৈরি — এটি Kubernetes-এর একটি মানক API (`gateway.networking.k8s.io`)। তিনটি মূল রিসোর্স: `GatewayClass` (কোন কন্ট্রোলার/বাস্তবায়ন), `Gateway` (শ্রোতা — কোন পোর্ট, কোন প্রোটোকল, কোন হোস্টনাম; সাধারণত ক্লাস্টার অ্যাডমিন বানান), `HTTPRoute` (কোন হোস্ট/পাথ/হেডার → কোন Service, কত ওজন; অ্যাপ টিম বানান)। এছাড়া TCP/UDP-এর জন্য `TCPRoute` / `UDPRoute`, gRPC-এর জন্য `GRPCRoute` আছে। Gateway API-তে canary (৯০% পুরোনো, ১০% নতুন সংস্করণে) সহজে করা যায়, আর ভূমিকার বিভাজন স্পষ্ট — অ্যাপ টিম Gateway বদলাতে পারে না, শুধু নিজের Route বানায়।

বিষয়	Ingress	Gateway API
রিসোর্স	একটি Ingress-এ সব নিয়ম	Gateway + HTTPRoute আলাদা
ভূমিকার বিভাজন	দুর্বল	স্পষ্ট (অ্যাডমিন বনাম অ্যাপ টিম)
Canary/ওজন-ভিত্তিক রাউটিং	annotation-নির্ভর, সীমিত	সরাসরি সমর্থিত
হেডার-ভিত্তিক রাউটিং	সীমিত	সরাসরি সমর্থিত

পোর্টেবিলিটি	annotation কন্ট্রোলার-নির্ভর	মানক API, বাস্তবায়ন বদলানো সহজ
পরিপক্কতা	বহুল ব্যবহৃত, স্থিতিশীল	নতুন, দ্রুত জনপ্রিয় হচ্ছে

বাস্তব পরামর্শ: আজকের প্রোডাকশনে Ingress (NGINX) নিরাপদ পছন্দ; নতুন প্রকল্পে Gateway API শিখে রাখুন — ভবিষ্যৎ সেদিকেই যাচ্ছে। দুটি একসঙ্গে চালানো যায়, একই কন্ট্রোলার (যেমন NGINX) দুটিই সমর্থন করতে পারে।

১৫.৬.৬ Canary ও ট্রাফিক বিভাজন — Gateway API-এর শক্তি

নতুন সংস্করণ ছাড়ার সময় সব ট্রাফিক একবারে নতুন Pod-এ পাঠানো ঝুঁকিপূর্ণ। Gateway API-তে HTTPRoute-এর `backendRefs`-এ ওজন (weight) দিয়ে ট্রাফিক ভাগ করা যায় — যেমন ৯০% অনুরোধ পুরোনো backend Service-এ, ১০% নতুন সংস্করণের Service-এ। নতুন সংস্করণে ত্রুটি দেখা গেলে ওজন শূন্য করে দিন, আর সব ঠিক থাকলে ধীরে ধীরে ১০০%-এ নিন। Ingress-এ এই কাজ annotation দিয়ে, কন্ট্রোলার-নির্ভরভাবে করতে হয় — তাই Canary-র জন্য Gateway API স্পষ্টভাবে এগিয়ে। TechNova-র মতো দলে এই পদ্ধতিতে রিলিজের ঝুঁকি অনেক কমে: প্রথমে অভ্যন্তরীণ ব্যবহারকারীদের হেডার দিয়ে নতুন সংস্করণে পাঠানো যায়, তারপর ধাপে ধাপে সবাইকে।

১৫.৬.৭ Ingress-এর সীমা ও কখন Service-ই যথেষ্ট

Ingress সব সমস্যার সমাধান নয়। অভ্যন্তরীণ সেবার মধ্যে যোগাযোগে Ingress দরকার নেই — ClusterIP Service-ই যথেষ্ট, কারণ সেখানে HTTP রাউটিংয়ের প্রয়োজন নেই। TCP-ভিত্তিক সেবা (যেমন ডাটাবেস, gRPC-এর কিছু ব্যবহার) Ingress দিয়ে যায় না — তার জন্য LoadBalancer বা Gateway API-এর TCPRoute দরকার। আর একটি Ingress Controller যদি একটিমাত্র Pod-এ চলে, সেটি নিজেই একক ব্যর্থতার বিন্দু (single point of failure) হয়ে যায় — তাই কন্ট্রোলারের অন্তত দুটি replica রাখুন, আর তার Service-টি LoadBalancer ধরনের করুন। Ingress বেছে নেওয়ার আগে প্রশ্ন করুন: ট্রাফিক কি HTTP? বাইরে থেকে আসছে কি? একাধিক হোস্ট/পাথ দরকার কি? তিনটির উত্তর “হ্যাঁ” হলে Ingress সঠিক, নইলে Service-ই যথেষ্ট।

১৫.৭ কনফিগারেশন উদাহরণ

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: technova-ingress
  namespace: technova
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /$2
    nginx.ingress.kubernetes.io/ssl-redirect: "true"
spec:
  ingressClassName: nginx
  tls:
  - hosts:
    - app.technova.example
    - api.technova.example
    secretName: technova-tls
  rules:
  - host: app.technova.example
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
```

```
            name: frontend
            port:
              number: 80
  - host: api.technova.example
    http:
      paths:
      - path: /api(/|$)(.*)
        pathType: ImplementationSpecific
        backend:
          service:
            name: backend
            port:
              number: 5000
---
# Gateway API উদাহরণ (gateway.networking.k8s.io/v1beta1)
apiVersion: gateway.networking.k8s.io/v1beta1
kind: Gateway
metadata:
  name: technova-gateway
  namespace: technova
spec:
  gatewayClassName: nginx
  listeners:
  - name: http
    protocol: HTTP
    port: 80
  - name: https
    protocol: HTTPS
    port: 443
    tls:
      mode: Terminate
      certificateRefs:
      - name: technova-tls
---
apiVersion: gateway.networking.k8s.io/v1beta1
kind: HTTPRoute
metadata:
  name: technova-routes
  namespace: technova
spec:
  parentRefs:
  - name: technova-gateway
  hostnames:
  - app.technova.example
  - api.technova.example
  rules:
  - matches:
    - path:
        type: PathPrefix
        value: /api
    backendRefs:
    - name: backend
      port: 5000
  - matches:
    - path:
        type: PathPrefix
        value: /
    backendRefs:
```

```
      - name: frontend
        port: 80
```

যাচাই: `kubectl get ingress -n technova`, `kubectl describe ingress technova-ingress -n technova`, `kubectl get gateway,httproute -n technova`।

১৫.৮ বাস্তব প্রকৌশল পরিস্থিতি

TechNova পরিস্থিতি: TechNova-র `api.technova.example` ডাকলে `404 Not Found` আসছে, অথচ সরাসরি Service ডাকলে backend ঠিক আছে। তদন্তে দেখা গেল Ingress-এ পাথ `/api` লেখা, কিন্তু backend আশা করে `/users` — Ingress `/api/users` পাঠাচ্ছে, backend-এর রাউটে `/api/users` নেই, শুধু `/users` আছে। সমাধান: `rewrite-target: /$2` annotation দিয়ে `/api` অংশ সরিয়ে দেওয়া। আরেকবার দেখা গেল Ingress-এর `ADDRESS` খালি — কারণ কন্ট্রোলার ইনস্টলই করা হয়নি; `kubectl -n ingress-nginx get pods`-এ কিছুই ছিল না। শিক্ষা: প্রথমে কন্ট্রোলার চলছে কি না দেখুন, তারপর নিয়ম, তারপর পাথ পুনর্লেখন।

> **WARNING:** TLS Secret-এর নাম ভুল হলে Ingress HTTPS-এ কাজ করবে না — কন্ট্রোলার ডিফল্ট (ভুয়া) সার্টিফিকেট দেবে, ব্রাউজার সতর্কতা দেখাবে। `kubectl describe ingress`-এ সতর্কতা দেখুন।

১৫.৯ ধাপে ধাপে ব্যবহারিক উদাহরণ

ধাপ ১: NGINX Ingress Controller ইনস্টল করুন (আপনার ক্লাস্টারের নির্দেশনা অনুযায়ী — kind/minikube-এ আলাদা পদ্ধতি থাকতে পারে)। **ধাপ ২:** `kubectl -n ingress-nginx get pods` দিয়ে কন্ট্রোলার চলছে কি না দেখুন। **ধাপ ৩:** ১৫.৭-এর Ingress তৈরি করুন (TLS অংশ ছাড়া প্রথমে — HTTP দিয়ে শুরু করুন)। **ধাপ ৪:** `kubectl get ingress -n technova`-এ ADDRESS আসা পর্যন্ত অপেক্ষা করুন। **ধাপ ৫:** আপনার `/etc/hosts`-এ `203.0.113.50 app.technova.example` যোগ করে (ADDRESS-এর আইপি বসান) `curl http://app.technova.example/` চালান। **ধাপ ৬:** একটি ভুল পাথ দিয়ে `curl` করে দেখুন কী হয়, তারপর `rewrite-target` যোগ করে পার্থক্য বুঝুন।

১৫.১০ সাধারণ ভুল ও সমস্যা সমাধান

লক্ষণ	কারণ	সমাধান
Ingress-এর ADDRESS খালি	কন্ট্রোলার ইনস্টল হয়নি; ভুল IngressClass	কন্ট্রোলার Pod যাচাই; `ingressClassName` মেলান
`404` — backend ঠিক আছে	পাথ মেলেনি; rewrite দরকার	pathType ও rewrite-target যাচাই
`502 Bad Gateway`	Service-এর Endpoints খালি; Pod প্রস্তুত নয়	অধ্যায় ১৩-এর Endpoints যাচাই
HTTPS-এ সার্টিফিকেট সতর্কতা	ভুল Secret; cert-manager সার্টিফিকেট দেয়নি	Secret আছে কি না দেখুন; cert-manager লগ দেখুন
HTTP HTTPS-এ যাচ্ছে না	ssl-redirect annotation নেই	annotation যোগ করুন
এক হোস্টের নিয়ম অন্য হোস্টে প্রভাব ফেলছে	ডিফল্ট backend বা হোস্ট বানান ভুল	প্রতিটি হোস্ট আলাদা rule-এ লিখুন

TROUBLE: Ingress সমস্যায় কন্ট্রোলারের লগ দেখুন — `kubectl -n ingress-nginx logs -l app.kubernetes.io/component=controller --tail=100`। বেশিরভাগ ভুল সেখানেই ধরা পড়ে।

১৫.১১ নিরাপত্তা বিবেচনা

Ingress হলো ইন্টারনেটের সামনে দাঁড়ানো দরজা — তাই এখানে নিরাপত্তা সবচেয়ে গুরুত্বপূর্ণ। সবসময় HTTPS ব্যবহার করুন, HTTP-কে HTTPS-এ redirect করুন। TLS সার্টিফিকেট cert-manager দিয়ে স্বয়ংক্রিয় নবায়ন করুন — মেয়াদোত্তীর্ণ সার্টিফিকেটে সাইট বন্ধ হয়ে যায়। সংবেদনশীল পাথ (`/admin`) শুধু Ingress-এ নয়, NetworkPolicy দিয়েও সুরক্ষিত করুন। Rate-limit annotation দিয়ে brute-force ঠেকান। Ingress Controller-এর Pod-কে সীমিত অনুমতি (RBAC) দিন — এটি ক্লাস্টারের শক্তিশালী কম্পোনেন্ট। আর কখনো ডাটাবেস Service-কে Ingress-এর পেছনে রাখবেন না।

১৫.১২ কর্মক্ষমতা ও খরচ বিবেচনা

একটি Ingress Controller দিয়ে অনেক সেবা চালানো যায় — LoadBalancer-এর খরচ একটিতেই সীমাবদ্ধ থাকে। কন্ট্রোলারের replica বাড়িয়ে উচ্চ প্রাপ্যতা নিশ্চিত করুন (অন্তত ২টি)। বড় ট্রাফিকে কন্ট্রোলারের CPU/মেমরি মনিটর করুন; NGINX-এর worker প্রক্রিয়া ও সংযোগ সীমা টিউন করতে হতে পারে। প্রতিটি Ingress পরিবর্তনে কন্ট্রোলার কনফিগ পুনরায় লোড করে — ঘন ঘন পরিবর্তনে সাময়িক বিলম্ব হতে পারে। Gateway API-তে একই কন্ট্রোলার আরও দক্ষ রাউটিং দিতে পারে, কারণ Route পরিবর্তন Gateway-কে ছোঁয় না।

১৫.১৩ সেরা অনুশীলন ও অ্যান্টি-প্যাটার্ন

সেরা অনুশীলন	অ্যান্টি-প্যাটার্ন
একটি Ingress Controller, অনেক Ingress/Route	প্রতিটি সেবায় আলাদা LoadBalancer
IngressClass সবসময় স্পষ্টভাবে লেখা	কোন কন্ট্রোলার ব্যবহার হবে তা অনুমান করা
TLS cert-manager দিয়ে স্বয়ংক্রিয় করা	হাতে সার্টিফিকেট বানিয়ে মেয়াদ ভুলে যাওয়া
পাথের জন্য Prefix ব্যবহার, Exact শুধু প্রয়োজনে	সব পাথে ImplementationSpecific
নতুন প্রকল্পে Gateway API বিবেচনা করা	annotation-নির্ভর জটিল Ingress-এ আটকে থাকা

PRO TIP: Ingress-এর আগে Service-টি ClusterIP দিয়ে সরাসরি পরীক্ষা করুন। Service ঠিক থাকলে তবেই Ingress-এর সমস্যা খুঁজুন — এতে সমস্যার স্তর দ্রুত আলাদা হয়।

১৫.১৪ অধ্যায়ের সারাংশ

Ingress হলো ক্লাস্টারের L7 প্রবেশপথ — হোস্ট ও পাথ দেখে ট্রাফিক সঠিক ClusterIP Service-এ পাঠায়। Ingress রিসোর্স শুধু নিয়ম; Ingress Controller (যেমন NGINX) সেই নিয়ম বাস্তবায়ন করে। TLS termination কন্ট্রোলারে হয়, সার্টিফিকেট আসে Secret থেকে — প্রোডাকশনে cert-manager দিয়ে স্বয়ংক্রিয় করুন। IngressClass বলে কোন কন্ট্রোলার, annotation দিয়ে কন্ট্রোলার-নির্দিষ্ট আচরণ বদলানো যায়। Gateway API (Gateway, HTTPRoute,

GatewayClass) হলো নতুন, আরও প্রকাশভঙ্গিসম্পন্ন উত্তরসূরি — ভূমিকার বিভাজন স্পষ্ট, canary ও হেডার-ভিত্তিক রাউটিং সরাসরি সমর্থিত। আজ Ingress নিরাপদ, আগামীকাল Gateway API।

১৫.১৫ মূল প্রযুক্তিগত পরিভাষা

Ingress: HTTP/HTTPS ট্রাফিকের হোস্ট ও পাথ-ভিত্তিক রাউটিং নিয়ম।

Ingress Controller: Ingress নিয়ম বাস্তবায়ন করা প্রক্সি (যেমন NGINX Ingress Controller)।

IngressClass: কোন কন্ট্রোলার একটি Ingress সামলাবে তা নির্ধারণ করা অবজেক্ট।

TLS Termination: প্রক্সিতে HTTPS ডিক্রিপ্ট করে ভেতরে HTTP পাঠানো।

cert-manager: সার্টিফিকেট স্বয়ংক্রিয়ভাবে সংগ্রহ ও নবায়ন করা টুল।

pathType: পাথ মেলানোর ধরন — Prefix, Exact বা ImplementationSpecific।

Gateway: Gateway API-তে শ্রোতা (পোর্ট, প্রোটোকল, হোস্টনাম) নির্ধারণ করা রিসোর্স।

HTTPRoute: Gateway API-তে হোস্ট/পাথ/হেডার থেকে Service-এ রাউটিং নির্ধারণ করা রিসোর্স।

GatewayClass: কোন কন্ট্রোলার Gateway API বাস্তবায়ন করবে তা নির্ধারণ করা রিসোর্স।

L7 রাউটিং: HTTP-স্তরের তথ্য (হোস্ট, পাথ, হেডার) দেখে রাউটিং করা।

১৫.১৬ জ্ঞান যাচাই

১. Ingress রিসোর্স ও Ingress Controller-এর পার্থক্য কী? কন্ট্রোলার ছাড়া Ingress তৈরি করলে কী হবে?

২. Service ও Ingress-এর পার্থক্য কী? কেন একটি LoadBalancer-এর বদলে Ingress ব্যবহার করবেন?

৩. pathType Prefix, Exact ও ImplementationSpecific-এর পার্থক্য কী?

৪. TLS termination কী? cert-manager এতে কীভাবে সাহায্য করে?

৫. IngressClass কেন দরকার? এক ক্লাস্টারে একাধিক কন্ট্রোলার থাকলে কী হয়?

৬. `/api/users` পাঠালে backend `/users` আশা করলে কী করবেন?

৭. Gateway API-এর Gateway, HTTPRoute ও GatewayClass-এর ভূমিকা কী কী?

৮. Ingress বনাম Gateway API — কোন পরিস্থিতিতে কোনটি বেছে নেবেন?

১৫.১৭ ব্যবহারিক অ্যাসাইনমেন্ট

অ্যাসাইনমেন্ট: আপনার টেস্ট ক্লাস্টারে NGINX Ingress Controller ইনস্টল করুন। তারপর TechNova-র জন্য একটি Ingress তৈরি করুন যাতে `app.technova.example` → frontend Service এবং `api.technova.example/api` → backend Service (rewrite-target দিয়ে `/api` সরিয়ে) যায়। `/etc/hosts`-এ ডোমেইন যোগ করে `curl` দিয়ে দুটি রুটই পরীক্ষা করুন। এরপর একই রাউটিং Gateway API-এর Gateway + HTTPRoute দিয়ে তৈরি করুন এবং দুটি পদ্ধতির YAML পাশাপাশি তুলনা করে একটি সংক্ষিপ্ত রিপোর্ট লিখুন — কোনটি আপনার কাছে সহজ মনে হলো এবং কেন।

অধ্যায় ১৬ • PART III — Networking, Storage & Traffic Management

Chapter 16 — PersistentVolumes, PersistentVolumeClaims, StorageClasses, and CSI

১৬.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: Pod ক্ষণস্থায়ী বলে ডাটাবেসের ডেটা কেন হারায় তা ব্যাখ্যা করতে; `PersistentVolume (PV)`, `PersistentVolumeClaim (PVC)` ও `StorageClass`-এর সম্পর্ক ও জীবনচক্র বোঝাতে; `static` বনাম `dynamic provisioning`-এর পার্থক্য বলতে; `access mode` (RWO, ROX, RWX) ও `reclaim policy` (Retain, Delete, Recycle) ব্যাখ্যা করতে; `CSI (Container Storage Interface)` ড্রাইভারের ধারণা দিতে; StatefulSet-এর `volumeClaimTemplates` ব্যবহার করতে; এবং MongoDB-র মতো ডাটাবেসের স্থায়ী স্টোরেজ, সমস্যা সমাধান ও ব্যাকআপ পরিকল্পনা করতে।

১৬.২ কেন এই বিষয়টি গুরুত্বপূর্ণ

Pod মরলে তার ভেতরের ফাইলসিস্টেমও মরে যায়। TechNova-র MongoDB Pod যদি নোড বদলায়, নতুন Pod-এ পুরোনো ডেটা থাকবে না — ব্যবহারকারীর অ্যাকাউন্ট, অর্ডার, সব হারিয়ে যাবে। কন্টেইনারের ভেতরে ডেটা রাখা মানে বালির ওপর বাড়ি বানানো।

স্থায়ী স্টোরেজ (persistent storage) এই সমস্যার সমাধান: ডেটা Pod-এর বাইরে, ক্লাস্টারের স্টোরেজ সিস্টেমে থাকে; Pod মরলেও ডেটা থাকে, নতুন Pod সেই একই ডিস্ক মাউন্ট করে আবার চালু হয়। এ ছাড়া ডাটাবেস, ফাইল আপলোড, ক্যাশ — কোনো স্থায়ী ডেটাই Kubernetes-এ চালানো যায় না। তাই স্টোরেজ বোঝা মানে প্রোডাকশন-উপযোগী অ্যাপ বোঝা।

> **CONCEPT:** Pod ক্ষণস্থায়ী, ডেটা স্থায়ী — এই দুটি আলাদা জীবনচক্রের মধ্যে সেতু হলো PV/PVC। Pod ডেটা “ধার” করে, মালিকানা স্টোরেজ সিস্টেমের।

১৬.৩ পূর্বশর্ত জ্ঞান

জানা থাকা দরকার: Pod, Deployment ও StatefulSet-এর ধারণা (StatefulSet অধ্যায়ে পরিচিত); Volume-এর প্রাথমিক ধারণা (`emptyDir` — Pod-এর জীবনকাল পর্যন্ত থাকে, Pod মরলে মুছে যায়); Linux-এ মাউন্ট ও ফাইলসিস্টেমের ধারণা; এবং `kubectl get pv,pvc` চালানোর অভ্যাস।

ধারণা	কেন দরকার
emptyDir	ক্ষণস্থায়ী বনাম স্থায়ী স্টোরেজের পার্থক্য বোঝার জন্য
StatefulSet	ডাটাবেস Pod-এর স্থায়ী পরিচয় ও স্টোরেজের জন্য
মাউন্ট	PV কীভাবে Pod-এর ভেতরে ফোল্ডার হয় তা বোঝার জন্য

১৬.৪ প্রথম নীতি থেকে ব্যাখ্যা

প্রথম নীতি থেকে ভাবুন: ডেটা রাখার জন্য দরকার (ক) একটি বাস্তব ডিস্ক, (খ) সেই ডিস্ক চাওয়ার একটি উপায়, (গ) ডিস্ক ও চাওয়ার মধ্যে মেলানোর নিয়ম, (ঘ) ডিস্ক ফেরত গেলে কী হবে তার নীতি। Kubernetes এই চারটি আলাদা অবজেক্টে ভাগ করেছে, যাতে স্টোরেজ অ্যাডমিন ও অ্যাপ ডেভেলপারের দায়িত্ব আলাদা থাকে।

`PersistentVolume (PV)` হলো বাস্তব ডিস্কের প্রতিনিধি — ক্লাস্টারে আগে থেকে তৈরি বা চাহিদায় তৈরি স্টোরেজের টুকরো। `PersistentVolumeClaim (PVC)` হলো অ্যাপের চাওয়া — “আমার ১০ গিগাবাইট দরকার, একটিমাত্র Pod লিখবে।” Kubernetes সেই চাওয়া মেটাতে উপযুক্ত PV খুঁজে বেঁধে দেয় (binding)। `StorageClass` হলো স্টোরেজের “শ্রেণি” — যেমন দ্রুত SSD বনাম সস্তা HDD, বা ক্লাউডের ডিস্কের ধরন — আর `provisioner` বলে কে সেই ডিস্ক তৈরি করবে। আর `CSI` হলো মানক ইন্টারফেস, যার মাধ্যমে যেকোনো স্টোরেজ কোম্পানির ড্রাইভার Kubernetes-এর সঙ্গে কথা বলে।

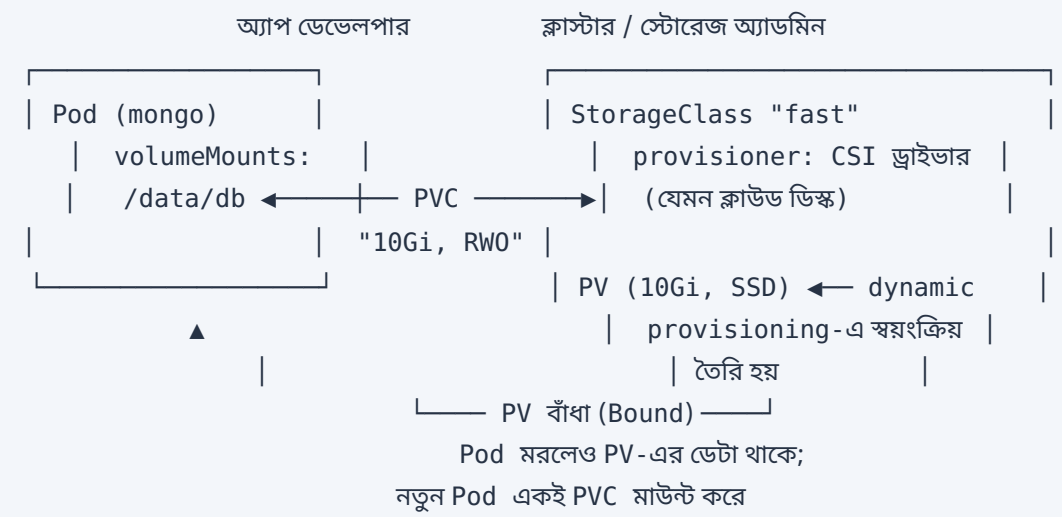
PersistentVolume (PV): ক্লাস্টারে উপলব্ধ স্থায়ী স্টোরেজের টুকরো; স্টোরেজ অ্যাডমিন বা dynamic provisioning তৈরি করে।

PersistentVolumeClaim (PVC): Pod-এর স্টোরেজের চাওয়া — আকার, access mode ও StorageClass উল্লেখ করে।

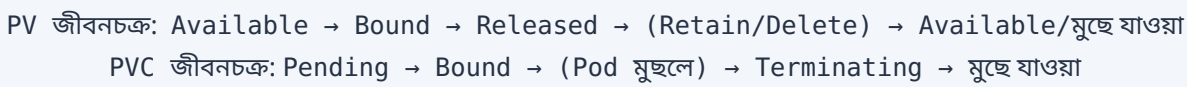
StorageClass: স্টোরেজের শ্রেণি ও provisioner নির্ধারণ করা অবজেক্ট; dynamic provisioning-এর নীলনকশা।

CSI (Container Storage Interface): Kubernetes ও স্টোরেজ ড্রাইভারের মধ্যে মানক ইন্টারফেস; যেকোনো স্টোরেজ সিস্টেম এতে প্লাগ করা যায়।

১৬.৫ আর্কিটেকচার / ধারণাগত চিত্র



চিত্র ১৬.১ — PVC, PV ও StorageClass-এর সম্পর্ক।



চিত্র ১৬.২ — PV ও PVC-এর জীবনচক্র।

১৬.৬ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

১৬.৬.১ PV বনাম PVC — সরবরাহ বনাম চাহিদা

PV হলো সরবরাহ, PVC হলো চাহিদা। অ্যাডমিন আগে থেকে PV তৈরি করতে পারেন (static provisioning) — যেমন একটি NFS শেয়ার বা ক্লাউড ডিস্কের PV। অথবা PVC তৈরি হলে StorageClass দেখে স্বয়ংক্রিয়ভাবে PV তৈরি হতে পারে (dynamic provisioning) — এটাই আধুনিক ও সাধারণ পদ্ধতি। Binding-এর নিয়ম: PVC-এর আকারের সমান বা বড় PV, একই access mode, একই StorageClass — প্রথম মেলা PV বাঁধা হয়। PVC-এর আকারের চেয়ে ছোট PV কখনো বাঁধা হয় না। একবার বাঁধা হলে PV অন্য PVC-কে দেওয়া হয় না, যতক্ষণ না PVC মুছে যায়।

১৬.৬.২ StorageClass ও Dynamic Provisioning

StorageClass বলে: কোন provisioner (CSI ড্রাইভার) ডিস্ক তৈরি করবে, কোন ধরনের ডিস্ক (যেমন SSD), কোন প্যারামিটার (যেমন IOPS), আর `reclaimPolicy` কী হবে। PVC-তে `storageClassName` লিখলে Kubernetes সেই ক্লাসের provisioner-কে ডাকে, provisioner ক্লাউডে/স্টোরেজে ডিস্ক তৈরি করে, তারপর PV তৈরি করে PVC-তে বাঁধে — সব স্বয়ংক্রিয়। ক্লাস্টারে একটি StorageClass-কে ডিফল্ট করা যায় — তখন PVC-তে ক্লাস না লিখলেও সেটি ব্যবহৃত হয়। ভিন্ন পরিবেশে ভিন্ন ক্লাস রাখুন: `fast-ssd` ডাটাবেসের জন্য, `standard` সাধারণ ফাইলের জন্য।

১৬.৬.৩ Access Mode — RWO, ROX, RWX

Access Mode	অর্থ	ব্যবহার
RWO (ReadWriteOnce)	একটি নোড থেকে পড়া ও লেখা	ডাটাবেস (MongoDB, PostgreSQL) — সবচেয়ে সাধারণ
ROX (ReadOnlyMany)	অনেক নোড থেকে শুধু পড়া	স্ট্যাটিক ফাইল, কনফিগ বিতরণ
RWX (ReadWriteMany)	অনেক নোড থেকে পড়া ও লেখা	শেয়ার করা ফাইল আপলোড, CMS — তবে সব স্টোরেজ দেয় না
RWOP (ReadWriteOncePod)	একটিমাত্র Pod থেকে পড়া ও লেখা	একক Pod-এর স্থানীয় ডিস্ক

সতর্কতা: RWX চাইলেই পাওয়া যায় না — ক্লাউডের ব্লক ডিস্ক সাধারণত শুধু RWO দেয়; RWX-এর জন্য NFS বা ফাইল স্টোরেজ (যেমন ক্লাউড ফাইল সিস্টেম) দরকার। RWX দরকার হলে প্রথমে আপনার StorageClass সেটি সমর্থন করে কি না যাচাই করুন।

১৬.৬.৪ Reclaim Policy — PVC মুছলে ডেটার কী হবে

PVC মুছে গেলে PV `Released` অবস্থায় যায়। তারপর কী হবে তা `reclaimPolicy` ঠিক করে। `Retain` মানে ডিস্ক ও ডেটা রেখে দেওয়া — অ্যাডমিন হাতে সিদ্ধান্ত নেবেন; ডাটাবেসের জন্য এটাই নিরাপদ। `Delete` মানে PV মুছলে ক্লাউড ডিস্কও মুছে যাবে — ডেটা চিরতরে হারাবে; অস্থায়ী পরিবেশে ঠিক, প্রোডাকশন ডাটাবেসে বিপজ্জনক। `Recycle` পুরোনো নীতি, এখন প্রায় ব্যবহৃত হয় না (ডেটা মুছে PV আবার ব্যবহারযোগ্য করা হতো)। প্রোডাকশনে ডাটাবেসের StorageClass-এ `Retain` রাখুন, আর PVC মুছবেন না — মুছতে হলে ব্যাকআপ নিয়ে মুছুন।

১৬.৬.৫ CSI ড্রাইভার — স্টোরেজের প্লাগইন ব্যবস্থা

আগে Kubernetes-এর ভেতরেই প্রতিটি স্টোরেজের কোড থাকত (in-tree) — নতুন স্টোরেজ যোগ করা কঠিন ছিল। CSI সেই কোডকে বাইরে নিয়ে এসেছে: স্টোরেজ কোম্পানি একটি CSI ড্রাইভার বানায়, Kubernetes সেই ড্রাইভারকে মানক API-তে ডাকে — ডিস্ক তৈরি করো, মাউন্ট করো, স্ন্যাপশট নাও, আকার বাড়াও। ফলে ক্লাউড ডিস্ক, NFS, Ceph, স্থানীয় ডিস্ক — সব একইভাবে কাজ করে। CSI ড্রাইভার সাধারণত `kube-system` বা আলাদা Namespace-এ DaemonSet/Deployment হিসেবে চলে।

১৬.৬.৬ StatefulSet ও volumeClaimTemplates

StatefulSet-এর প্রতিটি Pod-এর নিজস্ব স্থায়ী স্টোরেজ দরকার — `mongo-0`-এর ডিস্ক `mongo-1`-এর থেকে আলাদা। `volumeClaimTemplates` এই কাজটি করে: StatefulSet তৈরি হলে প্রতিটি Pod-এর জন্য আলাদা PVC স্বয়ংক্রিয়ভাবে তৈরি হয় (`data-mongo-0`, `data-mongo-1`), আর Pod মুছলেও PVC থাকে — নতুন Pod একই PVC মাউন্ট করে। Deployment-এ এটি নেই — Deployment-এর Pod-গুলো PVC ভাগ করে নেয়, যা ডাটাবেসের জন্য ভুল। তাই ডাটাবেস সবসময় StatefulSet + volumeClaimTemplates।

১৬.৬.৭ Volume-এর ধরন — emptyDir বনাম PersistentVolume

এই পার্থক্যটি পরীক্ষায় ও প্রোডাকশনে বারবার আসে। `emptyDir` তৈরি হয় Pod চালু হলে, মুছে যায় Pod মুছলে — ক্যাশ, অস্থায়ী ফাইল বা দুটি কন্টেইনারের মধ্যে ফাইল ভাগাভাগির জন্য উপযোগী। কিন্তু ডাটাবেসের ডেটা, আপলোড করা ফাইল বা যেকোনো দীর্ঘমেয়াদি ডেটার জন্য এটি একেবারেই অনুপযুক্ত। একটি সাধারণ ভুল হলো টিউটোরিয়ালে emptyDir দিয়ে ডাটাবেস চালিয়ে সেটিকে প্রোডাকশনে নিয়ে যাওয়া — টিউটোরিয়ালে Pod মরে না বলে সমস্যা ধরা পড়ে না, প্রোডাকশনে

প্রথম নোড রিস্টার্টেই ডেটা হারায়। তাই নিয়মটি সহজ: ডেটা যদি Pod-এর চেয়ে বেশি দিন বাঁচতে হয়, তাহলে সেটি PVC-তে রাখুন — কোনো ব্যতিক্রম নেই।

`emptyDir` Pod-এর জীবনকাল পর্যন্ত থাকে — ক্যাশ বা অস্থায়ী ফাইলের জন্য ঠিক। `hostPath` হোস্টের ফোল্ডার মাউন্ট করে — শুধু ডেভ/ডিবাগে, প্রোডাকশনে নয় (নোড বদলালে ডেটা হারায়, নিরাপত্তা ঝুঁকি)। `configMap` / `secret` ভলিউম কনফিগ ও গোপন তথ্য ফাইল হিসেবে দেয় — ডেটা স্টোরেজ নয়। স্থায়ী ডেটার জন্য শুধু PV/PVC।

১৬.৬.৮ স্ন্যাপশট ও ভলিউম সম্প্রসারণ

আধুনিক CSI ড্রাইভার দুটি গুরুত্বপূর্ণ সুবিধা দেয়: ভলিউম স্ন্যাপশট ও সম্প্রসারণ। `VolumeSnapshot` দিয়ে চলমান ডিস্কের একটি সময়ের ছবি (point-in-time copy) নেওয়া যায় — ব্যাকআপের ভিত্তি হিসেবে এটি খুবই কার্যকর, কারণ এতে অ্যাপ বন্ধ না করেই সামঞ্জস্যপূর্ণ কপি পাওয়া যায়। আর `allowVolumeExpansion: true` থাকলে PVC-এর `requests.storage` বাড়িয়ে `kubectl apply` করলেই ডিস্ক বড় হয় — Pod পুনরায় চালু করতে হতে পারে, তবে ডেটা হারায় না। মনে রাখবেন: স্ন্যাপশট নেওয়া মানেই ব্যাকআপ নয় — স্ন্যাপশট একই স্টোরেজ সিস্টেমে থাকে; আসল ব্যাকআপ মানে ডেটা অন্য জায়গায় (অন্য জোন বা অন্য স্টোরেজে) রাখা।

১৬.৭ কনফিগারেশন উদাহরণ

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: fast-ssd
provisioner: csi.example.com
parameters:
  type: ssd
reclaimPolicy: Retain
allowVolumeExpansion: true
volumeBindingMode: WaitForFirstConsumer
---
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: mongo-data
  namespace: technova
spec:
  accessModes:
  - ReadWriteOnce
  storageClassName: fast-ssd
  resources:
    requests:
      storage: 10Gi
---
# MongoDB StatefulSet — volumeClaimTemplates
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: mongo
  namespace: technova
spec:
  serviceName: mongo-headless
  replicas: 3
  selector:
    matchLabels:
      app: mongo
```

```
  template:
    metadata:
      labels:
        app: mongo
    spec:
      containers:
      - name: mongo
        image: mongo:7
        ports:
        - containerPort: 27017
        env:
        - name: MONGO_INITDB_ROOT_USERNAME
          value: admin
        - name: MONGO_INITDB_ROOT_PASSWORD
          value: CHANGE-ME-MONGO-PASSWORD
        volumeMounts:
        - name: data
          mountPath: /data/db
  volumeClaimTemplates:
  - metadata:
      name: data
    spec:
      accessModes: ["ReadWriteOnce"]
      storageClassName: fast-ssd
      resources:
        requests:
          storage: 10Gi
```

লক্ষ করুন: `volumeBindingMode: WaitForFirstConsumer` মানে PV তৈরি হবে Pod সময়সূচি (schedule) হওয়ার পর — এতে ডিস্ক Pod-এর নোডের কাছাকাছি তৈরি হয়, ভুল জোনে ডিস্ক তৈরি হয় না। `allowVolumeExpansion: true` মানে পরে PVC-এর আকার বাড়ানো যায়। পাসওয়ার্ড শুধু `CHANGE-ME-...` প্লেসহোল্ডার — আসল গোপন তথ্য Secret-এ রাখুন।

```
kubectl get storageclass
kubectl get pvc -n technova
kubectl get pv
kubectl describe pvc mongo-data -n technova
```

১৬.৮ বাস্তব প্রকৌশল পরিস্থিতি

TechNova পরিস্থিতি: TechNova-র MongoDB Pod নোড বদলানোর পর খালি ডাটাবেস নিয়ে চালু হলো — সব ব্যবহারকারীর ডেটা হারিয়ে গেছে বলে মনে হলো। তদন্তে দেখা গেল Deployment ব্যবহার করা হয়েছিল `emptyDir` ভলিউম দিয়ে — Pod মরতেই ডেটা মুছে গেছে। আসল ডেটা কোথাও ছিল না, ব্যাকআপও ছিল না। সমাধান: StatefulSet + PVC-তে পুনর্নির্মাণ, `reclaimPolicy: Retain` দিয়ে StorageClass, এবং দৈনিক ব্যাকআপ (mongodump) চালু করা। শিক্ষা: ডাটাবেস কখনো emptyDir বা hostPath-এ চালাবেন না; প্রথম দিন থেকেই PVC, StatefulSet ও ব্যাকআপ — তিনটি একসঙ্গে।

> **REAL WORLD:** প্রোডাকশনে PVC-এর আকার বাড়ানো যায় (`allowVolumeExpansion: true` থাকলে), কিন্তু কমানো যায় না। শুরুতে একটু বড় আকার নিন — ১০ Gi দিয়ে শুরু করে পরে ২০ Gi করা যায়, উল্টোটা যায় না।

১৬.৯ ধাপে ধাপে ব্যবহারিক উদাহরণ

ধাপ ১: `kubectl get storageclass` দিয়ে আপনার ক্লাস্টারের StorageClass দেখুন (kind/minikube-এ `standard` থাকে)। **ধাপ ২:** ১৬.৭-এর PVC তৈরি করুন (আপনার StorageClass-এর নাম বসান) এবং `kubectl get pvc -n technova`-এ `Bound` হওয়া পর্যন্ত অপেক্ষা করুন। **ধাপ ৩:** একটি Pod-এ PVC মাউন্ট করে একটি ফাইল লিখুন: `kubectl exec` দিয়ে `/data/hello.txt` তৈরি করুন। **ধাপ ৪:** Pod মুছে আবার তৈরি করুন — ফাইলটি এখনো আছে কি না দেখুন। **ধাপ ৫:** `kubectl get pv` দিয়ে PV-এর reclaimPolicy দেখুন। **ধাপ ৬:** PVC-এর আকার বাড়ানোর চেষ্টা করুন (`kubectl patch pvc ...`) — আপনার StorageClass expansion সমর্থন করে কি না দেখুন।

১৬.১০ সাধারণ ভুল ও সমস্যা সমাধান

লক্ষণ	কারণ	সমাধান
PVC `Pending`-এ আটকে	StorageClass নেই/ভুল; provisioner চলছে না; আকারের PV নেই	`kubectl describe pvc`; StorageClass ও CSI Pod যাচাই
Pod `Pending` — `unbound PVC`	PVC Bound হয়নি	PVC-এর অবস্থা আগে ঠিক করুন
Pod অন্য নোডে যেতে পারে না	RWO ভলিউম আগের নোডে আটকে আছে	আগের Pod সম্পূর্ণ মুছেছে কি না দেখুন; `volumeBindingMode` যাচাই
ডেটা হারিয়ে গেছে	emptyDir ব্যবহার; reclaimPolicy Delete; PVC ভুলে মুছে ফেলা	ব্যাকআপ থেকে পুনরুদ্ধার; ভবিষ্যতে Retain + ব্যাকআপ
RWX চেয়েও পেলেন না	StorageClass RWX সমর্থন করে না	NFS/ফাইল স্টোরেজের StorageClass ব্যবহার করুন
ডিস্ক পূর্ণ	লগ/ডেটা বেড়ে গেছে; আকার বাড়ানো যায়নি	PVC আকার বাড়ান; পুরোনো ডেটা পরিষ্কার করুন

TROUBLE: PVC Pending হলে `kubectl describe pvc`-এর Events অংশে আসল কারণ লেখা থাকে — “no persistent volumes available” নাকি “provisioner not found” — সেটি পড়লেই অর্ধেক সমস্যা মিটে যায়।

১৬.১১ নিরাপত্তা বিবেচনা

PV-তে ডাটাবেসের ডেটা থাকে — ডিস্কের এনক্রিপশন (encryption at rest) চালু আছে কি না যাচাই করুন, বিশেষ করে ক্লাউডে। ডাটাবেসের পাসওয়ার্ড কখনো YAML-এ সরাসরি লিখবেন না — Secret ব্যবহার করুন, আর এই বইয়ের উদাহরণের `CHANGE-ME-...` প্লেসহোল্ডার বাস্তবে বদলে নিন। PVC মুছলে `Delete` নীতিতে ডেটা মুছে যায় — প্রোডাকশন Namespace-এ PVC মুছতে RBAC দিয়ে সীমাবদ্ধ করুন। ব্যাকআপ ফাইলেও একই গোপন তথ্য থাকে — ব্যাকআপ এনক্রিপ্ট করে রাখুন। আর `hostPath` এড়িয়ে চলুন — এটি Pod-কে হোস্টের ফাইলসিস্টেমে পৌঁছে দেয়।

১৬.১২ কর্মক্ষমতা ও খরচ বিবেচনা

ডিস্কের ধরন ডাটাবেসের গতিতে সরাসরি প্রভাব ফেলে — SSD বনাম HDD-তে লেখার গতি কয়েকগুণ আলাদা হতে পারে। `volumeBindingMode: WaitForFirstConsumer` ব্যবহার করুন, যাতে ডিস্ক Pod-এর জোনে তৈরি হয় — ভুল জোনে

ডিস্ক হলে Pod চালু হতে দেরি হয় বা ব্যর্থ হয়। PVC-এর আকার বাড়ানো যায়, কমানো যায় না — তাই অতিরিক্ত বড় PVC নিলে ক্লাউড খরচ বাড়ে, আবার ছোট নিলে পরে বাড়াতে হয়। নিয়মিত ডিস্ক ব্যবহার মনিটর করুন (৮০% পূর্ণ হলে সতর্কতা)। স্ন্যাপশট ও ব্যাকআপের খরচও ধরুন — স্ন্যাপশট সংরক্ষণে আলাদা খরচ হয়।

১৬.১৩ সেরা অনুশীলন ও অ্যান্টি-প্যাটার্ন

সেরা অনুশীলন	অ্যান্টি-প্যাটার্ন
ডাটাবেস: StatefulSet + PVC + Retain + ব্যাকআপ	ডাটাবেস Deployment + emptyDir-এ চালানো
Dynamic provisioning ও StorageClass ব্যবহার	হাতে PV তৈরি করে প্রতিটি PVC মেলানো
WaitForFirstConsumer binding মোড	ভুল জোনে ডিস্ক তৈরি হওয়া
নিয়মিত ব্যাকআপ ও পুনরুদ্ধার পরীক্ষা	ব্যাকআপ নেওয়া কিন্তু কখনো ফিরিয়ে না দেখা
গোপন তথ্য Secret-এ রাখা	পাসওয়ার্ড YAML-এ সরাসরি লেখা

PRO TIP: ব্যাকআপ শুধু নিলেই হয় না — মাসে অন্তত একবার ব্যাকআপ থেকে পুনরুদ্ধার করে দেখুন। যে ব্যাকআপ ফিরিয়ে দেখা হয়নি, সেটি ব্যাকআপ নয়, আশা মাত্র।

১৬.১৪ অধ্যায়ের সারাংশ

Pod ক্ষণস্থায়ী, ডেটা স্থায়ী হতে হয় — তাই PV/PVC। PV হলো স্টোরেজের টুকরো, PVC হলো চাওয়া, StorageClass হলো dynamic provisioning-এর নীলনকশা, CSI হলো স্টোরেজ ড্রাইভারের মানক ইন্টারফেস। Access mode-এর মধ্যে RWO ডাটাবেসের জন্য, RWX শেয়ার করা ফাইলের জন্য — তবে সব স্টোরেজ RWX দেয় না। Reclaim policy-তে Retain ডেটা রাখে, Delete মুছে দেয় — প্রোডাকশন ডাটাবেসে Retain বেছে নিন। StatefulSet-এর volumeClaimTemplates প্রতিটি Pod-কে আলাদা PVC দেয়। আর সবচেয়ে গুরুত্বপূর্ণ: PVC থাকলেই ডেটা নিরাপদ নয় — নিয়মিত ব্যাকআপ ও পুনরুদ্ধার পরীক্ষা ছাড়া স্থায়ী স্টোরেজ অসম্পূর্ণ।

১৬.১৫ মূল প্রযুক্তিগত পরিভাষা

PersistentVolume (PV): ক্লাস্টারে উপলব্ধ স্থায়ী স্টোরেজের টুকরো।

PersistentVolumeClaim (PVC): Pod-এর স্টোরেজের চাওয়া — আকার, access mode ও StorageClass।

StorageClass: স্টোরেজের শ্রেণি, provisioner ও নীতি নির্ধারণ করা অবজেক্ট।

CSI: Container Storage Interface — Kubernetes ও স্টোরেজ ড্রাইভারের মানক ইন্টারফেস।

Dynamic Provisioning: PVC তৈরি হলে StorageClass দেখে স্বয়ংক্রিয়ভাবে PV তৈরি হওয়া।

Static Provisioning: অ্যাডমিনের আগে থেকে PV তৈরি করে রাখা।

Access Mode: ভলিউম কীভাবে মাউন্ট করা যাবে — RWO, ROX, RWX, RWOP।

Reclaim Policy: PVC মুছলে PV ও ডেটার কী হবে — Retain, Delete বা Recycle।

volumeClaimTemplates: StatefulSet-এ প্রতিটি Pod-এর জন্য আলাদা PVC তৈরি করার টেমপ্লেট।

emptyDir: Pod-এর জীবনকাল পর্যন্ত থাকা ক্ষণস্থায়ী ভলিউম; Pod মরলে মুছে যায়।

১৬.১৬ জ্ঞান যাচাই

১. Pod ক্ষণস্থায়ী হলে ডাটাবেসের ডেটা কেন হারায়? emptyDir কেন ডাটাবেসের জন্য উপযুক্ত নয়?

২. PV, PVC ও StorageClass-এর সম্পর্ক ব্যাখ্যা করুন। কে সরবরাহ, কে চাহিদা?

৩. Static provisioning ও dynamic provisioning-এর পার্থক্য কী?

৪. RWO, ROX ও RWX-এর অর্থ কী? MongoDB-র জন্য কোনটি ব্যবহার করবেন এবং কেন?

৫. Reclaim policy Retain ও Delete-এর পার্থক্য কী? প্রোডাকশন ডাটাবেসে কোনটি নিরাপদ?

৬. CSI কী? এটি কেন দরকার?

৭. StatefulSet-এ volumeClaimTemplates কী কাজ করে? Deployment-এ এটি নেই কেন?

৮. PVC Pending-এ আটকে থাকলে আপনি কী কী যাচাই করবেন?

১৬.১৭ ব্যবহারিক অ্যাসাইনমেন্ট

অ্যাসাইনমেন্ট: আপনার টেস্ট ক্লাস্টারে TechNova-র MongoDB-র জন্য একটি সম্পূর্ণ স্থায়ী স্টোরেজ সেটআপ তৈরি করুন: একটি StorageClass (`fast-ssd`, reclaimPolicy Retain, allowVolumeExpansion true), একটি StatefulSet (৩ replica, volumeClaimTemplates-এ ১০ Gi RWO PVC, মাউন্ট `/data/db`) এবং headless Service। MongoDB-তে কিছু নমুনা ডেটা লিখুন, তারপর একটি Pod মুছে দেখান নতুন Pod একই ডেটা নিয়ে ফিরে আসে। এরপর PVC-এর আকার বাড়ানোর চেষ্টা করুন এবং একটি `mongodump`-ভিত্তিক ব্যাকআপ পরিকল্পনা (কখন, কোথায়, কতদিন রাখবেন) এক পাতায় লিখুন।

PART IV • DEVOPS ENGINEERING MASTERY

Security & Operations

RBAC, ServiceAccount, Pod Security Standards, NetworkPolicy, ক্লাস্টার ট্রাবলশুটিং, মনিটরিং ও লগিং।

অধ্যায় 17 • PART IV — Security & Operations (ch 17–20)

Chapter 17 — RBAC, ServiceAccounts, Authentication, and Authorization

১৭.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন — **Authentication** (প্রমাণীকরণ: আপনি কে?) এবং **Authorization** (অনুমোদন: আপনি কী করতে পারবেন?) এর পার্থক্য স্পষ্টভাবে ব্যাখ্যা করতে; **ServiceAccount** কী, কেন Pod-এর নিজস্ব পরিচয় দরকার, এবং তার **token** কীভাবে কাজ করে তা বুঝতে; **RBAC** (Role-Based Access Control — ভূমিকা-ভিত্তিক প্রবেশ নিয়ন্ত্রণ) এর চারটি মূল অবজেক্ট — **Role**, **ClusterRole**, **RoleBinding**, **ClusterRoleBinding** — লিখতে ও প্রয়োগ করতে; **verb** (ক্রিয়া, যেমন `get`, `list`, `create`, `delete`), **resource** (সম্পদ, যেমন `pods`, `deployments`) এবং **apiGroup** দিয়ে ন্যূনতম অনুমতি (least privilege) নকশা করতে; এবং `kubectl auth can-i` দিয়ে যেকোনো অনুমতি যাচাই করতে। TechNova-র প্রেক্ষাপটে আপনি একজন জুনিয়র ডেভেলপারকে শুধু পড়ার অনুমতি, একজন CI/CD রোবটকে শুধু নির্দিষ্ট namespace-এ deploy করার অনুমতি, আর ক্লাস্টার অ্যাডমিনকে পূর্ণ নিয়ন্ত্রণ — এই তিনটি আলাদা করে দিতে পারবেন।

১৭.২ কেন এই বিষয়টি গুরুত্বপূর্ণ

Kubernetes-এর API server হলো পুরো ক্লাস্টারের দরজা। যে কেউ এই দরজা দিয়ে ঢুকতে পারলে Pod তৈরি করতে পারে, Secret পড়তে পারে, এমনকি পুরো ক্লাস্টার মুছে ফেলতে পারে। তাই দরজায় দুটি আলাদা প্রশ্নের উত্তর দরকার — প্রথমত, আপনি কে? দ্বিতীয়ত, আপনি কী করতে পারবেন? প্রথম প্রশ্নের উত্তর না দিলে দ্বিতীয় প্রশ্ন অর্থহীন; আবার পরিচয় জানা গেলেও সীমাহীন ক্ষমতা দেওয়া বিপজ্জনক। বাস্তব ঘটনা: অনেক প্রতিষ্ঠানে সবাইকে `cluster-admin` দিয়ে দেওয়া হয়, কারণ সেটাই সহজ। এর ফল — একজন ইন্টার্ন ভুল করে production namespace-এর সব Deployment মুছে ফেলে, অথবা একটি আপসকৃত (compromised) CI token দিয়ে আক্রমণকারী পুরো ক্লাস্টার দখল করে নেয়। TechNova-তে MERN অ্যাপের ডাটাবেসের পাসওয়ার্ড Secret-এ থাকে; যদি প্রতিটি ডেভেলপারের Pod সেই Secret পড়তে পারে, তবে একটি দুর্বল অ্যাপই পুরো ডাটাবেস ফাঁস করে দেবে। RBAC সঠিকভাবে প্রয়োগ করলে প্রত্যেকে শুধু ততটুকুই করতে পারে যতটুকু তার কাজের জন্য দরকার — এর নাম **least privilege** (ন্যূনতম সুবিধার নীতি)। এটি শুধু নিরাপত্তা নয়, এটি দলগত শৃঙ্খলা: কে কী ভেঙেছে, তা audit log থেকে খুঁজে পাওয়া যায়, কারণ প্রতিটি অনুরোধের সঙ্গে পরিচয় যুক্ত থাকে।

> **REAL-WORLD:** TechNova-র স্টেজিং ক্লাস্টারে একবার একজন ডেভেলপার পরীক্ষার জন্য `kubectl delete pods --all -n production` চালিয়ে ফেলেছিলেন — ভাগ্যক্রমে তিনি production-এ ছিলেন না, কারণ তার kubeconfig-এ শুধু staging namespace-এর অনুমতি ছিল। RBAC-ই সেদিন production বাঁচিয়েছিল।

১৭.৩ পূর্বশর্ত জ্ঞান

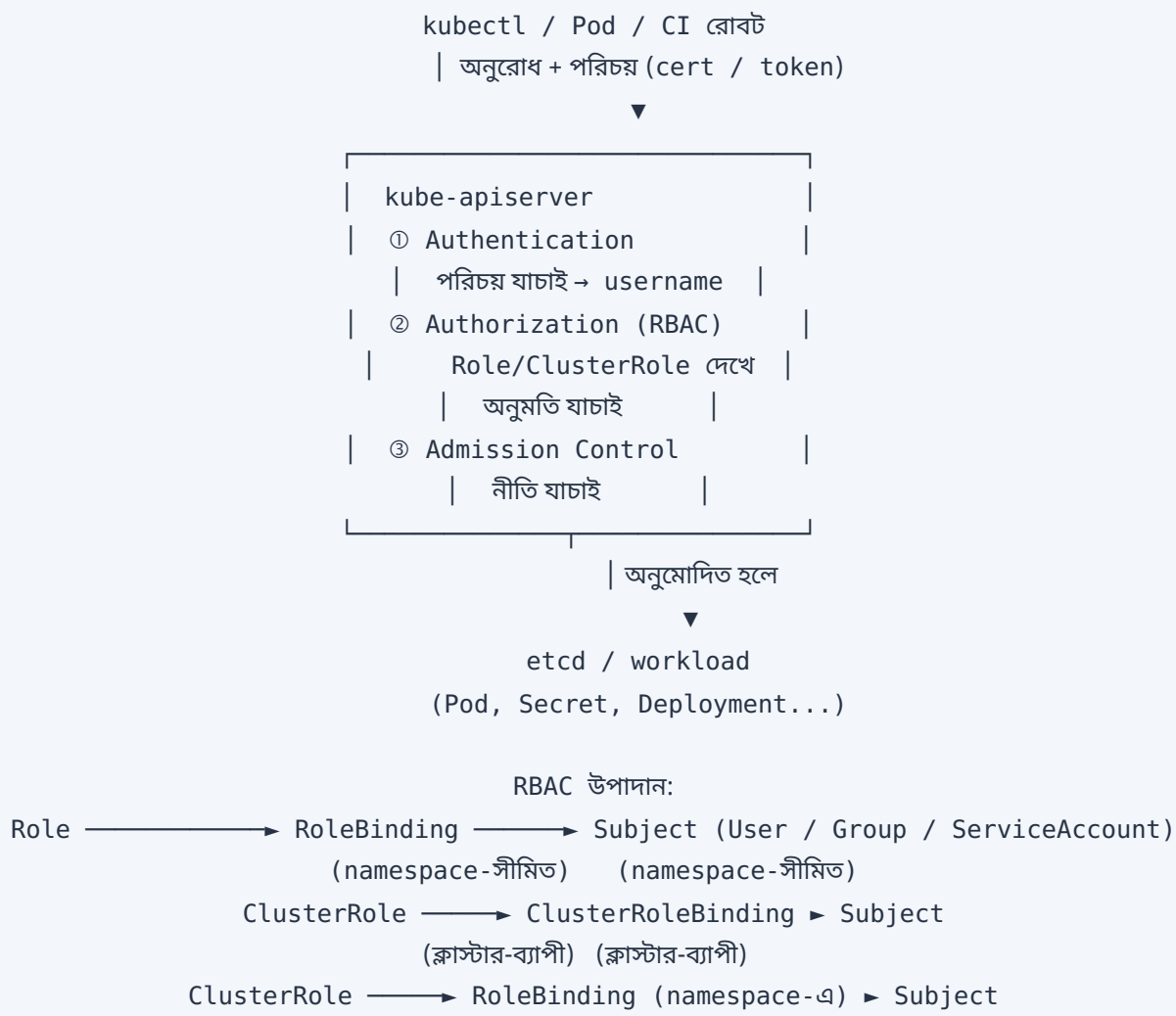
এই অধ্যায়ের জন্য আপনার জানা থাকা উচিত: Pod, Deployment, Namespace এবং Service-এর মৌলিক ধারণা (আগের অধ্যায়গুলো থেকে); `kubectl` দিয়ে ক্লাস্টারে সংযোগ এবং `kubeconfig` ফাইলের ধারণা; YAML-এর মৌলিক গঠন; এবং Linux-এর ফাইল অনুমতি (permission) সম্পর্কে সাধারণ ধারণা — যেমন একজন ব্যবহারকারীকে শুধু পড়ার অধিকার দেওয়া। API resource-এর নাম — যেমন `pods`, `services`, `deployments`, `secrets` — চেনা থাকলে ভালো হয়, না চিনলেও এই অধ্যায়েই শিখবেন।

১৭.৪ প্রথম নীতি থেকে ব্যাখ্যা

প্রথম নীতি থেকে ভাবুন: একটি কম্পিউটার সিস্টেমে যেকোনো কাজ করতে হলে দুটি জিনিস লাগে — পরিচয় এবং অনুমতি। আপনার বাড়ির দরজা ভাবুন। চাবি দিয়ে প্রমাণ হয় আপনি বাড়ির লোক (authentication)। কিন্তু বাড়ির লোক হলেই কি আপনি সিন্দুক খুলতে পারবেন? না — সিন্দুকের আলাদা চাবি লাগে (authorization)। Kubernetes-ও একইভাবে কাজ করে। **Authentication** হলো অনুরোধকারীর পরিচয় যাচাই: API server দেখে অনুরোধের সঙ্গে আসা client certificate, bearer token, অথবা OIDC token বৈধ কি না। পরিচয় যাচাই হলে API server একটি **username** এবং **groups** নির্ধারণ করে — যেমন `system:serviceaccount:technova:api-sa`। এরপর **Authorization** ধাপে API server RBAC নিয়ম দেখে: এই পরিচয়ের এই verb, এই resource, এই namespace-এ করার অধিকার আছে কি না? দুটি ধাপই পেরোলে তবেই অনুরোধ কার্যকর হয়। মাঝখানে আরও একটি ধাপ আছে — **Admission Control**, যা অনুরোধ বৈধ হলেও নীতির বিরুদ্ধে গেলে আটকে দেয় (যেমন Pod Security Standards)। মানুষ এবং Pod — দুই ধরনের পরিচয়ই আছে। মানুষ kubeconfig, certificate বা OIDC দিয়ে আসে; Pod আসে ServiceAccount token দিয়ে। ServiceAccount হলো Pod-এর পরিচয়পত্র — Pod যখন API server-কে কিছু জিজ্ঞেস করে (যেমন ConfigMap পড়া), তখন সে নিজের ServiceAccount token দেখায়।

> **CONCEPT:** Authentication = আপনি কে? Authorization = আপনি কী করতে পারবেন? দুটি সম্পূর্ণ আলাদা প্রশ্ন, আলাদা ব্যবস্থা। একটি ঠিক থাকলেই অন্যটি ঠিক হয় না।

১৭.৫ আর্কিটেকচার / ধারণাগত চিত্র



চিত্র ১৭.১ — API server-এ অনুরোধের তিন ধাপ এবং RBAC-এর চার অবজেক্টের সম্পর্ক।

উপরের চিত্রে লক্ষ করুন: Role শুধু একটি namespace-এর ভেতরে প্রযোজ্য, আর ClusterRole পুরো ক্লাস্টারে প্রযোজ্য। কিন্তু ClusterRole-কেও RoleBinding দিয়ে একটি namespace-এ বেঁধে দেওয়া যায় — তখন তার ক্ষমতা সেই namespace-এই সীমিত থাকে। Binding-ই হলো সংযোগ: কে (subject) কোন ভূমিকা (role) পেল, তা Binding বলে দেয়।

১৭.৬ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

১৭.৬.১ Authentication — পরিচয় যাচাইয়ের পদ্ধতি

Kubernetes-এর নিজস্ব ব্যবহারকারী ডাটাবেস নেই। API server বাইরের প্রমাণের ওপর নির্ভর করে। প্রধান পদ্ধতিগুলো: **Client certificate** — kubeconfig-এ থাকা X.509 সার্টিফিকেট; API server সার্টিফিকেটের Common Name (CN) থেকে username নেয়। **Bearer token** — `Authorization: Bearer <token>` হেডার; ServiceAccount token এই ধরনের। **OIDC** (OpenID Connect) — Google, GitHub, Okta বা কোম্পানির পরিচয়-প্রদানকারী (identity provider) থেকে আসা token; বড় দলে এটাই আদর্শ, কারণ কর্মী চলে গেলে এক জায়গা থেকে প্রবেশ বন্ধ করা যায়। **Webhook token authentication** — API server বাইরের সার্ভিসকে জিজ্ঞেস করে token বৈধ কি না। মনে রাখবেন: authentication

ব্যর্থ হলে API server `401 Unauthorized` দেয়; authorization ব্যর্থ হলে `403 Forbidden` দেয় — এই পার্থক্য দিয়ে সমস্যার মূল খুঁজে পাওয়া যায়।

১৭.৬.২ ServiceAccount — Pod-এর পরিচয়

প্রতিটি namespace-এ একটি `default` ServiceAccount থাকে। কোনো Pod যদি নিজের ServiceAccount না বলে, তবে সে `default` ব্যবহার করে। Pod চালু হলে Kubernetes তার ServiceAccount-এর token Pod-এর ভেতরে `/var/run/secrets/kubernetes.io/serviceaccount/token` ফাইলে মাউন্ট করে দেয় — অ্যাপ সেই token দিয়ে API server-কে ডাকতে পারে। কিন্তু শুধু token থাকলেই হবে না — সেই ServiceAccount-এর অনুমতি (RBAC) না থাকলে API server অনুরোধ ফিরিয়ে দেবে। তাই নিরাপদ নিয়ম: প্রতিটি অ্যাপের জন্য আলাদা ServiceAccount বানান, `default` ServiceAccount-কে কোনো অনুমতি দেবেন না, এবং যে Pod-এর API server-এর সঙ্গে কথা বলার দরকার নেই, তার `automountServiceAccountToken: false` করে দিন।

> **ServiceAccount:** Pod বা সিস্টেম উপাদানের জন্য Kubernetes-এর ভেতরে তৈরি পরিচয়। মানুষের অ্যাকাউন্টের মতো, তবে এটি ক্লাস্টারের ভেতরের কাজের জন্য ব্যবহৃত হয়।

১৭.৬.৩ RBAC-এর চার অবজেক্ট

অবজেক্ট	সীমা	কাজ	উদাহরণ
Role	একটি namespace	অনুমতির তালিকা	technova namespace-এ Pod পড়া
ClusterRole	পুরো ক্লাস্টার	অনুমতির তালিকা	সব namespace-এ Pod পড়া; nodes দেখা
RoleBinding	একটি namespace	Role/ClusterRole কে পেল	dev-কে technova-তে pod-reader Role দেওয়া
ClusterRoleBinding	পুরো ক্লাস্টার	ClusterRole কে পেল	admin-কে cluster-admin দেওয়া

প্রতিটি Role-এর **rules**-এ তিনটি জিনিস থাকে: `apiGroups` (যেমন `""` মানে core API, `apps` মানে Deployment-এর গ্রুপ), `resources` (যেমন `pods`, `services`, `secrets`), এবং `verbs` (যেমন `get`, `list`, `watch`, `create`, `update`, `patch`, `delete`)। `verbs: ["*"]` মানে সব ক্রিয়া — এটি প্রায় কখনোই দেবেন না, শুধু cluster-admin-এ থাকে। RBAC হলো **additive** (যোগমূলক): অনুমতি শুধু যোগ হয়, কখনো বিয়োগ হয় না; কোনো Role-এ "এই কাজটি করা যাবে না" বলার উপায় নেই। তাই নিরাপত্তা আসে অনুমতি না দেওয়া থেকে, নিষেধ করা থেকে নয়।

১৭.৬.৪ Least Privilege নকশা করা

শুরুতে ভাবুন: এই ব্যক্তি বা Pod-এর আসলে কী কী করা দরকার? একজন ডেভেলপারের হয়তো শুধু `get`, `list`, `logs` দরকার নিজের namespace-এ; CI রোবটের দরকার `create`, `update`, `patch` শুধু Deployment-এ; আর Secret পড়ার অধিকার শুধু সেই Pod-এর, যে Pod-এর সত্যিই Secret দরকার। ধাপে ধাপে অনুমতি বাড়ান, একবারে সব দেবেন না। `kubectl auth can-i --list` দিয়ে নিজের অনুমতি দেখে নিন, আর `kubectl auth can-i create deployments -n technova --as=dev-user` দিয়ে অন্যের অনুমতি পরীক্ষা করুন।

১৭.৬.৫ সাধারণ ভুল

সবচেয়ে বড় ভুল: সবাইকে `cluster-admin` দেওয়া। দ্বিতীয় ভুল: `default` ServiceAccount-কে শক্তিশালী Role দেওয়া — তাতে namespace-এর প্রতিটি Pod সেই ক্ষমতা পেয়ে যায়। তৃতীয় ভুল: Secret-এর ওপর `get` অনুমতি অযথা দেওয়া — Secret পড়তে পারলে ডাটাবেসের পাসওয়ার্ড, API key সব ফাঁস। চতুর্থ ভুল: `verbs: ["*"]` আর `resources:`

`["*"]` একসঙ্গে দেওয়া — এটি কার্যত cluster-admin-এর সমান। পঞ্চম ভুল: RoleBinding-এর বদলে ClusterRoleBinding ব্যবহার করে namespace-সীমিত কাজ ক্লাস্টার-ব্যাপী করে ফেলা।

১৭.৭ কনফিগারেশন উদাহরণ

```
# ServiceAccount: TechNova API Pod-এর পরিচয়
apiVersion: v1
kind: ServiceAccount
metadata:
  name: api-sa
  namespace: technova
automountServiceAccountToken: true
---
# Role: technova namespace-এ শুধু Pod ও ConfigMap পড়া
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: pod-reader
  namespace: technova
rules:
- apiGroups: [""]
  resources: ["pods", "configmaps"]
  verbs: ["get", "list", "watch"]
- apiGroups: [""]
  resources: ["pods/log"]
  verbs: ["get"]
---
# RoleBinding: pod-reader Role api-sa-কে দেওয়া
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: read-pods
  namespace: technova
subjects:
- kind: ServiceAccount
  name: api-sa
  namespace: technova
roleRef:
  kind: Role
  name: pod-reader
  apiGroup: rbac.authorization.k8s.io
---
# ClusterRole: শুধু Deployment তৈরি/হালনাগাদ (CI রোবটের জন্য)
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: deployer
rules:
- apiGroups: ["apps"]
  resources: ["deployments"]
  verbs: ["get", "list", "watch", "create", "update", "patch"]
---
# Pod-এ ServiceAccount যুক্ত করা
apiVersion: v1
kind: Pod
metadata:
  name: api-pod
```

```
  namespace: technova
spec:
  serviceAccountName: api-sa
  automountServiceAccountToken: true
  containers:
  - name: api
    image: technova/api:v1.2.0
```

```
# অনুমতি যাচাই
kubectl auth can-i list pods -n technova
kubectl auth can-i delete secrets -n technova
kubectl auth can-i create deployments -n technova --as=system:serviceaccount:technova:api-sa
kubectl auth can-i --list -n technova
# প্রয়োগ
kubectl apply -f rbac.yaml
kubectl get role,rolebinding -n technova
```

১৭.৮ বাস্তব প্রকৌশল পরিস্থিতি

TechNova-র তিনটি দল: ডেভেলপার দল শুধু `technova` namespace-এ কাজ করে; QA দল `staging` namespace-এ Pod-এর log দেখে; আর GitHub Actions থেকে আসা CI রোবট `technova` namespace-এ Deployment হালনাগাদ করে। আগে সবাই একটাই kubeconfig ব্যবহার করত, যাতে cluster-admin ছিল। একদিন CI-এর token ফাঁস হয়ে যায় (ভুল করে public লগে ছাপা হয়), আর আক্রমণকারী সেই token দিয়ে ক্লাস্টারের সব Secret পড়ার চেষ্টা করে। নতুন নকশায়: CI-এর ServiceAccount-কে শুধু `deployer` ClusterRole দেওয়া হয়েছে RoleBinding দিয়ে — শুধু `technova` namespace-এ, শুধু Deployment-এ, Secret পড়ার অধিকার একেবারেই নেই। ডেভেলপারদের দেওয়া হয়েছে `pod-reader` আর `edit`-এর মতো সীমিত Role। এখন token ফাঁস হলেও ক্ষতি সীমিত: আক্রমণকারী Deployment বদলাতে পারবে, কিন্তু Secret পড়তে বা Pod মুছতে পারবে না। এটাই least privilege-এর বাস্তব মূল্য — দুর্ঘটনা বা আক্রমণের blast radius (ক্ষতির পরিধি) ছোট রাখা।

১৭.৯ ধাপে ধাপে ব্যবহারিক উদাহরণ

ধাপ ১: namespace তৈরি করুন: `kubectl create namespace technova`। **ধাপ ২:** ServiceAccount তৈরি করুন: `kubectl create serviceaccount api-sa -n technova`। **ধাপ ৩:** উপরের `rbac.yaml` ফাইলটি লিখে `kubectl apply -f rbac.yaml` চালান। **ধাপ ৪:** যাচাই করুন: `kubectl auth can-i list pods -n technova --as=system:serviceaccount:technova:api-sa` — উত্তর `yes` আসা উচিত; `kubectl auth can-i delete pods -n technova --as=system:serviceaccount:technova:api-sa` — উত্তর `no` আসা উচিত। **ধাপ ৫:** একটি Pod-এ `serviceAccountName: api-sa` দিয়ে চালান, তারপর Pod-এর ভেতরে ঢুকে দেখুন: `kubectl exec -it api-pod -n technova -- cat /var/run/secrets/kubernetes.io/serviceaccount/token` — token দেখা যাবে। **ধাপ ৬:** Pod-এর ভেতর থেকে `curl` দিয়ে API ডাকার চেষ্টা করুন — পড়ার অনুরোধ সফল হবে, মুছে ফেলার অনুরোধ `403 Forbidden` দেবে। **ধাপ ৭:** ভুল করে `default` ServiceAccount-কে অনুমতি দিয়েছেন কি না, তা পরীক্ষা করুন: `kubectl get rolebindings -n technova`।

> **LAB:** নিজের নামে একটি Role তৈরি করুন যাতে শুধু `get` ও `list` আছে `services`-এর ওপর, তারপর `kubectl auth can-i` দিয়ে প্রমাণ করুন যে `create` কাজ করে না।

১৭.১০ সাধারণ ভুল ও সমস্যা সমাধান

সমস্যা / বার্তা	সম্ভাব্য কারণ	সমাধান
`403 Forbidden`	Role-এ verb বা resource নেই	Role-এর rules দেখুন; `kubectl auth can-i --list` চালান
`401 Unauthorized`	token/certificate মেয়াদোত্তীর্ণ বা ভুল	kubeconfig বা token নবায়ন করুন
Pod API ডাকতে পারছে না	ServiceAccount ভুল বা token মাউন্ট হয়নি	`serviceAccountName` ও token ফাইল পরীক্ষা করুন
সবাই সবকিছু করতে পারছে	কেউ cluster-admin দিয়ে রেখেছে	ClusterRoleBinding তালিকা দেখে অপ্রয়োজনীয় binding মুছুন
RoleBinding কাজ করছে না	namespace ভুল বা roleRef-এর নাম ভুল	`kubectl describe rolebinding -n technova` দেখুন

মনে রাখবেন: RBAC পরিবর্তন সঙ্গে সঙ্গে কার্যকর হয়, Pod রিস্টার্ট লাগে না। কিন্তু token-এর মেয়াদ বা kubeconfig-এর সমস্যা হলে নতুন করে সংযোগ করতে হয়।

১৭.১১ নিরাপত্তা বিবেচনা

kubeconfig ফাইল কখনো Git-এ, চ্যাটে বা public জায়গায় রাখবেন না — তাতে cluster-admin-এর চাবি ফাঁস হয়ে যায়। ServiceAccount token-এর মেয়াদ সীমিত রাখুন; Kubernetes ১.২১+ সংস্করণে token স্বয়ংক্রিয়ভাবে নবায়ন হয়। Secret পড়ার অনুমতি শুধু সেই Pod-কে দিন, যার সত্যিই দরকার। নিয়মিত audit করুন: `kubectl get clusterrolebindings` দেখে কে cluster-admin পেয়েছে, তা যাচাই করুন। API server কখনোই পাবলিক ইন্টারনেটে খুলে দেবেন না — শুধু VPN বা নির্দিষ্ট IP থেকে প্রবেশের ব্যবস্থা করুন। OIDC ব্যবহার করলে কর্মী চলে গেলে তার প্রবেশ এক জায়গা থেকে বন্ধ করুন।

> **WARNING:** `cluster-admin` হলো ক্লাস্টারের master key। এটি শুধু দুই-একজন বিশ্বস্ত অ্যাডমিনের জন্য। কাউকে সাময়িকভাবে দিলেও কাজ শেষে ফিরিয়ে নিন।

১৭.১২ কর্মক্ষমতা ও খরচ বিবেচনা

RBAC-এর কর্মক্ষমতার খরচ নগণ্য — API server প্রতিটি অনুরোধে নিয়ম মেলায়, তবে তা মিলিসেকেন্ডের ভগ্নাংশ। খরচের আসল দিক হলো ব্যবস্থাপনা: শত শত Role হাতে লেখা কঠিন, তাই Helm বা Kustomize দিয়ে RBAC টেমপ্লেট করুন (অধ্যায় ২১–২২)। ভুল RBAC-এর খরচ অনেক বেশি: একটি ভুল binding-এ production ডাউন হতে পারে, অথবা নিরাপত্তা লঙ্ঘনের জরিমানা আসতে পারে। তাই RBAC-কে কোড হিসেবে Git-এ রাখুন, পরিবর্তনের আগে পর্যালোচনা (review) করুন।

১৭.১৩ সেরা অনুশীলন ও অ্যান্টি-প্যাটার্ন

সেরা অনুশীলন	অ্যান্টি-প্যাটার্ন

প্রতিটি অ্যাপের জন্য আলাদা ServiceAccount	`default` ServiceAccount-কে শক্তিশালী Role দেওয়া
Least privilege: শুধু দরকারি verb/resource	`verbs: ["*"]`, `resources: ["*"]` দেওয়া
RoleBinding (namespace-সীমিত) আগে ভাবুন	সবকিছুতে ClusterRoleBinding ব্যবহার
RBAC Git-এ রাখুন, review করুন	হাতে `kubectl create clusterrolebinding` দিয়ে দেওয়া
নিয়মিত `auth can-i --list` দিয়ে যাচাই	অনুমতি না দেখে অনুমান করা

১৭.১৪ অধ্যায়ের সারাংশ

Authentication বলে আপনি কে, Authorization বলে আপনি কী করতে পারবেন। মানুষ kubeconfig, certificate বা OIDC দিয়ে, Pod ServiceAccount token দিয়ে পরিচয় দেয়। RBAC-এর চার অবজেক্ট — Role, ClusterRole, RoleBinding, ClusterRoleBinding — দিয়ে apiGroups, resources ও verbs-এর নিয়ম বেঁধে দেওয়া হয়। RBAC যোগমূলক, তাই নিরাপত্তা আসে অনুমতি না দেওয়া থেকে। Least privilege মেনে চলুন, cluster-admin সীমিত রাখুন, `kubectl auth can-i` দিয়ে যাচাই করুন।

১৭.১৫ মূল প্রযুক্তিগত পরিভাষা

Authentication: পরিচয় যাচাই — অনুরোধকারী আসলেই সে কি না, তা নিশ্চিত করা।

Authorization: অনুমোদন — পরিচয় জানা ব্যক্তি কোন কাজ করতে পারবে, তা নির্ধারণ করা।

RBAC: Role-Based Access Control — ভূমিকার ভিত্তিতে প্রবেশ নিয়ন্ত্রণের ব্যবস্থা।

ServiceAccount: Pod বা সিস্টেম উপাদানের জন্য Kubernetes-এর ভেতরের পরিচয়।

Role / ClusterRole: অনুমতির তালিকা; Role একটি namespace-এ, ClusterRole পুরো ক্লাস্টারে প্রযোজ্য।

RoleBinding / ClusterRoleBinding: কোন subject কোন Role পেল, সেই সংযোগ।

Least Privilege: কাজের জন্য যতটুকু দরকার, ঠিক ততটুকু অনুমতি দেওয়ার নীতি।

OIDC: OpenID Connect — বাইরের পরিচয়-প্রদানকারী দিয়ে লগইন যাচাইয়ের মানদণ্ড।

১৭.১৬ জ্ঞান যাচাই

১. Authentication ও Authorization-এর পার্থক্য নিজের ভাষায় লিখুন। ২. Pod কেন ServiceAccount ব্যবহার করে? ৩. Role ও ClusterRole-এর পার্থক্য কী? ৪. RoleBinding ও ClusterRoleBinding কখন কোনটি ব্যবহার করবেন? ৫. `verbs: ["*"]` দেওয়া কেন বিপজ্জনক? ৬. `kubectl auth can-i` কী কাজে লাগে? উদাহরণ দিন। ৭. `403 Forbidden` ও `401 Unauthorized`-এর পার্থক্য কী? ৮. একজন ডেভেলপারকে শুধু একটি namespace-এ Pod পড়ার অনুমতি দিতে কী কী অবজেক্ট লাগবে?

১৭.১৭ ব্যবহারিক অ্যাসাইনমেন্ট

TechNova-র জন্য একটি সম্পূর্ণ RBAC সেট তৈরি করুন: (ক) `technova` namespace-এ `api-sa` ServiceAccount; (খ) `pod-reader` Role (শুধু `get`, `list` Pod ও ConfigMap-এ); (গ) `deployer` ClusterRole (শুধু Deployment-এ `get`, `list`, `create`, `update`, `patch`) এবং তাকে RoleBinding দিয়ে শুধু `technova` namespace-এ বাঁধা; (ঘ) একটি CI ServiceAccount `ci-sa` তৈরি করে তাকে deployer দিন। তারপর `kubectl auth can-i` দিয়ে প্রমাণ করুন: `api-sa` Secret পড়তে পারে না, `ci-sa` Pod মুছতে পারে না। ফলাফল একটি সংক্ষিপ্ত প্রতিবেদনে লিখুন।

অধ্যায় 18 • PART IV — Security & Operations (ch 17–20)

Chapter 18 — Pod Security Standards, SecurityContext, and NetworkPolicies

১৮.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন — **Pod Security Standards** (Pod নিরাপত্তা মানদণ্ড) এর তিনটি স্তর — **Privileged**, **Baseline**, **Restricted** — ব্যাখ্যা ও প্রয়োগ করতে; **SecurityContext** দিয়ে Pod ও container-কে root ছাড়া চালানো, ফাইলসিস্টেম read-only করা এবং ক্ষমতা (capabilities) কমানো; **NetworkPolicy** দিয়ে Pod-এর মধ্যে নেটওয়ার্ক ট্রাফিক নিয়ন্ত্রণ করতে, বিশেষ করে **default deny** (ডিফল্টভাবে সব বন্ধ) নীতি লিখতে; এবং defense in depth (গভীর প্রতিরক্ষা) নীতিতে RBAC, Pod নিরাপত্তা ও নেটওয়ার্ক নীতি একসঙ্গে সাজাতে।

১৮.২ কেন এই বিষয়টি গুরুত্বপূর্ণ

একটি Pod আপসকৃত হলে কী হতে পারে, তা নির্ভর করে Pod-টি কতটা ক্ষমতা নিয়ে চলছে তার ওপর। Root ব্যবহারকারী হিসেবে চলা, host-এর ফাইলসিস্টেম দেখতে পারা, অথবা যেকোনো Pod-এর সঙ্গে কথা বলতে পারা Pod — এগুলো আক্রমণকারীর জন্য সোনার খনি। একটি দুর্বল MERN অ্যাপে ঢুকে আক্রমণকারী যদি host-এর root পেয়ে যায়, তবে সে পুরো node দখল করতে পারে, পাশের Pod-এর Secret চুরি করতে পারে, এমনকি ক্লাস্টারজুড়ে ছড়িয়ে পড়তে পারে। Pod Security Standards, SecurityContext ও NetworkPolicy — এই তিনটি একসঙ্গে Pod-কে একটি নিরাপদ খাঁচায় রাখে: Pod ভেঙে গেলেও ক্ষতি তার ভেতরেই আটকে থাকে। TechNova-তে গ্রাহকের তথ্য থাকে; একটি Pod-এর দুর্বলতা যেন পুরো প্ল্যাটফর্মের দুর্বলতা না হয়, সেটাই এই অধ্যায়ের লক্ষ্য।

১৮.৩ পূর্বশর্ত জ্ঞান

আপনার জানা থাকা উচিত: Pod, Deployment, Namespace ও Service-এর ধারণা; RBAC-এর মৌলিক ধারণা (অধ্যায় ১৭); Linux-এ root ব্যবহারকারী ও file permission-এর ধারণা; এবং নেটওয়ার্কের মৌলিক ধারণা — IP, port, ingress (ভেতরে আসা ট্রাফিক) ও egress (বাইরে যাওয়া ট্রাফিক)। YAML-এ Pod spec লেখার অভিজ্ঞতা থাকলে ভালো হয়।

১৮.৪ প্রথম নীতি থেকে ব্যাখ্যা

প্রথম নীতি: প্রতিটি Pod একটি ছোট কম্পিউটারের মতো, কিন্তু সে একটি বড় বাড়ির (node) ভেতরে ভাড়া থাকে। ভাড়াটে যদি বাড়ির মূল চাবি পায়, তবে সে সব ঘরে ঢুকতে পারবে। তাই ভাড়াটেকে শুধু নিজের ঘরের চাবি দিতে হয়। Linux-এ **root** হলো মূল চাবি — root যেকোনো ফাইল পড়তে, যেকোনো প্রক্রিয়া থামাতে পারে। **SecurityContext** বলে দেয় Pod কোন ব্যবহারকারী হিসেবে চলবে, সে root হতে পারবে কি না, ফাইলসিস্টেমে লিখতে পারবে কি না। **capabilities** হলো root-এর ক্ষমতাকে ছোট ছোট টুকরোয় ভাগ করা — যেমন শুধু নেটওয়ার্ক port বাঁধার ক্ষমতা। **Pod Security Standards** হলো বাড়ির নিয়মাবলি: কোন ভাড়াটে কী করতে পারবে, তার তিনটি স্তর। আর **NetworkPolicy** হলো বাড়ির দরজা ও করিডরের নিয়ম: কোন ঘর কোন ঘরের সঙ্গে কথা বলতে পারবে। এই তিন স্তর মিলে defense in depth তৈরি হয় — একটি স্তর ভেঙে গেলেও পরের স্তর আটকে দেয়।

CONCEPT: Defense in depth মানে একটিমাত্র দেয়ালের ওপর ভরসা না করে একাধিক স্তরে প্রতিরক্ষা সাজানো — RBAC, Pod নিরাপত্তা, নেটওয়ার্ক নীতি, আর ইমেজ স্ক্যানিং একসঙ্গে কাজ করে।

১৮.৫ আর্কিটেকচার / ধারণাগত চিত্র



চিত্র ১৮.১ — TechNova namespace-এ Pod নিরাপত্তা, SecurityContext ও NetworkPolicy একসঙ্গে।

১৮.৬ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

১৮.৬.১ Pod Security Standards — তিনটি স্তর

স্তর	অর্থ	কখন ব্যবহার করবেন
Privileged	প্রায় কোনো বিধিনিষেধ নেই; root, host access সম্ভব	শুধু সিস্টেম Pod (যেমন CNI, storage driver) — অ্যাপের জন্য নয়
Baseline	পরিচিত ঝুঁকিপূর্ণ কাজ বন্ধ (hostNetwork, hostPID, privileged container)	সাধারণ অ্যাপের সর্বনিম্ন মান
Restricted		production অ্যাপের লক্ষ্য

	সবচেয়ে কঠোর: non-root, read-only filesystem, সব capability বাদ	

Pod Security Admission হলো Kubernetes-এর অন্তর্নির্মিত ব্যবস্থা যা namespace-এর label দেখে Pod তৈরি আটকে দেয়। যেমন namespace-এ `pod-security.kubernetes.io/enforce: restricted` label থাকলে Restricted মান না মানা Pod তৈরি হতে পারবে না। তিনটি mode আছে: `enforce` (আটকে দেয়), `audit` (লগে লেখে কিন্তু চলতে দেয়), `warn` (সতর্কবার্তা দেয়)। নতুন namespace-এ প্রথমে `audit` দিয়ে শুরু করুন, তারপর `enforce` করুন — নইলে হঠাৎ সব Deployment ভেঙে যেতে পারে।

১৮.৬.২ SecurityContext — Pod ও container-এর নিরাপত্তা সেটিংস

SecurityContext দুই স্তরে থাকে: **pod-level** (`spec.securityContext`) — পুরো Pod-এর জন্য, যেমন `runAsUser: 1000`, `runAsNonRoot: true`, `fsGroup: 2000`; আর **container-level** (`spec.containers[].securityContext`) — একটি container-এর জন্য, যেমন `allowPrivilegeEscalation: false`, `readOnlyRootFilesystem: true`, `capabilities`। গুরুত্বপূর্ণ ক্ষেত্রগুলো: `runAsNonRoot: true` — root হিসেবে চলতে দেবে না; `runAsUser: 1000` — নির্দিষ্ট UID-তে চালায়; `readOnlyRootFilesystem: true` — root ফাইলসিস্টেমে লেখা বন্ধ, লেখার দরকার হলে `emptyDir` volume ব্যবহার করতে হয়; `allowPrivilegeEscalation: false` — প্রক্রিয়া নিজের ক্ষমতা বাড়াতে পারবে না; `capabilities: drop: ["ALL"]` — সব Linux capability বাদ, দরকার হলে শুধু একটি যোগ করুন (যেমন `NET_BIND_SERVICE`)। `seccompProfile: type: RuntimeDefault` — সিস্টেম কলের ওপর সীমা বসায়।

> **SecurityContext:** Pod বা container কীভাবে চলবে — কোন ব্যবহারকারী, কী ক্ষমতা, ফাইলসিস্টেমে লেখা যাবে কি না — তা নির্ধারণকারী সেটিংসের গুচ্ছ।

১৮.৬.৩ NetworkPolicy — Pod-এর মধ্যে ফায়ারওয়াল

ডিফল্টভাবে Kubernetes-এ সব Pod সব Pod-এর সঙ্গে কথা বলতে পারে — এটি সুবিধাজনক, কিন্তু নিরাপদ নয়। NetworkPolicy হলো Pod-স্তরের ফায়ারওয়াল: কোন Pod কোন Pod-কে, কোন port-এ ডাকতে পারবে, তা নিয়ম দিয়ে বেঁধে দেওয়া হয়। গুরুত্বপূর্ণ নিয়ম: একটি namespace-এ কোনো Pod-কে NetworkPolicy দিয়ে নির্বাচন (select) করা মাত্রই সেই Pod-এর জন্য **default deny** চালু হয়ে যায় — যা স্পষ্টভাবে অনুমোদিত নয়, তা সব বন্ধ। তাই প্রথম নীতিটি হওয়া উচিত সব ট্রাফিক বন্ধ করা, তারপর শুধু দরকারি পথ খুলে দেওয়া। NetworkPolicy-তে `podSelector` (কোন Pod-এর ওপর নীতি প্রযোজ্য), `ingress` (কারা ঢুকতে পারবে) ও `egress` (কোথায় বেরোতে পারবে) থাকে; `namespaceSelector` দিয়ে পুরো namespace-কে অনুমতি দেওয়া যায়, `ipBlock` দিয়ে বাইরের IP-কে নিয়ন্ত্রণ করা যায়। মনে রাখবেন: NetworkPolicy কাজ করে শুধু তখনই, যখন CNI plugin (যেমন Calico, Cilium) তা সমর্থন করে — সব CNI সমর্থন করে না।

১৮.৬.৪ Defense in Depth — স্তরে স্তরে প্রতিরক্ষা

RBAC ঠিক করে কে API-তে কী করতে পারে; Pod Security ঠিক করে Pod কতটা ক্ষমতা নিয়ে চলবে; NetworkPolicy ঠিক করে Pod কার সঙ্গে কথা বলতে পারবে; ইমেজ স্ক্যানিং ঠিক করে ভেতরে দুর্বলতা আছে কি না। একটি স্তর ভেঙে গেলেও পরের স্তর আটকে দেয়। যেমন আক্রমণকারী API Pod-এ ঢুকলেও, NetworkPolicy তাকে MongoDB Pod-এ যেতে দেবে না, আর read-only filesystem তাকে ম্যালওয়্যার লিখতে দেবে না।

১৮.৭ কনফিগারেশন উদাহরণ

```
# Namespace-এ Pod Security Standards চালু করা (enforce: restricted)
apiVersion: v1
kind: Namespace
metadata:
  name: technova
  labels:
    pod-security.kubernetes.io/enforce: restricted
    pod-security.kubernetes.io/audit: restricted
    pod-security.kubernetes.io/warn: restricted
---
# নিরাপদ Pod: SecurityContext সহ
apiVersion: v1
kind: Pod
metadata:
  name: api-secure
  namespace: technova
spec:
  securityContext:
    runAsNonRoot: true
    runAsUser: 1000
    fsGroup: 2000
    seccompProfile:
      type: RuntimeDefault
  containers:
  - name: api
    image: technova/api:v1.2.0
    ports:
    - containerPort: 3000
    securityContext:
      allowPrivilegeEscalation: false
      readOnlyRootFilesystem: true
      runAsNonRoot: true
      capabilities:
        drop: ["ALL"]
    volumeMounts:
    - name: tmp
      mountPath: /tmp
  volumes:
  - name: tmp
    emptyDir: {}
---
# NetworkPolicy: default deny (সব ingress ও egress বন্ধ)
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny
  namespace: technova
spec:
  podSelector: {}
  policyTypes: ["Ingress", "Egress"]
---
```

```
# NetworkPolicy: শুধু frontend থেকে api, আর api থেকে mongo
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: api-allow
  namespace: technova
spec:
  podSelector:
    matchLabels:
      app: api
  policyTypes: ["Ingress", "Egress"]
  ingress:
  - from:
    - podSelector:
        matchLabels:
          app: frontend
    ports:
    - protocol: TCP
      port: 3000
  egress:
  - to:
    - podSelector:
        matchLabels:
          app: mongo
    ports:
    - protocol: TCP
      port: 27017
  - to:
    - namespaceSelector:
        matchLabels:
          kubernetes.io/metadata.name: kube-system
    ports:
    - protocol: UDP
      port: 53
```

উপরের শেষ egress নিয়মটি DNS-এর জন্য: Pod-কে DNS সার্ভারে (port 53) যেতে দিতে হয়, নইলে নাম দিয়ে Service খুঁজে পাবে না। এটি ভুলে গেলে অ্যাপ হঠাৎ DNS ব্যর্থতায় ভুগবে — অধ্যায় ১৯-এ এর সমস্যা সমাধান আছে।

১৮.৮ বাস্তব প্রকৌশল পরিস্থিতি

TechNova-র frontend Pod একবার একটি দুর্বল লাইব্রেরির কারণে আপসকৃত হয়। আক্রমণকারী Pod-এর ভেতরে শেল পায়। কিন্তু Pod-টি `runAsNonRoot` ও `readOnlyRootFilesystem` দিয়ে চলছিল, তাই সে ফাইলসিস্টেমে কিছু লিখতে পারেনি, root হতে পারেনি। সে MongoDB-তে যাওয়ার চেষ্টা করে, কিন্তু NetworkPolicy শুধু api Pod-কে mongo-তে যাওয়ার অনুমতি দেয় — frontend থেকে mongo-তে যাওয়া বন্ধ। সে বাইরে তথ্য পাঠানোর চেষ্টা করে, কিন্তু egress নীতিতে শুধু DNS অনুমোদিত। ফলে আক্রমণ Pod-এর ভেতরেই আটকে যায়, আর লগ ও অ্যালার্ট থেকে দল দ্রুত তা ধরে ফেলে। এই ঘটনার পর TechNova সব namespace এ `restricted` enforce করে এবং default-deny NetworkPolicy বাধ্যতামূলক করে।

> **WARNING:** Default-deny NetworkPolicy চালু করার আগে DNS (port 53) ও দরকারি egress পথ খুলে নিন — নইলে বৈধ অ্যাপও কাজ করা বন্ধ করে দেবে।

১৮.৯ ধাপে ধাপে ব্যবহারিক উদাহরণ

ধাপ ১: একটি পরীক্ষামূলক namespace তৈরি করুন এবং `audit` mode-এ Restricted label দিন। **ধাপ ২:** একটি সাধারণ Pod (root হিসেবে চলা) তৈরি করার চেষ্টা করুন `warn` বার্তা দেখুন। **ধাপ ৩:** উপরের নিরাপদ Pod YAML প্রয়োগ করুন — দেখুন Pod চলছে। **ধাপ ৪:** default-deny NetworkPolicy প্রয়োগ করুন, তারপর frontend Pod থেকে api Pod-এ `curl` করার চেষ্টা করুন — ব্যর্থ হবে। **ধাপ ৫:** api-allow নীতি প্রয়োগ করুন — এবার শুধু অনুমোদিত পথে সংযোগ সফল হবে। **ধাপ ৬:** `kubectl get networkpolicy -n technova` ও `kubectl describe networkpolicy api-allow -n technova` দিয়ে নীতি যাচাই করুন। **ধাপ ৭:** একটি Pod-এ `kubectl exec` করে দেখুন সে root কি না — `id` কমান্ডে UID 1000 দেখা উচিত, 0 নয়।

১৮.১০ সাধারণ ভুল ও সমস্যা সমাধান

সমস্যা	কারণ	সমাধান
Pod তৈরি হচ্ছে না: `Forbidden: violates PodSecurity`	Restricted মান মানা হয়নি	SecurityContext যোগ করুন; প্রথমে audit mode ব্যবহার করুন
অ্যাপ লিখতে পারছে না: `read-only filesystem`	অ্যাপ /tmp বা লগ ফাইলে লিখতে চায়	emptyDir volume মাউন্ট করুন
Pod-এর মধ্যে সংযোগ ব্যর্থ	NetworkPolicy default deny চালু	ingress/egress নিয়মে অনুমোদিত পথ যোগ করুন
DNS কাজ করছে না	egress-এ DNS (53/UDP) বন্ধ	kube-system namespace-এ DNS অনুমতি দিন
NetworkPolicy কাজ করছে না	CNI plugin নীতি সমর্থন করে না	CNI (Calico/Cilium) সমর্থন যাচাই করুন
পুরোনো ইমেজ root দাবি করে	ইমেজে USER নির্দেশ নেই	Dockerfile-এ `USER 1000` যোগ করুন

১৮.১১ নিরাপত্তা বিবেচনা

Privileged Pod কখনোই production অ্যাপে ব্যবহার করবেন না — শুধু CNI বা storage-এর মতো সিস্টেম উপাদানে। HostPath volume এড়িয়ে চলুন, কারণ তা Pod-কে host-এর ফাইল দেখতে দেয়। ইমেজ সবসময় বিশ্বস্ত registry থেকে নিন, ট্যাগের বদলে digest ব্যবহার করুন, আর নিয়মিত vulnerability scan করুন। Secret কখনো environment variable-এর বদলে প্রয়োজনে volume হিসেবে মাউন্ট করার কথা ভাবুন, যাতে প্রক্রিয়ার তালিকায় তা না দেখা যায়। NetworkPolicy-কে Git-এ রাখুন, যাতে কেউ হাতে বদলে ফেললেও ধরা পড়ে।

১৮.১২ কর্মক্ষমতা ও খরচ বিবেচনা

SecurityContext ও Pod Security Standards-এর কর্মক্ষমতার খরচ প্রায় শূন্য — এগুলো শুধু Pod তৈরির সময় যাচাই হয়। NetworkPolicy-এর খরচ CNI-এর ওপর নির্ভর করে; Calico বা Cilium-এ eBPF-ভিত্তিক প্রয়োগে খরচ কম। Read-only filesystem-এর জন্য emptyDir ব্যবহার করলে node-এর ডিস্ক ব্যবহার বাড়ে, তবে তা সামান্য। আসল খরচ হলো ভুল কনফিগারেশনে ডাউনটাইম — তাই production-এ প্রয়োগের আগে staging-এ পরীক্ষা করুন।

১৮.১৩ সেরা অনুশীলন ও অ্যান্টি-প্যাটার্ন

সেরা অনুশীলন	অ্যান্টি-প্যাটার্ন
সব namespace-এ Restricted enforce করা	Privileged Pod অ্যাপে ব্যবহার করা
প্রথমে audit, তারপর enforce	হঠাৎ enforce করে সব Deployment ভেঙে ফেলা
Default-deny দিয়ে শুরু, তারপর পথ খোলা	কোনো NetworkPolicy না থাকা
capabilities: drop ALL, দরকারে একটাই যোগ	সব capability রেখে দেওয়া
readOnlyRootFilesystem + emptyDir	Root হিসেবে লেখা যায় এমন ফাইলসিস্টেম

১৮.১৪ অধ্যায়ের সারাংশ

Pod Security Standards তিন স্তরে Pod-এর ঝুঁকিপূর্ণ ক্ষমতা সীমিত করে; Restricted-ই production-এর লক্ষ্য। SecurityContext Pod-কে non-root হিসেবে চালায়, ফাইলসিস্টেম read-only করে, capability কমায়। NetworkPolicy Pod-স্তরের ফায়ারওয়াল — default deny দিয়ে শুরু করে শুধু দরকারি পথ খুলতে হয়। তিনটি মিলে defense in depth তৈরি করে, যাতে একটি Pod আপসকৃত হলেও ক্ষতি ছড়াতে না পারে।

১৮.১৫ মূল প্রযুক্তিগত পরিভাষা

Pod Security Standards: Pod-এর নিরাপত্তা মানদণ্ড — Privileged, Baseline, Restricted তিনটি স্তর।

SecurityContext: Pod বা container-এর নিরাপত্তা সেটিংস (ব্যবহারকারী, ক্ষমতা, ফাইলসিস্টেম)।

NetworkPolicy: কোন Pod কোন Pod-এর সঙ্গে নেটওয়ার্কে কথা বলতে পারবে, তা নিয়ন্ত্রণকারী নিয়ম।

Default Deny: স্পষ্টভাবে অনুমোদিত নয় এমন সব ট্রাফিক বন্ধ রাখার নীতি।

Capabilities: Linux root-এর ক্ষমতাকে ভাগ করা ছোট ছোট অনুমতি।

Defense in Depth: একাধিক স্তরে সাজানো প্রতিরক্ষা ব্যবস্থা।

seccompProfile: কোন সিস্টেম কল প্রক্রিয়া ডাকতে পারবে, তা সীমিত করার প্রোফাইল।

১৮.১৬ জ্ঞান যাচাই

১. Pod Security Standards-এর তিনটি স্তরের পার্থক্য লিখুন। ২ `runAsNonRoot` ও `allowPrivilegeEscalation: false` কী করে? ৩. readOnlyRootFilesystem চালু করলে অ্যাপের লেখার দরকার কীভাবে মেটাবেন? ৪. NetworkPolicy-তে default deny কেন গুরুত্বপূর্ণ? ৫. `podSelector` ও `namespaceSelector`-এর পার্থক্য কী? ৬. NetworkPolicy কাজ না করলে প্রথমে কী পরীক্ষা করবেন? ৭. Defense in depth বলতে কী বোঝায়? উদাহরণ দিন।

১৮.১৭ ব্যবহারিক অ্যাসাইনমেন্ট

TechNova-র `technova` namespace-এ সম্পূর্ণ নিরাপদ সেটআপ তৈরি করুন: (ক) namespace-এ Restricted enforce label; (খ) frontend, api ও mongo — তিনটি Deployment, প্রতিটিতে SecurityContext (non-root, read-

only filesystem, drop ALL); (গ) default-deny NetworkPolicy; (ঘ) শুধু frontend→api (port 3000) ও api→mongo (port 27017) এবং DNS egress অনুমোদনকারী নীতি। তারপর পরীক্ষা করুন: frontend থেকে mongo-তে সরাসরি সংযোগ ব্যর্থ হয় কি না, আর api থেকে mongo-তে সংযোগ সফল হয় কি না। ফলাফল লিখে রাখুন।

অধ্যায় 19 • PART IV — Security & Operations (ch 17–20)

Chapter 19 — Cluster Troubleshooting and Common Kubernetes Failures

১৯.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন — একটি পদ্ধতিগত সমস্যা সমাধান পদ্ধতি (describe → logs → events → status) প্রয়োগ করতে; সবচেয়ে সাধারণ ব্যর্থতাগুলো — **CrashLoopBackOff**, **ImagePullBackOff**, **Pending**, **OOMKilled**, probe ব্যর্থতা ও DNS ব্যর্থতা — চিনতে ও ঠিক করতে `kubectl describe`, `get`, `logs` ও `exec` দিয়ে তথ্য সংগ্রহ করতে; etcd ও control-plane ব্যর্থতার লক্ষণ চিনতে; এবং একটি সিদ্ধান্ত-প্রবাহ (decision flow) অনুসরণ করে দ্রুত মূল কারণে পৌঁছাতে।

১৯.২ কেন এই বিষয়টি গুরুত্বপূর্ণ

Kubernetes-এ কিছু ভুল হলে তা প্রায়ই রহস্যময় মনে হয়: Pod চলছে না, কিন্তু কেন — তা কোথাও স্পষ্টভাবে লেখা থাকে না। অনেকেই তখন এলোমেলোভাবে Pod মুছে, Deployment রিস্টার্ট করে, এমনকি পুরো ক্লাস্টার রিবুট করে — তাতে সমস্যা সাময়িকভাবে লুকায়, কিন্তু মূল কারণ থেকে যায়। Production-এ প্রতি মিনিট ডাউনটাইম মানে গ্রাহক হারানো ও আয় কমা। TechNova-র MERN অ্যাপ ডাউন থাকলে ব্যবহারকারীরা লগইন করতে পারে না, অর্ডার করতে পারে না। পদ্ধতিগত troubleshooting মানে আতঙ্ক নয়, ধাপে ধাপে প্রমাণ সংগ্রহ করা: Pod-এর অবস্থা কী বলছে, event-এ কী লেখা আছে, log-এ কী ভুল দেখা যাচ্ছে। এই দক্ষতা একজন অপারেটরকে অন্যদের থেকে আলাদা করে — কারণ ক্লাস্টার ভাঙবেই, প্রশ্ন হলো আপনি কত দ্রুত ঠিক করতে পারেন।

> **REAL-WORLD:** TechNova-র এক শুক্রবার রাতে api Pod CrashLoopBackOff-এ পড়ে। দল প্রথমে Pod মুছে ফেলে — নতুন Pod-ও একইভাবে ভাঙে। তারপর log দেখে বোঝা যায়: MongoDB-র পাসওয়ার্ড বদলানো হয়েছিল, কিন্তু Secret হালনাগাদ করা হয়নি। Pod মোছা সমাধান ছিল না; Secret ঠিক করাই ছিল সমাধান।

১৯.৩ পূর্বশর্ত জ্ঞান

আপনার জানা থাকা উচিত: Pod, Deployment, Service, ConfigMap ও Secret-এর ধারণা, `kubectl`-এর মৌলিক কমান্ড; container image ও registry-র ধারণা; এবং resource requests/limits-এর ধারণা (আগের অধ্যায়গুলো থেকে)। Linux-এ log পড়ার অভিজ্ঞতা থাকলে ভালো হয়।

১৯.৪ প্রথম নীতি থেকে ব্যাখ্যা

প্রথম নীতি: Kubernetes একটি ঘোষণামূলক (declarative) সিস্টেম — আপনি বলেন কী চান, আর নিয়ন্ত্রক (controller) সেটি বাস্তবায়নের চেষ্টা করে। কোনো কিছু না চললে প্রশ্ন তিনটি: (১) আপনি কী চেয়েছিলেন? (২) সিস্টেম কী দেখছে? (৩) দুটির মধ্যে পার্থক্য কোথায়? এই পার্থক্যই সমস্যার মূল। Pod-এর **status** বলে সিস্টেম কী দেখছে, **events** বলে কী ঘটেছে; **logs** বলে অ্যাপ ভেতরে কী বলছে। এই তিনটি উৎস একসঙ্গে পড়লে প্রায় সব সমস্যার মূল পাওয়া যায়। যেমন Pod Pending মানে scheduler Pod-কে কোথাও বসাতে পারেনি — হয় জায়গা নেই, নয়তো নিয়ম (taint, affinity) বাধা দিচ্ছে। CrashLoopBackOff মানে অ্যাপ চালু হয়ে বারবার মরছে — সমস্যা অ্যাপে বা তার পরিবেশে (environment), Kubernetes-এ নয়। এইভাবে লক্ষণ থেকে স্তর (layer) চিহ্নিত করা — অ্যাপ, Pod, node, নেটওয়ার্ক, নাকি control plane — এটাই troubleshooting-এর প্রথম নীতি।

> **CONCEPT:** লক্ষণ নয়, মূল কারণ ঠিক করুন। Pod মোছা লক্ষণ ঢাকে; log ও event পড়া মূল কারণ খুঁজে দেয়।

১৯.৫ আর্কিটেকচার / ধারণাগত চিত্র



চিত্র ১৯.১ — পদ্ধতিগত `troubleshooting`-এর সিদ্ধান্ত-প্রবাহ।

১৯.৬ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

১৯.৬.১ পদ্ধতিগত পদ্ধতি: Describe → Logs → Events → Status

সবসময় একই ক্রমে এগোন। প্রথম `kubectl get pods -n technova` দিয়ে STATUS ও RESTARTS দেখুন। তারপর `kubectl describe pod <নাম>` — এর শেষে Events অংশে scheduler, kubelet ও probe-এর বার্তা থাকে; এটাই সবচেয়ে তথ্যবহুল জায়গা। তারপর `kubectl logs <pod>` — অ্যাপের নিজের বার্তা; আর `kubectl logs <pod> --previous` — আগের ভাঙা

container-এর লগ, যা CrashLoopBackOff-এ অমূল্য। সবশেষে kubectl exec দিয়ে ভেতরে ঢুকে পরিবেশ পরীক্ষা করুন। এই ক্রম মানলে আপনি এলোমেলো অনুমান এড়াতে পারবেন।

১৯.৬.২ CrashLoopBackOff

Pod চালু হয়ে কয়েক সেকেন্ডের মধ্যে মরে যাচ্ছে, Kubernetes বারবার চেষ্টা করছে, প্রতিবার অপেক্ষা বাড়ছে — এটাই CrashLoopBackOff। কারণ প্রায়ই অ্যাপের ভেতরে: ভুল environment variable, ডাটাবেসে সংযোগ ব্যর্থ, পোর্ট ইতিমধ্যে ব্যবহৃত, অথবা কোডে exception। সমাধান: kubectl logs --previous দিয়ে শেষ ভুলটি পড়ুন। সাধারণত log-এর শেষ লাইনেই উত্তর থাকে — যেমন MongoServerSelectionError বা EADDRINUSE।

১৯.৬.৩ ImagePullBackOff ও ErrImagePull

kubelet ইমেজ টানতে (pull) পারছে না। কারণ: ইমেজের নাম বা ট্যাগ ভুল, private registry imagePullSecrets নেই, registry ডাউন, অথবা নেটওয়ার্কে registry-তে পৌঁছানো যাচ্ছে না। kubectl describe pod-এর Events-এ স্পষ্ট বার্তা থাকে — যেমন Failed to pull image "technova/api:v9.9.9": rpc error: code = NotFound। সমাধান: নাম-ট্যাগ যাচাই করুন, Secret যাচাই করুন, registry-তে হাতে docker pull করে দেখুন।

১৯.৬.৪ Pending — Scheduling ব্যর্থতা

Pod কোনো node-এ বসতে পারছে না। Events-এ scheduler-এর বার্তা দেখুন 0/3 nodes are available: 3 Insufficient memory (resource চাপ), node(s) had taints that the pod didn't tolerate (taint), অথবা node(s) didn't match Pod's node affinity। সমাধান: kubectl describe nodes দিয়ে node-এর বাকি resource দেখুন, kubectl top nodes দিয়ে ব্যবহার দেখুন, অথবা requests কমান / node বাড়ান।

CrashLoopBackOff: Pod বারবার চালু হয়ে মরে যাওয়ার অবস্থা; Kubernetes প্রতিবার বেশি অপেক্ষা করে আবার চেষ্টা করে।

OOMKilled: Out Of Memory Killed — container-এর memory limit ছাড়িয়ে গেলে kernel তাকে হত্যা করে।

১৯.৬.৫ OOMKilled

kubectl get pods-এ OOMKilled দেখা গেলে বুঝুন container memory limit ছাড়িয়েছে। kubectl describe pod-এ Last State: Terminated, Reason: OOMKilled, Exit Code: 137 দেখা যায়। কারণ: memory limit খুব

কম, অথবা অ্যাপে memory leak। সমাধান `kubectl top pods` দিয়ে আসল ব্যবহার দেখুন, limit বাড়ান, অথবা leak ঠিক করুন। মনে রাখবেন: limit না দিলে Pod node-এর সব memory খেয়ে ফেলতে পারে — তাই limit দেওয়া জরুরি।

১৯.৬.৬ Probe ব্যর্থতা

Liveness probe ব্যর্থ হলে Kubernetes container রিস্টার্ট করে; readiness probe ব্যর্থ হলে Pod Service থেকে সরিয়ে নেওয়া হয় (ট্রাফিক পায় না, কিন্তু চলে)। লক্ষণ: Pod Running, কিন্তু READY 0/1। কারণ: probe-এর পথ (path) ভুল, অ্যাপ ধীরে চালু হয় (initialDelaySeconds কম), অথবা অ্যাপ আসলেই অসুস্থ। সমাধান `kubectl describe pod`-এ probe-এর বার্তা দেখুন, `kubectl exec` দিয়ে ভেতর থেকে health endpoint-এ curl করুন।

১৯.৬.৭ DNS ব্যর্থতা

Pod Service-এর নাম দিয়ে ডাকতে পারছে না (যেমন `mongo.technova.svc.cluster.local`)। পরীক্ষা: Pod-এর ভেতরে `nslookup mongo` বা `getent hosts mongo` চালান। কারণ: CoreDNS Pod ডাউন, NetworkPolicy DNS বন্ধ করে দিয়েছে (অধ্যায় ১৮), অথবা Service-এর selector ভুল। সমাধান: `kubectl get pods -n kube-system` দিয়ে CoreDNS দেখুন, `kubectl get svc mongo -n technova` দিয়ে Service-এর ClusterIP ও endpoints যাচাই করুন।

১৯.৬.৮ etcd ও Control-Plane ব্যর্থতা

সব Pod ঠিক আছে, কিন্তু `kubectl` ধীর বা ব্যর্থ, নতুন Pod তৈরি হচ্ছে না — সন্দেহ control plane-এ। লক্ষণ: API server-এর log-এ etcd সংযোগের ভুল, etcd ডিস্ক পূর্ণ, অথবা control-plane node-এর resource চাপ। `kubectl get nodes` দিয়ে node-এর অবস্থা দেখুন। `etcdctl endpoint health` দিয়ে etcd স্বাস্থ্য যাচাই করুন (অধ্যায় ২৩-এ বিস্তারিত)। etcd-এর ডিস্ক পূর্ণ হলে API server লেখা বন্ধ করে দেয় — তখন পুরোনো snapshot মুছে জায়গা করতে হয়, কিন্তু সাবধানে।

১৯.৭ কনফিগারেশন উদাহরণ

```
# সমস্যা নির্ণয়ের প্রধান কমান্ড
kubectl get pods -n technova -o wide
kubectl describe pod api-7d9c4f6b5-abc12 -n technova
kubectl logs api-7d9c4f6b5-abc12 -n technova --previous
kubectl logs api-7d9c4f6b5-abc12 -n technova -f --tail=100
kubectl exec -it api-7d9c4f6b5-abc12 -n technova -- sh
kubectl get events -n technova --sort-by=.lastTimestamp
kubectl get nodes
kubectl top pods -n technova
kubectl top nodes
# Service ও Endpoint যাচাই
kubectl get svc mongo -n technova
kubectl get endpoints mongo -n technova
# Pod-এর ভেতর থেকে DNS ও সংযোগ পরীক্ষা (চিত্রণমূলক নমুনা আউটপুট)
```

```
# $ nslookup mongo.technova.svc.cluster.local
# Server:    10.96.0.10
# Address:   10.96.0.10#53
# Name: mongo.technova.svc.cluster.local
# Address: 10.244.1.15
```

```
# সঠিক probe সেটিংসের উদাহরণ (ধীরে চালু হওয়া অ্যাপের জন্য)
livenessProbe:
  httpGet:
    path: /health
    port: 3000
  initialDelaySeconds: 15
  periodSeconds: 10
  failureThreshold: 3
readinessProbe:
  httpGet:
    path: /ready
    port: 3000
  initialDelaySeconds: 5
  periodSeconds: 5
```

১৯.৮ বাস্তব প্রকৌশল পরিস্থিতি

TechNova-র production-এ হঠাৎ frontend Pod-গুলো READY 0/1 হয়ে যায় — অ্যাপ চলছে, কিন্তু ট্রাফিক পাচ্ছে না। দল প্রথমে Deployment রিস্টার্ট করে, কোনো লাভ হয় না। তার `kubectl describe pod`-এ দেখা যায় readiness probe ব্যর্থ হচ্ছে: `Readiness probe failed: HTTP probe failed with statuscode: 500`। Pod-এর ভেতরে ঢুকে দেখা যায় `/ready` endpoint MongoDB-তে সংযোগ পরীক্ষা করে, আর MongoDB Pod তখন node চাপে Pending অবস্থায় ছিল। মূল কারণ ছিল node-এর memory পূর্ণ — একটি Pod-এর memory limit না থাকায় সে সব memory খেয়ে ফেলেছিল। সমাধান: সেই Pod-এ limit বসানো, MongoDB-কে আলাদা node-এ সরানো, আর readiness probe-কে শুধু অ্যাপের নিজের স্বাস্থ্য পরীক্ষা করতে বলা, ডাটাবেসের নয়। এই ঘটনা শেখায়: লক্ষণ (readiness ব্যর্থ) আর মূল কারণ (memory চাপ) এক জায়গায় নাও থাকতে পারে।

১৯.৯ ধাপে ধাপে ব্যবহারিক উদাহরণ

দৃশ্য: নতুন Deployment-এর Pod ImagePullBackOff-এ আছে। **ধাপ ১:** `kubectl get pods -n technova` দিয়ে STATUS নিশ্চিত করুন। **ধাপ ২:** `kubectl describe pod <নাম> -n technova` — Events-এ ইমেজের নাম ও ভুলটি পড়ুন। **ধাপ ৩:** ইমেজের নাম-ট্যাগ registry-তে আছে কি না যাচাই করুন। **ধাপ ৪:** private registry হলে `kubectl get secret regcred -n technova` দিয়ে imagePullSecrets আছে কি না দেখুন। **ধাপ ৫:** YAML ঠিক করে `kubectl apply` করুন, তারপর `kubectl get pods -w` দিয়ে Pod Running হওয়া দেখুন। **ধাপ ৬:** একই পদ্ধতিতে একটি CrashLoopBackOff Pod-এর `--previous` লগ পড়ে মূল ভুলটি খুঁজে বের করুন। প্রতিটি ধাপের ফলাফল লিখে রাখুন — এটাই আপনার incident নোট।

TROUBLESHOOTING: `kubectl get events --sort-by=.lastTimestamp` চালালে সাম্প্রতিক ঘটনাগুলো একসঙ্গে দেখা যায় — প্রায়ই এখানেই উত্তর লুকিয়ে থাকে।

১৯.১০ সাধারণ ভুল ও সমস্যা সমাধান

লক্ষণ	সম্ভাব্য কারণ	পরীক্ষা
CrashLoopBackOff	ভুল env, DB সংযোগ ব্যর্থ, কোডে exception	`logs --previous`
ImagePullBackOff	ভুল ট্যাগ, Secret নেই, registry ডাউন	`describe pod` Events
Pending	Resource নেই, taint, affinity	`describe pod`, `describe nodes`
OOMKilled (137)	Memory limit কম বা leak	`top pods`, limit বাড়ান
READY 0/1	Readiness probe ব্যর্থ	`describe pod`, ভেতরে curl
Service-এ সংযোগ নেই	Selector ভুল, endpoints খালি, DNS বন্ধ	`get endpoints`, nslookup
kubectl ধীর/ব্যর্থ	etcd বা API server চাপ	node ও control-plane লগ

১৯.১১ নিরাপত্তা বিবেচনা

Troubleshooting-এর সময় `kubectl exec` দিয়ে Pod-এর ভেতরে ঢোকা শক্তিশালী ক্ষমতা — এটি শুধু অনুমোদিত ব্যক্তিদের দিন (RBAC, অধ্যায় ১৭)। লগে Secret বা পাসওয়ার্ড ছাপাবেন না; অ্যাপের লগে সংবেদনশীল তথ্য না আসে, তা নিশ্চিত করুন। সমস্যার সময় সাময়িকভাবে Pod-কে privileged করা বা SecurityContext শিথিল করা বিপজ্জনক — পরীক্ষা শেষে তা ফিরিয়ে নিন। Audit log চালু রাখুন, যাতে কে কখন Pod-এর ভেতরে ঢুকেছে, তা জানা যায়।

১৯.১২ কর্মক্ষমতা ও খরচ বিবেচনা

ঘন ঘন রিস্টার্ট হওয়া Pod (CrashLoopBackOff) node-এর CPU খায় এবং লগের পরিমাণ বাড়িয়ে স্টোরেজ খরচ। `kubectl logs -f` দীর্ঘক্ষণ চালিয়ে রাখলে নেটওয়ার্ক ও API server-এর চাপ বাড়ে। বড় ক্লাস্টারে `kubectl get pods --all-namespaces` বারবার চালানো API server-এর ওপর চাপ ফেলে — নির্দিষ্ট namespace-এ সীমিত রাখুন। Metrics Server (অধ্যায় ২০) চালু থাকলে resource চাপ দ্রুত ধরা যায়, ফলে সমস্যা বড় হওয়ার আগেই ঠিক করা যায়।

১৯.১৩ সেরা অনুশীলন ও অ্যান্টি-প্যাটার্ন

সেরা অনুশীলন	অ্যান্টি-প্যাটার্ন
একই ক্রমে: `get` → `describe` → `logs` → `exec`	কারণ না জেনে Pod মুছে ফেলা
`--previous` লগ পড়া	শুধু বর্তমান লগ দেখে সিদ্ধান্ত নেওয়া
Events পড়া, শেষ বার্তা গুরুত্ব দেওয়া	Events উপেক্ষা করা
মূল কারণ লিখে রাখা (incident নোট)	ঠিক করে ভুলে যাওয়া
Probe-এ সঠিক delay দেওয়া	অ্যাপ চালু হওয়ার আগেই probe চালু করা

১৯.১৪ অধ্যায়ের সারাংশ

Troubleshooting মানে পদ্ধতিগত প্রমাণ সংগ্রহ: Pod-এর STATUS দেখা, describe-এর Events পড়া, logs (বিশেষ করে --previous) পড়া, প্রয়োজনে exec দিয়ে ভেতরে পরীক্ষা করা। CrashLoopBackOff মানে অ্যাপের ভেতরের ভুল; ImagePullBackOff মানে ইমেজ বা registry সমস্যা; Pending মানে scheduling বাধা; OOMKilled মানে memory limit; probe ব্যর্থতা মানে অ্যাপ বা probe সেটিংসের সমস্যা; DNS ব্যর্থতা মানে CoreDNS, NetworkPolicy বা Service সমস্যা। Control-plane সমস্যায় etcd ও API server-এর স্বাস্থ্য দেখুন।

১৯.১৫ মূল প্রযুক্তিগত পরিভাষা

CrashLoopBackOff: Pod বারবার চালু হয়ে মরে যাওয়া; প্রতিবার পুনঃচেষ্টার ব্যবধান বাড়ে।

ImagePullBackOff: kubelet ইমেজ টানতে না পারার অবস্থা।

OOMKilled: Memory সীমা ছাড়িয়ে যাওয়ায় container বন্ধ হয়ে যাওয়া (exit code 137)।

Liveness Probe: Container বেঁচে আছে কি না, তা পরীক্ষা; ব্যর্থ হলে রিস্টার্ট।

Readiness Probe: Pod ট্রাফিক নেওয়ার জন্য প্রস্তুত কি না, তা পরীক্ষা।

Events: ক্লাস্টারে কী ঘটেছে, তার সময়-ক্রমিক রেকর্ড।

১৯.১৬ জ্ঞান যাচাই

১. Troubleshooting-এর সঠিক ক্রমটি লিখুন ও কেন তা গুরুত্বপূর্ণ, ব্যাখ্যা করুন। ২. CrashLoopBackOff-এ `--previous` লগ কেন দরকার? ৩. Pending Pod-এর তিনটি সম্ভাব্য কারণ লিখুন। ৪. OOMKilled-এর exit code কত, আর এর অর্থ কী? ৫. Liveness ও readiness probe ব্যর্থতার ফলাফলের পার্থক্য কী? ৬. Service-এর নামে সংযোগ না হলে কী কী পরীক্ষা করবেন? ৭. `kubectl describe pod`-এর কোন অংশটি সবচেয়ে তথ্যবহুল? ৮. etcd ব্যর্থতার লক্ষণ কী কী?

১৯.১৭ ব্যবহারিক অ্যাসাইনমেন্ট

ইচ্ছাকৃতভাবে তিনটি ভাঙা Pod তৈরি করুন: (ক) ভুল ইমেজ ট্যাগ দিয়ে ImagePullBackOff; (খ) ভুল environment variable (ভুল DB host) দিয়ে CrashLoopBackOff; (গ) খুব কম memory limit দিয়ে OOMKilled। প্রতিটির জন্য describe, events ও logs সংগ্রহ করে একটি প্রতিবেদন লিখুন: লক্ষণ, মূল কারণ, আর সমাধান। তারপর তিনটি ঠিক করে Pod Running করুন। এই অনুশীলনটি আপনার troubleshooting পেশিশক্তি গড়ে তুলবে।

অধ্যায় 20 • PART IV — Security & Operations (ch 17–20)

Chapter 20 — Monitoring, Logging, Metrics Server, and Resource Observability

২০.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন — **Metrics Server** ও `kubectl top` দিয়ে CPU ও memory ব্যবহার দেখতে; **requests বনাম প্রকৃত ব্যবহার** (usage)-এর পার্থক্য ব্যাখ্যা করতে; Pod, node ও ক্লাস্টার-স্তরের logging স্থাপত্য বুঝতে এবং **Fluent Bit**-এর ভূমিকা জানতে; **Prometheus ও Grafana**-কে উচ্চ স্তরে পরিচিতি হিসেবে ব্যাখ্যা করতে; alerting-এর মৌলিক নীতি জানতে; এবং control plane, node ও workload — তিন স্তরে কী কী পর্যবেক্ষণ (monitor) করতে হবে, তা তালিকা করতে।

২০.২ কেন এই বিষয়টি গুরুত্বপূর্ণ

যা মাপা যায় না, তা পরিচালনা করা যায় না। Kubernetes-এ Pod হঠাৎ মরে গেলে, অ্যাপ ধীর হলে, অথবা খরচ হঠাৎ বেড়ে গেলে — কারণ জানতে হলে মেট্রিক্স ও লগ দরকার। Metrics ছাড়া আপনি জানবেন না কোন Pod কত CPU খাচ্ছে, কোন node পূর্ণ হয়ে আসছে, অথবা কোন Deployment-এর memory ধীরে ধীরে বাড়ছে (leak)। লগ ছাড়া জানবেন না অ্যাপ ভেতরে কী ভুল বলছে। TechNova-র MERN অ্যাপে ব্যবহারকারী অভিযোগ করল "সাইট ধীর" — মেট্রিক্স দেখে বোঝা গেল API Pod-এর CPU ৯৫% ছুঁয়েছে, কারণ একটি Pod-এর requests খুব কম দেওয়া হয়েছিল। পর্যবেক্ষণ (observability) মানে শুধু সমস্যা ধরা নয়, সমস্যা আসার আগেই সতর্ক হওয়া — যেমন ডিস্ক ৮০% পূর্ণ হলে অ্যালার্ট পাওয়া, ১০০% হওয়ার আগেই ব্যবস্থা নেওয়া।

> **REAL-WORLD:** TechNova-তে একবার MongoDB Pod-এর memory ধীরে ধীরে বাড়ছিল, কিন্তু কেউ লক্ষ করেনি — কারণ `kubectl top` কেউ চালাত না। একদিন Pod OOMKilled হয়ে ডাটাবেস ডাউন হয়। এরপর থেকে Grafana ড্যাশবোর্ডে memory-এর গ্রাফ সবসময় চোখের সামনে থাকে, আর ৮০% ছুঁলেই অ্যালার্ট আসে।

২০.৩ পূর্বশর্ত জ্ঞান

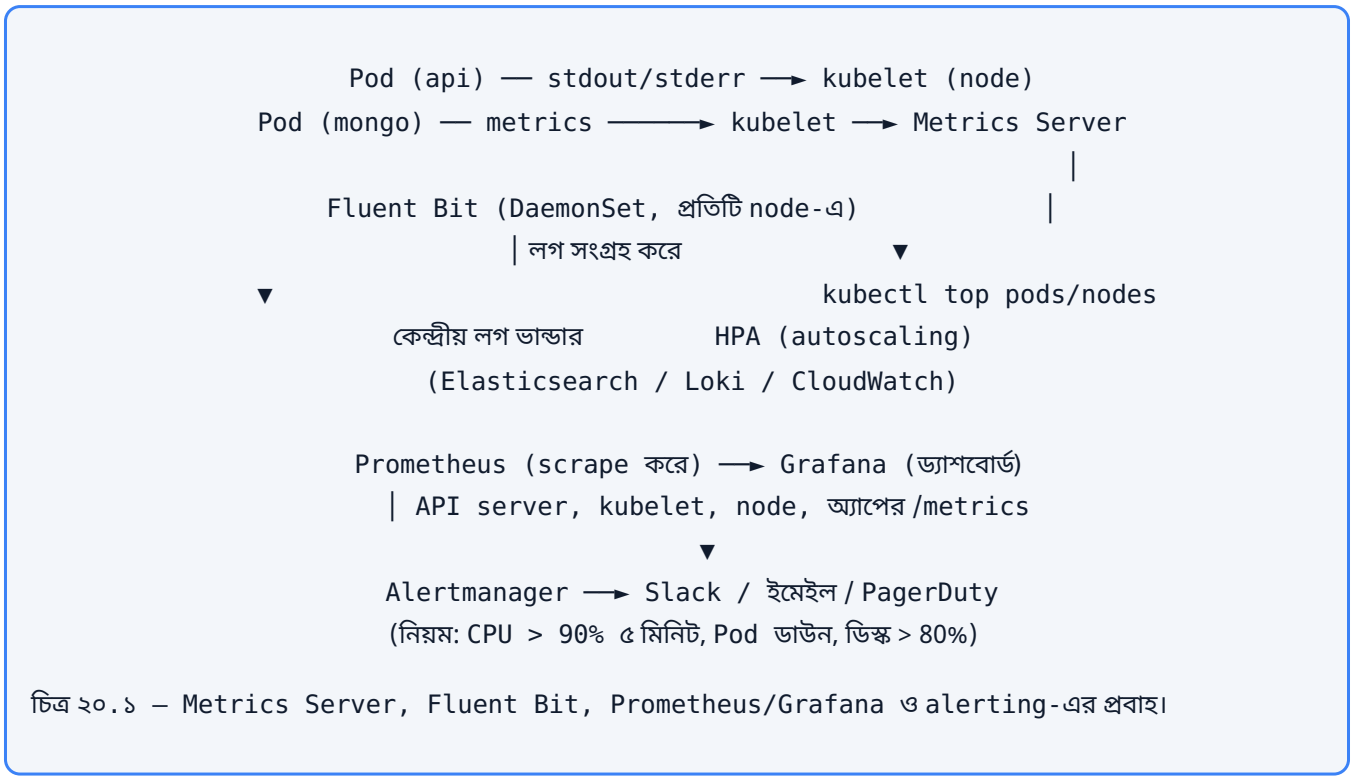
আপনার জানা থাকা উচিত: Pod, Deployment, Namespace ও resource requests/limits-এর ধারণা; `kubectl`-এর মৌলিক কমান্ড; এবং লগ (log) কী, তা সাধারণ ধারণা। Prometheus বা Grafana আগে ব্যবহার না করলেও চলবে — এই অধ্যায়ে সেগুলো উচ্চ স্তরে পরিচয় করিয়ে দেওয়া হয়েছে, পূর্ণ টিউটোরিয়াল নয়।

২০.৪ প্রথম নীতি থেকে ব্যাখ্যা

প্রথম নীতি: একটি চলমান সিস্টেম সম্পর্কে তিনটি প্রশ্নের উত্তর দরকার — (১) এখন কী হচ্ছে? (২) আগে কী হয়েছিল? (৩) কিছু ভুল হলে কীভাবে জানব? প্রথম প্রশ্নের উত্তর দেয় **metrics** (পরিমাপ) — সংখ্যা, যেমন CPU ৫০%, memory ২ Gi, অনুরোধ প্রতি সেকেন্ডে ১০০টি। দ্বিতীয় প্রশ্নের উত্তর দেয় **logs** (লগ) — ঘটনার লিখিত বিবরণ, যেমন "ডাটাবেসে সংযোগ ব্যর্থ"। তৃতীয় প্রশ্নের উত্তর দেয় **alerting** (সতর্কবার্তা) — নিয়ম, যেমন "CPU ৯০%-এর বেশি ৫ মিনিট থাকলে জানাও"। Kubernetes-এ metrics আসে kubelet থেকে — প্রতিটি node-এর kubelet তার Pod-গুলোর ব্যবহার মাপে। **Metrics Server** সেই তথ্য সংগ্রহ করে API server-কে দেয়, যাতে `kubectl top` কাজ করে। আর লগ আসে container-এর stdout/stderr থেকে — kubelet সেগুলো node-এর ডিস্কে রাখে, আর Fluent Bit-এর মতো এজেন্ট সেগুলো সংগ্রহ করে কেন্দ্রীয় জায়গায় পাঠায়। এই তিনটি — metrics, logs, alerts — একসঙ্গে মিলে **observability** (পর্যবেক্ষণযোগ্যতা) তৈরি করে।

> **CONCEPT:** Observability মানে শুধু ড্যাশবোর্ড নয় — সিস্টেমের ভেতরে কী হচ্ছে, তা বাইরে থেকে বোঝার ক্ষমতা। Metrics বলে কতটা, logs বলে কেন, alerts বলে কখন জানাতে হবে।

২০.৫ আর্কিটেকচার / ধারণাগত চিত্র



চিত্র ২০.১ — Metrics Server, Fluent Bit, Prometheus/Grafana ও alerting-এর প্রবাহ।

২০.৬ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

২০.৬.১ Metrics Server ও kubectl top

Metrics Server হলো ক্লাস্টারের হালকা মেট্রিক্স সংগ্রাহক। এটি kubelet-এর Summary API থেকে CPU ও memory ব্যবহার সংগ্রহ করে Metrics API-তে প্রকাশ ক`kubectl top pods -n technova` দিয়ে Pod-এর বর্তমান ব্যবহার, অ`kubectl top nodes` দিয়ে node-এর ব্যবহার দেখা যায়। মনে রাখবেন: Metrics Server শুধু বর্তমান ব্যবহার দেখায়, ইতিহাস রাখে না — ইতিহাসের জন্য Prometheus দরকার। আর Metrics Server না থাকলে Horizontal Pod Autoscaler (HPA) কাজ করবে না, কারণ HPA-ও এই API ব্যবহার করে।

২০.৬.২ Requests বনাম Usage

Requests হলো scheduler-কে দেওয়া প্রতিশ্রুতি: "এই Pod-এর এতটুকু CPU/memory দরকার। **Limits** হলো সর্বোচ্চ সীমা। **Usage** হলো আসল ব্যবহার। তিনটি আলাদা সংখ্যা — এবং এদের পার্থক্যই অনেক সমস্যার মূল। Requests খুব বেশি দিলে node-এর জায়গা নষ্ট হয় (Pod বসতে পারে না, Pending হয়); requests খুব কম দিলে Pod প্রয়োজনের সময় CPU পায় না, ধীর হয়। নিয়ম: প্রথমে requests = গড় ব্যবহারের কাছাকাছি, limits = সর্বোচ্চ ব্যবহারের কাছাকাছি রাখুন, তা`kubectl top` দিয়ে কয়েক দিন পর্যবেক্ষণ করে সমন্বয় করুন।

Metrics Server: kubelet থেকে CPU ও memory ব্যবহার সংগ্রহ করে Metrics API-তে প্রকাশকারী ক্লাস্টার উপাদান; kubectl top ও HPA এটি ব্যবহার করে।

Golden Signals: সেবার স্বাস্থ্য মাপার চারটি মূল সংকেত — latency (বিলম্ব), traffic (ট্রাফিক), errors (ভুল), saturation (সম্পৃক্তি)।

২০.৬.৩ Logging স্থাপত্য

তিনটি স্তর: **Pod logs** — `kubectl logs <pod>` দিয়ে দেখা যায়; Pod মুছে গেলে তার লগও চলে যায়। **Node logs** — kubelet ও container runtime-এর লগ node-এ `/var/log`-এ থাকে। **Cluster-level logging** — Fluent Bit-এর মতো এজেন্ট (সাধারণত DaemonSet হিসেবে প্রতিটি node-এ চলে) সব Pod-এর লগ সংগ্রহ করে কেন্দ্রীয় ভান্ডারে (যেমন Elasticsearch, Loki, বা ক্লাউডের লগ সেবা) পাঠায়। সেখানে লগ খোঁজা, ফিল্টার করা ও দীর্ঘদিন রাখা যায়। নিয়ম: অ্যাপের লগে সংবেদনশীল তথ্য (পাসওয়ার্ড, token) ছাপাবেন না, আর লগের ফরম্যাটে সময়, স্তর (info/error) ও অনুরোধের শনাক্তকারী রাখুন।

২০.৬.৪ Prometheus ও Grafana — উচ্চ স্তরের পরিচিতি

Prometheus হলো মেট্রিক্স সংগ্রহ ও সতর্কবার্তার জন্য বহুল ব্যবহৃত ওপেন-সোর্স সিস্টেম। এটি নির্দিষ্ট সময় পরপর `/metrics` endpoint থেকে (scrape করে) সংখ্যা সংগ্রহ করে, সংরক্ষণ করে, আর PromQL ভাষায় জিজ্ঞাসা করতে দেয়। **Grafana** হলো সেই সংখ্যাকে সুন্দর ড্যাশবোর্ড ও গ্রাফে দেখানোর টুল। একসঙ্গে তারা ক্লাস্টারের স্বাস্থ্য, অ্যাপের কার্যক্ষমতা ও খরচের চিত্র দেয়। এই বইয়ে এদের পূর্ণ টিউটোরিয়াল দেওয়া হয়নি — উচ্চ স্তরে জানুন: Prometheus সংগ্রহ করে, Grafana দেখায়, Alertmanager সতর্ক করে। Managed Kubernetes-এ (অধ্যায় ২৩) ক্লাউড-প্রদত্ত পর্যবেক্ষণ সেবাও একই কাজ করে।

২০.৬.৫ Alerting-এর মৌলিক নীতি

ভালো অ্যালার্ট মানে: সঠিক সময়ে, সঠিক ব্যক্তিকে, কার্যকর তথ্য দেওয়া। নিয়ম: (১) ব্যবহারকারীর ওপর প্রভাব পড়ে, এমন জিনিসে অ্যালার্ট দিন (যেমন Pod ডাউন, error rate বেশি) — শুধু সংখ্যা বেশি হলেই নয়। (২) প্রতিটি অ্যালার্টের সঙ্গে একটি runbook (কী করতে হবে) যুক্ত করুন (অধ্যায় ২৪)। (৩) অ্যালার্টের ক্লান্তি (alert fatigue) এড়ান — খুব বেশি অ্যালার্ট এলে মানুষ সব উপেক্ষা করতে শুরু করে। (৪) গুরুত্ব অনুযায়ী ভাগ করুন: critical (এখনই ব্যবস্থা), warning (কাজের সময়ে দেখুন)।

২০.৬.৬ কী কী পর্যবেক্ষণ করবেন

স্তর	কী দেখবেন	কেন
Control plane	API server বিলম্ব, etcd স্বাস্থ্য, scheduler সারি	ক্লাস্টার নিজেই সুস্থ কি না
Node	CPU, memory, ডিস্ক, নেটওয়ার্ক, kubelet স্বাস্থ্য	Pod বসানোর জায়গা আছে কি না
Workload	Pod অবস্থা, restart সংখ্যা, CPU/memory ব্যবহার	অ্যাপ সুস্থ কি না
অ্যাপ (Golden Signals)	Latency, traffic, errors, saturation	ব্যবহারকারীর অভিজ্ঞতা কেমন

২০.৭ কনফিগারেশন উদাহরণ

```
# Metrics Server ইনস্টল (অফিসিয়াল নির্দেশনা অনুযায়ী)
kubectl apply -f https://github.com/kubernetes-sigs/metrics-server/releases/latest/download/components.yaml
kubectl get deployment metrics-server -n kube-system
# ব্যবহার দেখা (চিত্রণমূলক নমুনা আউটপুট)
# $ kubectl top pods -n technova
```

```
# NAME                    CPU(cores)   MEMORY(bytes)
# api-7d9c4f6b5-abc12     45m          128Mi
# mongo-0                 120m         512Mi
# $ kubectl top nodes
# NAME       CPU(cores)   CPU%   MEMORY(bytes)   MEMORY%
# node-1     350m         17%    1200Mi          15%
# সাজানো দেখা
kubectl top pods -n technova --sort-by=cpu
kubectl top pods -n technova --sort-by=memory
```

```
# Fluent Bit DaemonSet-এর ধারণাগত অংশ (সংক্ষিপ্ত)
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: fluent-bit
  namespace: logging
spec:
  selector:
    matchLabels:
      app: fluent-bit
  template:
    metadata:
      labels:
        app: fluent-bit
    spec:
      containers:
      - name: fluent-bit
        image: fluent/fluent-bit:2.2
        volumeMounts:
        - name: varlog
          mountPath: /var/log
      volumes:
      - name: varlog
        hostPath:
          path: /var/log
```

Fluent Bit-এর পূর্ণ কনফিগারেশন (input, filter, output) পরিবেশভেদে বদলায়; উপরের অংশটি শুধু DaemonSet হিসেবে চলার ধারণা দেয়। কেন্দ্রীয় ভান্ডারের ঠিকানা আপনার পরিবেশ অনুযায়ী বসাতে হবে।

২০.৮ বাস্তব প্রকৌশল পরিস্থিতি

TechNova-র Black Friday-এর মতো ব্যস্ত দিনে হঠাৎ API-এর latency (বিলম্ব) বেড়ে যায়। Grafana ড্যাশবোর্ডে দেখা যায়: অনুরোধের সংখ্যা (traffic) তিন গুণ বেড়েছে, কিন্তু Pod-এর সংখ্যা একই আছে — HPA-এর সীমা কম দেওয়া ছিল। একই সঙ্গে error rate বেড়েছে, কারণ CPU সম্পৃক্ত (saturation) হয়ে গেছে। দল HPA-এর সর্বোচ্চ Pod সংখ্যা বাড়ায়, আর একটি অ্যালার্ট যোগ করে: traffic ৫০% বাড়লেই আগেভাগে সতর্কবার্তা আসবে। পরের ব্যস্ত দিনে অ্যালার্ট আগেই আসে, দল আগেভাগে Pod বাড়ায় — ব্যবহারকারীরা কিছুই টের পায় না। এটাই observability-এর মূল্য: সমস্যা ব্যবহারকারী জানার আগেই ধরা।

২০.৯ ধাপে ধাপে ব্যবহারিক উদাহরণ

ধাপ ১: Metrics Server ইনস্টল আছে কি না দেখুন: `kubectl get deployment metrics-server -n kube-system`। **ধাপ ২:** `kubectl top pods -n technova` ও `kubectl top nodes` চালিয়ে বর্তমান ব্যবহার লিখে রাখুন। **ধাপ ৩:** একটি Pod-এর requests/limits দেখুন: `kubectl describe pod <নাম> -n technova` — Limits ও Requests অংশে। **ধাপ ৪:** ব্যবহার requests-এর চেয়ে বেশি কি না, তা তুলনা করুন — বেশি হলে requests বাড়ানোর কথা ভাবুন। **ধাপ ৫:** `kubectl logs <pod> -n technova --tail=50` দিয়ে সাম্প্রতিক লগ দেখুন; ভুলের (error) লাইন খুঁজুন। **ধাপ ৬:** একটি সহজ Grafana ড্যাশবোর্ডের ধারণা আঁকুন: কোন চারটি গ্রাফ থাকবে (CPU, memory, অনুরোধের সংখ্যা, error rate) — কাগজে আঁকলেই চলবে।

> **PRO-TIP:** `kubectl top` দিয়ে সিদ্ধান্ত নেওয়ার আগে কয়েকবার, কয়েক দিন ধরে দেখুন — এক মুহূর্তের সংখ্যা বিভ্রান্ত করতে পারে।

২০.১০ সাধারণ ভুল ও সমস্যা সমাধান

সমস্যা	কারণ	সমাধান
`kubectl top` ব্যর্থ: metrics not available	Metrics Server নেই বা চালু হয়নি	Metrics Server ইনস্টল ও লগ দেখুন
Pod-এর ব্যবহার requests-এর চেয়ে অনেক বেশি	Requests কম দেওয়া	Requests সমন্বয় করুন
Pod মুছে গেলে লগ হারিয়ে যায়	শুধু Pod-স্তরের লগ	Cluster-level logging (Fluent Bit) চালু করুন
খুব বেশি অ্যালার্ট আসে	সীমা খুব কম বা ভুল নিয়ম	শুধু ব্যবহারকারীর ওপর প্রভাব পড়া জিনিসে অ্যালার্ট দিন
ড্যাশবোর্ডে ইতিহাস নেই	Metrics Server ইতিহাস রাখে না	Prometheus ব্যবহার করুন

২০.১১ নিরাপত্তা বিবেচনা

লগে কখনো পাসওয়ার্ড, token বা ব্যক্তিগত তথ্য ছাপাবেন না — লগ ভান্ডারে সেগুলো অনেকের কাছে পৌঁছে যায়। Metrics endpoint-এ সংবেদনশীল তথ্য রাখবেন না। Grafana ও Prometheus-এর প্রবেশ RBAC ও প্রমাণীকরণ দিয়ে সুরক্ষিত করুন —

পর্যবেক্ষণ ব্যবস্থা নিজেই আক্রমণের লক্ষ্য হতে পারে, কারণ তাতে পুরো সিস্টেমের চিত্র থাকে। লগ ভান্ডারের প্রবেশ শুধু অনুমোদিত ব্যক্তিদের দিন।

২০.১২ কর্মক্ষমতা ও খরচ বিবেচনা

Metrics Server হালকা, কিন্তু Prometheus দীর্ঘ ইতিহাস রাখলে ডিস্ক খরচ বাড়ে — retention (কতদিন রাখবেন) ঠিক করুন। লগের পরিমাণ বিশাল হতে পারে; অপ্রয়োজনীয় debug লগ production-এ বন্ধ রাখুন, আর লগের retention সীমিত রাখুন। Managed Kubernetes-এ ক্লাউড-প্রদত্ত পর্যবেক্ষণ সেবা সহজ, কিন্তু তার খরচ আলাদাভাবে দেখুন — লগের পরিমাণ বাড়লে খরচও বাড়ে। সঠিক requests/limits-ই আসল খরচ বাঁচায়: অতিরিক্ত requests মানে অপ্রয়োজনীয় node-এর খরচ।

২০.১৩ সেরা অনুশীলন ও অ্যান্টি-প্যাটার্ন

সেরা অনুশীলন	অ্যান্টি-প্যাটার্ন
Metrics Server + Prometheus + Grafana একসঙ্গে	শুধু kubectl top -এর ওপর নির্ভর করা
Cluster-level logging চালু রাখা	Pod মুছে গেলে লগ হারানো
Golden Signals পর্যবেক্ষণ করা	শুধু CPU দেখা
প্রতিটি অ্যালার্টে runbook যুক্ত করা	কারণ ছাড়া অ্যালার্ট পাঠানো
Requests নিয়মিত সমন্বয় করা	একবার দিয়ে আর না দেখা

২০.১৪ অধ্যায়ের সারাংশ

Metrics Server বর্তমান CPU/memory ব্যবহার দেয়, `kubectl top` তা দেখায়। Requests হলো প্রতিশ্রুতি, usage হলো আসল ব্যবহার — দুটি মিলিয়ে সমন্বয় করতে হয়। লগ তিন স্তরে: Pod, node, ও ক্লাস্টার-স্তর (Fluent Bit দিয়ে কেন্দ্রীয় ভান্ডারে)। Prometheus মেট্রিক্স সংগ্রহ করে, Grafana দেখায়, Alertmanager সতর্ক করে। পর্যবেক্ষণ করতে হবে control plane, node, workload ও Golden Signals — আর প্রতিটি অ্যালার্টের সঙ্গে runbook থাকতে হবে।

২০.১৫ মূল প্রযুক্তিগত পরিভাষা

Metrics Server: kubelet থেকে CPU ও memory ব্যবহার সংগ্রহকারী ক্লাস্টার উপাদান।

Observability: সিস্টেমের ভেতরের অবস্থা বাইরে থেকে বোঝার ক্ষমতা (metrics, logs, traces)।

Fluent Bit: প্রতিটি node-এ চলা হালকা লগ সংগ্রাহক এজেন্ট।

Prometheus: মেট্রিক্স সংগ্রহ, সংরক্ষণ ও সতর্কবার্তার ওপেন-সোর্স সিস্টেম।

Grafana: মেট্রিক্সকে ড্যাশবোর্ড ও গ্রাফে দেখানোর টুল।

Golden Signals: Latency, traffic, errors, saturation — সেবার স্বাস্থ্যের চার সংকেত।

Alerting: নির্দিষ্ট শর্ত পূরণ হলে স্বয়ংক্রিয় সতর্কবার্তা পাঠানোর ব্যবস্থা।

২০.১৬ জ্ঞান যাচাই

১. Metrics Server কী করে, আর `kubectl top` ছাড়া আর কী এটি ব্যবহার করে? ২. Requests, limits ও usage-এর পার্থক্য লিখুন। ৩. Pod মুছে গেলে তার লগ কেন হারিয়ে যায়, আর এর সমাধান কী? ৪. Fluent Bit কীভাবে কাজ করে? ৫. Prometheus ও Grafana-এর ভূমিকার পার্থক্য কী? ৬. Golden Signals চারটি কী কী? ৭. ভালো অ্যালার্টের তিনটি বৈশিষ্ট্য লিখুন। ৮. Control plane-এ কী কী পর্যবেক্ষণ করবেন?

২০.১৭ ব্যবহারিক অ্যাসাইনমেন্ট

TechNova-র জন্য একটি পর্যবেক্ষণ পরিকল্পনা লিখুন: (ক) Metrics Server দি `kubectl top` -এর আউটপুট সংগ্রহ করে প্রতিটি Pod-এর requests বনাম usage তুলনা করুন; (খ) চারটি Golden Signals-এর জন্য কোন মেট্রিক্স দেখবেন, তা তালিকা করুন; (গ) পাঁচটি অ্যালার্ট নিয়ম লিখুন (যেমন Pod ডাউন, CPU ৯০% ৫ মিনিট, ডিস্ক ৮০%, error rate বেশি, restart বেশি) এবং প্রতিটির জন্য একটি করে runbook ধাপ লিখুন; (ঘ) Fluent Bit দিয়ে লগ কেন্দ্রীয় ভান্ডারে পাঠানোর স্থাপত্যের একটি চিত্র আঁকুন।

PART V • DEVOPS ENGINEERING MASTERY

Production Kubernetes Engineering

Helm, Kustomize, ক্লাস্টার আপগ্রেড, etcd ব্যাকআপ, Disaster Recovery, প্রোডাকশন আর্কিটেকচার, GitOps ও অপারেশন রানবুক।

অধ্যায় 21 • PART V — Production Kubernetes Engineering (ch 21–24)

Chapter 21 — Helm — Kubernetes Package Management

২১.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন — **Helm** কী এবং কেন দরকার, তা ব্যাখ্যা করতে; **chart** (প্যাকেজ), **release** (ইনস্টল করা সংস্করণ), **values** (কনফিগারেশন মান) ও **templates** (টেমপ্লেট) এর ধারণা বুঝতে; chart-এর গঠন পড়তে ও লিখতে; `helm install`, `upgrade`, `rollback` ও `history` চালাতে; `values.yaml` ও `--set` দিয়ে কনফিগারেশন বদলাতে; এবং Helm বনাম কাঁচা YAML-এর সুবিধা-অসুবিধা বিচার করতে।

২১.২ কেন এই বিষয়টি গুরুত্বপূর্ণ

একটি MERN অ্যাপ deploy করতে কতগুলো YAML লাগে ভাবুন: Deployment, Service, ConfigMap, Secret, Ingress, NetworkPolicy, ServiceAccount, RBAC — হয়তো দশটির বেশি ফাইল। এবার সেই একই অ্যাপ dev, staging ও production — তিন পরিবেশে deploy করতে হলে কী করবেন? তিনবার ফাইল কপি করে হাতে বদলাবেন? ভুল হবেই — কোথাও image ট্যাগ পুরোনো থাকবে, কোথাও replica সংখ্যা ভুল হবে। Helm এই সমস্যার সমাধান: একবার chart লিখুন, তারপর values বদলে যেকোনো পরিবেশে ইনস্টল করুন। Helm আরও দেয় সংস্করণ ব্যবস্থাপনা (versioning) — কোন সংস্করণ ইনস্টল আছে, আগের সংস্করণে ফেরা (rollback) — যা কাঁচা YAML-এ হাতে করা কঠিন। TechNova-তে প্রতিদিন deploy হয়; Helm ছাড়া প্রতিটি deploy হতো ভুলের ঝুঁকি।

> **REAL-WORLD:** TechNova-তে আগে staging ও production-এর YAML আলাদা ফোল্ডারে রাখা হতো। একদিন production-এর ফাইলে staging-এর ডাটাবেসের ঠিকানা থেকে যায় — production অ্যাপ staging ডাটাবেসে লিখতে শুরু করে। Helm-এ আসার পর ডাটাবেসের ঠিকানা শুধু values ফাইলে থাকে, আর প্রতিটি পরিবেশের values আলাদা — এমন ভুল আর হয়নি।

২১.৩ পূর্বশর্ত জ্ঞান

আপনার জানা থাকা উচিত: Deployment, Service, ConfigMap, Secret ও Ingress-এর YAML লেখা; `kubectl apply` দিয়ে deploy করা; এবং YAML-এর মৌলিক গঠন। টেমপ্লেট (template) ধারণা — যেমন একটি ফাইলে জায়গাধারক (placeholder) রেখে পরে মান বসানো — জানা থাকলে ভালো হয়, না জানলেও এই অধ্যায়ে শিখবেন।

২১.৪ প্রথম নীতি থেকে ব্যাখ্যা

প্রথম নীতি: একই জিনিস বারবার লেখা ভুলের উৎস। প্রোগ্রামিংয়ে আমরা ফাংশন লিখি — একবার লিখে বারবার ডাকি, প্যারামিটার বদলে। Helm Kubernetes YAML-এর জন্য সেই ফাংশন। **Chart** হলো ফাংশনের সংজ্ঞা — টেমপ্লেট ফাইলের গুচ্ছ, যাতে জায়গাধারক আছে। **Values** হলো ফাংশনের আর্গুমেন্ট — যেমন image ট্যাগ, replica সংখ্যা, পরিবেশের নাম। **Release** হলো ফাংশন ডাকার ফল — একটি নির্দিষ্ট values দিয়ে chart ইনস্টল করলে ক্লাস্টারে যা তৈরি হয়। **revision** (সংস্করণ) হলো প্রতিটি ইনস্টল বা আপগ্রেডের ইতিহাস — ভুল হলে আগের revision-এ ফেরা যায়। Helm-এর ভেতরে **Go templates** কাজ করে: `{{ .Values.replicaCount }}` লেখা থাকলে Helm সেখানে values.yaml-এর মান বসিয়ে দেয়। এটাই পুরো জাদু — বাকিটা শুধু এই ধারণার প্রয়োগ।

> **CONCEPT:** Chart = রেসিপি, Values = উপকরণের পরিমাণ, Release = রান্না করা খাবার। একই রেসিপিতে উপকরণ বদলে ভিন্ন ভিন্ন খাবার বানানো যায়।

২১.৫ আর্কিটেকচার / ধারণাগত চিত্র



চিত্র ২১.১ — Chart, values, release ও revision-এর সম্পর্ক।

২১.৬ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

২১.৬.১ Chart-এর গঠন

প্রতিটি chart-এ `Chart.yaml` থাকে — নাম, সংস্করণ (`version`), অ্যাপের সংস্করণ (`appVersion`) ও বিবরণ। `values.yaml`-এ ডিফল্ট মান থাকে — যেমন `replicaCount: 2`, `image: technova/api:v1.2.0`। `templates/` ফোল্ডারে Kubernetes YAML থাকে, তবে তাতে Go template নির্দেশনা (`{{ ... }}`) থাকে। `_helpers.tpl`-এ পুনঃব্যবহারযোগ্য ছোট টেমপ্লেট থাকে — যেমন নাম তৈরির নিয়ম। `charts/` ফোল্ডারে নির্ভরশীল (dependency) chart থাকে — যেমন MongoDB-র chart।

২১.৬.২ Values ও --set

Values বদলানোর তিনটি উপায়, অগ্রাধিকারের ক্রমে (পরেরটি আগেরটিকে ছাপিয়ে যায়): (১) chart `values.yaml`; (২) `-f values-prod.yaml` দিয়ে দেওয়া ফাইল; (৩) `--set image.tag=v1.3.0` কমান্ড লাইনে। production-এ `--set`

দিয়ে Secret বসাবেন না — তা shell history-তে থেকে যায়; Secret-এর জন্য আলাদা ব্যবস্থা (যেমন Sealed Secrets বা External Secrets) ব্যবহার করুন, অথবা values ফাইল Git-এ না রেখে নিরাপদে রাখুন।

২১.৬.৩ Install, Upgrade, Rollback, History

`helm install <release-name> <chart>` প্রথমবার ইনস্টল করে।`helm upgrade <release-name> <chart>` নতুন values বা chart দিয়ে হালনাগাদ করে — নতুন revision তৈরি হয়।`helm history <release-name>` দিয়ে সব revision দেখা যায়।`helm rollback <release-name> <revision>` দিয়ে আগের revision-এ ফেরা যায় — deploy ভুল হলে এটাই দ্রুততম উদ্ধার।`helm uninstall` দিয়ে release মুছে ফেলা যা।`helm template` দিয়ে আসলে কী YAML তৈরি হবে, তা ইনস্টল না করেই দেখা যায় — পরীক্ষার জন্য অমূল্য।

Chart: Kubernetes resource-এর টেমপ্লেটসহ প্যাকেজ — Helm-এর বিতরণের একক।

Release: ক্লাস্টারে ইনস্টল করা chart-এর একটি নির্দিষ্ট সংস্করণের উদাহরণ।

Values: Chart-এর টেমপ্লেটে বসানোর জন্য কনফিগারেশন মান।

২১.৬.৪ Templating Helpers

Go template-এ `{{ .Values.image.repository }}` দিয়ে মান বসানো যায়, `{{ if .Values.ingress.enabled }}` দিয়ে শর্ত দেওয়া যায়, `{{ range }}` দিয়ে তালিকা ঘোরানো যায়, আর `{{ include "technova.fullname" . }}` দিয়ে helper ডাকা যা।`required` ফাংশন দিয়ে বাধ্যতামূলক মান চাওয়া যায় — মান না দিলে Helm ভুল দেখিয়ে থেমে যাবে, যা ভুল deploy আটকায়।

২১.৬.৫ Chart Versioning ও Helm বনাম কাঁচা YAML

Chart-এর `version` বাড়ানো হয় chart বদলালে,`appVersion` বাড়ানো হয় অ্যাপ বদলালে — দুটি আলাদা। Helm-এর সুবিধা: পুনঃব্যবহার, পরিবেশভেদে values, rollback, ইতিহাস। অসুবিধা: টেমপ্লেটের জটিলতা, ভুল খুঁজে পাওয়া কঠিন (তৈরি হওয়া YAML দেখতে `helm template` লাগে), আর খুব সহজ কাজে অতিরিক্ত ভার। নিয়ম: একটি অ্যাপ একাধিক পরিবেশে deploy করলে Helm; একটিমাত্র সহজ YAML হলে কাঁচা YAML-ই যথেষ্ট।

২১.৭ কনফিগারেশন উদাহরণ

```
# Chart.yaml
apiVersion: v2
name: technova
description: TechNova MERN app Helm chart
type: application
version: 0.1.0
appVersion: "1.2.0"
---
# values.yaml
replicaCount: 2
image:
  repository: technova/api
  tag: "v1.2.0"
  pullPolicy: IfNotPresent
service:
  type: ClusterIP
  port: 3000
ingress:
  enabled: true
  host: app.technova.example
resources:
  limits:
    cpu: 500m
    memory: 512Mi
  requests:
    cpu: 100m
    memory: 128Mi
```

```
# templates/deployment.yaml (সংক্ষিপ্ত)
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ include "technova.fullname" . }}
  labels:
    app: {{ .Chart.Name }}
spec:
  replicas: {{ .Values.replicaCount }}
  selector:
    matchLabels:
      app: {{ .Chart.Name }}
  template:
    metadata:
      labels:
        app: {{ .Chart.Name }}
    spec:
      containers:
      - name: api
        image: "{{ .Values.image.repository }}:{{ .Values.image.tag }}"
        imagePullPolicy: {{ .Values.image.pullPolicy }}
        ports:
        - containerPort: {{ .Values.service.port }}
```

```
        resources:
          {{- toYaml .Values.resources | nindent 10 }}
```

```
# প্রধান কমান্ড
helm create technova-chart
helm lint ./technova-chart
helm template technova ./technova-chart -f values-prod.yaml
helm install technova ./technova-chart -n technova --create-namespace
helm upgrade technova ./technova-chart --set image.tag=v1.3.0 -n technova
helm history technova -n technova
helm rollback technova 1 -n technova
helm list -n technova
helm uninstall technova -n technova
```

২১.৮ বাস্তব প্রকৌশল পরিস্থিতি

TechNova-র নতুন ফিচার deploy করার সময় production-এ ভুল image ট্যাগ চলে যায় — অ্যাপ চালু হয় না। আগে কাঁচা YAML ব্যবহারের সময় ফিরতে হতো Git-এর পুরোনো commit খুঁজে, হাতে apply করে — ২০ মিনিট লাগত। Helm-এর ‘helm history technova -n technova দিয়ে দেখা যায় revision 3 ভুল, revision 2 ঠিক ছিল;helm rollback technova 2 -n technova এক কমান্ডে ৩০ সেকেন্ডে আগের সংস্করণ ফিরে আসে। এই ঘটনার পর দল নিয়ম করে: প্রতিটি deploy Helm upgrade দিয়ে, আর প্রতিটি chart সংস্করণ Git tag-এর সঙ্গে মিলিয়ে রাখা হয়।

২১.৯ ধাপে ধাপে ব্যবহারিক উদাহরণ

ধাপ ১: helm create technova-chart দিয়ে একটি নমুনা chart তৈরি করুন, তার গঠন দেখুন।**ধাপ ২:** values.yaml-এ image ও replicaCount বদলান।**ধাপ ৩:** helm template technova ./technova-chart দিয়ে তৈরি হওয়া YAML দেখুন — টেমপ্লেট ঠিকমতো মান বসিয়েছে কি না, যাচাই করুন।**ধাপ ৪:** helm install technova ./technova-chart -n technova --create-namespace দিয়ে ইনস্টল করুন।**ধাপ ৫:** --set replicaCount=3 দিয়ে upgrade করুন, তারপর helm history দিয়ে দুটি revision দেখুন।**ধাপ ৬:** helm rollback technova 1 দিয়ে আগের revision-এ ফিরুন।**ধাপ ৭:** helm uninstall দিয়ে পরিষ্কার করুন।

> **LAB:** একটি values-prod.yaml ফাইল তৈরি করুন যাতে replicaCount 3, আর values-dev.yaml-এ replicaCount 1 — একই chart দিয়ে দুটি release ইনস্টল করে দেখান।

২১.১০ সাধারণ ভুল ও সমস্যা সমাধান

সমস্যা	কারণ	সমাধান
`Error: release already exists`	একই নামে release আছে	`helm upgrade` ব্যবহার করুন, অথবা নাম বদলান
Template-এ ভুল: `nil pointer`	values-এ মান নেই	`helm template` দিয়ে দেখুন; ডিফল্ট মান দিন
Upgrade-এ পুরোনো Pod থেকে যায়	Deployment-এর selector বদলানো হয়েছে	Selector বদলাবেন না; নতুন release নাম ব্যবহার করুন
Secret `--set`-এ দেওয়া	Shell history-তে ফাঁস	Values ফাইল নিরাপদে রাখুন, Secret ব্যবস্থা ব্যবহার করুন
Chart সংস্করণ না বাড়ানো	একই version-এ ভিন্ন বিষয়বস্তু	প্রতিটি পরিবর্তনে version বাড়ান

২১.১১ নিরাপত্তা বিবেচনা

Helm chart-এ Secret-এর আসল মান রাখবেন না — শুধু CHANGE-ME-... ধরনের placeholder রাখুন, আসল মান deploy-এর সময় নিরাপদ উৎস থেকে দিন। Chart-এর উৎস যাচাই করুন — অপরিচিত chart ইনস্টল করার আগে তার টেমপ্লেট পড়ুন, কারণ chart যেকোনো RBAC বা privileged Pod তৈরি করতে পারে। Helm-এর release তথ্য Secret হিসেবে সংরক্ষিত হয় — তাতে সংবেদনশীল values থাকলে তা Secret-এর মতোই সুরক্ষিত রাখতে হবে।

> **WARNING:** অপরিচিত Helm chart ইনস্টল করার আ`helm template` দিয়ে তৈরি হওয়া সব YAML পড়ুন — chart-এর ভেতরে লুকানো ClusterRoleBinding থাকতে পারে।

২১.১২ কর্মক্ষমতা ও খরচ বিবেচনা

Helm নিজে হালকা — এটি শুধু YAML তৈরি করে API server-কে দেয়। খরচের দিক: chart-এ ভুল resources (অতিরিক্ত requests) থাকলে node-এর খরচ বাড়ে; তাই values-এ resources সাবধানে বসান। বড় chart-এ অনেক টেমপ্লেট থাকলে `helm template` দিয়ে পরীক্ষা করতে সময় লাগে, তবে তা deploy-এর আগেই ধরা যায়। Chart-কে OCI registry-তে রাখলে (যেমন `helm push`) সংরক্ষণের খরচ সামান্য।

২১.১৩ সেরা অনুশীলন ও অ্যান্টি-প্যাটার্ন

সেরা অনুশীলন	অ্যান্টি-প্যাটার্ন
Chart Git-এ রাখুন, সংস্করণ বাড়ান	Chart-এর version না বাড়িয়ে বদলানো
প্রতি পরিবেশে আলাদা values ফাইল	একই values সব পরিবেশে ব্যবহার
Deploy-এর আগে helm template ও lint	না দেখে install করা
Secret values নিরাপদ উৎস থেকে দেওয়া	--set -এ পাসওয়ার্ড দেওয়া
Rollback পরীক্ষা করা	Rollback কখনো পরীক্ষা না করা

২১.১৪ অধ্যায়ের সারাংশ

Helm Kubernetes-এর প্যাকেজ ব্যবস্থাপক: chart হলো টেমপ্লেটসহ প্যাকেজ, values হলো কনফিগারেশন, release হলো ইনস্টল করা উদাহরণ, revision হলো প্রতিটি পরিবর্তনের ইতিহাস। helm install, upgrade, rollback, history দিয়ে deploy পরিচালনা করা যায়। values.yaml ও --set দিয়ে পরিবেশভেদে কনফিগারেশন বদলানো যায়। কাঁচা YAML সহজ কাজে ভালো, Helm পুনঃব্যবহার ও সংস্করণ ব্যবস্থাপনায় ভালো।

২১.১৫ মূল প্রযুক্তিগত পরিভাষা

Helm: Kubernetes-এর প্যাকেজ ব্যবস্থাপক — chart ইনস্টল, আপগ্রেড ও rollback করার টুল।

Chart: Kubernetes resource-এর টেমপ্লেটসহ প্যাকেজ।

Release: ক্লাস্টারে ইনস্টল করা chart-এর নির্দিষ্ট উদাহরণ।

Values: টেমপ্লেটে বসানোর কনফিগারেশন মান (values.yaml বা --set)।

Revision: একটি `release`-এর প্রতিটি ইনস্টল/আপগ্রেডের ক্রমিক সংস্করণ।

Go Templates: Helm-এর টেমপ্লেট ভাষা — `{{ }}` নির্দেশনা দিয়ে মান বসানো হয়।

২১.১৬ জ্ঞান যাচাই

১. Chart, release ও values-এর পার্থক্য নিজের ভাষায় লিখুন। ২. `helm upgrade` ও `helm rollback` কখন ব্যবহার করবেন? ৩. values বদলানোর তিনটি উপায় ও তাদের অগ্রাধিকার লিখুন। `helm template` কেন দরকার? ৫. Chart-এর version ও appVersion-এর পার্থক্য কী? ৬. Helm বনাম কাঁচা YAML — কখন কোনটি ভালো? `--set`-এ Secret দেওয়া কেন বিপজ্জনক?

২১.১৭ ব্যবহারিক অ্যাসাইনমেন্ট

TechNova MERN অ্যাপের জন্য একটি Helm chart তৈরি করুন: Deployment, Service, ConfigMap ও Ingress টেমপ্লেটসহ; values.yaml-এ replicaCount, image repository/tag, service port ও ingress host রাখুন। তারপর values-dev.yaml (replica 1) ও values-prod.yaml (replica 3) তৈরি করে দুটি release ইনস্টল করুন। `helm history` দিয়ে revision দেখান, একটি upgrade করুন, তারপর rollback করে দেখান। পুরো প্রক্রিয়ার কমান্ড ও আউটপুট লিখে রাখুন।

অধ্যায় 22 • PART V — Production Kubernetes Engineering (ch 21–24)

Chapter 22 — Kustomize and Environment Overlays

২২.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন **Kustomize** কী এবং কী সমস্যার সমাধান করে, তা ব্যাখ্যা করতে; **base** (ভিত্তি) ও **overlay** (উপরিস্তর) ধারণা দিয়ে dev/staging/prod পরিবেশ সাজাতে; **patches** (প্যাচ) ও **generators** (জেনারেটর) ব্যবহার করতে; `kubectl -k` দিয়ে deploy করতে; এবং Kustomize বনাম Helm-এর তুলনা করে কখন কোনটি ব্যবহার করবেন, তা সিদ্ধান্ত নিতে।

২২.২ কেন এই বিষয়টি গুরুত্বপূর্ণ

Helm শক্তিশালী, কিন্তু তার জন্য টেমপ্লেট ভাষা শিখতে হয়, আর টেমপ্লেটে ভুল হলে আসল YAML দেখা কঠিন হয়। অনেক দল চায় সহজ কিছু: কাঁচা YAML-ই থাকবে, শুধু পরিবেশভেদে ছোট ছোট বদল করা যাবে — যেমন dev-এ replica 1, prod-এ replica 3; dev-এ image ট্যাগ latest, prod-এ v1.2.0। Kustomize ঠিক এটাই করে, টেমপ্লেট ছাড়াই। এটি Kubernetes-এর অন্তর্নির্মিত (kubectl-এর সঙ্গে আসে) — আলাদা টুল ইনস্টল লাগে না। TechNova-তে যারা Helm-এর জটিলতা চায় না, তারা Kustomize দিয়ে একই base থেকে তিন পরিবেশ চালায়। দুটি টুলই বৈধ; প্রশ্ন হলো আপনার দলের জন্য কোনটি সহজ।

> **REAL-WORLD:** TechNova-র একটি ছোট দল Helm chart লিখতে গিয়ে টেমপ্লেটের ভুলে দুই দিন নষ্ট করে। তারা Kustomize-এ ফিরে আসে: base ফোল্ডারে আসল YAML, প্রতিটি পরিবেশের ফোল্ডারে শুধু প্যাচ — একদিনেই সব পরিবেশ চালু হয়।

২২.৩ পূর্বশর্ত জ্ঞান

আপনার জানা থাকা উচিত: Deployment, Service, ConfigMap-এর YAML; `kubectl apply`; এবং Helm-এর মৌলিক ধারণা (অধ্যায় ২১) — তুলনা বুঝতে। YAML-এর patch ধারণা (একটি ফাইলের অংশ বদলানো) জানা থাকলে ভালো হয়।

২২.৪ প্রথম নীতি থেকে ব্যাখ্যা

প্রথম নীতি: বেশিরভাগ পরিবেশের মধ্যে পার্থক্য ছোট — replica সংখ্যা, image ট্যাগ, resource সীমা, পরিবেশের নাম। পুরো ফাইল কপি করে বদলানো মানে একই জিনিস তিন জায়গায় রাখা — এক জায়গায় ভুল ঠিক করলে অন্য জায়গায় থেকে যায়। Kustomize এর নীতি: **একটি base** (সাধারণ অংশ) রাখুন, আর প্রতিটি পরিবেশের জন্য **শুধু পার্থক্য** লিখুন। Kustomize base + overlay মিলিয়ে চূড়ান্ত YAML তৈরি করে — মূল base ফাইল বদলায় না। এটি টেমপ্লেট না, **transformation** (রূপান্তর): base YAML পড়ে, তার ওপর প্যাচ, নামের উপসর্গ (prefix), label, image বদল প্রয়োগ করে। যেমন overlay-এ `namePrefix: prod-` বললে সব resource-এর নামের আগে prod- বসে, `images` বললে image ট্যাগ বদলে যায়। এইভাবে DRY (Don't Repeat Yourself — নিজেকে পুনরাবৃত্তি কোরো না) নীতি মানা হয়, টেমপ্লেট ভাষা ছাড়াই।

> **CONCEPT:** Helm বলে "টেমপ্লেটে মান বসাও", Kustomize বলে "আসল YAML নাও, তার ওপর শুধু পার্থক্য প্রয়োগ করো"। দুটি ভিন্ন দর্শন, একই লক্ষ্য — পরিবেশভেদে কনফিগারেশন।

২২.৫ আর্কিটেকচার / ধারণাগত চিত্র



চিত্র ২২.১ — Base + overlays গঠন এবং kubectl -k-এর প্রবাহ।

২২.৬ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

২২.৬.১ Base ও Overlay

Base হলো সাধারণ YAML-এর ফোল্ডার, যাতে একটি `kustomization.yaml` থাকে যেখানে `resources`-এ সব ফাইলের তালিকা থাকে। Overlay হলো আরেকটি ফোল্ডার, যার `kustomization.yaml`-এ `bases` বা `resources` দিয়ে base-এর পথ বলা থাকে, আর তার সঙ্গে পরিবেশের পার্থক্য। Overlay base-কে বদলায় না — সে শুধু নিজের মতো করে রূপান্তরিত সংস্করণ তৈরি করে। একই base থেকে অনেক overlay বানানো যায়।

২২.৬.২ Patches — লক্ষ্যভিত্তিক বদল

Patch মানে নির্দিষ্ট resource-এর নির্দিষ্ট অংশ বদলানো। **Strategic merge patch** — YAML-এর অংশ লিখে বদলানো (যেমন replica সংখ্যা)। **JSON patch** — `op: replace, path: /spec/replicas, value: 3` ধরনের নির্দেশনা। Kustomize-

এর `patches` ক্ষেত্রে `target` দিয়ে কোন resource (kind, name) বদলাবেন, তা বলা হয়। সাধারণ কা `replicas`, `images`, `namePrefix`, `commonLabels` — এই অন্তর্নির্মিত ক্ষেত্রগুলোই যথেষ্ট; জটিল বদলে patch লিখতে হয়।

২২.৬.৩ Generators — ConfigMap ও Secret তৈরি

Generators দিয়ে ConfigMap ও Secret তৈরি করা যায়: `configMapGenerator` দিয়ে ফাইল বা key-value থেকে ConfigMap, `secretGenerator` দিয়ে Secret। সুবিধা: প্রতিটি পরিবর্তনে Kustomize নামের শেষে hash যোগ করে দেয় — ফলে ConfigMap বদলালে Pod স্বয়ংক্রিয়ভাবে নতুন ConfigMap পায় (নাম বদলে যায় বলে Deployment-এর Pod টেমপ্লেট বদলায়)। মনে রাখবেন: Secret generator-এর মান Git-এ রাখবেন না — শুধু placeholder রাখুন, আসল মান নিরাপদ উৎস থেকে দিন।

Base: সাধারণ Kubernetes YAML-এর ভিত্তি, যা সব পরিবেশে একই থাকে।

Overlay: একটি পরিবেশের জন্য base-এর ওপর প্রয়োগ করা পার্থক্যের স্তর।

Patch: নির্দিষ্ট resource-এর নির্দিষ্ট অংশ বদলানোর নির্দেশনা।

২২.৬.৪ kubectl -k ও kustomize build

`kubectl apply -k overlays/prod` দিয়ে overlay প্রয়োগ করা যায় — kubectl ভেতরে Kustomize চাল `kubectl kustomize overlays/prod` বা `kustomize build overlays/prod` দিয়ে চূড়ান্ত YAML দেখা যায়, প্রয়োগ না করেই — পরীক্ষার জন্য অমূল্য। GitOps-এ (অধ্যায় ২৪) Argo CD ও Flux Kustomize সরাসরি সমর্থন করে, তাই overlay Git-এ রেখে স্বয়ংক্রিয় deploy করা যায়।

২২.৬.৫ Kustomize বনাম Helm — তুলনা

দিক	Kustomize	Helm
ধরন	YAML রূপান্তর, টেমপ্লেট নয়	Go টেমপ্লেট + প্যাকেজ
শেখার ভার	কম — শুধু YAML	বেশি — টেমপ্লেট ভাষা শিখতে হয়
পরিবেশ ব্যবস্থাপনা	Base + overlay — খুব ভালো	Values ফাইল — ভালো

Rollback / ইতিহাস	নেই (Git-এর ওপর নির্ভর)	আছে (revision, history)
প্যাকেজ বিতরণ	নেই	আছে (chart repository, OCI)
kubectl-এ অন্তর্নির্মিত	হ্যাঁ (`-k`)	না, আলাদা টুল
কখন ব্যবহার করবেন	নিজের অ্যাপ, পরিবেশভেদে ছোট পার্থক্য	তৃতীয় পক্ষের অ্যাপ, সংস্করণ ও rollback দরকার

অনেক দল দুটিই একসঙ্গে ব্যবহার করে: Helm chart-কে Kustomize overlay দিয়ে পরিবেশভেদে বদলানো যায়। কোনোটি "ভালো" নয় — প্রশ্ন হলো আপনার দলের জন্য কোনটি সহজ ও টেকসই।

২২.৭ কনফিগারেশন উদাহরণ

```
# base/kustomization.yaml
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
resources:
- deployment.yaml
- service.yaml
commonLabels:
  app: technova
---
# base/deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: api
spec:
  replicas: 2
  selector:
    matchLabels:
      app: api
  template:
    metadata:
      labels:
        app: api
    spec:
      containers:
      - name: api
        image: technova/api:v1.2.0
        ports:
        - containerPort: 3000
---
# overlays/prod/kustomization.yaml
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
resources:
- ../../base
namePrefix: prod-
commonLabels:
```

```
  env: prod
replicas:
- name: api
  count: 3
images:
- name: technova/api
  newTag: v1.2.0
patches:
- path: patch-resources.yaml
  target:
    kind: Deployment
    name: api
---
# overlays/prod/patch-resources.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: api
spec:
  template:
    spec:
      containers:
      - name: api
        resources:
          limits:
            cpu: 500m
            memory: 512Mi
          requests:
            cpu: 100m
            memory: 128Mi
---
# overlays/dev/kustomization.yaml
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
resources:
- ../../base
namePrefix: dev-
commonLabels:
  env: dev
replicas:
- name: api
  count: 1
```

```
# ব্যবহার
kubectl kustomize overlays/prod
kubectl apply -k overlays/dev
kubectl apply -k overlays/prod
kubectl diff -k overlays/prod
```

২২.৮ বাস্তব প্রকৌশল পরিস্থিতি

TechNova-র dev, staging ও prod — তিন পরিবেশে একই MERN অ্যাপ চলে, কিন্তু পার্থক্য আছে: dev-এ replica 1, staging-এ 2, prod-এ 3; dev-এ resource limit কম, prod-এ বেশি; প্রতিটি পরিবেশের ConfigMap-এ ডাটাবেসের ঠিকানা আলাদা। আগে তিনটি আলাদা ফোল্ডারে পুরো YAML কপি করা থাকত — একদিন base-এ securityContext যোগ করা হয়, কিন্তু prod-এর কপিতে যোগ করা হয়নি, ফলে prod Pod root হিসেবে চলতে থাকে। Kustomize-এ আসার পর base-এ একবার যোগ করলেই তিন পরিবেশে চলে যায়; overlay-তে শুধু পার্থক্য থাকে। এখন নতুন পরিবেশ যোগ করতে শুধু একটি নতুন overlay ফোল্ডার বানাতে হয় — দশ মিনিটের কাজ।

২২.৯ ধাপে ধাপে ব্যবহারিক উদাহরণ

ধাপ ১: `base/` ফোল্ডারে deployment.yaml, service.yaml ও kustomization.yaml লিখুন**ধাপ ২:** `overlays/dev/` ও `overlays/prod/` ফোল্ডার বানান, প্রতিটিতে kustomization.yaml লিখুন (dev-এ replica 1, prod-এ replica 3)। **ধাপ ৩:** `kubectl kustomize overlays/prod` দিয়ে চূড়ান্ত YAML দেখুন — নামের আগে prod- বসেছে কি না, replica 3 হয়েছে কি না, যাচাই করুন**ধাপ ৪:** `kubectl apply -k overlays/dev` দিয়ে dev deploy করুন**ধাপ ৫:** base-এ একটি label যোগ করুন, তারপর `kubectl diff -k overlays/prod` দিয়ে দেখুন prod-এও সেই পরিবর্তন আসছে কি না। **ধাপ ৬:** একটি ConfigMap generator যোগ করে দেখুন নামের শেষে hash বদলায় কি না।

> **PRO-TIP:** kubectl diff -k দিয়ে প্রয়োগের আগে কী বদলাবে, তা দেখে নিন — ভুল deploy আটকানোর সবচেয়ে সহজ উপায়।

২২.১০ সাধারণ ভুল ও সমস্যা সমাধান

সমস্যা	কারণ	সমাধান
kustomization.yaml not found	ভুল ফোল্ডারে -k চালানো	overlay ফোল্ডারের পথ যাচাই করুন
Patch কাজ করছে না	target-এর kind/name ভুল	base-এর resource-এর নাম যাচাই করুন
namePrefix-এ Service-এর নাম বদলে গেছে	Prefix সব resource-এ বসে	এটাই স্বাভাবিক; Service-এর নাম overlay অনুযায়ী বদলায়
Secret Git-এ ফাঁস	secretGenerator-এ আসল মান রাখা	Placeholder রাখুন, আসল মান নিরাপদ উৎস থেকে দিন
Base বদলে overlay ভেঙে গেছে	Patch base-এর পুরোনো গঠন ধরে নিয়েছে	kustomize build দিয়ে পরীক্ষা করুন

২২.১১ নিরাপত্তা বিবেচনা

Kustomize-এর Secret generator-এ আসল Secret রাখবেন না — Git-এ শুধু placeholder রাখুন। Overlay-তে পরিবেশের নাম label হিসেবে দিলে ভুল পরিবেশে deploy ধরা সহজ হয়। Base-এর SecurityContext ও NetworkPolicy সব overlay-তে চলে যায় — তাই base-এ নিরাপত্তা ঠিক রাখলে সব পরিবেশ নিরাপদ থা`kubectl apply -k` করার আগে `kubectl diff -k` দিয়ে দেখে নিন, যাতে ভুল করে production-এ dev-এর overlay না যায়।

২২.১২ কর্মক্ষমতা ও খরচ বিবেচনা

Kustomize-এর কর্মক্ষমতার খরচ নগণ্য — এটি শুধু YAML তৈরি করে। খরচের আসল দিক: overlay-তে ভুল resources (অতিরিক্ত requests) থাকলে node-এর খরচ বাড়ে। Dev overlay-তে replica 1 ও কম resources রাখলে dev-এর খরচ কমে — এটাই overlay-এর বড় সুবিধা: প্রতিটি পরিবেশের খরচ আলাদাভাবে নিয়ন্ত্রণ করা যায়।

২২.১৩ সেরা অনুশীলন ও অ্যান্টি-প্যাটার্ন

সেরা অনুশীলন	অ্যান্টি-প্যাটার্ন
Base-এ সাধারণ অংশ, overlay-তে শুধু পার্থক্য	প্রতিটি পরিবেশে পুরো YAML কপি করা
প্রয়োগের আগে `kustomize build` ও `diff`	না দেখে apply করা
পরিবেশের label (env: prod) দেওয়া	কোন পরিবেশের Pod, তা না জানা
ConfigMap generator-এর hash ব্যবহার	ConfigMap বদলে Pod রিস্টার্ট না করা
Helm-এর সঙ্গে প্রয়োজনে মিলিয়ে ব্যবহার	একটি টুলই সব জায়গায় জোর করে ব্যবহার

২২.১৪ অধ্যায়ের সারাংশ

Kustomize টেমপ্লেট ছাড়াই YAML রূপান্তর করে: একটি base থেকে অনেক overlay বানানো যায়, প্রতিটি পরিবেশের শুধু পার্থক্য লিখতে হয়। Patches দিয়ে নির্দিষ্ট অংশ বদলানো যায়, generators দিয়ে ConfigMap/Secret তৈরি করা `kubectl -k` দিয়ে deploy করা যায়, `kustomize build` দিয়ে চূড়ান্ত YAML দেখা যায়। Helm প্যাকেজ ও rollback-এ ভালো, Kustomize সরলতা ও পরিবেশভেদে ভালো — প্রয়োজনে দুটিই একসঙ্গে ব্যবহার করা যায়।

২২.১৫ মূল প্রযুক্তিগত পরিভাষা

Kustomize: টেমপ্লেট ছাড়াই Kubernetes YAML রূপান্তরকারী টুল; kubectl-এ অন্তর্নির্মিত।

Base: সাধারণ YAML-এর ভিত্তি, যা সব পরিবেশে একই।

Overlay: একটি পরিবেশের জন্য base-এর ওপর পার্থক্যের স্তর।

Patch: নির্দিষ্ট resource-এর নির্দিষ্ট অংশ বদলানোর নির্দেশনা।

Generator: ConfigMap বা Secret স্বয়ংক্রিয়ভাবে তৈরি করার ব্যবস্থা।

kustomization.yaml: Kustomize-এর কনফিগারেশন ফাইল — resources, patches, overlays ঘোষণা করে।

২২.১৬ জ্ঞান যাচাই

১. Base ও overlay-এর ধারণা নিজের ভাষায় লিখুন। ২. Kustomize Helm থেকে কীভাবে আলাদা? ৩. Patch কখন দরকার, উদাহরণ দিন। ৪. ConfigMap generator-এর hash কেন উপকারী? ৫ `kubectl -k` ও `kubectl apply -f`-এর পার্থক্য কী? ৬. Kustomize বনাম Helm তুলনা সারণি থেকে তিনটি পার্থক্য লিখুন। ৭. নতুন পরিবেশ যোগ করতে Kustomize-এ কী করতে হয়?

২২.১৭ ব্যবহারিক অ্যাসাইনমেন্ট

TechNova অ্যাপের জন্য Kustomize গঠন তৈরি করুন: base-এ Deployment ও Service; overlays/dev (replica 1, namePrefix dev-), overlays/staging (replica 2), overlays/prod (replica 3, resource limits-এর

patch, image tag v1.2.0)। `kubectl kustomize` দিয়ে প্রতিটি overlay-এর চূড়ান্ত YAML দেখান, তারপর dev ও prod apply করুন। একটি সংক্ষিপ্ত নোট লিখুন: Helm-এর বদলে Kustomize কখন বেছে নেবেন, আর কখন Helm ভালো?

অধ্যায় 23 • PART V — Production Kubernetes Engineering (ch 21–24)

Chapter 23 — Cluster Upgrades, etcd Backup, Disaster Recovery, and Managed Kubernetes

২৩.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন — ক্লাস্টার আপগ্রেডের সঠিক কৌশল ব্যাখ্যা করতে (control plane আগে, node পরে; version skew নীতি); **etcdctl snapshot** দিয়ে etcd-এর ব্যাকআপ নেওয়া ও ফেরানো; **RTO** (Recovery Time Objective) ও **RPO** (Recovery Point Objective) দিয়ে দুর্যোগ-পুনরুদ্ধার পরিকল্পনা লিখতে; এবং **managed Kubernetes** — EKS, AKS, GKE, IBM Cloud — এর তুলনা করতে: control plane-এর দায়িত্ব, node ব্যবস্থাপনা, নেটওয়ার্কিং, IAM, স্টোরেজ, লোড ব্যালান্সার ও খরচের দিক থেকে, কোনো উদ্ভাবিত মূল্য বা বৈশিষ্ট্য ছাড়াই।

২৩.২ কেন এই বিষয়টি গুরুত্বপূর্ণ

ক্লাস্টার একবার চালু করলেই শেষ নয় — তাকে হালনাগাদ করতে হয়, ব্যাকআপ নিতে হয়, আর দুর্যোগের জন্য প্রস্তুত থাকতে হয়। Kubernetes প্রতি কয়েক মাসে নতুন সংস্করণ প্রকাশ করে; পুরোনো সংস্করণে থাকলে নিরাপত্তা প্যাচ পাওয়া যায় না। আবার ভুল আপগ্রেডে পুরো ক্লাস্টার ডাউন হতে পারে। etcd হলো ক্লাস্টারের মস্তিষ্ক — তাতে সব অবস্থা (state) থাকে; etcd হারিয়ে গেলে ক্লাস্টার হারিয়ে যায়। আর দুর্যোগ — ডাটা সেন্টারে আগুন, অঞ্চল (region) ডাউন, ভুল করে সব মুছে ফেলা — কখনোই "হবে না" ভাবা যায় না। TechNova-র গ্রাহকের তথ্য etcd ও ডাটাবেসে থাকে; ব্যাকআপ না থাকলে একটি ভুলে সব হারিয়ে যেতে পারে। Managed Kubernetes এই বোঝার অনেকটাই ক্লাউড প্রদানকারীর কাছে সরিয়ে দেয় — কিন্তু কী সরিয়ে দেয়, আর কী আপনার দায়িত্বে থাকে, তা জানা জরুরি।

> **REAL-WORLD:** TechNova-র staging ক্লাস্টারে একবার ভুল করে সব namespace মুছে ফেলা হয়। etcd ব্যাকআপ ছিল না, তবে Git-এ সব YAML ছিল — তাই অ্যাপ ফিরিয়ে আনা যায়, কিন্তু ডাটাবেসের তথ্য হারিয়ে যায়। এরপর থেকে প্রতিদিন etcd snapshot ও ডাটাবেস ব্যাকআপ — দুটিই বাধ্যতামূলক।

২৩.৩ পূর্বশর্ত জ্ঞান

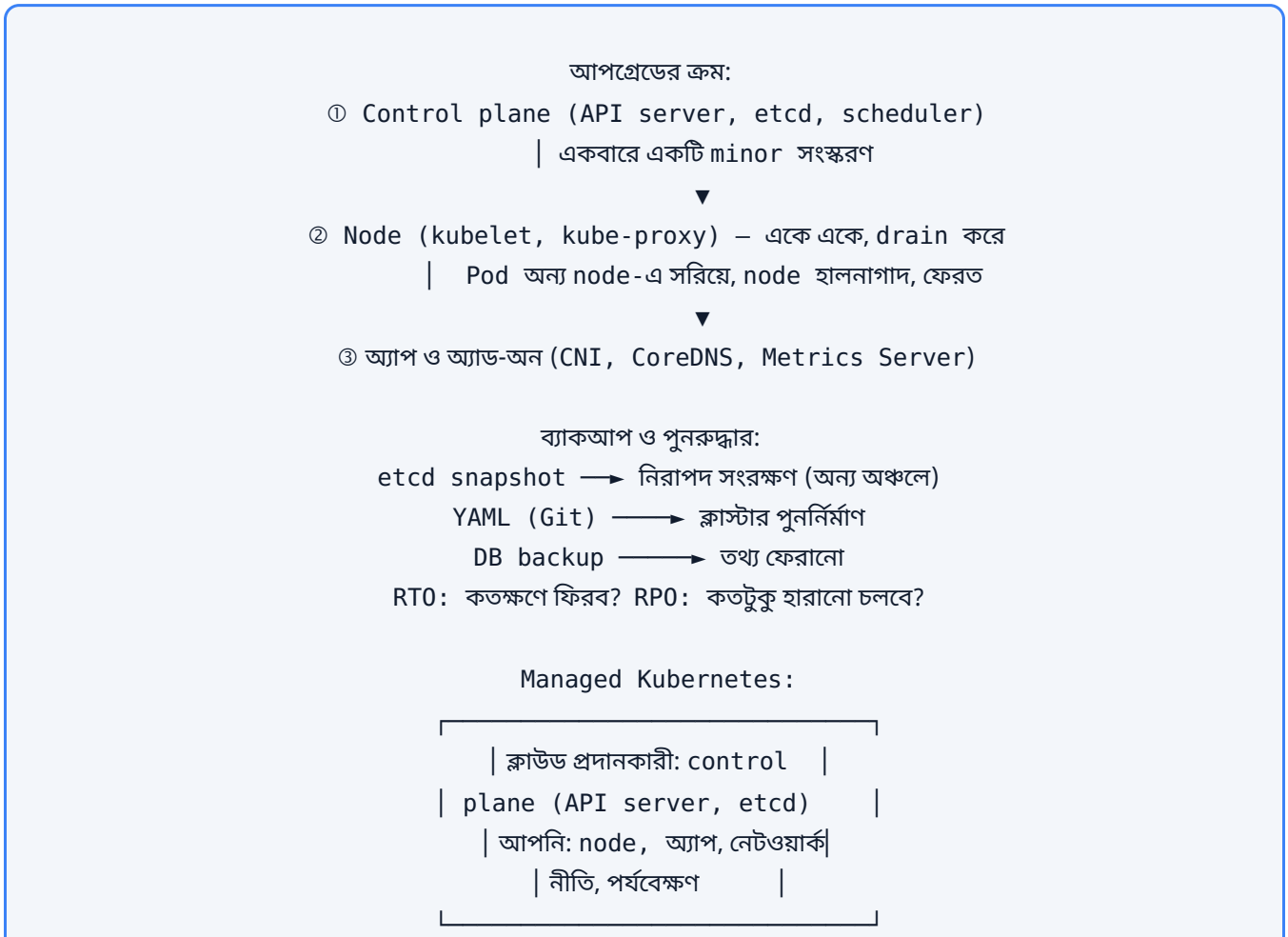
আপনার জানা থাকা উচিত: control plane-এর উপাদান (API server, etcd, scheduler, controller-manager) ও node-এর উপাদান (kubelet, kube-proxy); Deployment ও StatefulSet-এর ধারণা; এবং ক্লাউডের মৌলিক ধারণা — region, availability zone, managed service। etcd-এর ধারণা আগের অধ্যায়গুলো থেকে জানা থাকলে ভালো হয়।

২৩.৪ প্রথম নীতি থেকে ব্যাখ্যা

প্রথম নীতি: ক্লাস্টারের সব অবস্থা একটি জায়গায় থাকে — etcd-এ। Pod, Service, Secret, Deployment — সবকিছুর ঘোষণা etcd-এ সংরক্ষিত। API server শুধু etcd-এর সামনের দরজা। তাই etcd-এর ব্যাকআপ মানে ক্লাস্টারের ঘোষণার ব্যাকআপ। কিন্তু etcd-এ ডাটাবেসের ভেতরের তথ্য (যেমন MongoDB-র তথ্য) থাকে না — সেটি আলাদাভাবে ব্যাকআপ নিতে হয়। আপগ্রেডের নীতি: control plane আগে, node পরে — কারণ নতুন control plane পুরোনো node-কে বোঝে, কিন্তু পুরোনো control plane নতুন node-কে নাও বুঝতে পারে। **Version skew policy** বলে: kubelet-এর সংস্করণ API server-এর চেয়ে তিনটি minor সংস্করণের বেশি পুরোনো হতে পারবে না। দুর্যোগ-পুনরুদ্ধারের দুটি সংখ্যা: **RTO** — কতক্ষণের মধ্যে ফিরতে হবে; **RPO** — কতটুকু তথ্য হারানো চলবে। RTO ১ ঘণ্টা মানে ১ ঘণ্টার মধ্যে সেবা ফিরতে হবে; RPO ১ ঘণ্টা মানে সর্বোচ্চ ১ ঘণ্টার তথ্য হারানো চলবে। এই দুটি সংখ্যা করলেই ব্যাকআপের ঘনত্ব ও পুনরুদ্ধারের পদ্ধতি ঠিক হয়।

> **CONCEPT:** etcd ব্যাকআপ = ক্লাস্টারের ঘোষণার ব্যাকআপ। ডাটাবেসের তথ্যের ব্যাকআপ আলাদা। দুটি না থাকলে পুনরুদ্ধার অসম্পূর্ণ।

২৩.৫ আর্কিটেকচার / ধারণাগত চিত্র



চিত্র ২৩.১ — আপগ্রেডের ক্রম, ব্যাকআপের তিন স্তর ও managed Kubernetes-এর দায়িত্ব ভাগ।

২৩.৬ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

২৩.৬.১ আপগ্রেড কৌশল

নিয়ম: (১) একবারে একটি minor সংস্করণ (যেমন ১.৩০ থেকে ১.৩১) — দুটি লাফ দেবেন না। (২) Control plane আগে, node পরে। (৩) প্রথমে staging-এ পরীক্ষা করুন, তারপর production-এ। (৪) আপগ্রেডের আগে etcd ব্যাকআপ নিন। (৫) Node আপগ্রেডের সময় `kubectl drain` দিয়ে Pod সরিয়ে নিন, তারপর node হালনাগাদ ক `kubectl uncordon` করুন। (৬) অ্যাড-অন (CNI, CoreDNS) নতুন সংস্করণের সঙ্গে সামঞ্জস্যপূর্ণ কি না, তা যাচাই করুন। kubeadm ব্যবহার ঃ `kubeadm upgrade plan` দিয়ে পরিকল্পনা দেখা যায়, `kubeadm upgrade apply` দিয়ে control plane হালনাগাদ করা যায়।

২৩.৬.২ etcd ব্যাকআপ ও পুনরুদ্ধার

etcd-এর ব্যাকআপ নেওয়া হয় `etcdctl snapshot save` দিয়ে। Control-plane node-এ etcd-এর শংসাপত্র (certificate) ও endpoint দিয়ে কমান্ড চালাতে হয়। Snapshot ফাইলটি নিরাপদ জায়গায় রাখুন — অন্য অঞ্চলে বা আলাদা স্টোরেজে, কারণ একই node-এ রাখলে node হারালে ব্যাকআপও হারাবে। পুনরুদ্ধার (`etcdctl snapshot restore`) একটি ধ্বংসাত্মক কাজ — etcd বন্ধ করে, snapshot থেকে নতুন ডাটা ডিরেক্টরি তৈরি করে, তারপর etcd চালু করতে হয়। নিয়ম: পুনরুদ্ধার নিয়মিত পরীক্ষা করুন — ব্যাকআপ আছে, কিন্তু ফেরানো যায় না, এমন ব্যাকআপের কোনো মূল্য নেই। Managed ক্লাস্টারে etcd-এর ব্যাকআপ প্রায়শই প্রদানকারী নিজেই নেয় — তবে তা যাচাই করা আপনার দায়িত্ব: প্রদানকারীর নথি (documentation) দেখে নিশ্চিত হোন ব্যাকআপ কতদিন রাখা হয়, আর কীভাবে ফেরানো যায়।

etcd: ক্লাস্টারের সব অবস্থা সংরক্ষণকারী বিতরণকৃত key-value ভান্ডার।

RTO: Recovery Time Objective — দুর্যোগের পর কতক্ষণের মধ্যে সেবা ফিরতে হবে।

RPO: Recovery Point Objective — সর্বোচ্চ কতটুকু তথ্য হারানো চলবে (কত ঘন ঘন ব্যাকআপ দরকার)।

২৩.৬.৩ দুর্যোগ-পুনরুদ্ধার পরিকল্পনা

পরিকল্পনায় থাকতে হবে: (১) RTO ও RPO — ব্যবসার সঙ্গে ঠিক করা সংখ্যা। (২) ব্যাকআপের তালিকা — etcd, ডাটাবেস, YAML (Git), Secret (নিরাপদ ভান্ডার)। (৩) পুনরুদ্ধারের ধাপ — কে কী করবে, কোন ক্রমে। (৪) পরীক্ষা — প্রতি তিন মাসে একবার পুনরুদ্ধারের মহড়া (drill)। (৫) যোগাযোগ পরিকল্পনা — গ্রাহককে কে জানাবে। মনে রাখবেন: Git-এ YAML থাকলে ক্লাস্টার পুনর্নির্মাণ করা যায়, কিন্তু ডাটাবেসের তথ্য Git-এ থাকে না — তার জন্য আলাদা ব্যাকআপ দরকার।

২৩.৬.৪ Managed Kubernetes তুলনা — EKS, AKS, GKE, IBM Cloud

Managed Kubernetes-এ ক্লাউড প্রদানকারী control plane (API server, etcd, scheduler) চালায় ও হালনাগাদ করে; আপনি node ও অ্যাপ চালান। চারটি প্রধান সেবার তুলনা — কোনো উদ্ভাবিত মূল্য বা বৈশিষ্ট্য ছাড়া, শুধু সাধারণ ধারণা:

দিক	EKS (AWS)	AKS (Azure)	GKE (Google Cloud)	IBM Cloud Kubernetes
Control plane	AWS পরিচালনা করে	Azure পরিচালনা করে	Google পরিচালনা করে	IBM পরিচালনা করে
Node ব্যবস্থাপনা	Managed node group / Fargate	Node pool	Node pool / Autopilot	Worker pool

IAM সংযোগ	AWS IAM	Azure AD / Entra ID	Google IAM	IBM Cloud IAM
নেটওয়ার্কিং	VPC CNI	Azure CNI / Kubenet	GKE CNI	Calico / VPC
লোড ব্যালান্সার	AWS LB Controller / ELB	Azure LB	GKE Ingress / Cloud LB	IBM LB / Ingress
স্টোরেজ	EBS CSI driver	Azure Disk/ File CSI	Persistent Disk CSI	Block/File CSI

খরচের বিবেচনা: control plane-এর জন্য প্রদানকারী আলাদা ফি নিতে পারে, node-এর জন্য কম্পিউট খরচ আলাদা, আর ডাটা স্থানান্তর, লোড ব্যালান্সার ও স্টোরেজের খরচও আলাদা — সঠিক মূল্য প্রদানকারীর মূল্য পৃষ্ঠা (pricing page) থেকে যাচাই করুন, এই বইয়ে কোনো মূল্য দেওয়া হয়নি। সুবিধা: control plane-এর রক্ষণাবেক্ষণ, etcd ব্যাকআপ ও আপগ্রেডের বোঝা কমে। অসুবিধা: প্রদানকারীর ওপর নির্ভরতা (vendor lock-in), কিছু সেটিংসে নিয়ন্ত্রণ কম, আর খরচের পূর্বাভাস কঠিন হতে পারে। নিজে চালানো (self-managed) ক্লাস্টারে পূর্ণ নিয়ন্ত্রণ থাকে, কিন্তু control plane, etcd ও আপগ্রেডের সব দায়িত্ব আপনার।

২৩.৭ কনফিগারেশন উদাহরণ

```
# etcd snapshot নেওয়া (control-plane node-এ; পথ পরিবেশভেদে বদলায়)
ETCDCTL_API=3 etcdctl --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key \
  snapshot save /backup/etcd-snapshot-$(date +%F).db
# snapshot যাচাই
ETCDCTL_API=3 etcdctl snapshot status /backup/etcd-snapshot-2026-11-09.db
# পুনরুদ্ধার (সাবধানে — etcd বন্ধ করে করতে হয়)
ETCDCTL_API=3 etcdctl snapshot restore /backup/etcd-snapshot-2026-11-09.db \
  --data-dir=/var/lib/etcd-restored
```

```
# Node drain ও আপগ্রেড
kubectl drain node-1 --ignore-daemonsets --delete-emptydir-data
# node হালনাগাদ করার পর
kubectl uncordon node-1
kubectl get nodes
# kubeadm দিয়ে আপগ্রেড পরিকল্পনা
kubeadm upgrade plan
kubeadm upgrade apply v1.31.0
```

উপরের শংসাপত্রের পথ kubeadm-ভিত্তিক ক্লাস্টারের জন্য; অন্য ইনস্টলেশনে পথ আলাদা হতে পারে — আপনার পরিবেশের নথি দেখে নিন। Managed ক্লাস্টারে etcd-এ সরাসরি প্রবেশ থাকে না; সেখানে প্রদানকারীর ব্যাকআপ ব্যবস্থা ব্যবহার করতে হয়।

২৩.৮ বাস্তব প্রকৌশল পরিস্থিতি

TechNova-র ক্লাস্টার ১.৩০ সংস্করণে চলছিল। দল আপগ্রেডের পরিকল্পনা করে: প্রথমে staging-এ ১.৩১-এ উন্নীত করা হয়, এক সপ্তাহ পর্যবেক্ষণ করা হয়, তারপর production-এ control plane হালনাগাদ করা হয়, তারপর একে একে node drain করে হালনাগাদ করা হয়। আপগ্রেডের আগে etcd snapshot নেওয়া হয়, আর PodDisruptionBudget (অধ্যায় ২৪) নিশ্চিত করে যে একসঙ্গে খুব বেশি Pod ডাউন হয় না। পুরো প্রক্রিয়ায় ব্যবহারকারীরা কিছু টের পায় না। এর বিপরীতে, একটি প্রতিষ্ঠান এক লাফে দুটি সংস্ক এগিয়ে যায় — API server ও kubelet-এর অসামঞ্জস্যে Pod তৈরি বন্ধ হয়ে যায়। শিক্ষা: ধীরে, এক ধাপে, ব্যাকআপ নিয়ে এগোন।

২৩.৯ ধাপে ধাপে ব্যবহারিক উদাহরণ

ধাপ ১: বর্তমান সংস্করণ দেখুন: `kubectl version` ও `kubectl get nodes`। **ধাপ ২:** etcd snapshot নিন (উপরের কমান্ড অনুযায়ী) এবং snapshot ফাইলটি নিরাপদ জায়গায় কপি করু**ধাপ ৩:** `kubeadm upgrade plan` দিয়ে দেখুন কোন সংস্করণে যাওয়া যায়**ধাপ ৪:** একটি RTO/RPO সারণি লিখুন: TechNova-র জন্য RTO ২ ঘণ্টা, RPO ১ ঘণ্টা হলে ব্যাকআপ কত ঘন ঘন নিতে হবে? **ধাপ ৫:** একটি node drain করার মহড়া দিন (staging-এ) `kubectl drain`, Pod অন্য node-এ গেছে কি না দেখুন, তারপর `uncordon` করুন। **ধাপ ৬:** আপনার ক্লাউড প্রদানকারীর নথি থেকে managed ব্যাকআপের বিবরণ লিখে রাখুন।

> **WARNING:** etcd snapshot restore একটি ধ্বংসাত্মক কাজ — production-এ করার আগে staging-এ মহড়া দিন, আর snapshot-এর অনুলিপি নিরাপদে রাখুন।

২৩.১০ সাধারণ ভুল ও সমস্যা সমাধান

সমস্যা	কারণ	সমাধান
আপগ্রেডের পর Pod তৈরি হচ্ছে না	Version skew লঙ্ঘন	kubelet ও API server-এর সংস্করণ মেলান
etcd ডিস্ক পূর্ণ	পুরোনো revision জমে আছে	etcd defrag ও compaction; snapshot নিয়ে জায়গা করুন
ব্যাকআপ আছে, ফেরানো যাচ্ছে না	পুনরুদ্ধার কখনো পরীক্ষা করা হয়নি	নিয়মিত মহড়া দিন
Node drain-এ Pod আটকে আছে	PodDisruptionBudget বাধা দিচ্ছে	PDB যাচাই করুন; প্রয়োজনে সাময়িকভাবে সমন্বয় করুন

Managed ক্লাস্টারে etcd-এ প্রবেশ নেই	প্রদানকারী etcd পরিচালনা করে	প্রদানকারীর ব্যাকআপ ব্যবস্থা ব্যবহার করুন

২৩.১১ নিরাপত্তা বিবেচনা

etcd snapshot-এ সব Secret-এর আসল মান থাকে — তাই snapshot ফাইলটি Secret-এর মতোই সুরক্ষিত রাখুন, এনক্রিপ্ট করে সংরক্ষণ করুন, আর প্রবেশ সীমিত রাখুন। Snapshot কখনো public স্টোরেজে রাখবেন না। আপগ্রেডের সময় পুরোনো সংস্করণের নিরাপত্তা দুর্বলতা থেকে যায় — তাই আপগ্রেড দেরি করবেন না। Managed ক্লাস্টারে IAM-এর মাধ্যমে কে ক্লাস্টারে প্রবেশ করতে পারে, তা নিয়ন্ত্রণ করুন; API server কখনোই পাবলিক ইন্টারনেটে খুলে দেবেন না।

২৩.১২ কর্মক্ষমতা ও খরচ বিবেচনা

etcd-এর ডিস্ক ধীর হলে API server ধীর হয় — etcd-এর জন্য দ্রুত SSD ব্যবহার করুন। ঘন ঘন snapshot নিলে ডিস্ক ও নেটওয়ার্ক খরচ বাড়ে; RPO অনুযায়ী ঘনত্ব ঠিক করুন। Managed Kubernetes-এ control plane-এর ফি, node-এর কম্পিউট খরচ, লোড ব্যালান্সার ও স্টোরেজ — সব মিলিয়ে খরচ হিসাব করুন; সঠিক মূল্য প্রদানকারীর মূল্য পৃষ্ঠা থেকে যাচাই করুন। Multi-AZ node রাখলে নির্ভরযোগ্যতা বাড়ে, কিন্তু খরচও বাড়ে — ব্যবসার প্রয়োজনে ভারসাম্য রাখুন।

২৩.১৩ সেরা অনুশীলন ও অ্যান্টি-প্যাটার্ন

সেরা অনুশীলন	অ্যান্টি-প্যাটার্ন
একবারে একটি minor সংস্করণ আপগ্রেড	দুটি সংস্করণ লাফ দেওয়া
আপগ্রেডের আগে etcd ব্যাকআপ	ব্যাকআপ ছাড়া আপগ্রেড
প্রথমে staging, তারপর production	সরাসরি production-এ আপগ্রেড
নিয়মিত পুনরুদ্ধারের মহড়া	ব্যাকআপ নিয়ে কখনো পরীক্ষা না করা
RTO/RPO লিখিত পরিকল্পনা	মৌখিক "ব্যাকআপ আছে" ভরসা

২৩.১৪ অধ্যায়ের সারাংশ

আপগ্রেড করুন ধীরে: control plane আগে, node পরে, একবারে একটি minor সংস্করণ, প্রথমে staging-এ। etcd হলো ক্লাস্টারের অবস্থার ভান্ডার — `etcdctl snapshot` দিয়ে ব্যাকআপ নিন, নিরাপদে রাখুন, আর পুনরুদ্ধারের মহড়া দিন। দুর্যোগ-

পুনরুদ্ধার পরিকল্পনা RTO ও RPO দিয়ে লিখুন। Managed Kubernetes-এ (EKS, AKS, GKE, IBM Cloud) প্রদানকারী control plane চালায়, আপনি node ও অ্যাপ চালান — খরচ ও দায়িত্বের ভাগ বুঝে সিদ্ধান্ত নিন।

২৩.১৫ মূল প্রযুক্তিগত পরিভাষা

etcd: ক্লাস্টারের সব অবস্থা সংরক্ষণকারী বিতরণকৃত key-value ভান্ডার।

Version Skew Policy: control plane ও node-এর সংস্করণের মধ্যে সর্বোচ্চ অনুমোদিত পার্থক্যের নিয়ম।

RTO: Recovery Time Objective — কতক্ষণের মধ্যে সেবা ফিরতে হবে।

RPO: Recovery Point Objective — সর্বোচ্চ কতটুকু তথ্য হারানো চলবে।

Managed Kubernetes: ক্লাউড প্রদানকারীর পরিচালিত Kubernetes সেবা (যেমন EKS, AKS, GKE)।

kubectl drain: Node থেকে Pod সরিয়ে node-কে রক্ষণাবেক্ষণের জন্য প্রস্তুত করার কমান্ড।

২৩.১৬ জ্ঞান যাচাই

১. আপগ্রেডে control plane আগে কেন, node পরে কেন? ২. Version skew policy কী? ৩. etcd snapshot কীভাবে নেবেন ও যাচাই করবেন? ৪. RTO ও RPO-এর পার্থক্য লিখুন। ৫. etcd ব্যাকআপে কী থাকে, আর কী থাকে না? ৬. Managed বনাম self-managed Kubernetes-এর দুটি সুবিধা ও দুটি অসুবিধা লিখুন। ৭. EKS, AKS, GKE-এর IAM সংযোগের পার্থক্য কী? ৮. পুনরুদ্ধারের মহড়া কেন দরকার?

২৩.১৭ ব্যবহারিক অ্যাসাইনমেন্ট

TechNova-র জন্য একটি দুর্যোগ-পুনরুদ্ধার পরিকল্পনা লিখুন: (ক) RTO ২ ঘণ্টা, RPO ১ ঘণ্টা ধরে ব্যাকআপের সময়সূচি লিখুন (etcd, ডাটাবেস, Git); (খ) আপগ্রেডের ধাপে ধাপে পরিকল্পনা লিখুন (১.৩০ থেকে ১.৩১); (গ) EKS, AKS, GKE ও IBM Cloud-এর তুলনা সারণি পূরণ করে TechNova-র জন্য একটি সুপারিশ লিখুন (মূল্য ছাড়া, শুধু দায়িত্ব, IAM ও পরিচালনার দিক থেকে); (ঘ) একটি node drain-এর মহড়া staging-এ করে তার ফলাফল লিখুন।

অধ্যায় 24 • PART V — Production Kubernetes Engineering (ch 21–24)

Chapter 24 — Production Architecture, Reliability, GitOps Introduction, and Operations Runbooks

২৪.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন — production-এর জন্য উচ্চ-প্রাপ্যতা (HA) স্থাপত্য নকশা করতে (HA control plane, multi-AZ node, PodDisruptionBudget, topology spread); নির্ভরযোগ্যতার উপাদান — probe, resource ব্যবস্থাপনা, anti-affinity — একসঙ্গে সাজাতে; **GitOps** পরিচিতি ব্যাখ্যা করতে (Git-এ desired state, Argo CD/Flux দিয়ে reconciliation); এবং deploy, rollback, incident ও backup যাচাইয়ের জন্য **operations runbook** লিখতে। সিরিজের সমাপ্তি হিসেবে আপনি পরবর্তী ধাপ — ক্লাউড প্রদানকারীর ভলিউমগুলো — সম্পর্কেও ধারণা পাবেন।

২৪.২ কেন এই বিষয়টি গুরুত্বপূর্ণ

একটি অ্যাপ চালানো আর production-এ নির্ভরযোগ্যভাবে চালানো — দুটি আলাদা জিনিস। Production মানে: একটি node মরে গেলেও সেবা চলবে, একটি অঞ্চল (zone) ডাউন হলেও ব্যবহারকারীরা টের পাবে না, ভুল deploy হলে কয়েক মিনিটে ফেরা যাবে, আর রাত ৩টায় সমস্যা হলে দায়িত্বপ্রাপ্ত ব্যক্তি জানবে কী করতে হবে। TechNova-র গ্রাহকরা ২৪ ঘণ্টা সেবা আশা করে; একটি zone-এর ব্যর্থতায় পুরো সাইট ডাউন হওয়া গ্রহণযোগ্য নয়। GitOps ও runbook এই নির্ভরযোগ্যতাকে ব্যক্তিনির্ভরতা থেকে ব্যবস্থানির্ভরতায় বদলে দেয় — কে আছে, তার ওপর নয়; কী লেখা আছে, তার ওপর ভরসা করা যায়। এই অধ্যায় পুরো সিরিজের সারসংক্ষেপ: আগের ২৩ অধ্যায়ের সবকিছু এখানে একসঙ্গে মিলে একটি production ব্যবস্থা হয়।

> **REAL-WORLD:** TechNova-র একটি availability zone-এ একবার বিদ্যুৎ বিভ্রাট হয়। কিন্তু Pod-গুলো তিনটি zone-এ ছড়িয়ে ছিল (topology spread), আর PodDisruptionBudget নিশ্চিত করেছিল যে একসঙ্গে অর্ধেকের বেশি Pod ডাউন হয় না — ব্যবহারকারীরা শুধু সামান্য ধীরগতি টের পায়, সাইট ডাউন হয় না।

২৪.৩ পূর্বশর্ত জ্ঞান

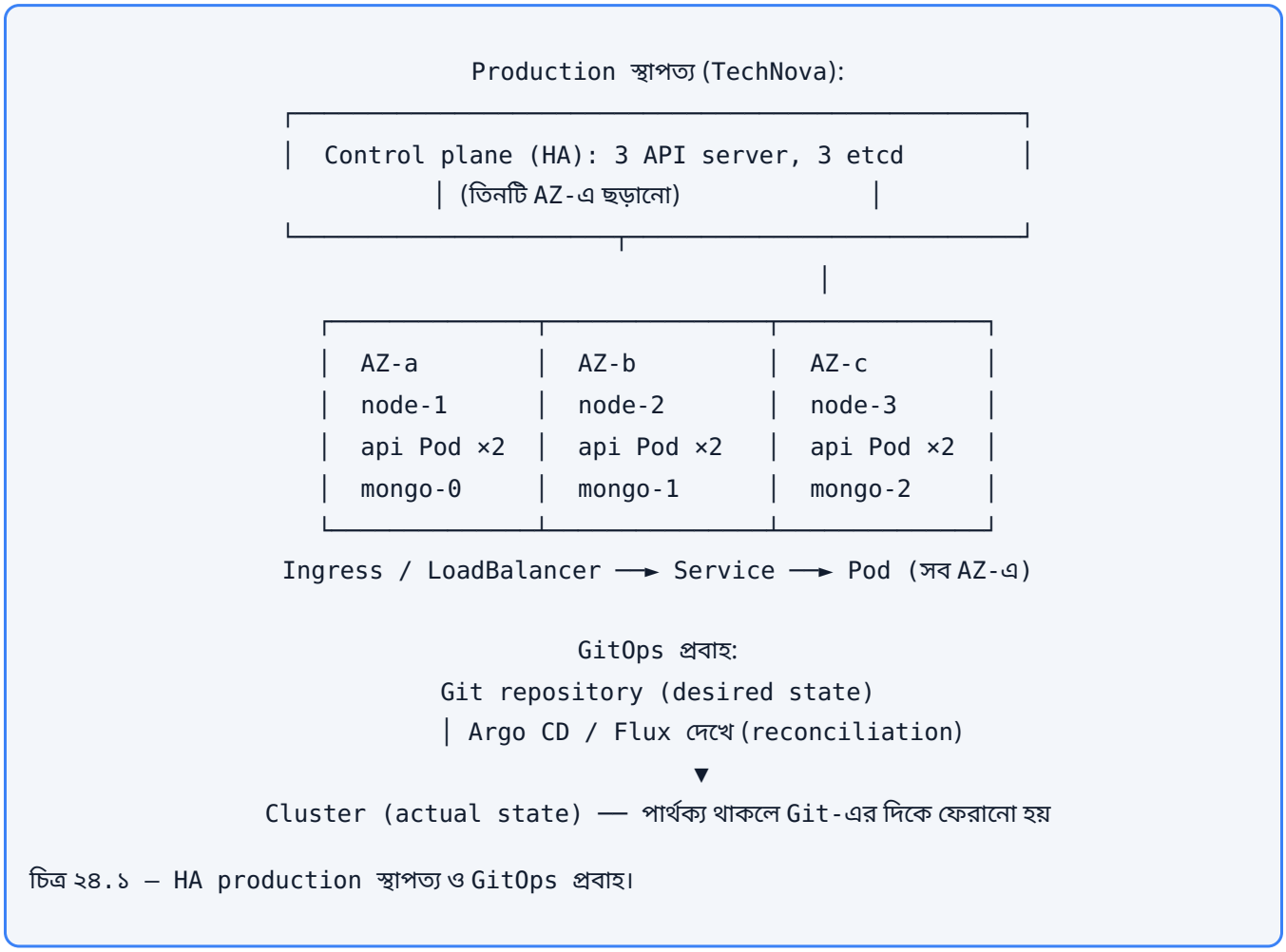
আপনার জানা থাকা উচিত: এই সিরিজের আগের সব অধ্যায় — বিশেষ করে Deployment, Service, Ingress, RBAC, Pod Security, NetworkPolicy, Monitoring, Helm/Kustomize, আপগ্রেড ও ব্যাকআপ। Git-এর মৌলিক ধারণা (commit, repository) জানা থাকলে GitOps বুঝতে সুবিধা হবে।

২৪.৪ প্রথম নীতি থেকে ব্যাখ্যা

প্রথম নীতি: যেকোনো একক উপাদান ব্যর্থ হবে — node মরবে, zone ডাউন হবে, মানুষ ভুল করবে। নির্ভরযোগ্যতা মানে একক ব্যর্থতা (single point of failure) দূর করা। HA control plane মানে API server ও etcd একাধিক জায়গায় চলা; multi-AZ node মানে Pod একাধিক zone-এ ছড়িয়ে থাকা। **PodDisruptionBudget (PDB)** মানে স্বেচ্ছায় (যেমন drain) কতগুলো Pod একসঙ্গে বন্ধ করা যাবে, তার সীমা। **topology spread** মানে Pod-গুলোকে zone ও node-জুড়ে সমানভাবে ছড়িয়ে দেওয়া। **GitOps**-এর প্রথম নীতি: ক্লাস্টারের কাঙ্ক্ষিত অবস্থা (desired state) Git-এ লেখা থাকে; একটি এজেন্ট (Argo CD বা Flux) ক্রমাগত Git ও ক্লাস্টার মেলায় — পার্থক্য থাকলে Git-এর দিকে ক্লাস্টারকে ফিরিয়ে আনে (reconciliation)। এতে deploy মানে Git-এ commit করা, আর ক্লাস্টারে হাতে বদলানো কিছু টেকে না — কারণ এজেন্ট তা ফিরিয়ে দেবে। **Runbook** হলো লিখিত পদ্ধতি: সমস্যা হলে ধাপে ধাপে কী করতে হবে — যাতে রাত ৩টায়ও কেউ অনুমান না করে, লেখা অনুসরণ করে।

> **CONCEPT:** GitOps = Git-ই সত্যের উৎস (source of truth)। ক্লাস্টার Git-কে অনুসরণ করে, উল্টোটা নয়। হাতে বদলানো কিছু টেকে না — এটাই এর শক্তি।

২৪.৫ আর্কিটেকচার / ধারণাগত চিত্র



চিত্র ২৪.১ — HA production স্থাপত্য ও GitOps প্রবাহ।

২৪.৬ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

২৪.৬.১ HA Control Plane ও Multi-AZ Node

Production control plane-এ কমপক্ষে তিনটি etcd সদস্য ও একাধিক API server থাকা উচিত, তিনটি availability zone-এ ছড়ানো — যাতে একটি zone ডাউন হলেও quorum (সংখ্যাগরিষ্ঠতা) থাকে। etcd-এর quorum নীতি: তিন সদস্যের মধ্যে দুটি বেঁচে থাকলে ক্লাস্টার চলে; তাই সদস্য সংখ্যা বিজোড় রাখুন (৩ বা ৫)। Node-ও একাধিক AZ-এ রাখুন, আর Pod-গুলোকে `topologySpreadConstraints` দিয়ে zone-জুড়ে ছড়িয়ে দিন `podAntiAffinity` দিয়ে একই অ্যাপের Pod-গুলোকে একই node-এ বসতে বাধা দিন।

২৪.৬.২ PodDisruptionBudget (PDB)

PDB বলে স্বেচ্ছায় ব্যাহতকরণে (যেমন node drain, আপগ্রেড) কমপক্ষে কতগুলো Pod চলতে থাকবে। `minAvailable: 2` মানে সবসময় অন্তত ২টি Pod চলবে; অথবা `maxUnavailable: 1` মানে একসঙ্গে ১টির বেশি Pod বন্ধ করা যাবে না। PDB অনিচ্ছাকৃত

ব্যর্থতায় (যেমন node হঠাৎ মরে যাওয়া) কাজ করে না — শুধু স্বেচ্ছায় কাজে বাধা দেয়। PDB ছাড়া drain করলে একসঙ্গে সব Pod বন্ধ হয়ে যেতে পারে।

PodDisruptionBudget (PDB): স্বেচ্ছায় ব্যাহতকরণে কমপক্ষে কতগুলো Pod চলবে, তা নির্ধারণকারী নীতি।

GitOps: Git-কে কাঙ্ক্ষিত অবস্থার উৎস ধরে, স্বয়ংক্রিয় এজেন্ট দিয়ে ক্লাস্টারকে সেই অবস্থায় রাখার পদ্ধতি।

Reconciliation: Git-এর কাঙ্ক্ষিত অবস্থা ও ক্লাস্টারের আসল অবস্থার পার্থক্য মিলিয়ে নেওয়ার ক্রমাগত প্রক্রিয়া।

২৪.৬.৩ নির্ভরযোগ্যতার উপাদান একসঙ্গে

Probe (liveness/readiness) — অসুস্থ Pod ধরা ও সরানো; resource requests/limits — একটি Pod যেন অন্যের resource না খায়; PDB — drain-এ ন্যূনতম প্রাপ্যতা; anti-affinity ও topology spread — Pod ছড়িয়ে রাখা; HPA — চাহিদা অনুযায়ী Pod বাড়ানো; আর Pod-এর সংখ্যা — কমপক্ষে ২–৩ replica, যাতে একটি মরলেও সেবা চলে। এগুলো একসঙ্গে মিলে নির্ভরযোগ্যতা তৈরি করে — কোনো একটিই যথেষ্ট নয়।

২৪.৬.৪ GitOps পরিচিতি — Argo CD / Flux

GitOps-এ deploy-এর প্রবাহ: ডেভেলপার Git-এ YAML বা Helm chart হালনাগাদ করে, pull request-এ পর্যালোচনা হয়, merge হলে Argo CD বা Flux তা দেখে ক্লাস্টারে প্রয়োগ করে। সুবিধা: (১) সব পরিবর্তনের ইতিহাস Git-এ থাকে; (২) rollback মানে Git-এ আগের commit-এ ফেরা; (৩) ক্লাস্টারে হাতে বদলানো কিছু ধরা পড়ে (drift detection) ও ফিরিয়ে দেওয়া হয়; (৪) কে কী বদলেছে, তা জানা যায়। Argo CD-এর একটি ওয়েব UI আছে যেখানে অ্যাপের অবস্থা দেখা যায়; Flux হালকা ও Git-কেন্দ্রিক। দুটিই Helm ও Kustomize সমর্থন করে। এই বইয়ে এদের পূর্ণ টিউটোরিয়াল দেওয়া হয়নি — উচ্চ স্তরের পরিচিতি হিসেবে জানুন: Git-এ desired state, এজেন্ট reconciliation করে।

২৪.৬.৫ Operations Runbook — লিখিত পদ্ধতি

Runbook হলো একটি পরিস্থিতির জন্য লিখিত ধাপে ধাপে নির্দেশনা — যাতে জরুরি মুহূর্তে কেউ অনুমান না করে। ভালো runbook-এ থাকে: (১) লক্ষণ ও কীভাবে শনাক্ত করবেন; (২) প্রভাব (কোন ব্যবহারকারী ক্ষতিগ্রস্ত); (৩) তাৎক্ষণিক ব্যবস্থা (সেবা ফেরানো); (৪) মূল কারণ খোঁজার ধাপ; (৫) যোগাযোগ — কাকে জানাবেন; (৬) সমাপ্তির শর্ত। Runbook নিয়মিত হালনাগাদ করুন, আর মহড়া দিন — লেখা থাকলেই হবে না, চর্চা করতে হবে।

২৪.৭ কনফিগারেশন উদাহরণ

```
# PodDisruptionBudget: সবসময় অন্তত 2টি Pod চলবে
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: api-pdb
  namespace: technova
spec:
  minAvailable: 2
  selector:
    matchLabels:
      app: api
---
# Topology Spread: Pod তিনটি zone-এ সমানভাবে ছড়ানো
apiVersion: apps/v1
kind: Deployment
metadata:
  name: api
  namespace: technova
spec:
  replicas: 6
  selector:
    matchLabels:
      app: api
  template:
    metadata:
      labels:
        app: api
    spec:
      topologySpreadConstraints:
      - maxSkew: 1
        topologyKey: topology.kubernetes.io/zone
        whenUnsatisfiable: DoNotSchedule
        labelSelector:
          matchLabels:
            app: api
      affinity:
        podAntiAffinity:
          preferredDuringSchedulingIgnoredDuringExecution:
          - weight: 100
            podAffinityTerm:
              labelSelector:
                matchLabels:
                  app: api
              topologyKey: kubernetes.io/hostname
      containers:
      - name: api
        image: technova/api:v1.2.0
        ports:
        - containerPort: 3000
        livenessProbe:
          httpGet:
            path: /health
            port: 3000
          initialDelaySeconds: 15
```

```
          periodSeconds: 10
        readinessProbe:
          httpGet:
            path: /ready
            port: 3000
          initialDelaySeconds: 5
          periodSeconds: 5
        resources:
          limits:
            cpu: 500m
            memory: 512Mi
          requests:
            cpu: 100m
            memory: 128Mi
```

```
# GitOps-এর ধারণাগত Argo CD Application (সংক্ষিপ্ত)
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: technova
  namespace: argocd
spec:
  project: default
  source:
    repoURL: https://git.technova.example/technova/k8s.git
    targetRevision: main
    path: overlays/prod
  destination:
    server: https://kubernetes.default.svc
    namespace: technova
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
```

উপরের Argo CD উদাহরণটি ধারণাগত — repoURL-টি TechNova-র কাল্পনিক উদাহরণ; আপনার নিজের Git repository-এর ঠিকানা ব্যবহার করুন। `selfHeal: true` মানে ক্লাস্টারে হাতে বদলানো কিছু Argo CD Git-এর অবস্থায় ফিরিয়ে দেবে।

২৪.৮ বাস্তব প্রকৌশল পরিস্থিতি

TechNova-র production-এ একবার ভুল deploy হয় — নতুন সংস্করণে একটি bug, ব্যবহারকারীরা লগইন করতে পারছে না। GitOps থাকায় সমাধান সহজ: Git-এ আগের commit-এ revert করা হয়, Argo CD স্বয়ংক্রিয়ভাবে আগের সংস্করণ ক্লাস্টারে ফিরিয়ে আনে — ৫ মিনিটে সেবা ফিরে আসে। একই সঙ্গে runbook অনুযায়ী দল গ্রাহকদের জানায়, incident নোট লেখে, আর bug-এর মূল কারণ খুঁজে বের করে। আগে (GitOps-এর আগে) একই ঘটনায় হাতে kubectl apply করতে হতো, কে কী করেছে তা জানা যেত না, আর ফিরতে ৩০ মিনিট লাগত। এই ঘটনার পর দল নিয়ম করে: production-এ কোনো হাতে পরিবর্তন নয় — সব Git দিয়ে।

২৪.৯ ধাপে ধাপে ব্যবহারিক উদাহরণ

Deploy runbook: (১) Git-এ পরিবর্তন commit করুন, pull request-এ পর্যালোচনা নিন। (২) Staging-এ deploy করে পরীক্ষা করুন। (৩) Production-এ deploy করুন (Helm upgrade বা GitOps sync)। (৪ `kubectl rollout status` দিয়ে deploy পর্যবেক্ষণ করুন। (৫) Grafana-তে error rate ও latency দেখুন। (৬) সমস্যা হলে rollback করুন। **Rollback runbook:** (১) `helm rollback` বা Git revert করুন। (২) Pod Running কি না দেখুন। (৩) গ্রাহককে জানান।**Incident runbook:** (১) লক্ষণ লিখুন। (২) `kubectl get pods`, describe, logs দিয়ে তথ্য সংগ্রহ করুন (অধ্যায় ১৯)। (৩) তাৎক্ষণিক ব্যবস্থা নিন (সেবা ফেরানো)। (৪) মূল কারণ খুঁজুন। (৫) Incident নোট লিখুন।**Backup যাচাই runbook:** (১) etcd snapshot নিন। (২) Staging-এ পুনরুদ্ধারের মহড়া দিন। (৩) ফলাফল লিখে রাখুন।

> **LAB:** TechNova-র জন্য একটি এক পৃষ্ঠার incident runbook লিখুন: "API Pod CrashLoopBackOff" — লক্ষণ, পরীক্ষার কমান্ড, তাৎক্ষণিক ব্যবস্থা, আর যোগাযোগের তালিকা।

২৪.১০ সাধারণ ভুল ও সমস্যা সমাধান

সমস্যা	কারণ	সমাধান
একটি zone ডাউন হলে সেবা ডাউন	সব Pod এক zone-এ	Topology spread ও multi-AZ node
Drain-এ সব Pod বন্ধ	PDB নেই	PDB তৈরি করুন
GitOps sync হচ্ছে না	Git-এর পথ বা credential ভুল	Argo CD/Flux-এর লগ ও Application অবস্থা দেখুন
হাতে বদলানো কিছু বারবার ফিরে আসে	GitOps self-heal কাজ করছে	এটাই স্বাভাবিক — বদল Git-এ করুন
Rollback-এও সমস্যা থেকে যায়	ডাটাবেসের পরিবর্তন ফেরানো হয়নি	ডাটাবেস migration-এর rollback পরিকল্পনা আলাদা রাখুন

২৪.১১ নিরাপত্তা বিবেচনা

Production-এ API server কখনোই পাবলিক ইন্টারনেটে খুলে দেবেন না — শুধু VPN বা নির্দিষ্ট IP থেকে প্রবেশ দিন। GitOps repository-এর প্রবেশ সীমিত রাখুন — যে Git-এ লিখতে পারে, সে production-এ deploy করতে পারে। Argo CD ও Flux-এর প্রবেশ RBAC দিয়ে সুরক্ষিত করুন। Secret Git-এ রাখবেন না — Sealed Secrets বা External Secrets ব্যবহার করুন। নিয়মিত নিরাপত্তা আপডেট (আপগ্রেড) করুন, আর Pod Security Standards enforce করুন (অধ্যায় ১৮)।

> **WARNING:** GitOps মানে Git-এ লেখা সবকিছুই production-এ যাবে — তাই Git repository-এর প্রবেশ নিয়ন্ত্রণ production-এর প্রবেশ নিয়ন্ত্রণের সমান গুরুত্বপূর্ণ।

২৪.১২ কর্মক্ষমতা ও খরচ বিবেচনা

HA control plane ও multi-AZ node নির্ভরযোগ্যতা বাড়ায়, কিন্তু খরচও বাড়ায় — তিনটি zone-এ node রাখা এক zone-এর চেয়ে ব্যয়বহুল। Replica সংখ্যা বাড়ালে প্রাপ্যতা বাড়ে, কিন্তু খরচও বাড়ে — ব্যবসার প্রয়োজনে ভারসাম্য রাখুন। HPA দিয়ে চাহিদা অনুযায়ী Pod বাড়ানো-কমানো খরচ বাঁচায়। GitOps-এর খরচ কম — Argo CD বা Flux হালকা, কিন্তু তাদের নিজেদেরও পর্যবেক্ষণ দরকার। নিয়মিত `kubectl top` দিয়ে অতিরিক্ত requests খুঁজে বের করুন — অতিরিক্ত requests মানে অপ্রয়োজনীয় node-এর খরচ।

২৪.১৩ সেরা অনুশীলন ও অ্যান্টি-প্যাটার্ন

সেরা অনুশীলন	অ্যান্টি-প্যাটার্ন
HA control plane, multi-AZ, PDB, topology spread	একটি node/zone-এ সব Pod রাখা
GitOps: সব পরিবর্তন Git দিয়ে	Production-এ হাতে kubectl apply
Runbook লেখা ও মহড়া দেওয়া	জরুরি মুহূর্তে অনুমান করা
Deploy-এর আগে staging-এ পরীক্ষা	সরাসরি production-এ পরীক্ষা
নিয়মিত ব্যাকআপ যাচাই	ব্যাকআপ নিয়ে কখনো পরীক্ষা না করা

২৪.১৪ অধ্যায়ের সারাংশ

Production মানে একক ব্যর্থতা দূর করা: HA control plane, multi-AZ node, PDB, topology spread, anti-affinity, probe ও resource ব্যবস্থাপনা একসঙ্গে কাজ করে। GitOps-এ Git-ই সত্যের উৎস; Argo CD বা Flux ক্রমাগত ক্লাস্টারকে Git-এর অবস্থায় রাখে। Runbook লিখিত পদ্ধতি দেয় — deploy, rollback, incident ও backup যাচাইয়ের জন্য। এই অধ্যায় দিয়ে সিরিজের Kubernetes অংশ শেষ হলো — পরবর্তী ধাপে ক্লাউড প্রদানকারীর ভলিউমগুলোতে (EKS, AKS, GKE-এর গভীরে) এই জ্ঞান প্রয়োগ করবেন।

সিরিজের সমাপ্তি: আপনি এখন container থেকে Pod, Deployment থেকে Service, RBAC থেকে NetworkPolicy, Helm থেকে Kustomize, আপগ্রেড থেকে GitOps — পুরো যাত্রা জানেন। পরবর্তী ধাপ: একটি ছোট production ক্লাস্টার নিজে তৈরি

করুন, TechNova অ্যাপ deploy করুন, আর প্রতিটি অধ্যায়ের অ্যাসাইনমেন্ট সম্পূর্ণ করুন। শুভকামনা — আপনার DevOps যাত্রা এখান থেকে শুরু।

২৪.১৫ মূল প্রযুক্তিগত পরিভাষা

High Availability (HA): একক উপাদান ব্যর্থ হলেও সেবা চলতে থাকার নকশা।

PodDisruptionBudget: স্বেচ্ছায় ব্যাহতকরণে কমপক্ষে কতগুলো Pod চলবে, তা নির্ধারণকারী নীতি।

Topology Spread: Pod-গুলোকে zone ও node-জুড়ে সমানভাবে ছড়িয়ে দেওয়ার নিয়ম।

GitOps: Git-কে কাঙ্ক্ষিত অবস্থার উৎস ধরে ক্লাস্টার পরিচালনার পদ্ধতি।

Argo CD / Flux: GitOps এজেন্ট — Git ও ক্লাস্টারের মধ্যে reconciliation করে।

Runbook: একটি পরিস্থিতির জন্য লিখিত ধাপে ধাপে পদ্ধতি।

Reconciliation: কাঙ্ক্ষিত ও আসল অবস্থার পার্থক্য মিলিয়ে নেওয়ার প্রক্রিয়া।

২৪.১৬ জ্ঞান যাচাই

১. HA control plane-এ etcd সদস্য সংখ্যা বিজোড় কেন রাখা হয়? ২. PDB কী, আর এটি কখন কাজ করে না? ৩. Topology spread ও anti-affinity-এর পার্থক্য কী? ৪. GitOps-এ "সত্যের উৎস" কী, আর কেন? ৫.

Reconciliation বলতে কী বোঝায়? ৬. ভালো runbook-এ কী কী থাকা উচিত? ৭. Rollback-এ ডাটাবেসের পরিবর্তন কেন আলাদাভাবে ভাবতে হয়? ৮. Production-এ হাতে kubectl apply করা কেন নিরুৎসাহিত করা হয়?

২৪.১৭ ব্যবহারিক অ্যাসাইনমেন্ট

TechNova-র জন্য একটি সম্পূর্ণ production পরিকল্পনা লিখুন: (ক) HA স্থাপত্যের চিত্র (3 AZ, PDB, topology spread); (খ) GitOps প্রবাহের চিত্র (Git → Argo CD/Flux → ক্লাস্টার); (গ) চারটি runbook — deploy, rollback, incident (API ডাউন), backup যাচাই — প্রতিটি এক পৃষ্ঠা; (ঘ) সিরিজের পরবর্তী ধাপ হিসেবে আপনার নিজের শেখার পরিকল্পনা: ক্লাউড প্রদানকারীর কোন ভলিউমটি প্রথমে পড়বেন, আর কেন? এই পরিকল্পনাটি আপনার production প্রস্তুতির নীলনকশা।